

The Open Source Packet Analyzer

TCP/IP Network Analysis

presented by Rolf Leutert in cooperation with



and

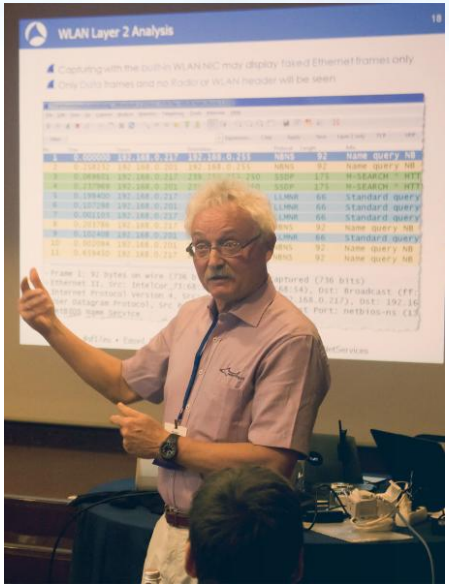


Remote Training: 20./21. October 2026

Morning 9:00 to 12:00 / 3 sessions @ 50 Min.
Afternoon 13:00 to 17:00 / 4 sessions @ 50 Min.
Starting on the hour

In case of connection problems, send an e-mail to
leutert@netsniffing.ch or call **P&W +49 2364 10538-0**

Instructor



Rolf Leutert, El. Ing. HTL
Network Protocol Expert
Leutert NetServices
Zürich-Flughafen

- Network Analysis & Troubleshooting
- Trainings TCP/IP, QUIC, WLAN, VoIP, IPv6

- Certified Wireless Analysis Professional 2026
- Certified Wireless Network Administrator 2026
- Wireshark® Certified Analyst 2025
- Wireshark® Certified Network Analyst 2010
- Wireshark® Instructor 2006
- Sniffer® certified Instructor 1990

leutert@netsniffing.ch
www.netsniffing.ch

Sniffer Certified MASTER
Sniffer Certified INSTRUCTOR
Wireshark University WIRESHARK CERTIFIED NETWORK ANALYST
CERTIFIED ANALYST WIRESHARK
CWAP Certified Wireless Analysis Professional
CWNA Certified Wireless Network Administrator

© Leutert NetServices 2026 / TCP / V4.1f

Copyright © - Information



All the material provided during this training is intended for your **personal use only**.

All rights reserved. No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the publisher.



Above restrictions are also valid for the **Wireshark Trace-Files** provided with the training.

If you wish to use any material for another than the above specified purpose, write a permission requests to the publisher at the address below.

Table of Contents

- Course Objectives 5
- Wireshark Installation, Architecture and new Functions 7
- TCP/IP Protocol Family 24
- Internet Protocol (IP) 26
- Transmission Control Protocol (TCP) 31
- TCP Analysis using the Wireshark Expert and TCP Stream Graph 60
- TCP Extensions for High Performance 79
- TCP Analysis with two Interfaces 94
- TCP Reassembly and Export Functions 99
- Analyzing MTU problems and TCP Segmentation Offloading 102
- Domain Name Service (DNS) 110
- Transport Layer Security (TLS) 118
- Secure Shell (SSH) 132
- User Datagram Protocol (UDP) 134
- Simple Network Management Protocol (SNMP) 140
- NetFlow 152
- New: Quick UDP Internet Connections (QUIC) 157
- Internet Control Message Protocol (ICMP) 174
- Miscellaneous Functions of Wireshark 183

Course Objectives

You will learn how to use the Wireshark Network Analyzer for narrowing down errors, optimizing data transfers and analysing TCP/IP protocol structure and processes.

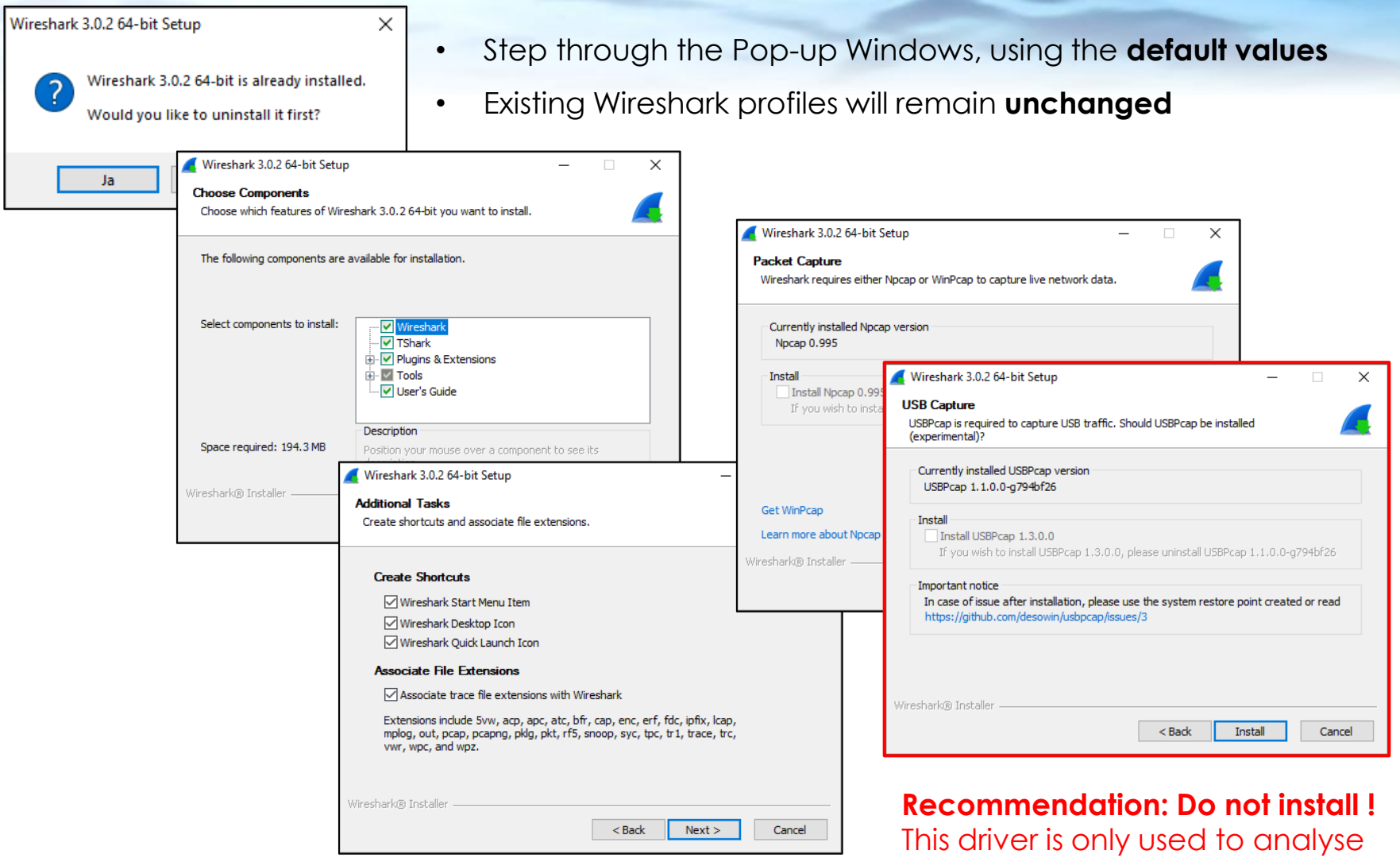
The course covers the following topics:

- New features and functions of Wireshark
- TCP/IP Protocol family
- TCP Analyzing and Tuning
- TCP Expert functions of Wireshark
- Domain Name Service (DNS)
- Transport Layers Security (TLS) and Secure Shell (SSH)
- Analyzing Application Layer Protocols (HTTP)
- Simple Network Management Protocol (SNMP)
- NetFlow Protocol
- Miscellaneous Functions of Wireshark

© Leutert NetServices 2026 / TCP / V4.1f

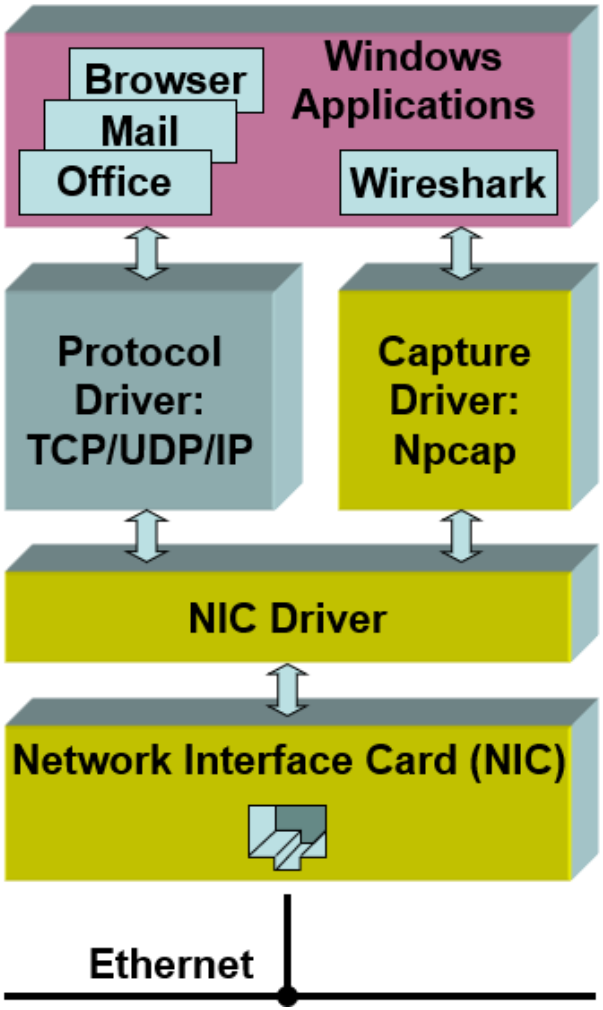
Installation and Setup of Wireshark

- Step through the Pop-up Windows, using the **default values**
- Existing Wireshark profiles will remain **unchanged**



Recommendation: Do not install !
This driver is only used to analyse traffic on USB interfaces.

Wireshark for Ethernet on Windows, MacOS, Linux



- The **NIC** by default is **filtering frames** using the **MAC Destination address**
- Only frames to own **MAC address, broadcast and multicast** are passed.
- To **capture all frames**, the NIC has to support **Promiscuous Mode**

➢ On **Windows** the **Npcap driver** is required https://nmap.org/npcap/

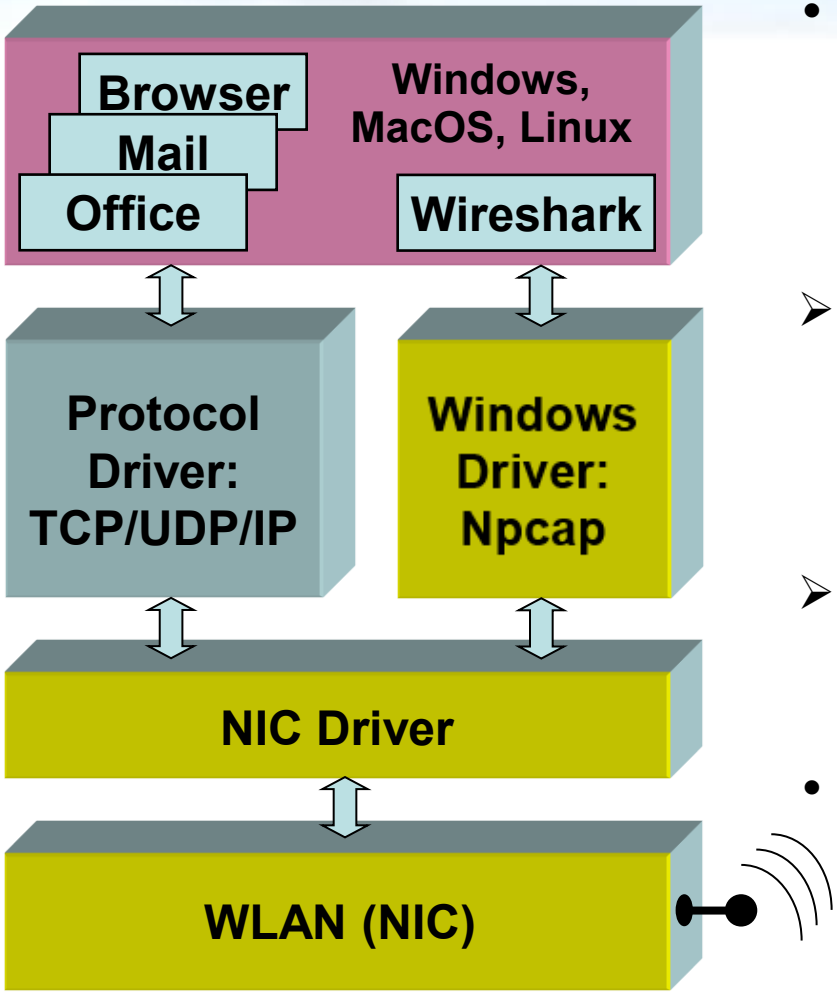
➢ **MacOS** and **Linux drivers** supports **Promiscuous Mode by default**

- Wireshark can run **simultaneously** with other applications, even though it is **not recommended** for several technical reasons (Performance)

+

© Leutert NetServices 2026 / TCP / V4.1f

Wireshark for WLAN on Windows, MacOS, Linux



- To capture the **WLAN Management & Control Frames**, the **Monitor Mode** is required.
  https://wiki.wireshark.org/CaptureSetup/WLAN

➢ On Windows the **Npcap** Driver supports **monitor mode only** on **specified WLAN NICS.**
https://secwiki.org/w/Npcap/WiFi_adapters

➢ On **MacOS & Linux** most NICs are supporting the **WLAN Monitor Mode**

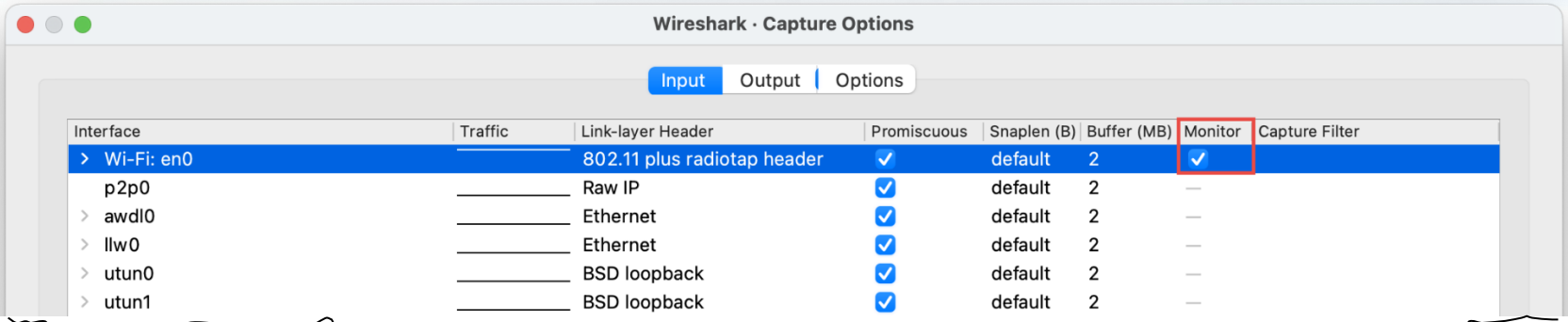
- To capture **simultaneously in multiple channels** (for roaming problems), special hardware is required.
  (i.e. Ekahau Sidekick, Hamina Nomad)

+

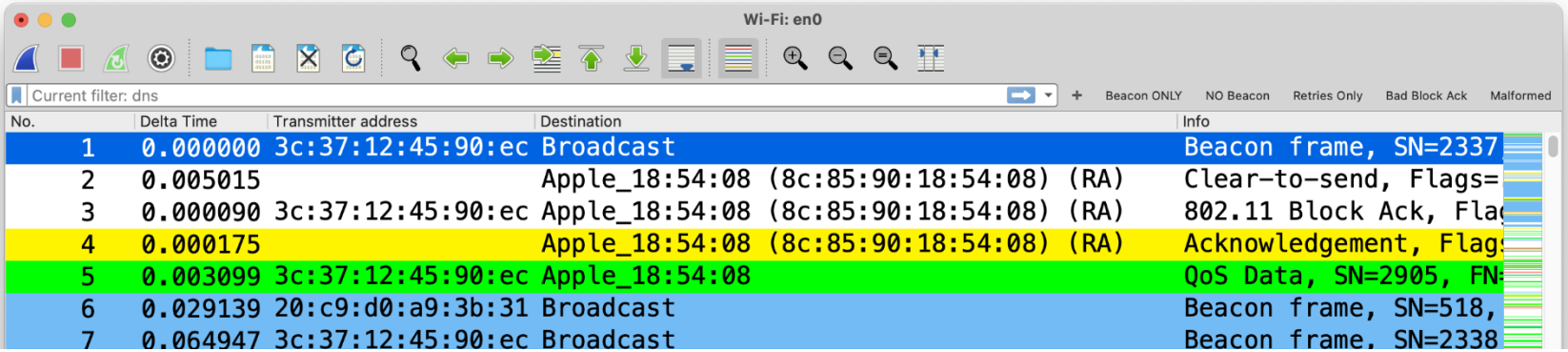
© Leutert NetServices 2026 / TCP / V4.1f

Wireshark for WLAN with Linux, MacOS etc.

- With **Linux/MAC OS** most Adapters support the **Monitor Mode**

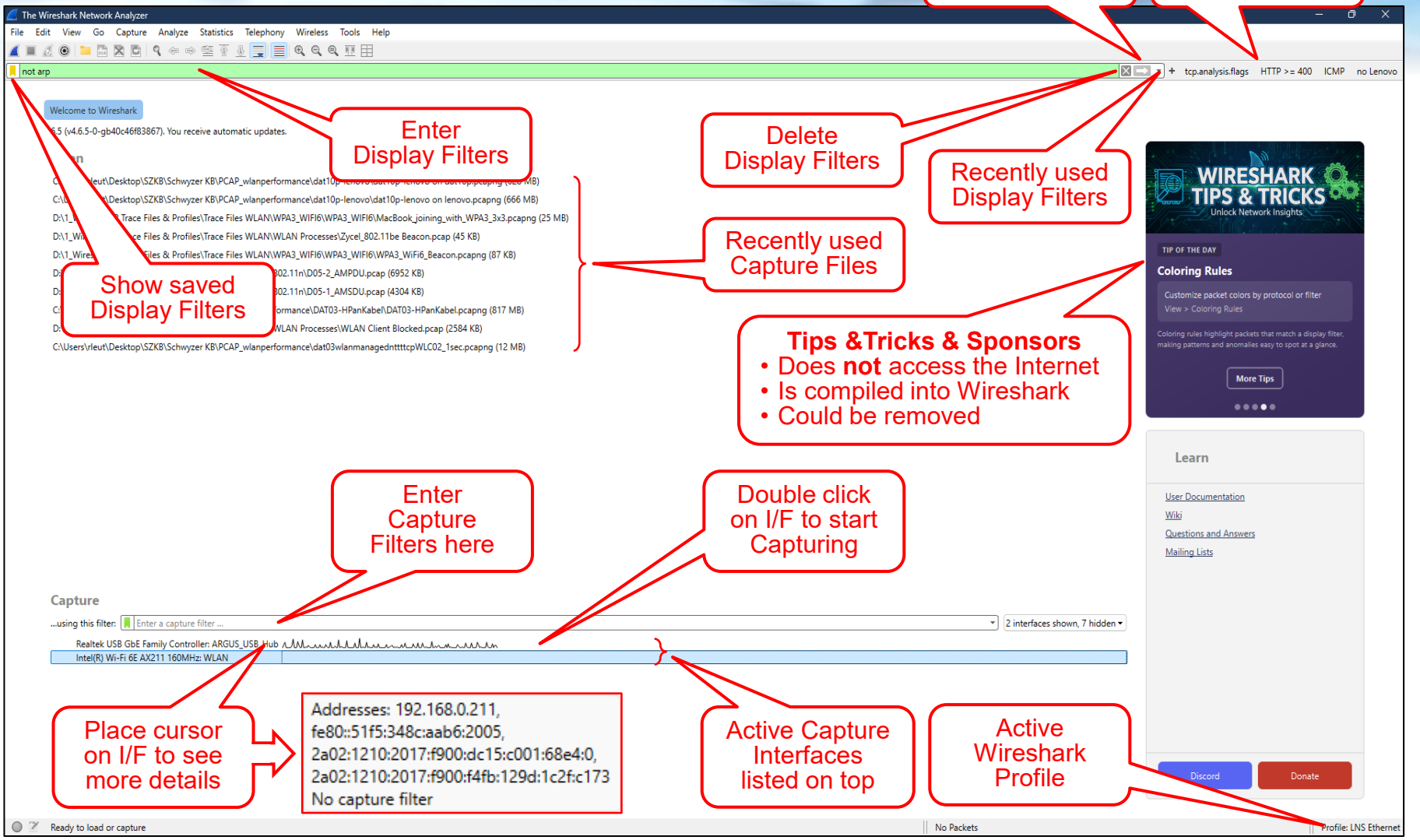


- Monitor Mode shows **Mgmt. & Control Frames** like **Beacon**, **Acks** etc.



© Leutert NetServices 2026 / TCP / V4.1f

Wireshark Welcome Page since V. 4.6.5



Added functions since Wireshark Version 1

Wireshark is constantly being enhanced by the developer community:

Version 2: → Newsletter Jan 2016

- Toolkit **QT** replaces **GTK+**
- Native **MAC OS support** added
- **Language support** added
- **Related Packets** added
- **Intelligent Scrollbar** added
- **USB Interface Driver** added



Version 3: → Newsletter Jan 2020

- **Npcap** replaces **WinPcap** Driver
- **Language support** added
- **Drag & Drop** für **Filter & Column** added

Version 4: → Newsletter Jan 2023

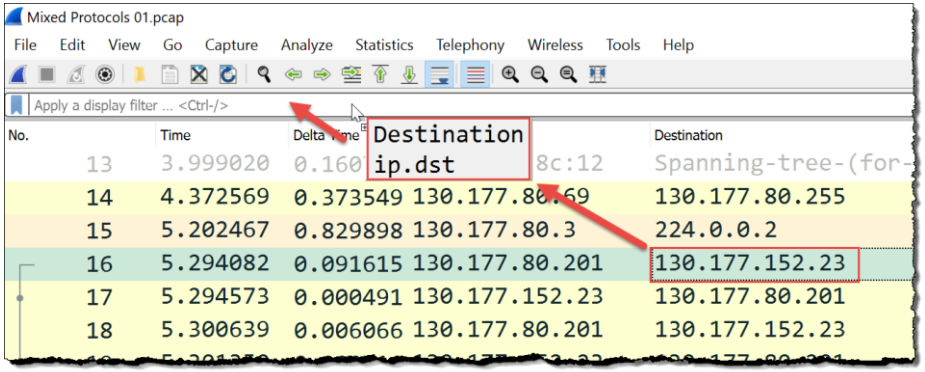
- Active **Interfaces** shown **on top** of interface list.
- **New arrangement** of windows **Packet List, Packet Details, Packet Bytes**
- Many new **Display Filters** like **Layer Operator, Membership, Slice Operator** etc.

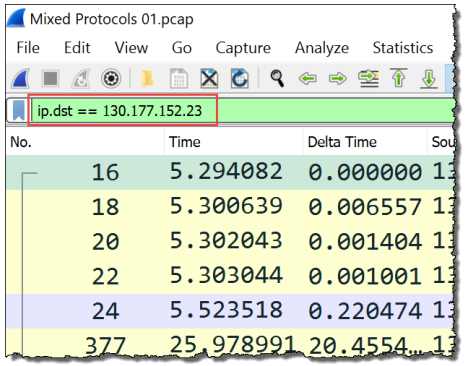
© Leutert NetServices 2026 / TCP / V4.1f

New Wireshark Functions

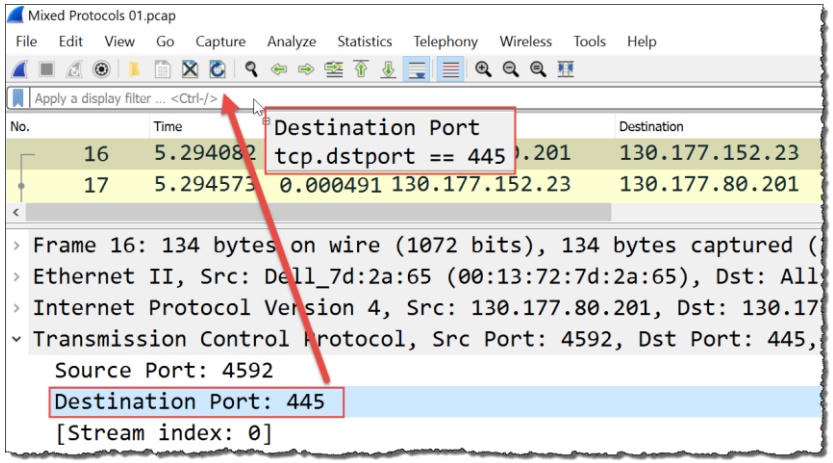
Drag & Drop for **Display Filter**

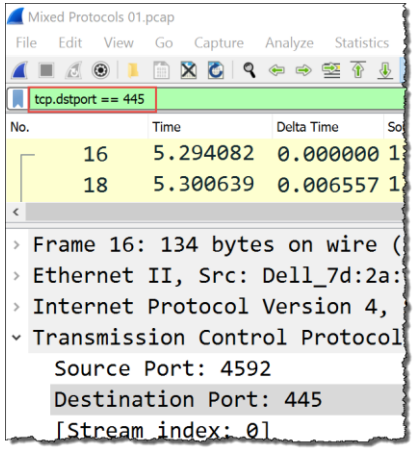
- Click on a field in the and **Packet List** windows, drag & drop the value into the filter bar to apply as a filter. (Exceptions: Info and Protocol columns are summaries)





- The same is possible with fields from the **Packet Details** windows.

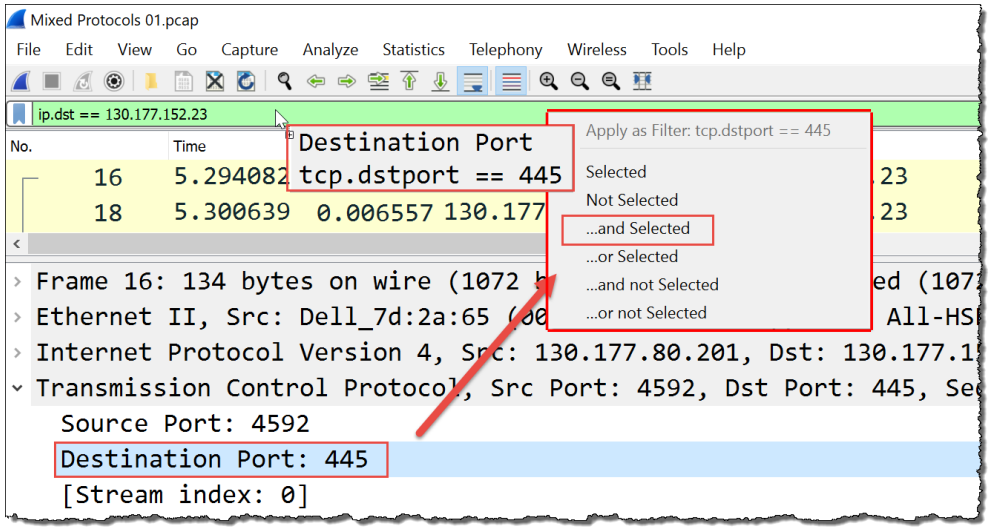


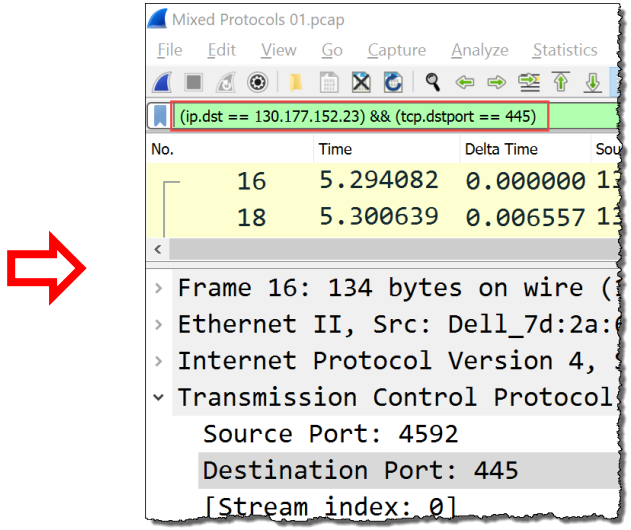


New Wireshark Functions

Drag & Drop for **Display Filter**

- Existing filters can be modified or replaced with any other field
- Drag & Drop a new field to the filter bar; if a filter already exists, a choice of options will appear

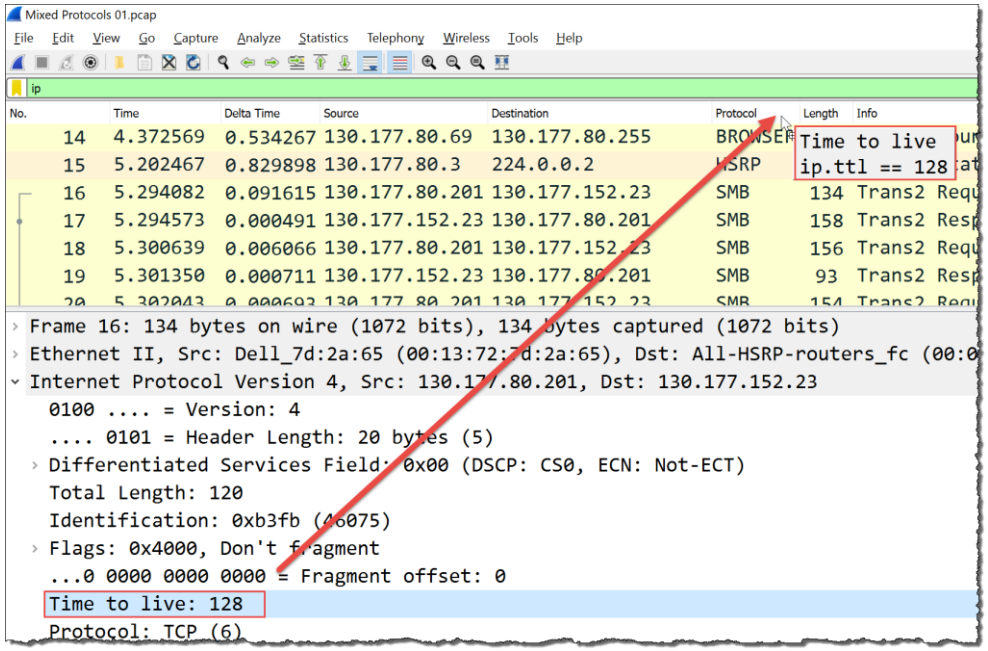




New Wireshark Functions

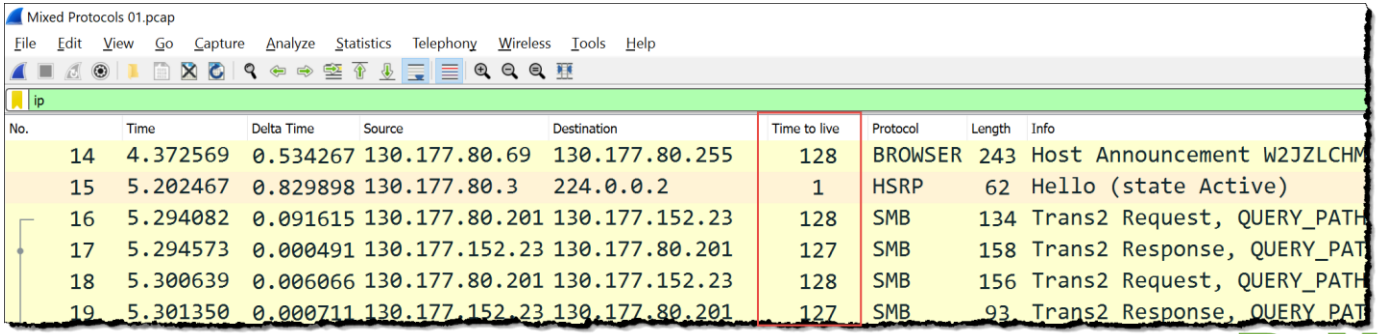
Drag & Drop für **zusätzliche Spalten**

- Additional Columns can be added by dragging and dropping a field to the column title line



- Select the field to add as a new column (i.e. TTL field)
- Drag & Drop the field on an existing column title (i.e. Protocol)
- The new column will be inserted to the right of the existing column

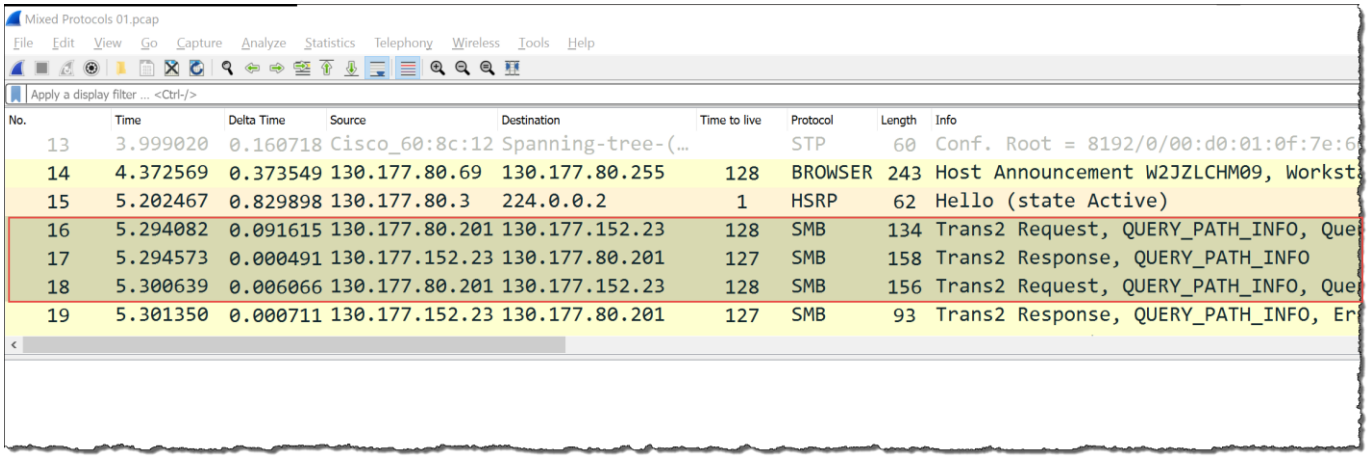




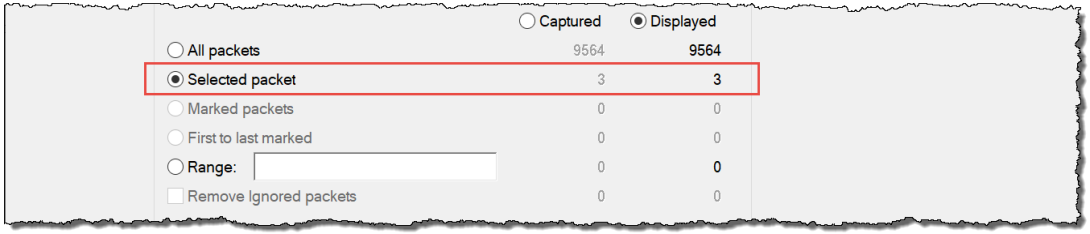
New Wireshark Functions

Marking or Selecting multiple Packets

- A packet can be **marked** by pressing the **centre mouse button** (faster than using the right click)
- In previous Wireshark versions, only one packet could be **selected.** By holding the **Shift key**, multiple packets can be selected now
- If more than one packet is selected, Packet Details and Packet Bytes windows remains empty

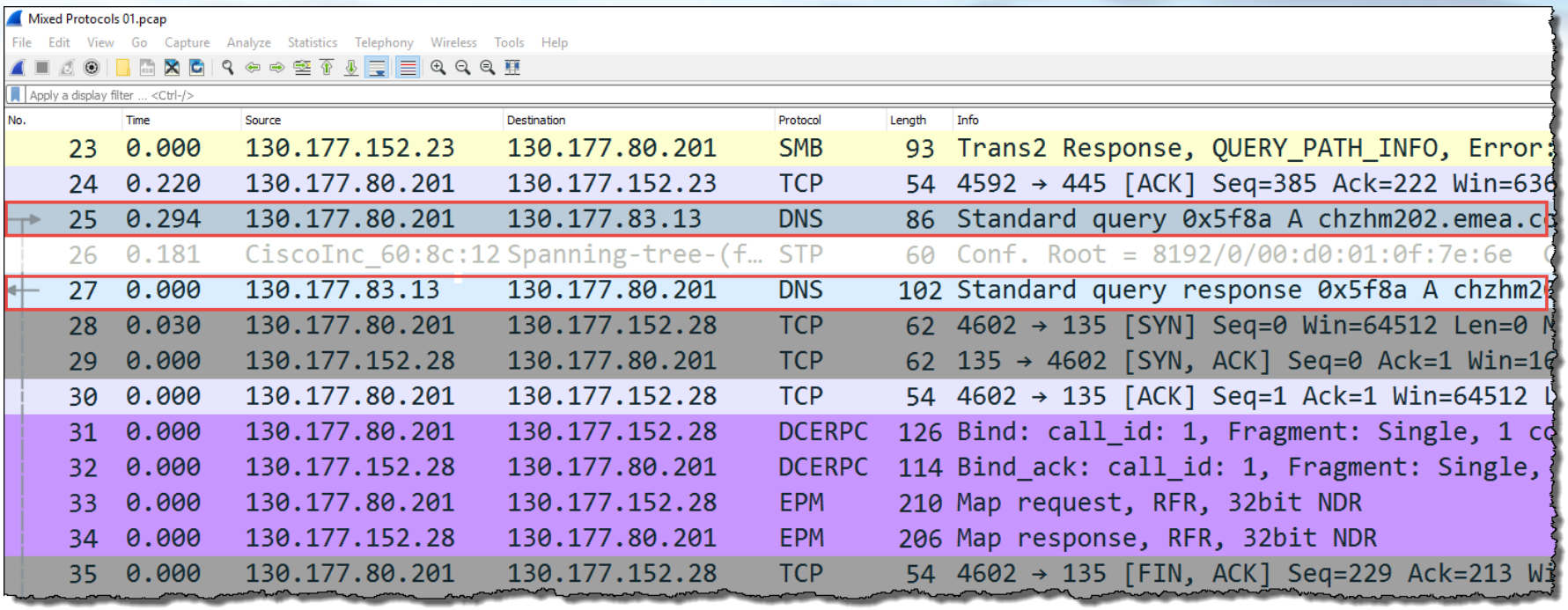


- **Marked** or **selected** packet can be saved with → File → Export Specified Packets…



New Wireshark Functions

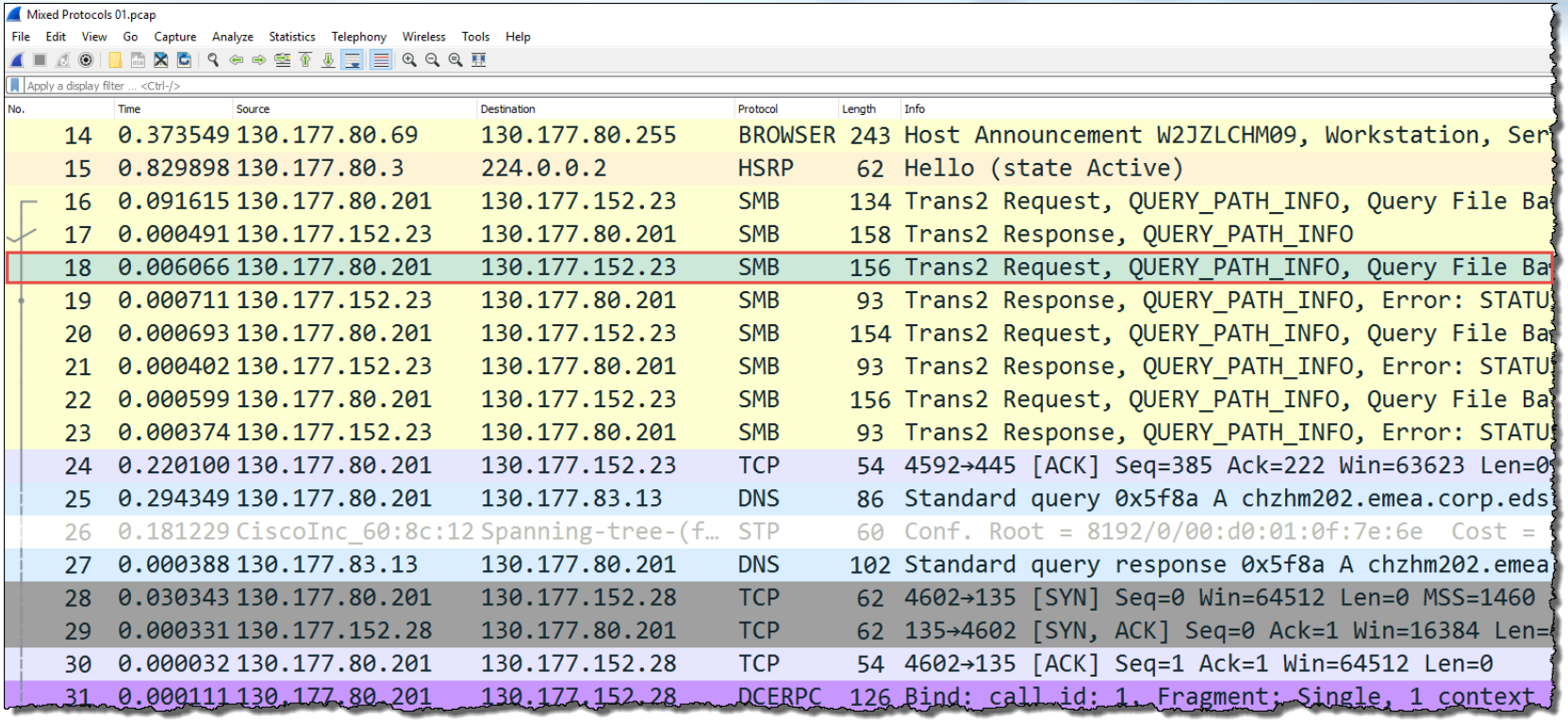
Marking Related Packets



- I selecting a DNS Query or Query Response, Wireshark marks the corresponding DSN packet in the left column with arrows.
- An ongoing broken line indicates that more DNS packets are following further down.

New Wireshark Functions

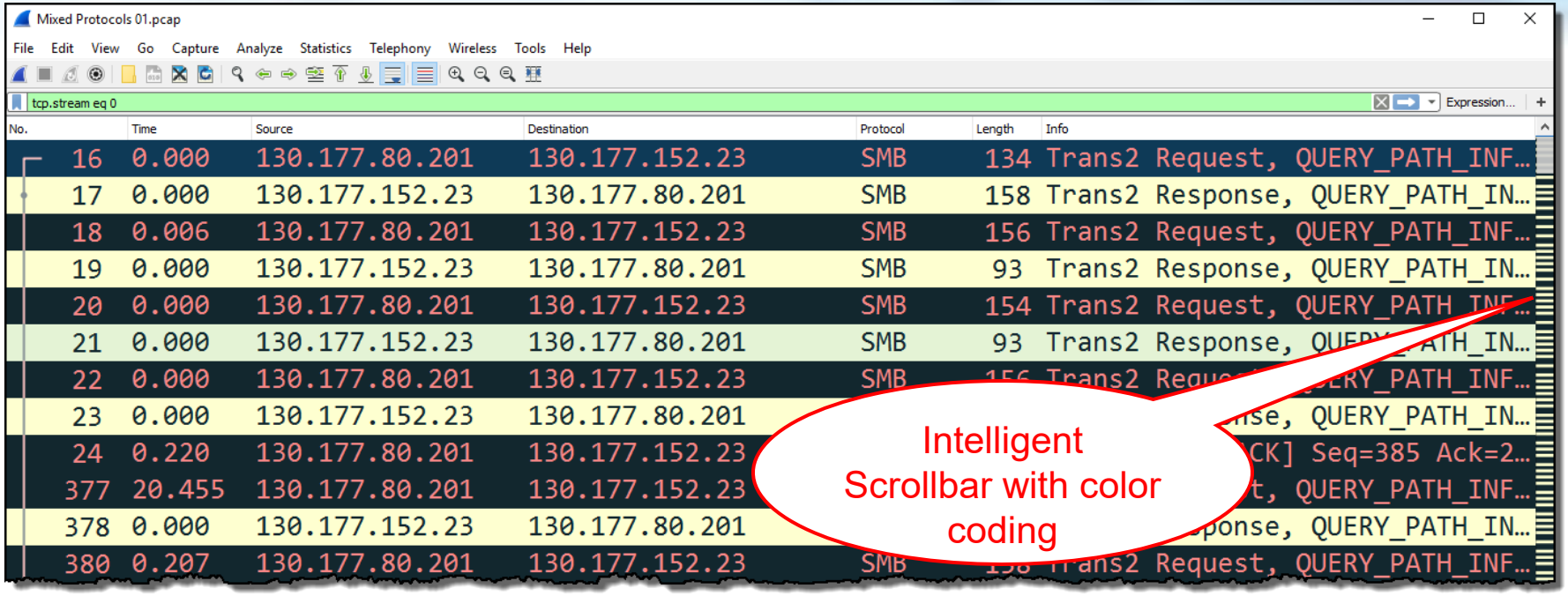
Marking Related Packets



- Selecting a packet belonging to a TCP session, Wireshark marks all packet belonging to the same TCP session with a continuous line.
- Packets not belonging to the selected TCP session are marked with a broken line.

New Wireshark Functions

Intelligent Scrollbar

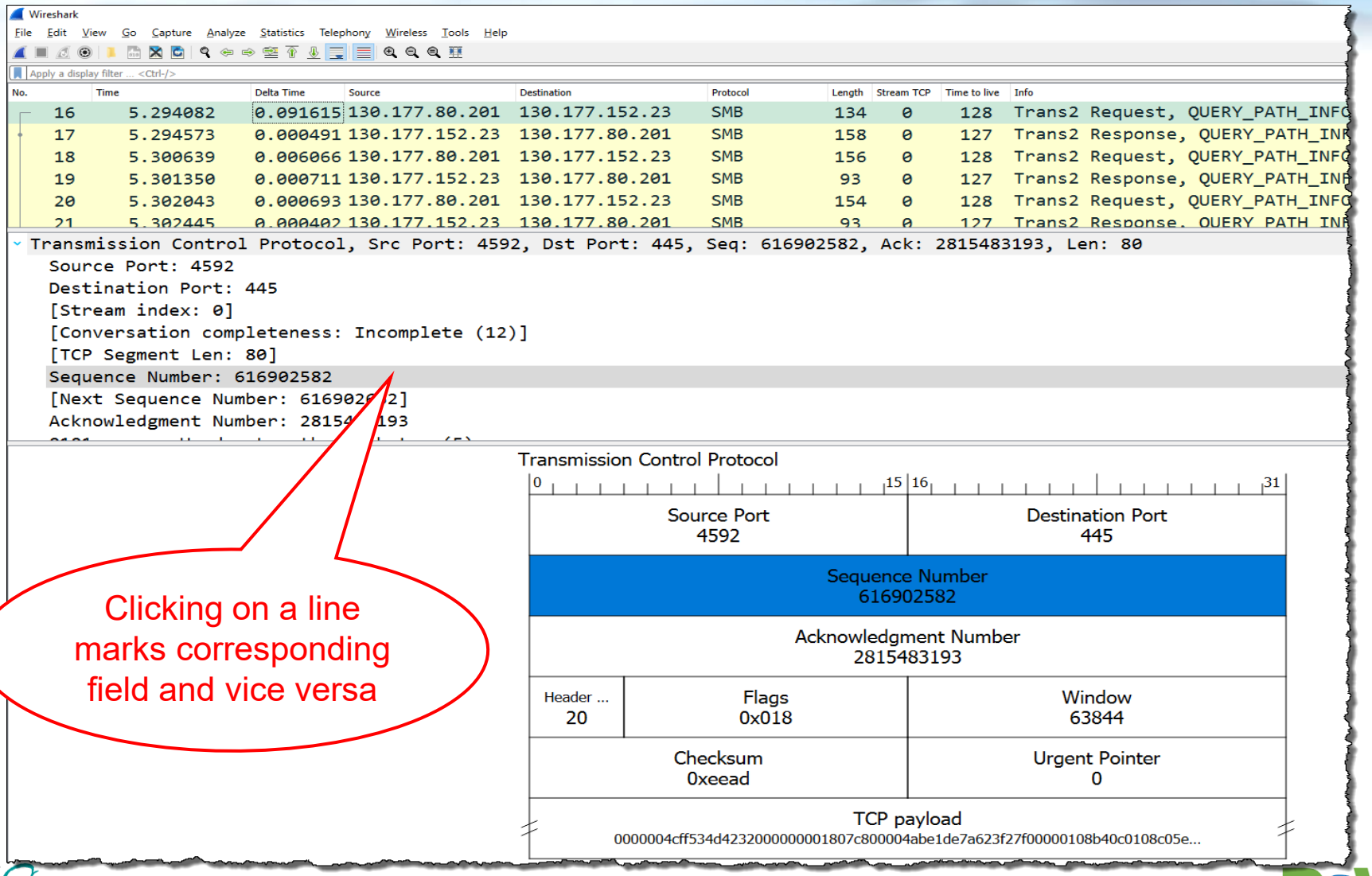


- The color-coded scrollbar shows a preview of packets around the selected one.
- This features enables faster location of areas with errors (black marked packets)
- If the number of frames is too large to display in the scrollbar, only the frames closest to the selected area are shown

© Leutert NetServices 2026 / TCP / V4.1f

New Wireshark Functions

Edit → Preferences → Layout → Packet Diagram



Windows Network Sniffer - **pktmon** (formerly known as "netsh trace")

What is pktmon (Packet Monitor):

- **pktmon** is a Windows built-in, CLI-based network sniffer
- Comparable to **tcpdump** in Linux
- **Captures network packets** on Windows servers & PCs



Implemented in:

- Windows Server 2019 / 2022
- Windows PCs 10 /11
- Azure Stack Hub, Azure, Azure Stack HCI
- Pktmon.exe requires administrative privileges
- Located in **C:\Windows\System32\PktMon.exe**

Features:

- pktmon creates **Event Trace Log (.etl)** files,
- ETL files contains event logs generated by Microsoft Operating System Kernel
- ETL logs include application and system-level errors, warnings, and other events data
- ETL files are helpful in analyzing and troubleshooting system level problems
- ETL files can be opened with the **Windows Performance Analyzer (WPA)** tool

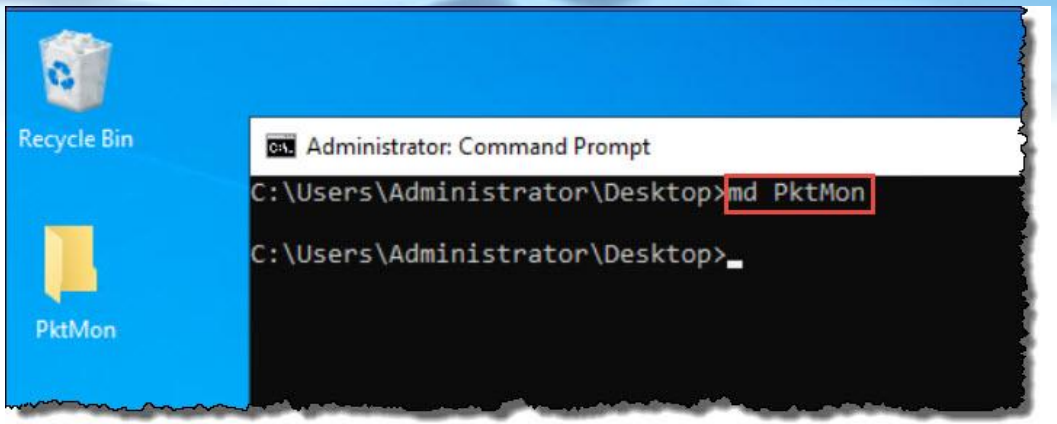
➢ **ETL files can be converted to other file formats such as TXT, CSV or PCAPNG**

© Leutert NetServices 2026 / TCP / V4.1f

Windows Network Sniffer - pktmon

PktMon first steps:

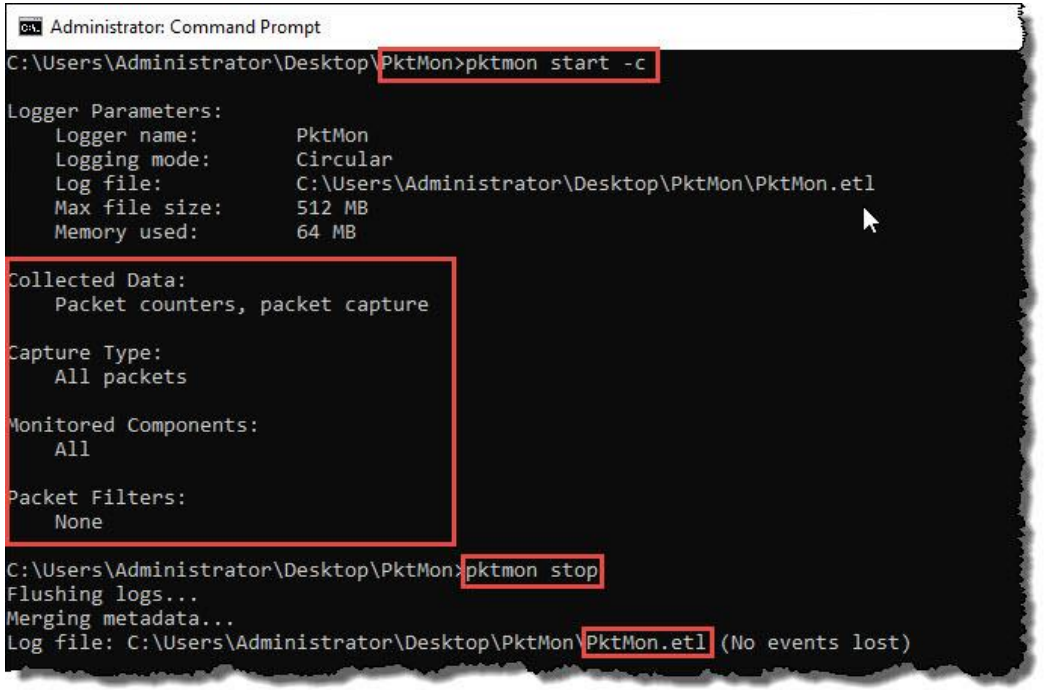
- **Open** Command Prompt with **admin privileges**
- **Change** to location where you want to store .etl files (i.e. Desktop)
- Create new folder



Start capture with default values:

➢ ***pktmon start -c***

- Starts capture on all active interfaces
- Limits packets to 128 bytes



➢ ***pktmon stop***

- Stops and stores capture file
- Default file name is ***PktMon.etl***
- Option ***-f*** for another file name
- **File is stored** in the folder path where PktMon is started from.

© Leutert NetServices 2026 / TCP / V4.1f

Windows Network Sniffer - pktmon

Capture from specified interface(s)

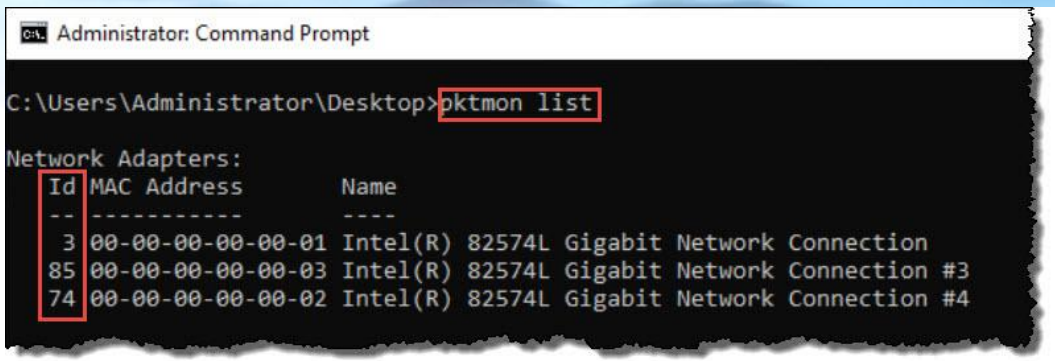
- ***Pktmon list***
- Shows **IDs** of all interfaces
- Use ID# to select interface

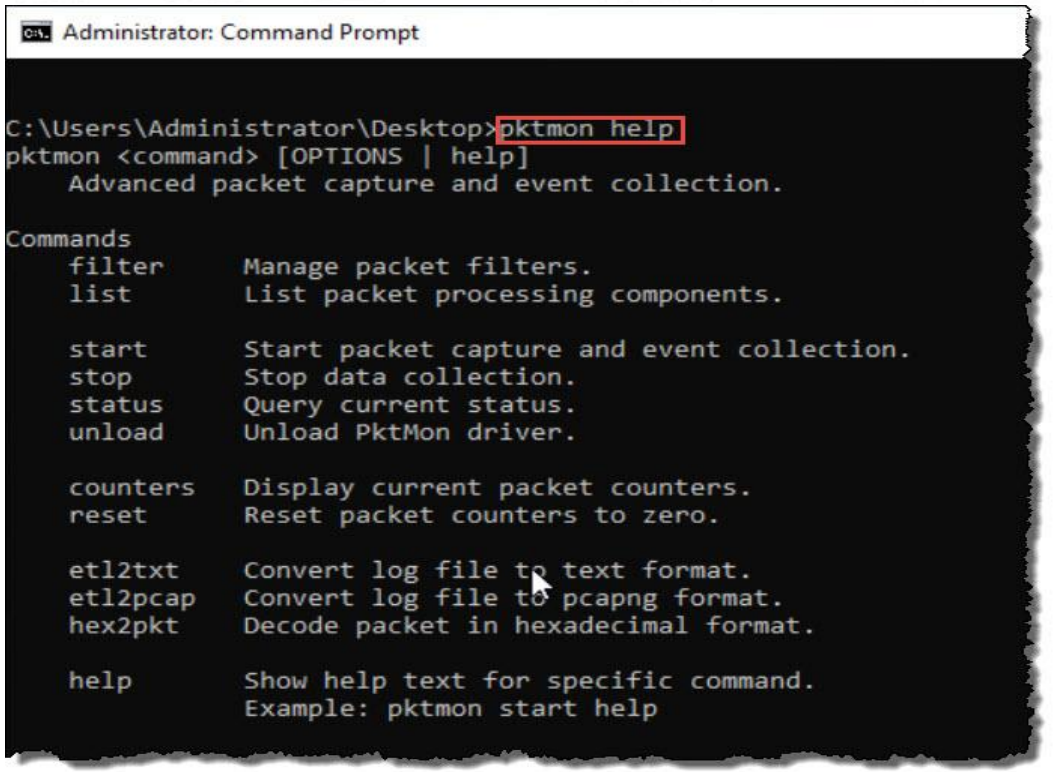
- ***pktmon start -c --comp 3***
- Captures from component **ID 3**
- Only the **first 128 B** of packet
- Default file name is **pktmon.etl**

- ***pktmon start -c --comp 3 --pkt-size 0***
- Starts capture on **ID 3** with **full** packet length.

PktMon help function:

- PktMon offers extensive help on several command levels.
- Enter ***help*** after any command





© Leutert NetServices 2026 / TCP / V4.1f

Windows Network Sniffer - pktmon

Applying Capture filter

- ***pktmon filter add HTTP –p 80 443***
- ***pktmon filter add DNS -p 53***
- Captures only HTTP and DNS frames
- Up to 32 filters are supported

- ***pktmon filter list***
- Shows active capture filters

- ***pktmon filter remove***
- Deletes all filters
- Deleting a single filter is not possible

```
Administrator: Eingabeaufforderung

C:\WINDOWS\system32>pktmon filter add HTTP -p 80 443
Filter hinzugefügt.

C:\WINDOWS\system32>pktmon filter add DNS -p 53
Filter hinzugefügt.

C:\WINDOWS\system32>pktmon filter list
Paketfilter:
     # Name Port
     - ---- ----
     1 HTTP   80
            443
     2 DNS    53

C:\WINDOWS\system32>pktmon filter remove
Alle Filter wurden entfernt.
```

Converting ETL file to PCAPNG format

- ***pktmon pcapng PktMon.etl -o PktMon.pcapng***
- Use option -o and specify target file name

```
C:\WINDOWS\system32>pktmon pcapng PktMon.etl -o PktMon.pcapng
Verarbeitung erfolgt...

Pakete insgesamt:                270
Anzahl der verworfenen Pakete:     0
Formatierte Pakete:              270
Formatierte Datei:       PktMon.pcapng
```

➢ Go to **3_Reference_Guides** for **pktmon_quick_reference-v2.0.pdf**

TCP/IP Protocol Family

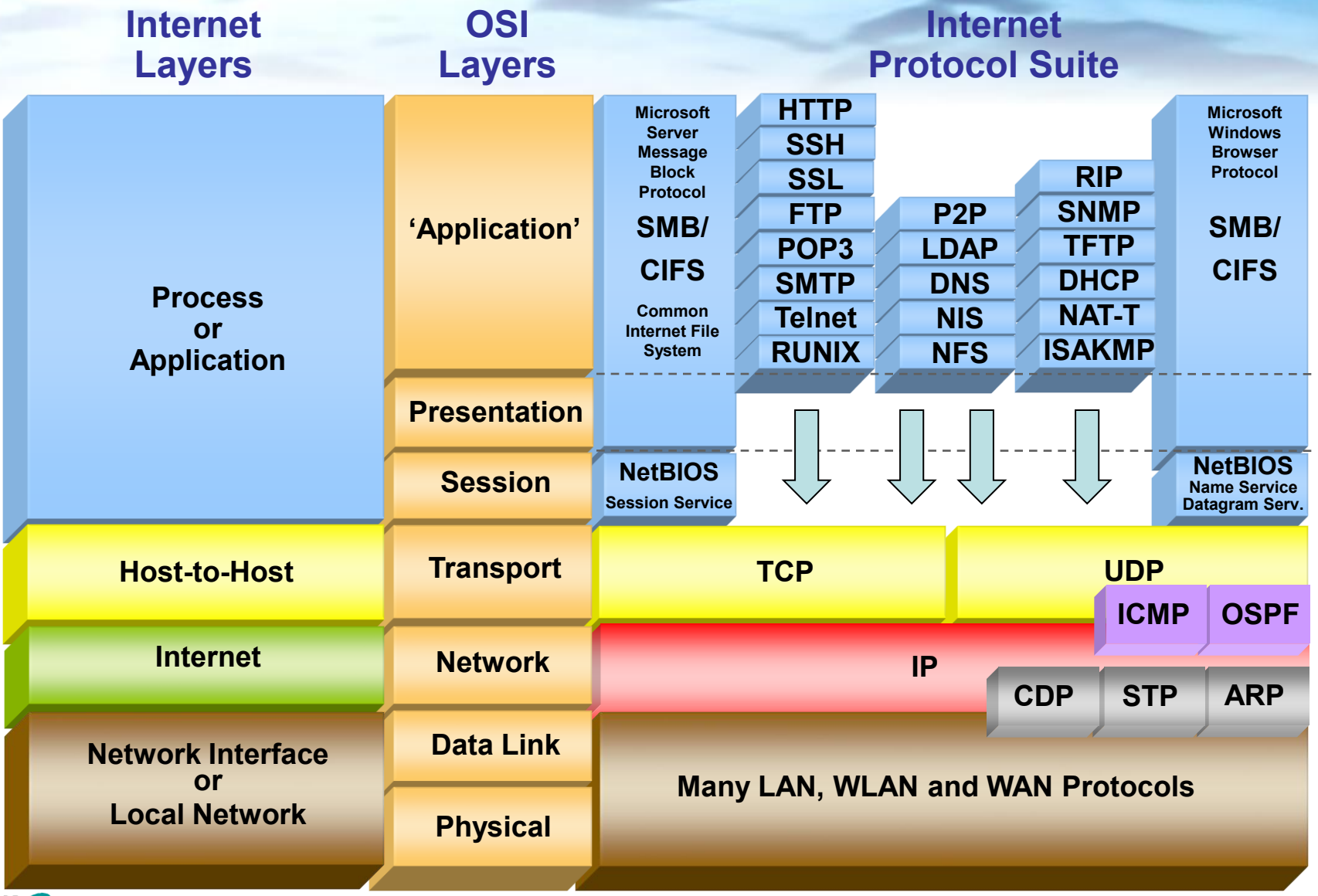
The protocols of the TCP/IP family dominate today's networking techniques.

Understanding this protocol family in detail by means of the Wireshark Analyser makes is possible to isolate problems precisely and to troubleshoot efficiently.



© Leutert NetServices 2026 / TCP / V4.1f

TCP/IP Protocol Family

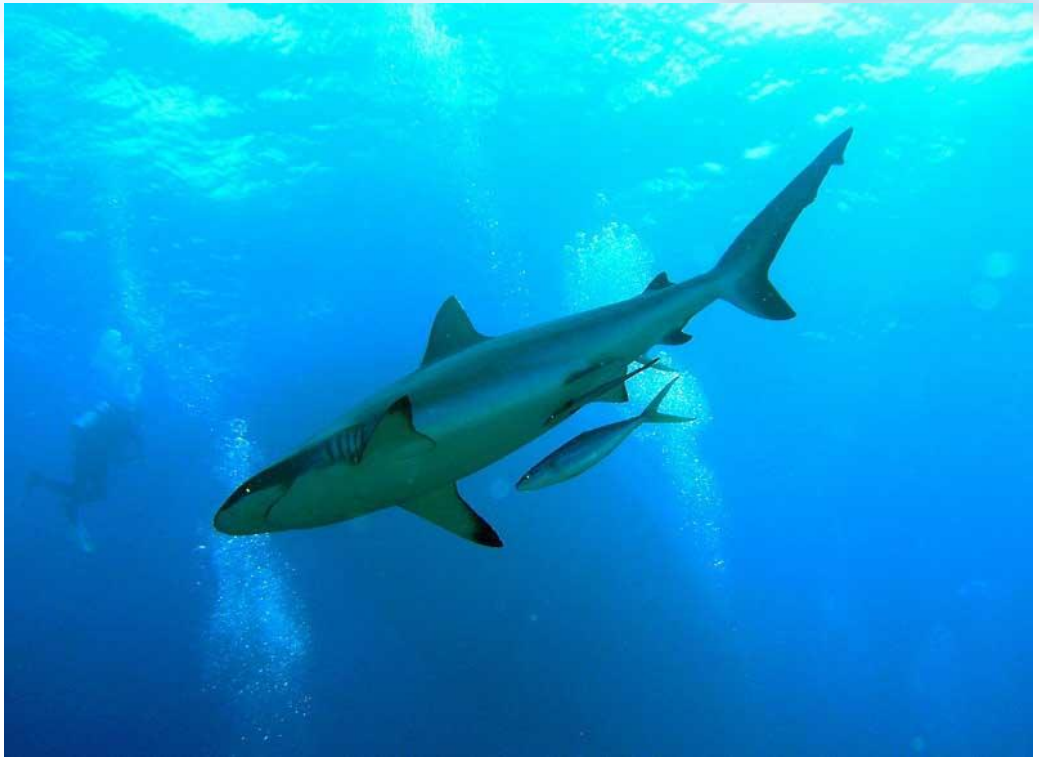


Internet Protocol (IP)

The Internet Protocol forms the basis of the TCP/IP family.

The main purpose if IP is to carry the addresses which are used by the routers to forward the packets across the network.

IP only provides best effort delivery and is connection-less, reliability must be provided by higher protocols i.e. TCP



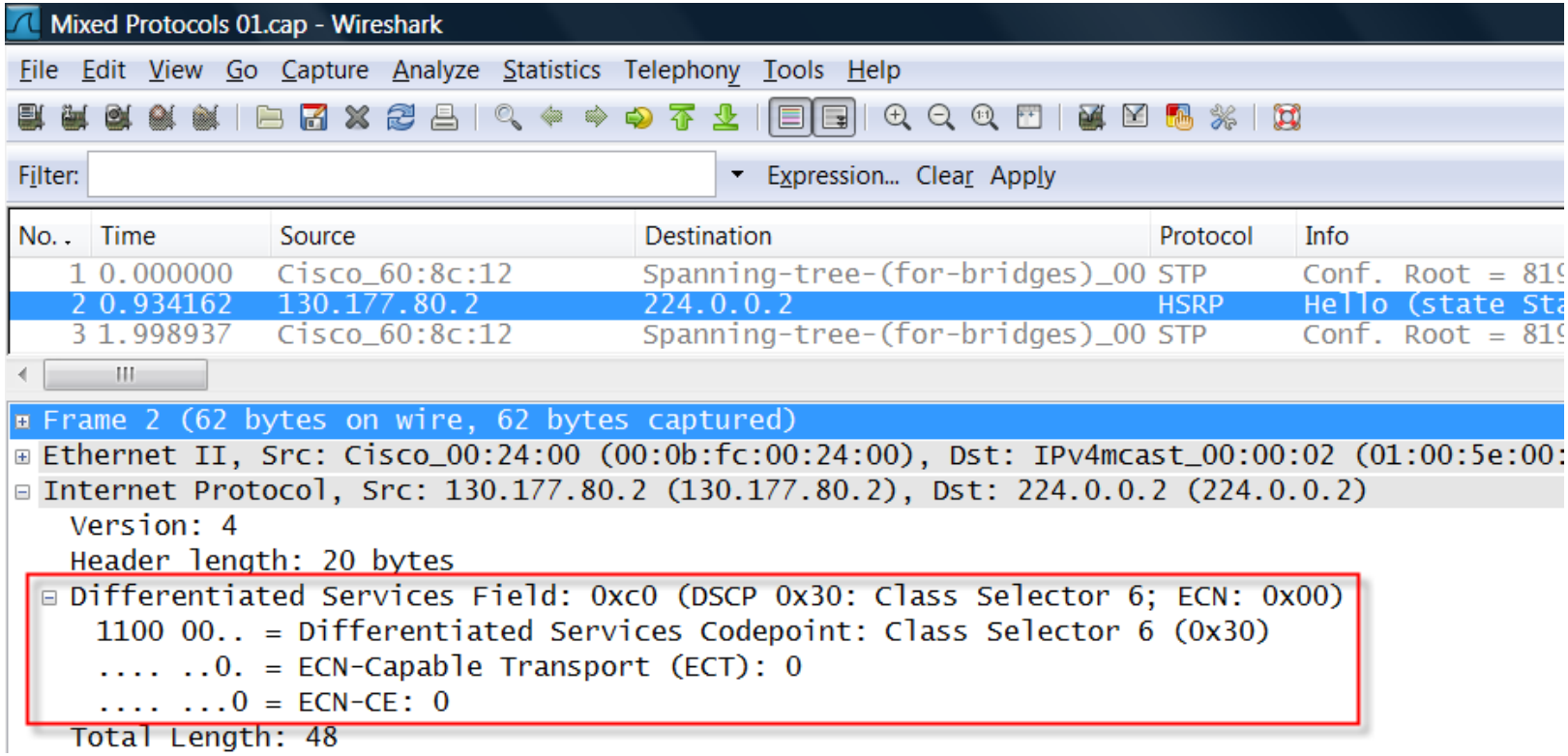
Profound knowledge about IP addressing & subnetting is a prerequisite for troubleshooting networks, but due to time reasons not part of this training.

© Leutert NetServices 2026 / TCP / V4.1f

Internet Protocol (IP)

DiffServ (Differentiated Services Code Point) Header
Since September 2001 (RFC 3168) the TOS byte is newly defined.

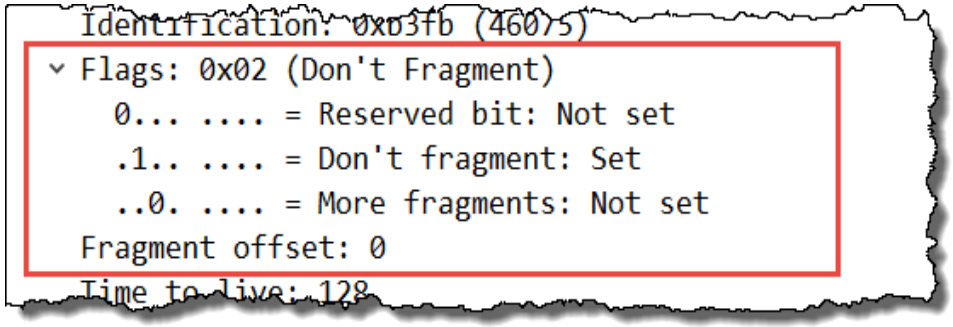
The following partition is applied:
Bits 0-5: DSCP (Differentiated Services Code Point)
Bits 6-7: ECN (Explicit Congestion Notification - IP Flow Control)



Internet Protocol (IP)

The **Flags Byte**

- IP datagrams may be splitted into smaller units, called fragmentation
- Router may fragment a packet if it is to large for the target network
- The sender may prohibit fragmentation by setting the Don't fragment bit
- Application protocols (i.e. SMB, VoIP) do not allow fragmentation by default



Bit 0 : Reserved, set to 0
Bit 1 (DF) : 0 = Fragmentation allowed
: 1 = Fragmentation not allowed
Bit 2 (MF) : 0 = More Fragments will follow
: 1 = Last Fragment

- Fragmenting a packet requires to set the More bit and calculate to offset value

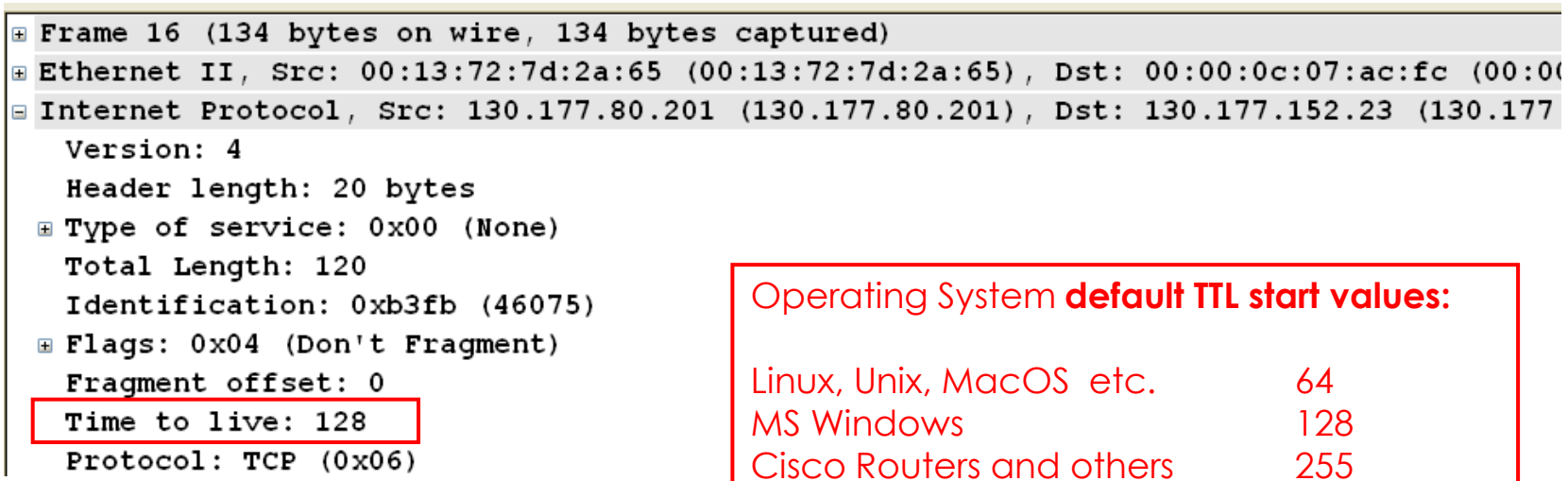
Note:

- With IPv6, routers do no support fragmentation anymore
- If an IPv6 packet is to large to forward, the router drops the packet
- The router then sends an ICMP Packet too big message back to the source
- With IPv6, fragmentation has to be done by the source

Internet Protocol (IP)

TTL (Time to Live) Header

- A value indicating how many hops (routers) a packet is allowed to cross
- The sender of a packet can define how far a packet is allowed to travel
- Each router on the packet path decrements the TTL value by 1 (one)
- The TTL value is decrement at the incoming interface of the router
- When the TTL value reaches zero, the packet will be discarded by the router
- By this rule, an endless forward of packets, e.g. in a **Routing Loop** can be avoided. Routing Loops could lead to a network overload.

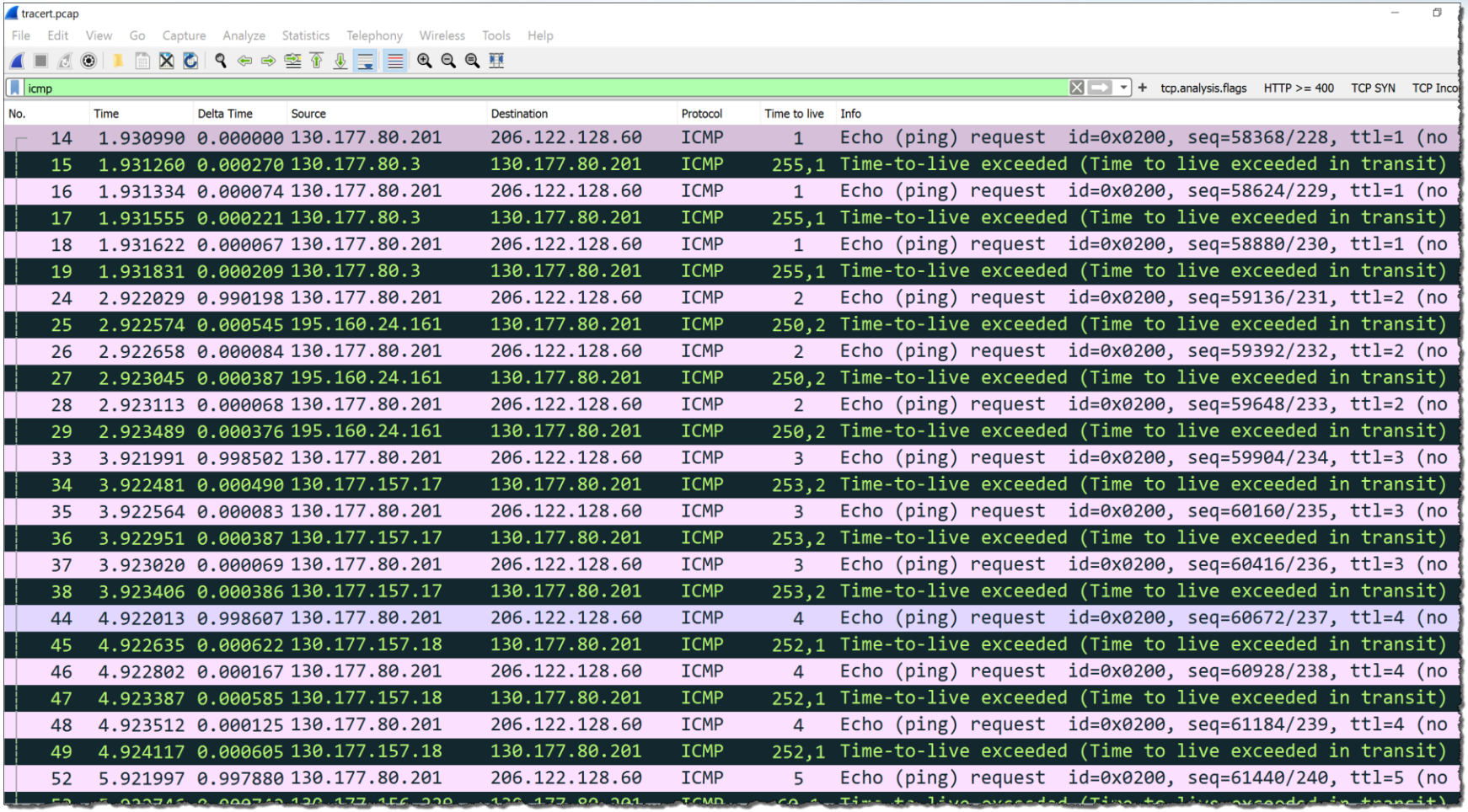


Operating System **default TTL start values:**

Linux, Unix, MacOS etc.	64
MS Windows	128
Cisco Routers and others	255

Internet Protocol (IP)

The Windows command **tracert <ip address>** lists the hops to a target IP



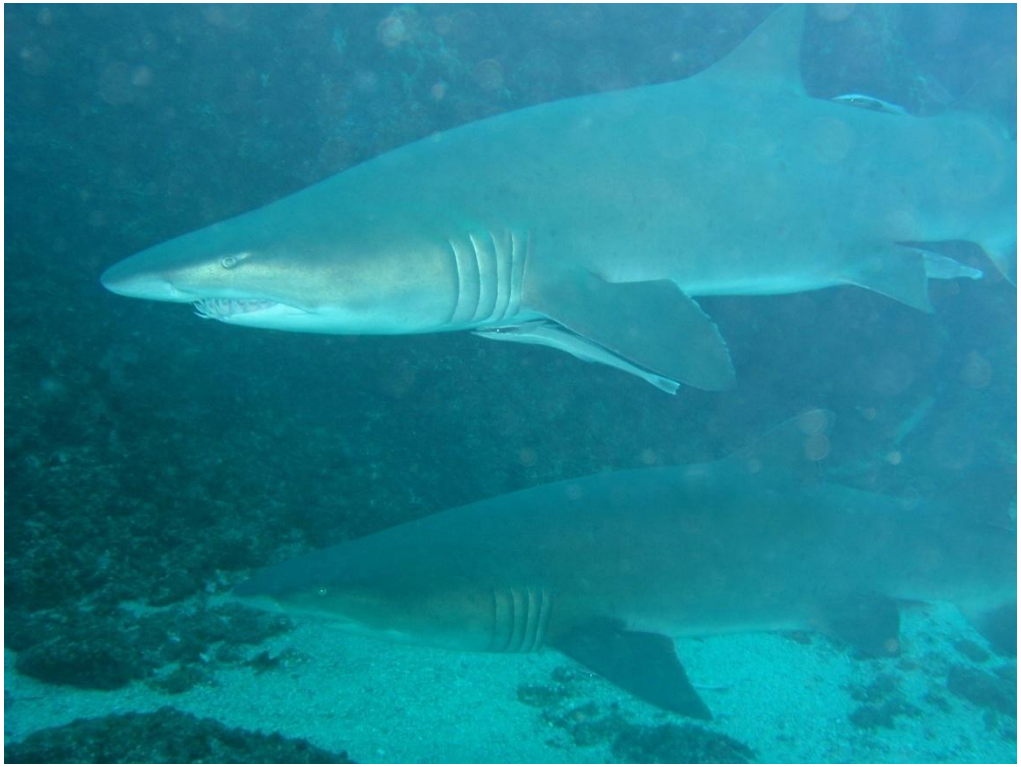
No.	Time	Delta Time	Source	Destination	Protocol	Time to live	Info
14	1.930990	0.000000	130.177.80.201	206.122.128.60	ICMP	1	Echo (ping) request id=0x0200, seq=58368/228, ttl=1 (no
15	1.931260	0.000270	130.177.80.3	130.177.80.201	ICMP	255,1	Time-to-live exceeded (Time to live exceeded in transit)
16	1.931334	0.000074	130.177.80.201	206.122.128.60	ICMP	1	Echo (ping) request id=0x0200, seq=58624/229, ttl=1 (no
17	1.931555	0.000221	130.177.80.3	130.177.80.201	ICMP	255,1	Time-to-live exceeded (Time to live exceeded in transit)
18	1.931622	0.000067	130.177.80.201	206.122.128.60	ICMP	1	Echo (ping) request id=0x0200, seq=58880/230, ttl=1 (no
19	1.931831	0.000209	130.177.80.3	130.177.80.201	ICMP	255,1	Time-to-live exceeded (Time to live exceeded in transit)
24	2.922029	0.990198	130.177.80.201	206.122.128.60	ICMP	2	Echo (ping) request id=0x0200, seq=59136/231, ttl=2 (no
25	2.922574	0.000545	195.160.24.161	130.177.80.201	ICMP	250,2	Time-to-live exceeded (Time to live exceeded in transit)
26	2.922658	0.000084	130.177.80.201	206.122.128.60	ICMP	2	Echo (ping) request id=0x0200, seq=59392/232, ttl=2 (no
27	2.923045	0.000387	195.160.24.161	130.177.80.201	ICMP	250,2	Time-to-live exceeded (Time to live exceeded in transit)
28	2.923113	0.000068	130.177.80.201	206.122.128.60	ICMP	2	Echo (ping) request id=0x0200, seq=59648/233, ttl=2 (no
29	2.923489	0.000376	195.160.24.161	130.177.80.201	ICMP	250,2	Time-to-live exceeded (Time to live exceeded in transit)
33	3.921991	0.998502	130.177.80.201	206.122.128.60	ICMP	3	Echo (ping) request id=0x0200, seq=59904/234, ttl=3 (no
34	3.922481	0.000490	130.177.157.17	130.177.80.201	ICMP	253,2	Time-to-live exceeded (Time to live exceeded in transit)
35	3.922564	0.000083	130.177.80.201	206.122.128.60	ICMP	3	Echo (ping) request id=0x0200, seq=60160/235, ttl=3 (no
36	3.922951	0.000387	130.177.157.17	130.177.80.201	ICMP	253,2	Time-to-live exceeded (Time to live exceeded in transit)
37	3.923020	0.000069	130.177.80.201	206.122.128.60	ICMP	3	Echo (ping) request id=0x0200, seq=60416/236, ttl=3 (no
38	3.923406	0.000386	130.177.157.17	130.177.80.201	ICMP	253,2	Time-to-live exceeded (Time to live exceeded in transit)
44	4.922013	0.998607	130.177.80.201	206.122.128.60	ICMP	4	Echo (ping) request id=0x0200, seq=60672/237, ttl=4 (no
45	4.922635	0.000622	130.177.157.18	130.177.80.201	ICMP	252,1	Time-to-live exceeded (Time to live exceeded in transit)
46	4.922802	0.000167	130.177.80.201	206.122.128.60	ICMP	4	Echo (ping) request id=0x0200, seq=60928/238, ttl=4 (no
47	4.923387	0.000585	130.177.157.18	130.177.80.201	ICMP	252,1	Time-to-live exceeded (Time to live exceeded in transit)
48	4.923512	0.000125	130.177.80.201	206.122.128.60	ICMP	4	Echo (ping) request id=0x0200, seq=61184/239, ttl=4 (no
49	4.924117	0.000605	130.177.157.18	130.177.80.201	ICMP	252,1	Time-to-live exceeded (Time to live exceeded in transit)
52	5.921997	0.997880	130.177.80.201	206.122.128.60	ICMP	5	Echo (ping) request id=0x0200, seq=61440/240, ttl=5 (no

Transmission Control Protocol (TCP)

The TCP Protocol contains all important components which are necessary for a reliable data transmission.

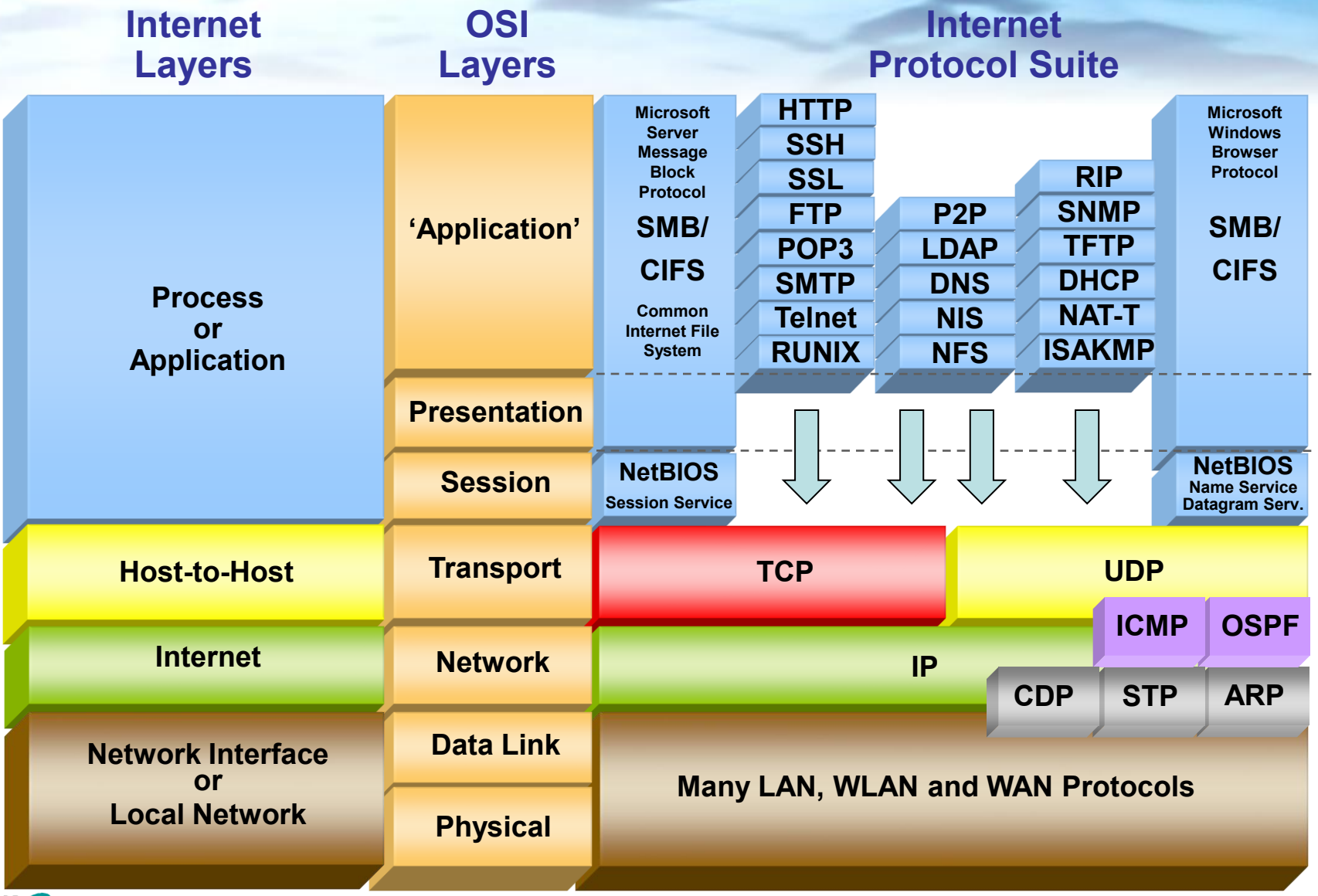
Reliable protocols consists of three phases: connection establishment, secured data transmission and connection tear down.

TCP offers these functions as a service for the Application Layer Protocols.



© Leutert NetServices 2026 / TCP / V4.1f

Transmission Control Protocol (TCP)



Transmission Control Protocol (TCP)

TCP is described as connection-oriented, reliable byte stream service

- Comprises all functions important for a secured end-to-end transmission
- Guaranteed delivery: Acknowledges
- Measures against lost segments: Retransmission
- Error corrections: Checksum Test; Retransmission
- Flow control: Window Size
- Efficient data transmission: Full Duplex; Sliding Window
- Transmission of large amounts of data: Window Scaling
- Parallel data transmission: Different Source Port

© Leutert NetServices 2026 / TCP / V4.1f

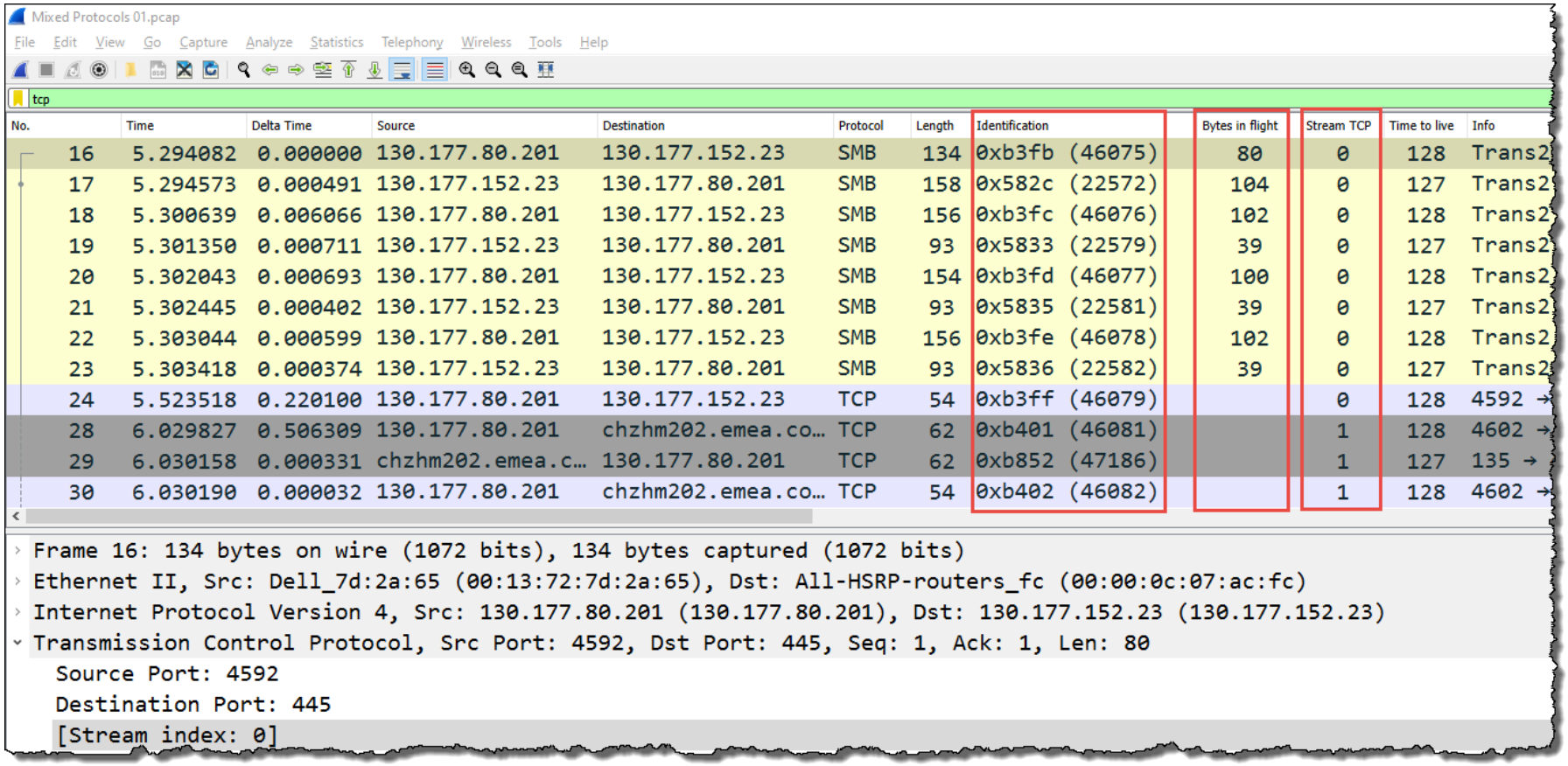
Transmission Control Protocol (TCP) continuation

- Adaptation to bad line quality: Slow Start
- Adaptation to round trip times: Adaptive Timeout; Retransmission Timers
- Transparency for Application Layer Data: Binary, ASCII, EBCDIC etc.
- Detection of out-of-sequence Packets: Segment Numbering
- Detection of ‚Duplicated Frames‘: Segment Numbering
- Setting priorities with the receiver: PUSH Flag
- Marking important data: URGENT Flag; URGENT Pointer

© Leutert NetServices 2026 / TCP / V4.1f

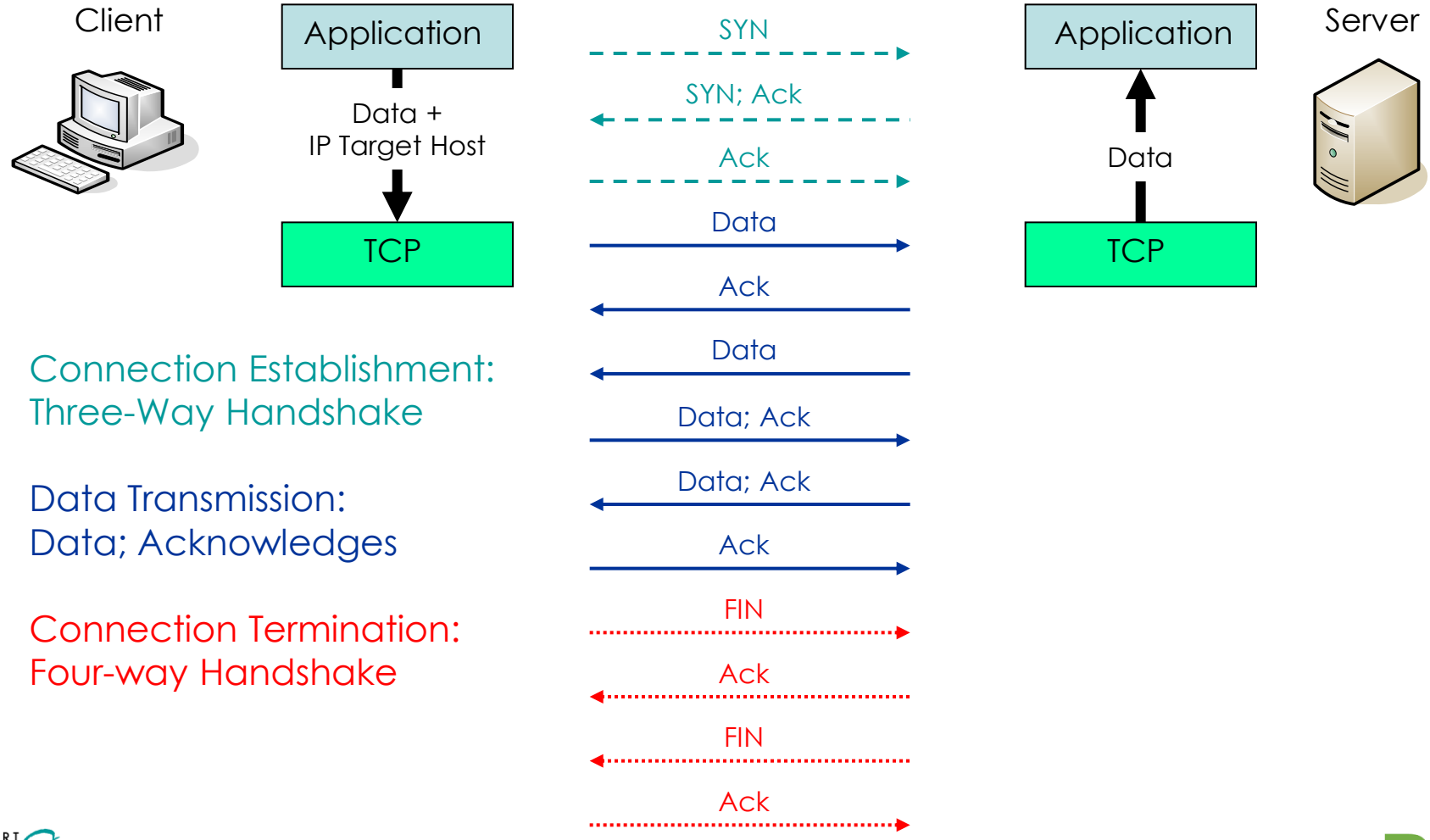
Creating a Wireshark Profile for TCP

- **Stream Index:** Wireshark numbers TCP sessions in order of detection
- **Bytes in flight:** Transmitted, but not yet acknowledged bytes
- **IP Identification:** Number increased by sender with each IP packet



Transmission Control Protocol (TCP)

The three phases of data transmission:



Connection Establishment:
Three-Way Handshake

Data Transmission:
Data; Acknowledges

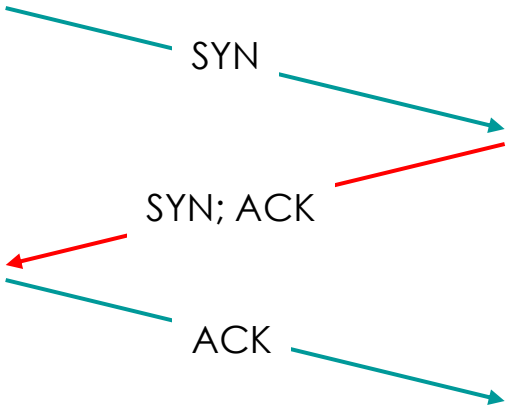
Connection Termination:
Four-way Handshake

Transmission Control Protocol (TCP) Connection Establishment

TCP ‚Three-way Handshake'

Client SYN
- Start Sequence Number
- Window Size

Options:
- Maximum Segment Size
- Window Scaling
- Selective Acknowledges
- Timestamp
- PAWS (Protection against wrapped sequence #)



Server SYN; ACK
- Start Sequence Number
- Acknowledge Client Sequence Number
- Window Size

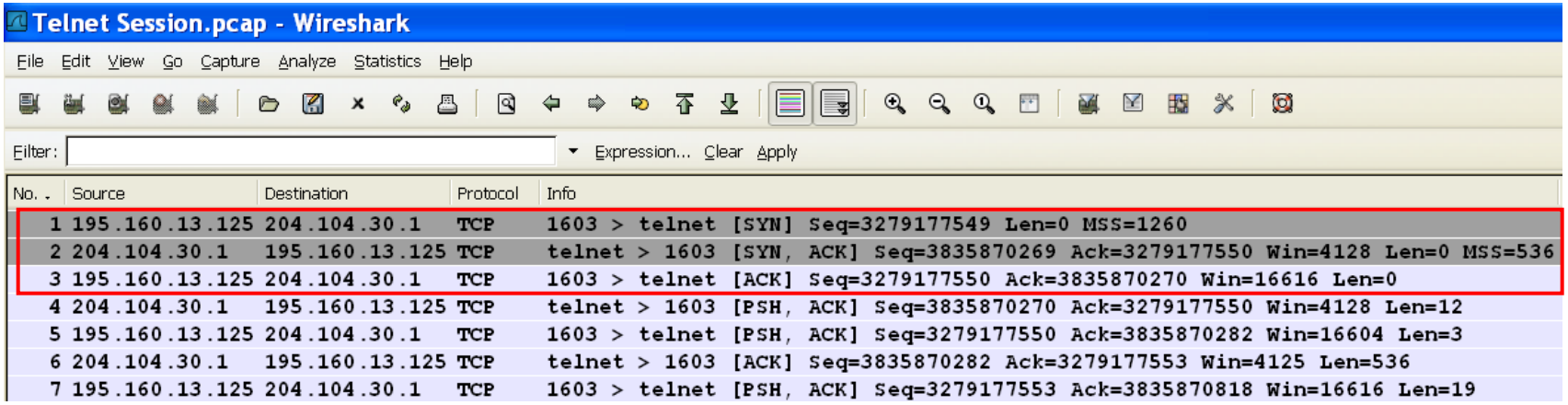
Options:
- Maximum Segment Size
- Window Scaling
- Selective Acknowledges
- Timestamp
- PAWS (Protection against wrapped sequence #)

Client ACK
- Acknowledge Server Sequence Number

Transmission Control Protocol (TCP) Connection Establishment

TCP ‚Three-way Handshake'

The TCP connection establishment consists of three segments



(File: Telnet Session.pcap)

Transmission Control Protocol (TCP) Connection Establishment

‚SYN' Segment of the client (Frame # 1)

```
Transmission Control Protocol, Src Port: 1603 (1603), Dst Port: telnet (23), Seq: 3279177549, Len: 0
    Source port: 1603 (1603)
    Destination port: telnet (23)
    Sequence number: 3279177549
    Header length: 28 bytes
  Flags: 0x02 (SYN)
      0... .... = Congestion Window Reduced (CWR): Not set
      .0.. .... = ECN-Echo: Not set
      ..0. .... = Urgent: Not set
      ...0 .... = Acknowledgment: Not set
      .... 0... = Push: Not set
      .... .0.. = Reset: Not set
      .... ..1. = Syn: Set
      .... ...0 = Fin: Not set
    Window size: 16384
    Checksum: 0x7744 [validation disabled]
  Options: (8 bytes)
      Maximum segment size: 1260 bytes
      NOP
      NOP
      SACK permitted
```

The client indicates its start sequence number

The client marks frame as SYN

The client informs about its receiver buffer size

The client wants no segments bigger than 1260 bytes

The client supports ‚Selective Acknowledges'

(File: Telnet Session.pcap)

Transmission Control Protocol (TCP) Connection Establishment

‚SYN; ACK' Segment of the Server (Frame # 2)

```
Transmission Control Protocol, Src Port: telnet (23), Dst Port: 1603 (1603), Seq: 3835870269, Ack: 3279177550, Len: 0
  Source port: telnet (23)
  Destination port: 1603 (1603)
  Sequence number: 3835870269
  Acknowledgement number: 3279177550
  Header length: 28 bytes
Flags: 0x12 (SYN, ACK)
    0... .... = Congestion Window Reduced (CWR): Not set
    .0.. .... = ECN-Echo: Not set
    ..0. .... = Urgent: Not set
    ...1 .... = Acknowledgment: Set
    .... 0... = Push: Not set
    .... .0.. = Reset: Not set
    .... ..1. = Syn: Set
    .... ...0 = Fin: Not set
  Window size: 4128
  Checksum: 0x755e [validation disabled]
Options: (8 bytes)
    Maximum segment size: 536 bytes
```

The server indicates its start sequence number

The server confirms the client start sequence number

The server marks the frame as ACK

The server marks the frame as SYN

The server informs about its receiver buffer size

The server wants no segments bigger than 536 bytes

The server does not support ‚Selective Acknowledges'

TCP options will only be activated if client and server correspond.
As Maximum Segment Size (MSS) the smaller of both values will be used.

(File: Telnet Session.pcap)

Transmission Control Protocol (TCP) Connection Establishment

‚ACK' Segment of the Client (Frame # 3)

```
Transmission Control Protocol, Src Port: 1603 (1603), Dst Port: telnet (23), Seq: 3279177550, Ack: 3835870270, Len: 0
  Source port: 1603 (1603)
  Destination port: telnet (23)
  Sequence number: 3279177550
  Acknowledgement number: 3835870270
  Header length: 20 bytes
  Flags: 0x10 (ACK)
    0... .... = Congestion Window Reduced (CWR): Not set
    .0.. .... = ECN-Echo: Not set
    ..0. .... = Urgent: Not set
    ...1 .... = Acknowledgment: Set
    .... 0... = Push: Not set
    .... .0.. = Reset: Not set
    .... ..0. = Syn: Not set
    .... ...0 = Fin: Not set
  Window size: 16616
  Checksum: 0x0167 [validation disabled]
```

The client indicates its start sequence number plus one

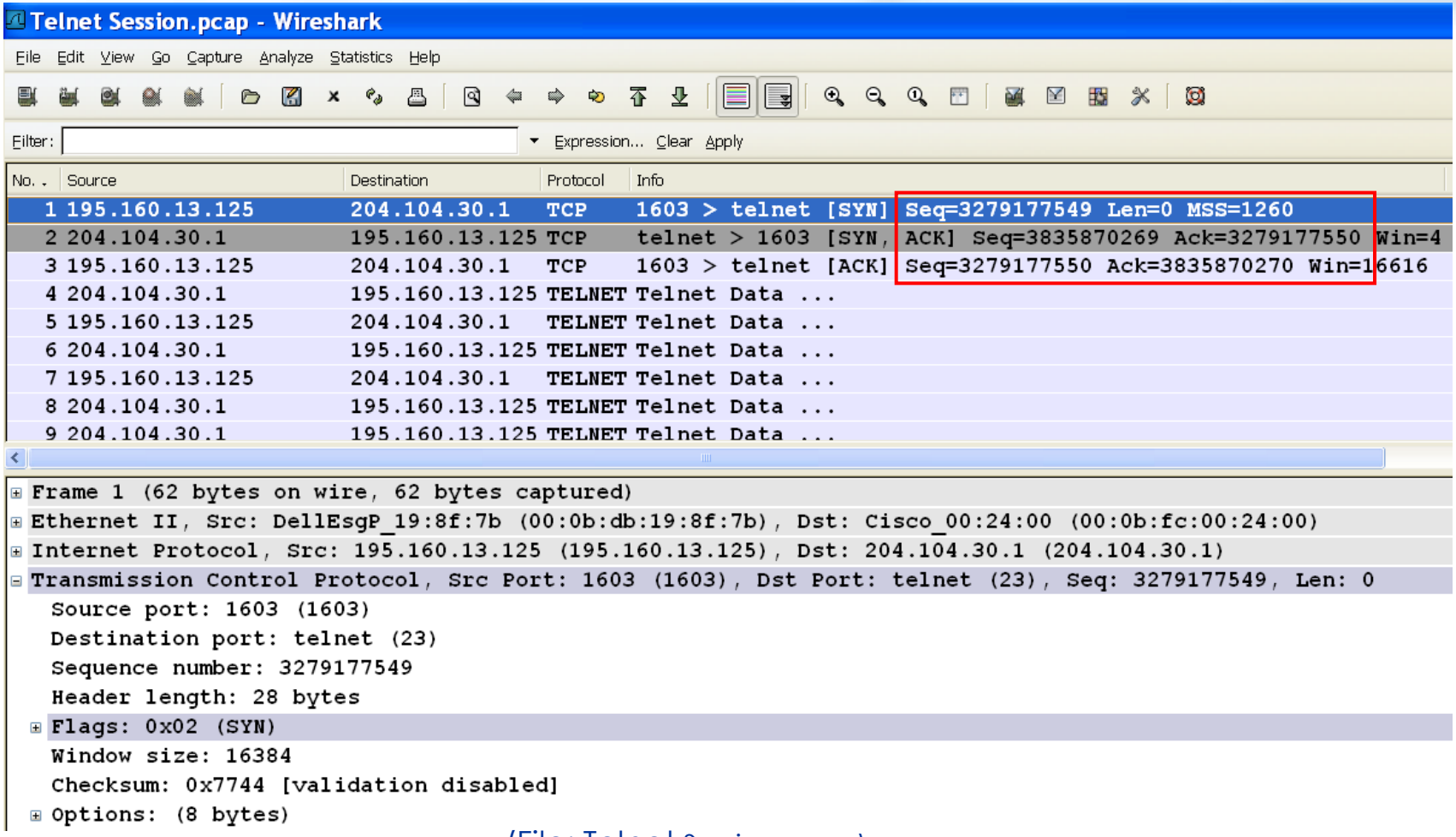
The client confirms the server start sequence number

The client marks the frame as ACK

(File: Telnet Session.pcap)

Transmission Control Protocol (TCP)

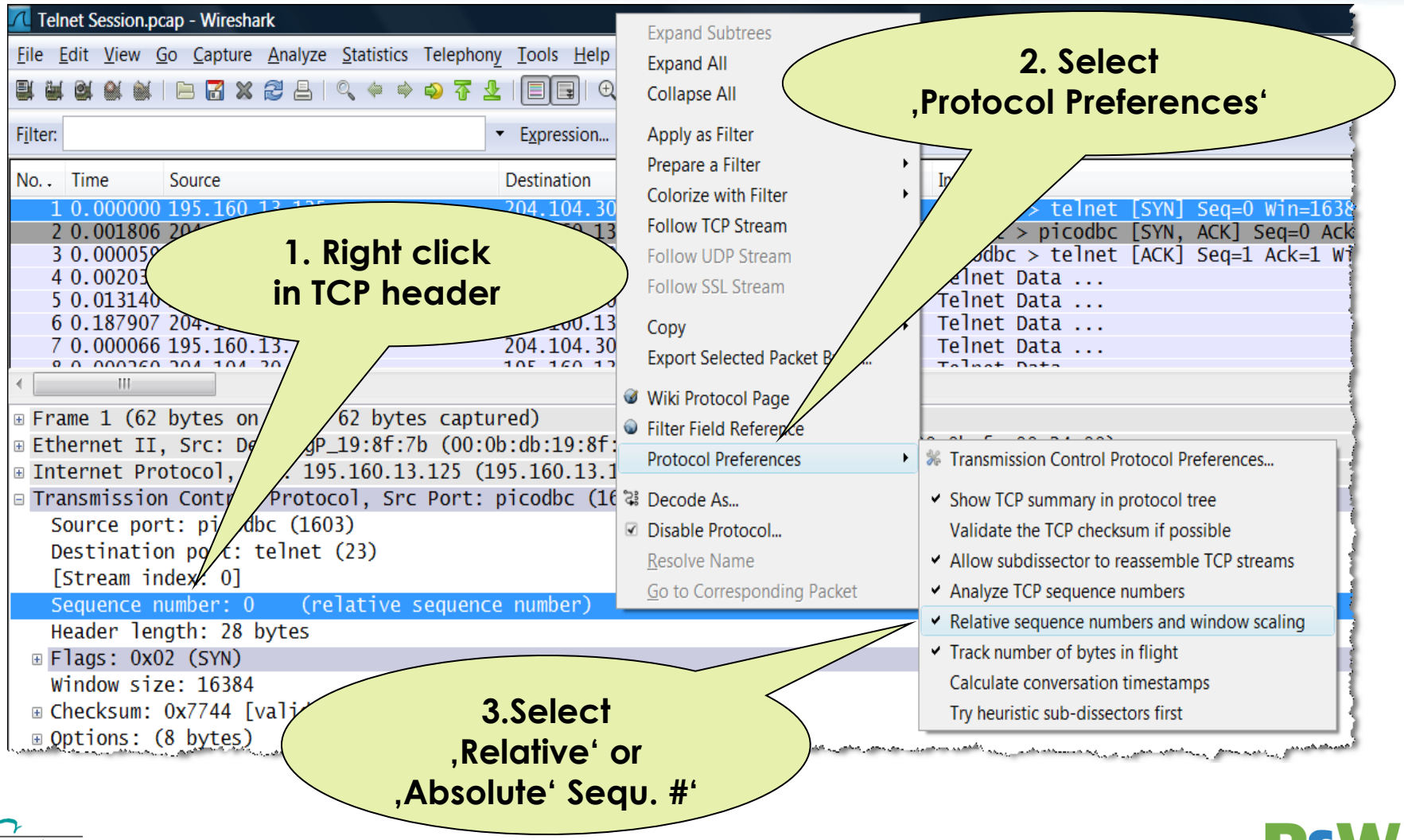
Telnet Session with ‚Absolute Sequence No.'



(File: Telnet Session.pcap)

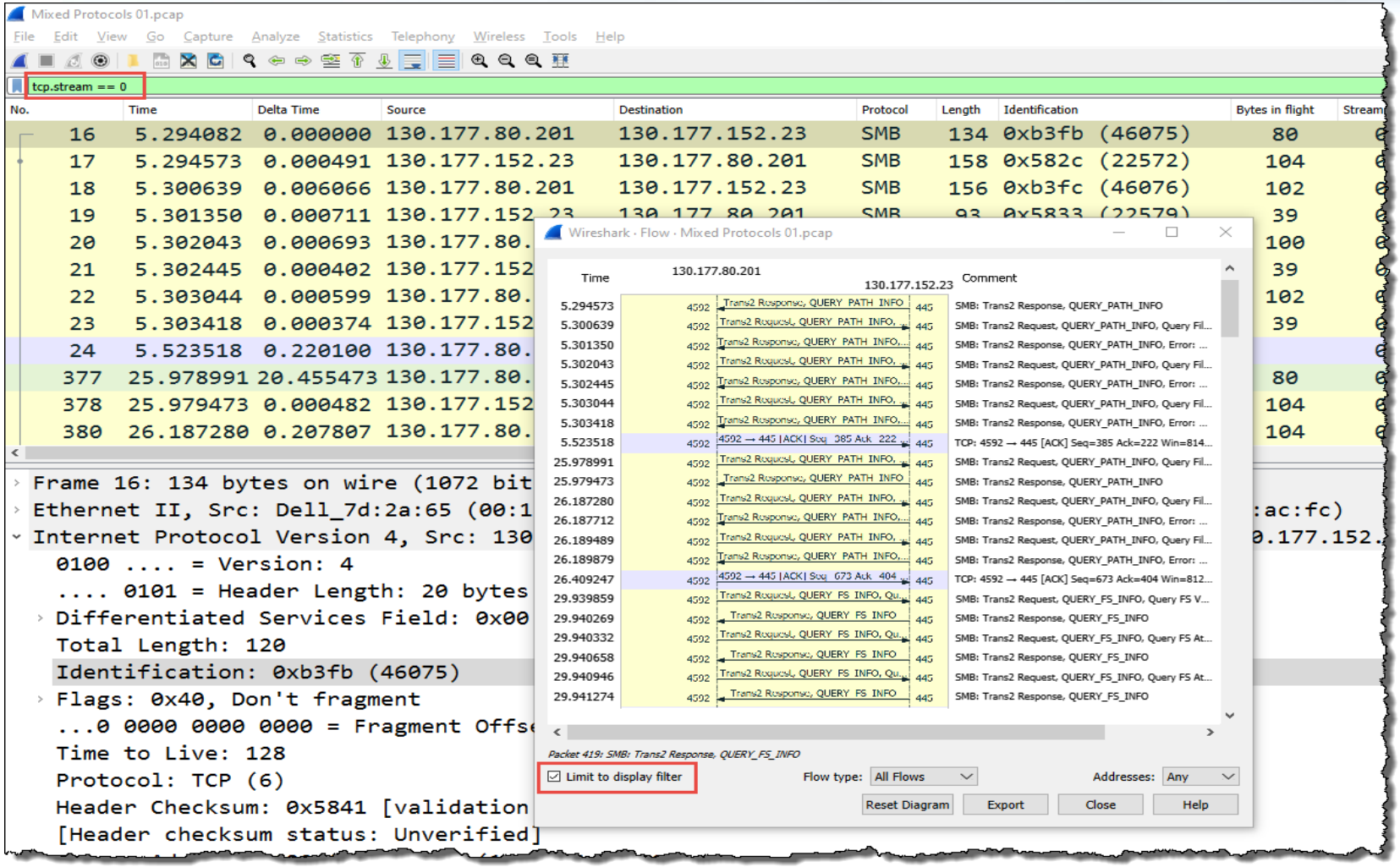
Transmission Control Protocol (TCP)

Display of ‚Relative' or ‚Absolut' TCP Sequence-numbers



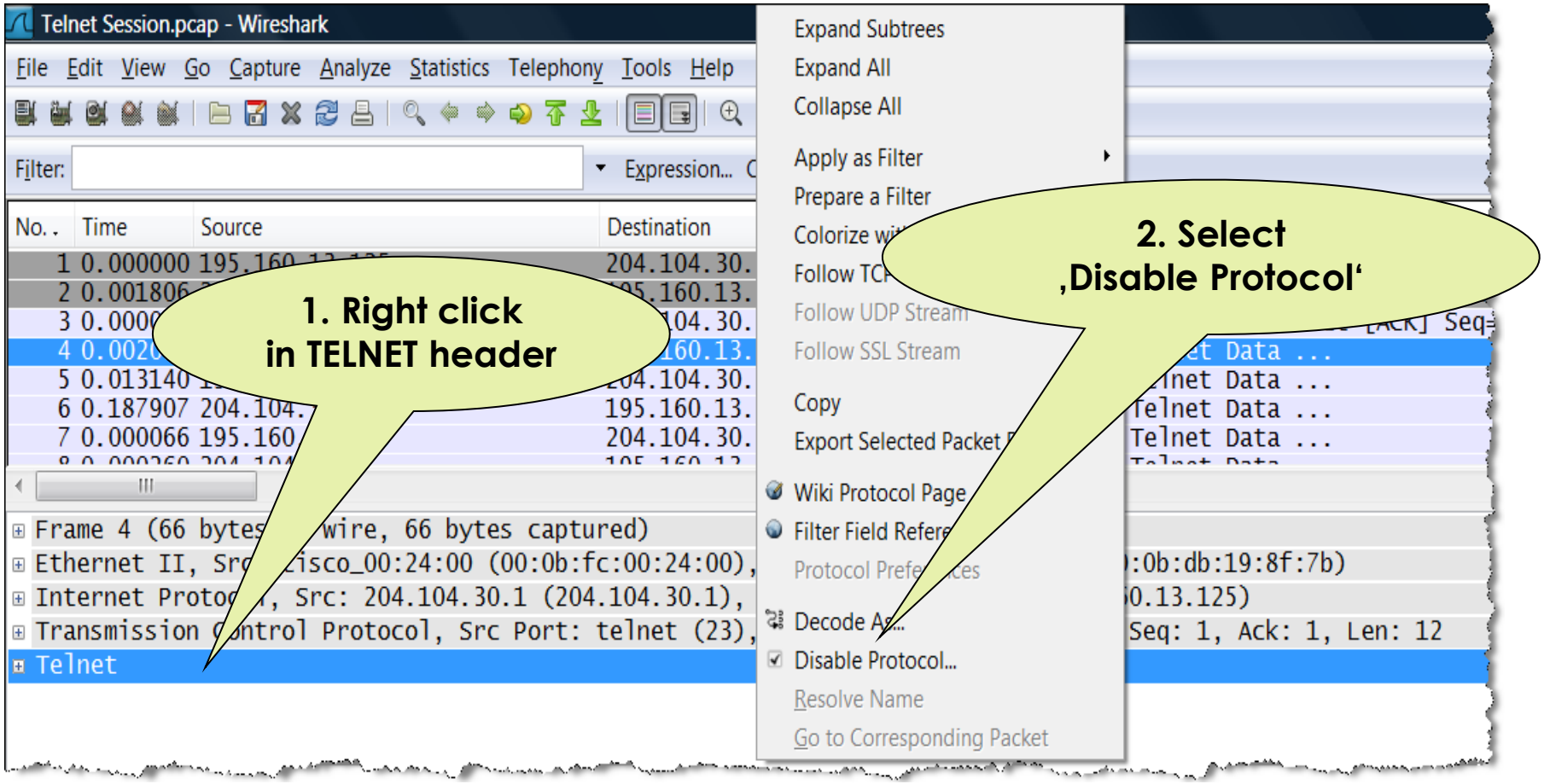
Transmission Control Protocol (TCP)

→ Statistics → Flow Graph



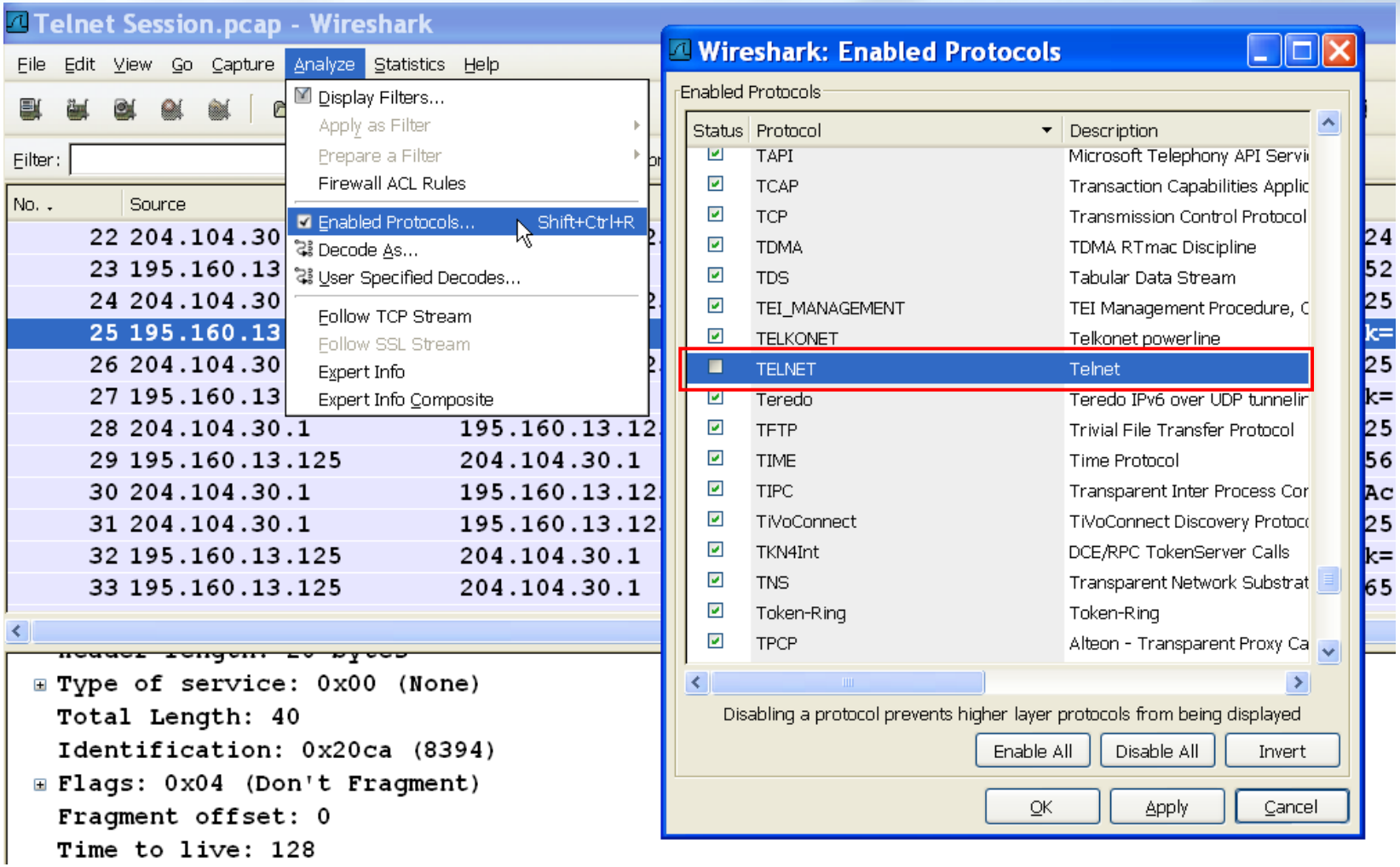
Transmission Control Protocol (TCP) Data Transmission

Disable displaying protocols



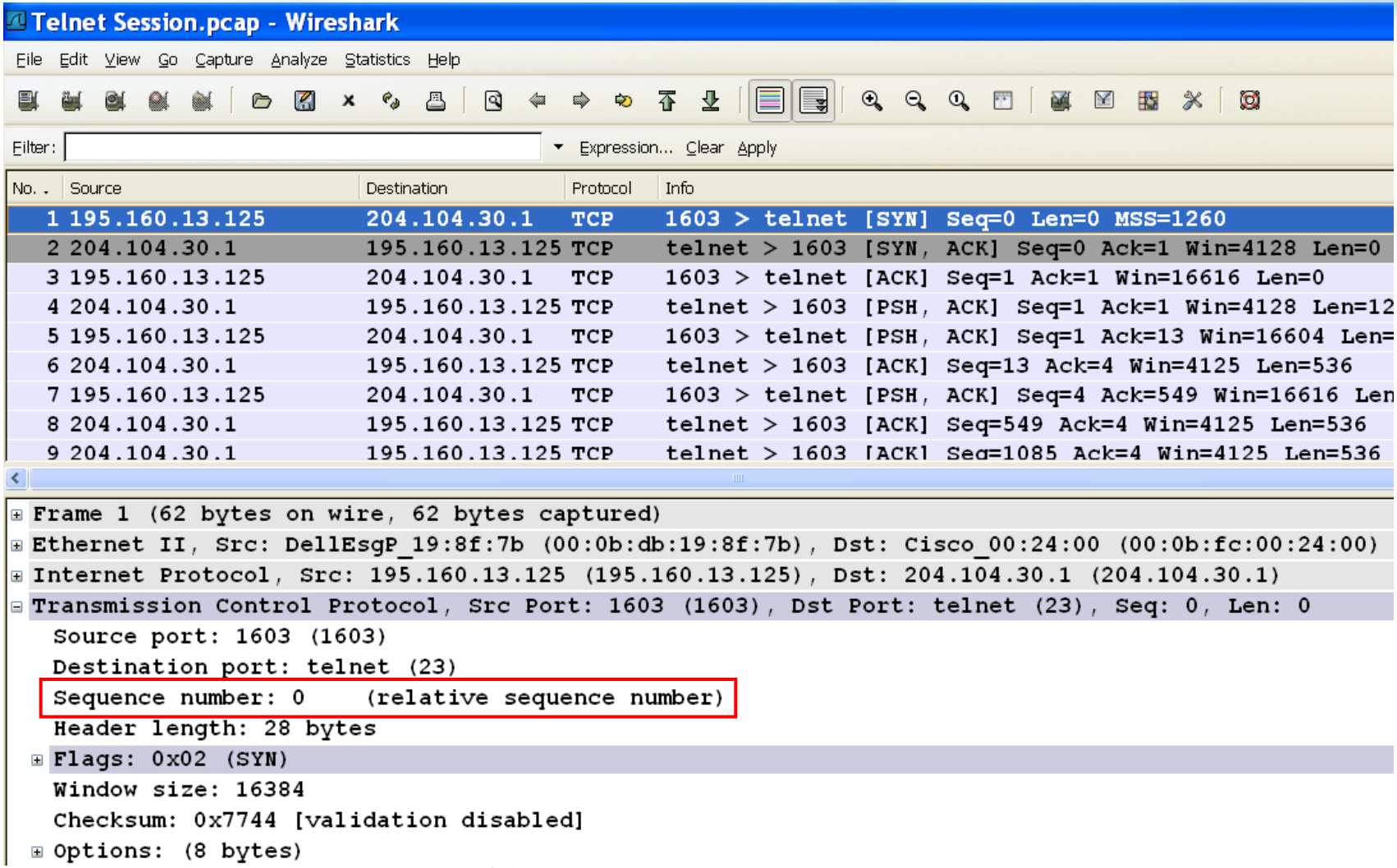
Transmission Control Protocol (TCP)

Re-enabling protocol display



Transmission Control Protocol (TCP)

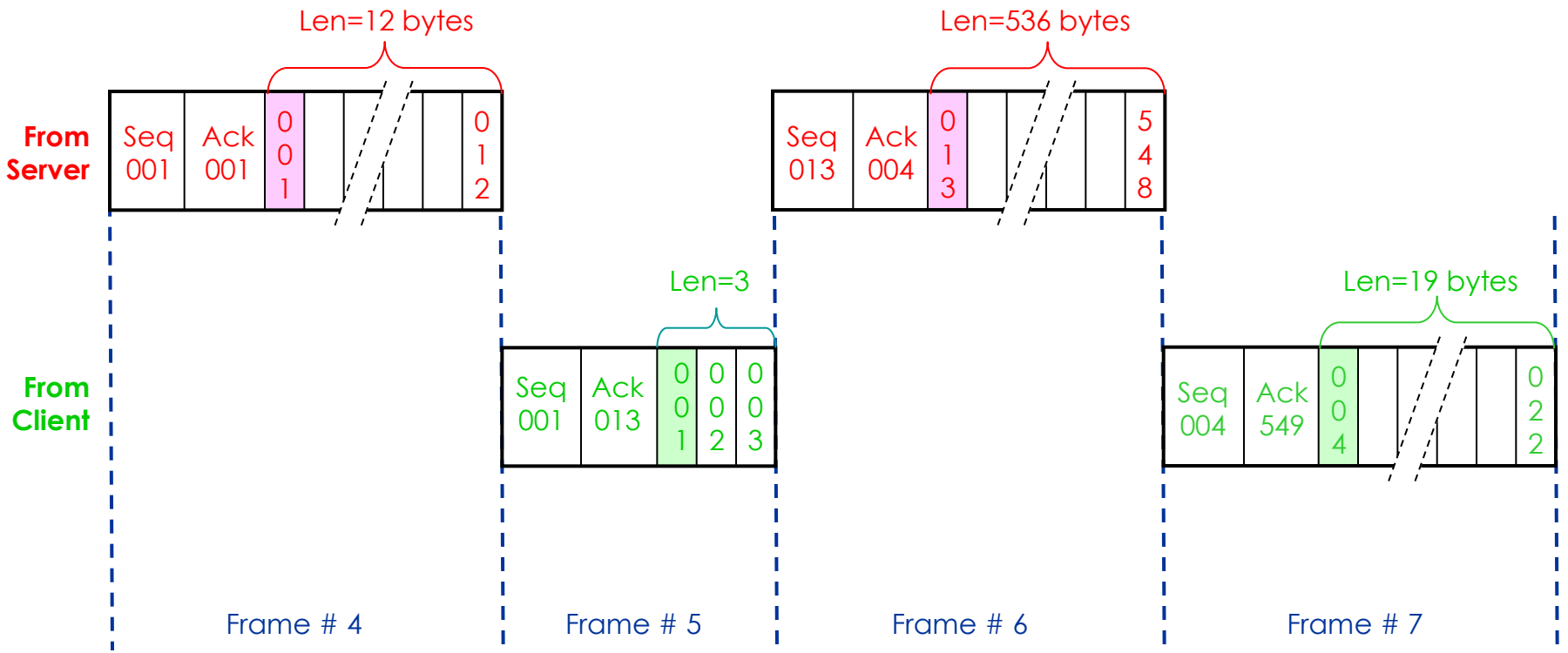
Telnet Session with ‚Relative Sequence No.'



(File: Telnet Session.pcap)

Transmission Control Protocol (TCP) Data Transmission

TCP Sequence Numbering (relative). The sequence number marks the first data byte of the segment. The acknowledge number refers to the next data byte to be received (ping-pong dialogue).

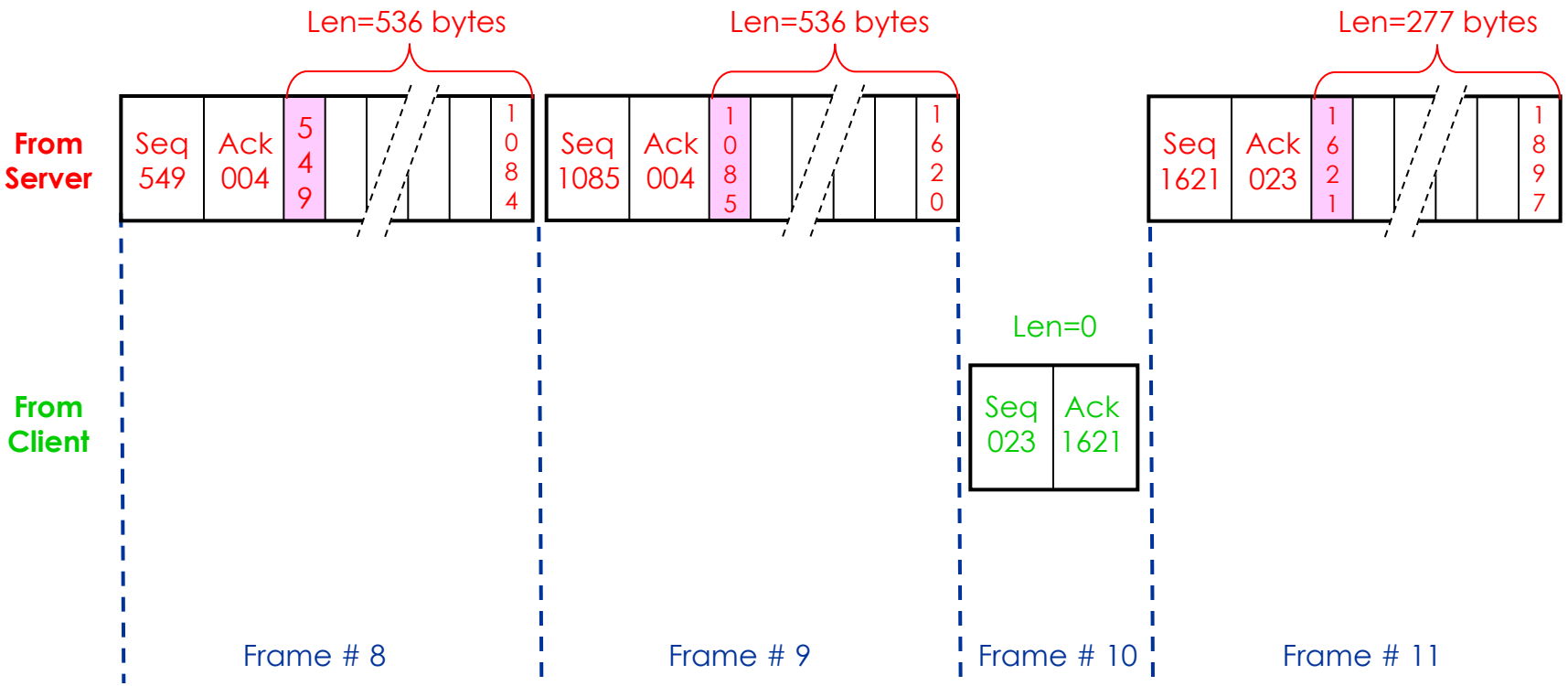


(File: Telnet Session.pcap)

© Leutert NetServices 2026 / TCP / V4.1f

Transmission Control Protocol (TCP) Data Transmission

A ‚TCP Acknowledge' is able to confirm a number of received segments in one go (bursting data).

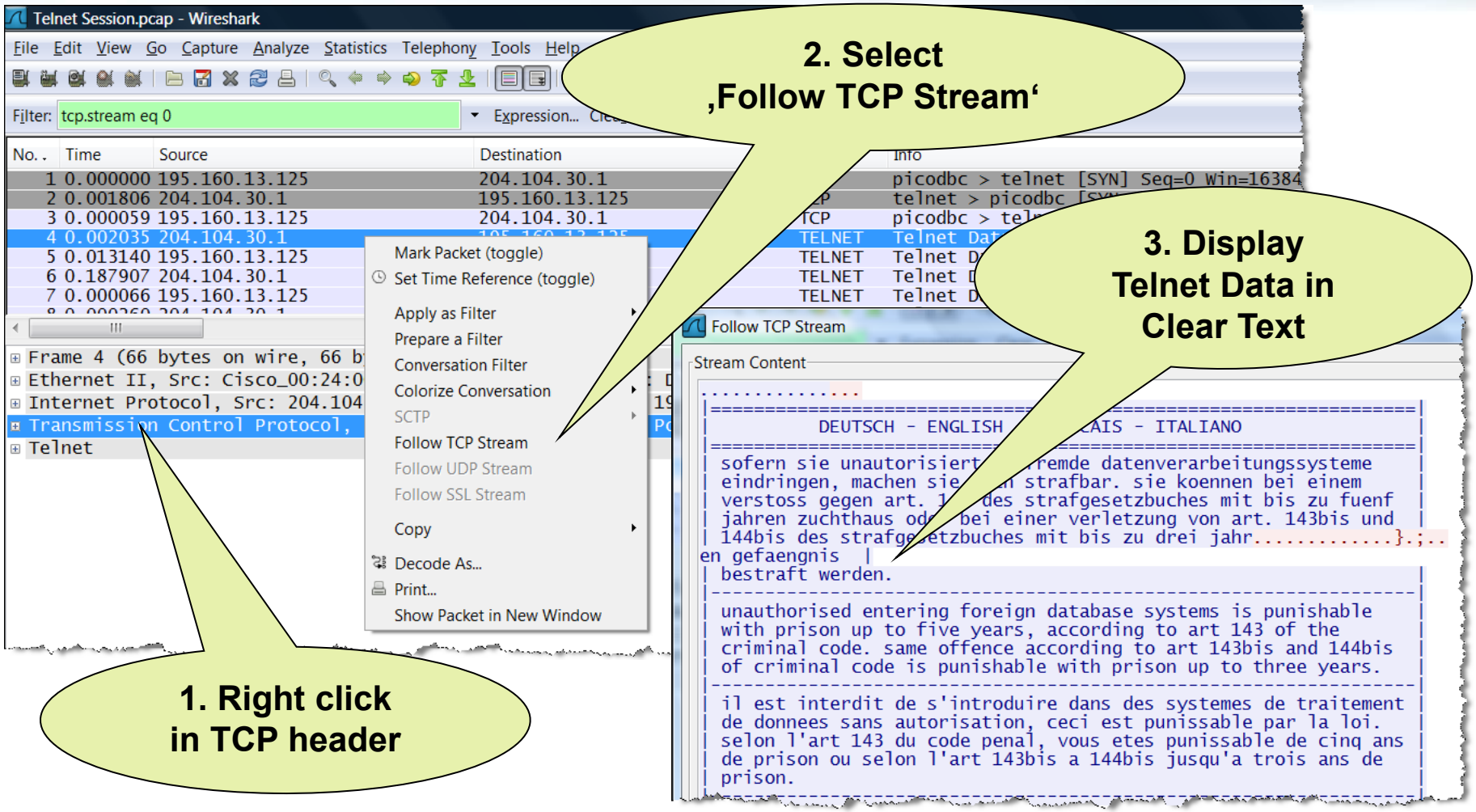


(File: Telnet Session.pcap)

© Leutert NetServices 2026 / TCP / V4.1f

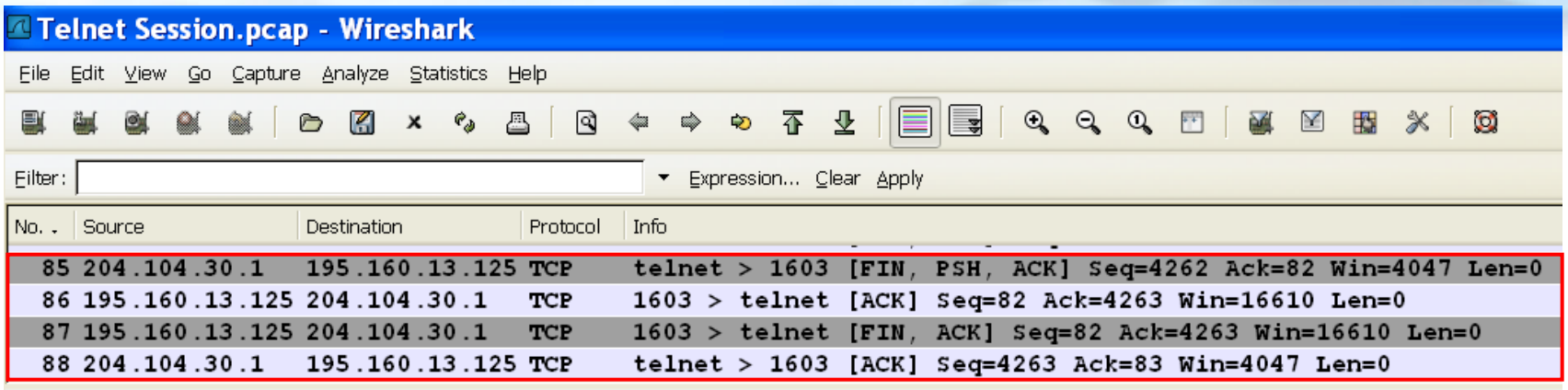
Transmission Control Protocol (TCP) Data Transmission

The Wireshark function ‚Follow TCP Stream‘

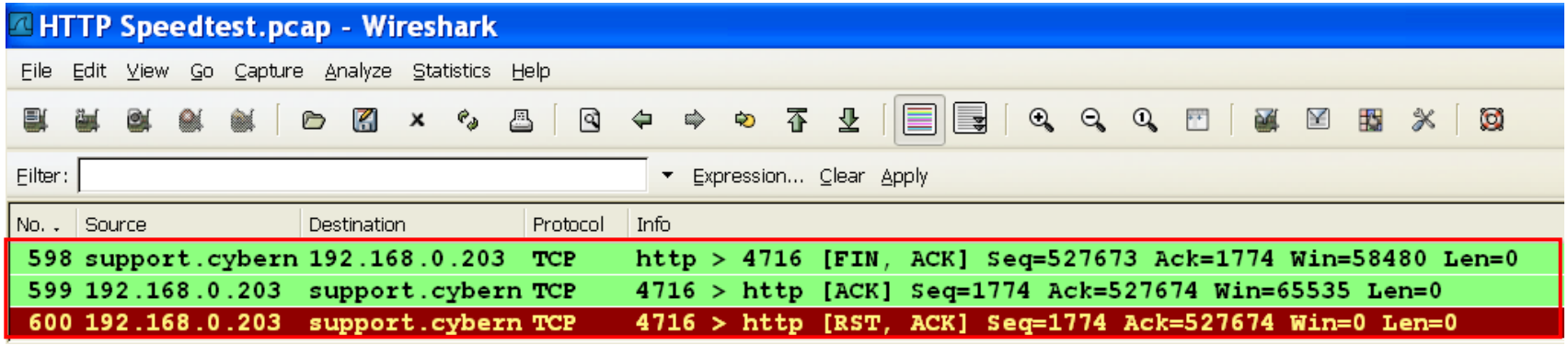


Transmission Control Protocol (TCP) Terminating Connection

Traditionally with ‚Four-way Handshake'



Variation with ‚RESET Flag'



Transmission Control Protocol (TCP) Reliability Functions

TCP contains numerous features for a reliable data transmission as well as the error correction.

One important feature is the repetition of lost, not confirmed segments (Retransmissions).

The sender starts a ‚Retransmission Time-Out' (RTO) for each segment sent. If no confirmation arrives within this time the segment is transmitted again.

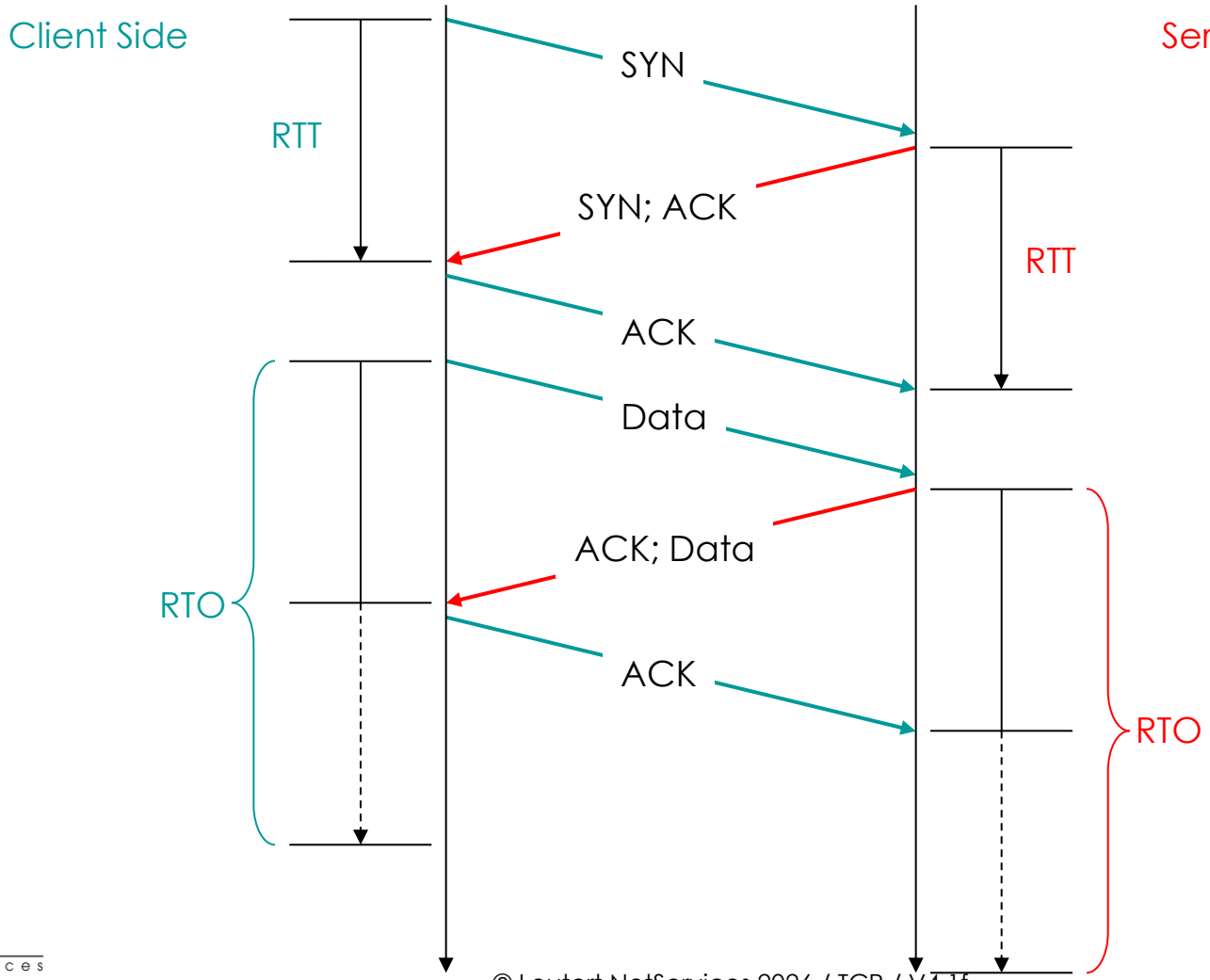
The value of this ‚RTO' has substantial influence on the efficiency of the data throughput. If the ‚RTO' is to long, valuable time is lost, if it is to short the connection will be overloaded with unnecessary ‚Duplicate Segments'.

TCP solves this problem by using an ‚adaptive RTO'.

The ‚Round Trip Time' (RTT) which is taken during the ‚Three-way Handshake' serves as a basis for the RTO.

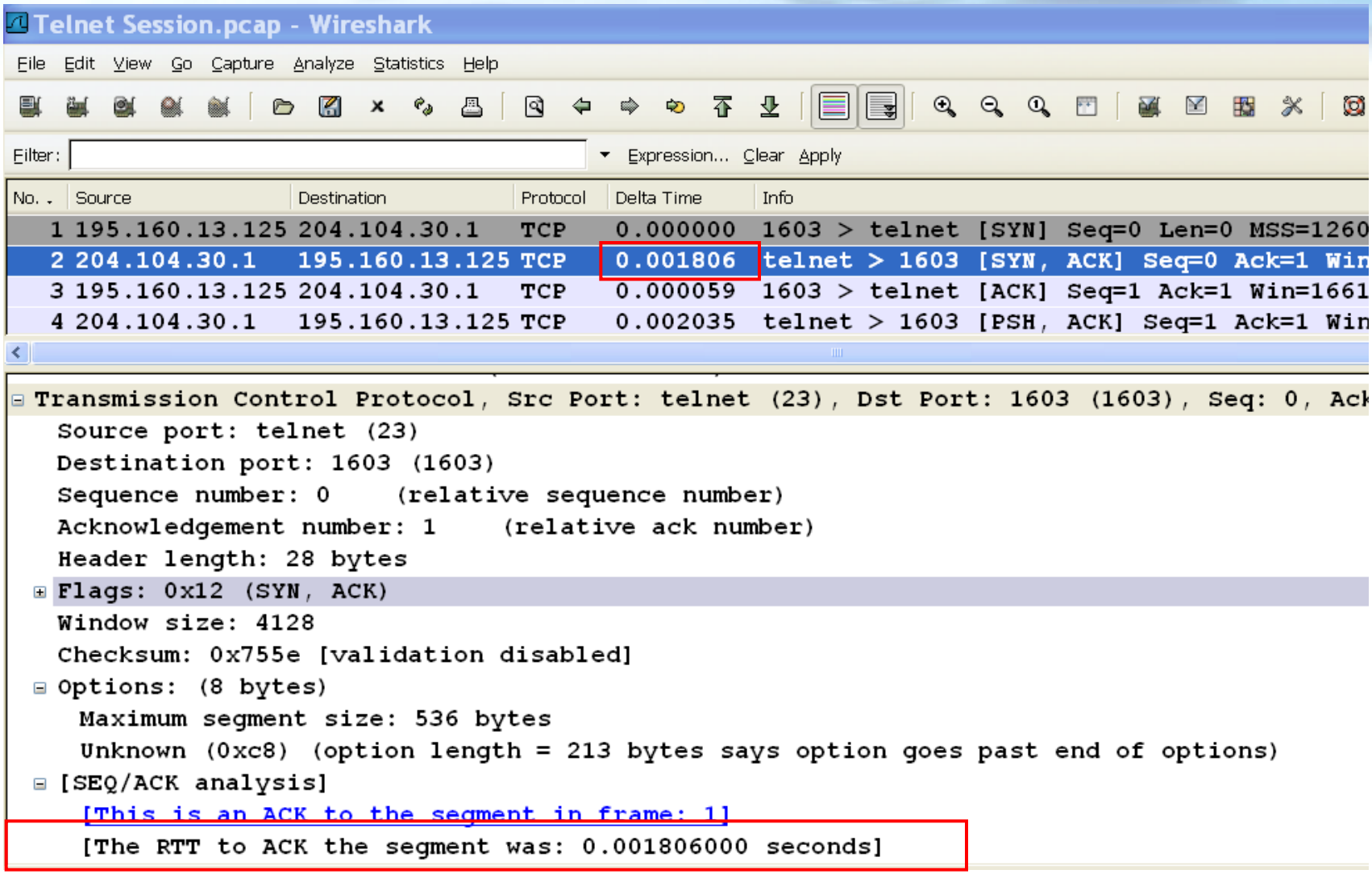
Transmission Control Protocol (TCP) Reliability Functions

‚Round-Trip Time' (RTT) measure during the ‚Three-way Handshake' in order to determine the ‚Retransmission Time-Out' (RTO).



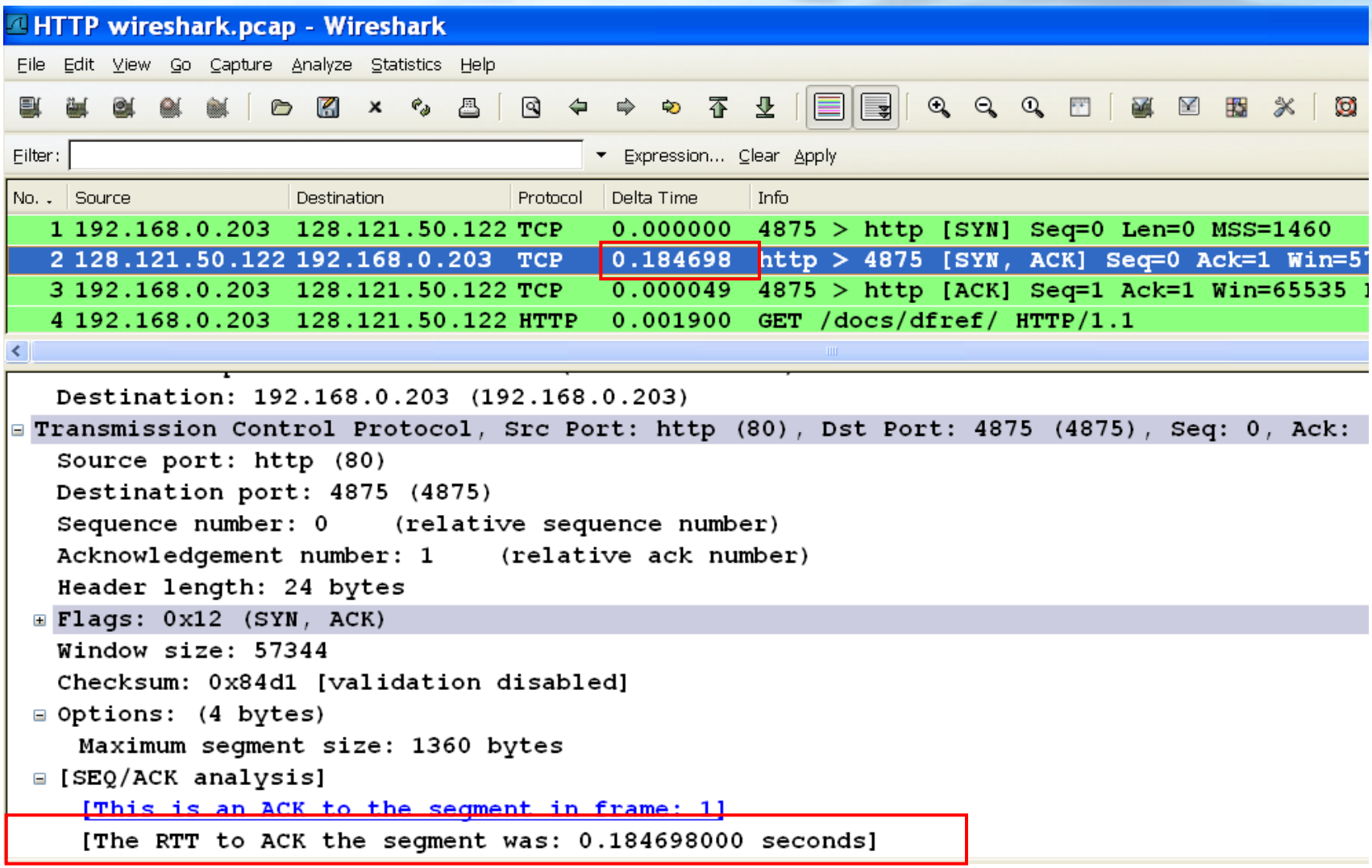
Transmission Control Protocol (TCP) Reliability Functions

‚Round-Trip Time' (RTT) measure with ‚Delta Time' (Local network)



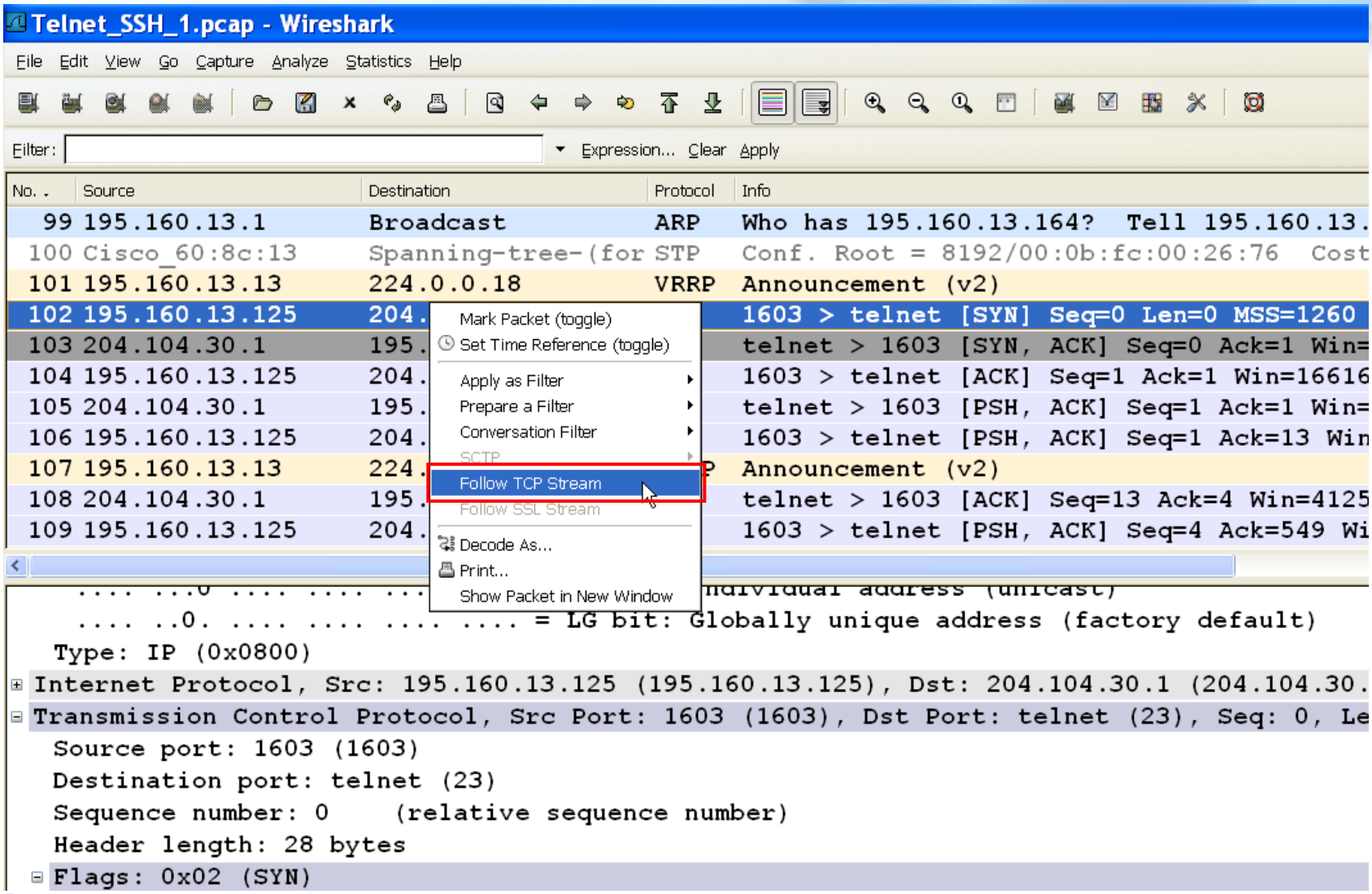
Transmission Control Protocol (TCP) Reliability Functions

‚Round-Trip Time' (RTT) measure with ‚Delta Time' (Remote network)



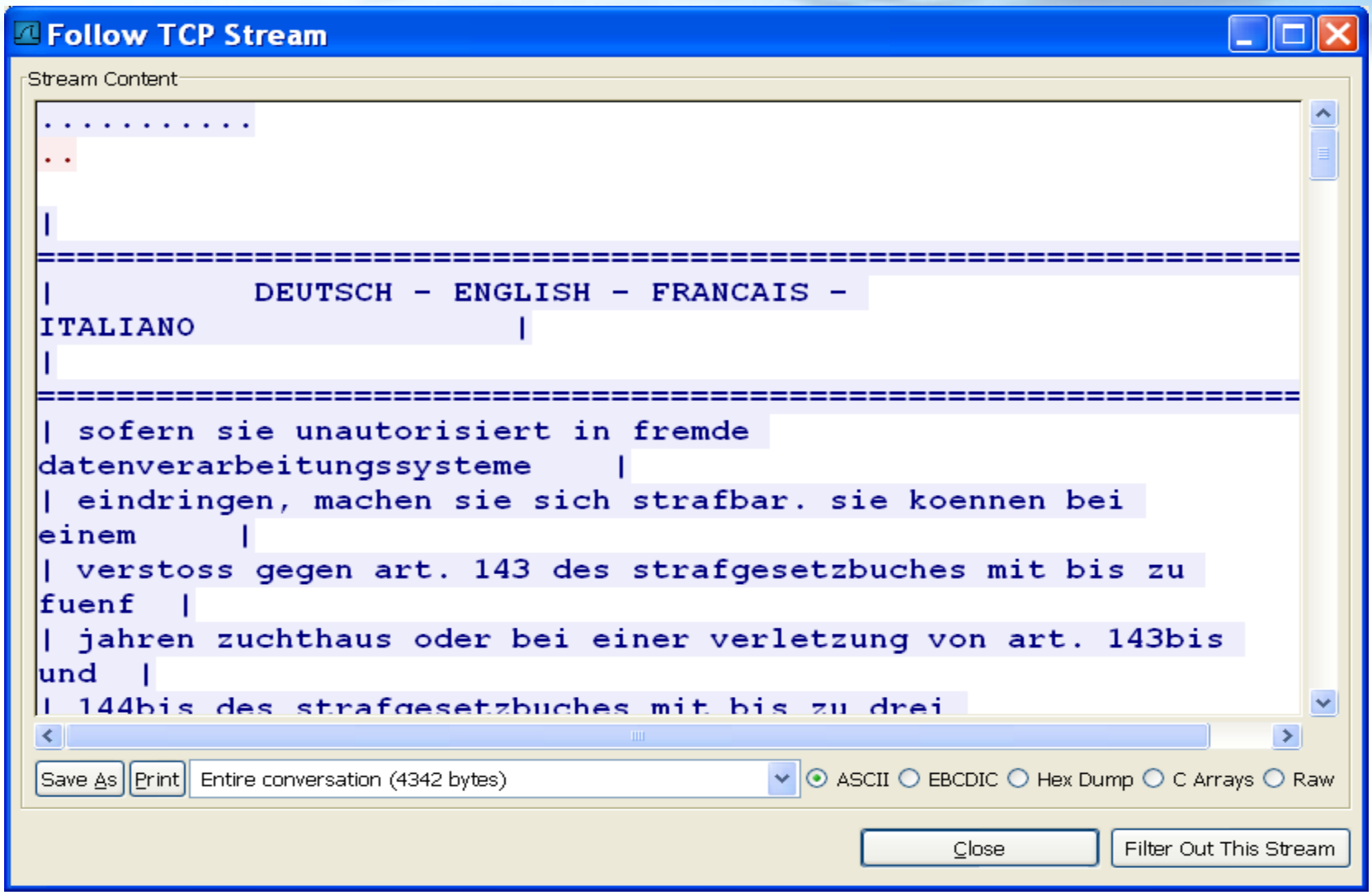
Transmission Control Protocol (TCP) Wireshark Basic Analysis

The Wireshark Function ‚Follow TCP Stream‘



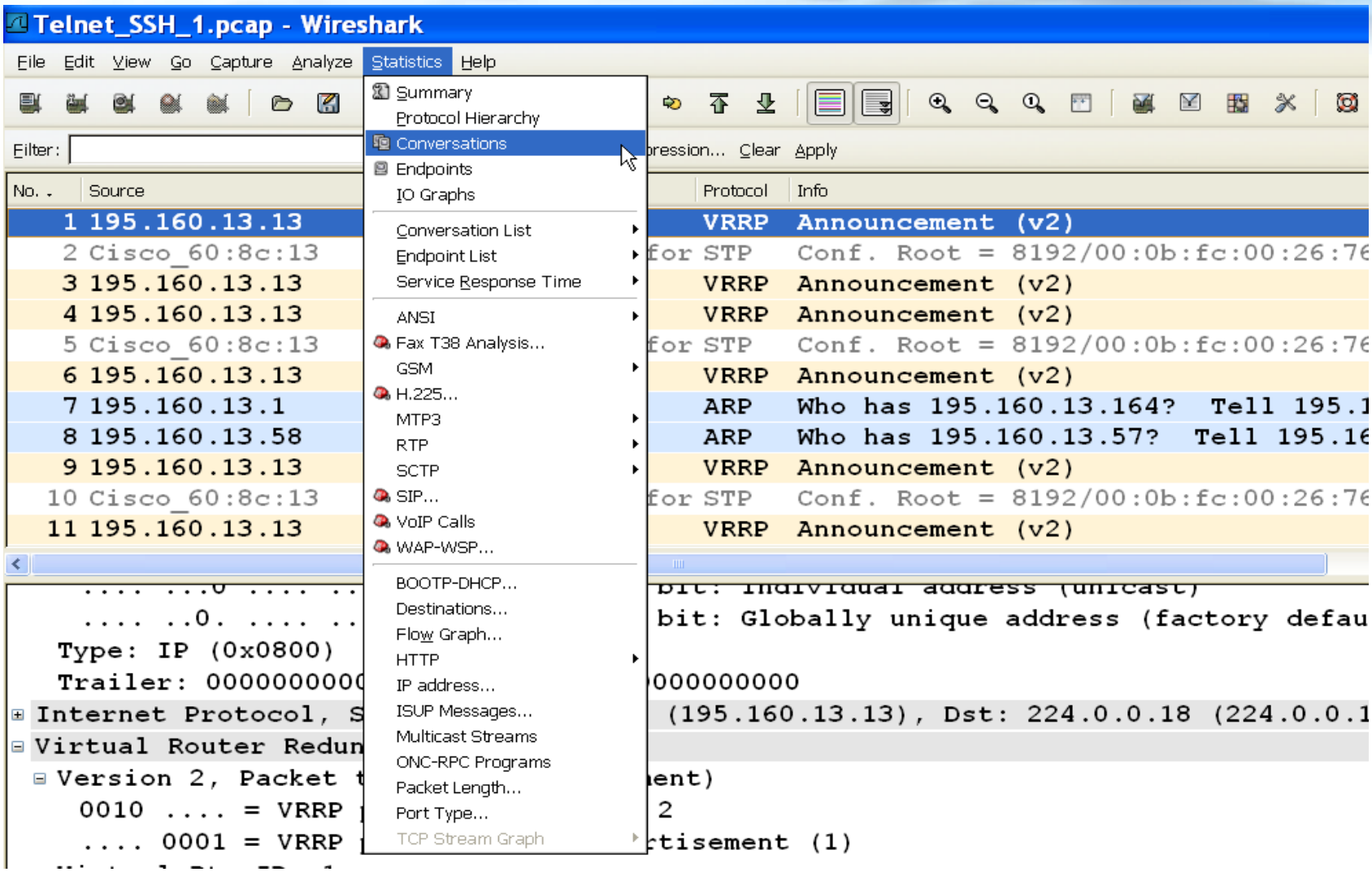
Transmission Control Protocol (TCP) Wireshark Basic Analysis

The Wireshark Function ‚Follow TCP Stream‘



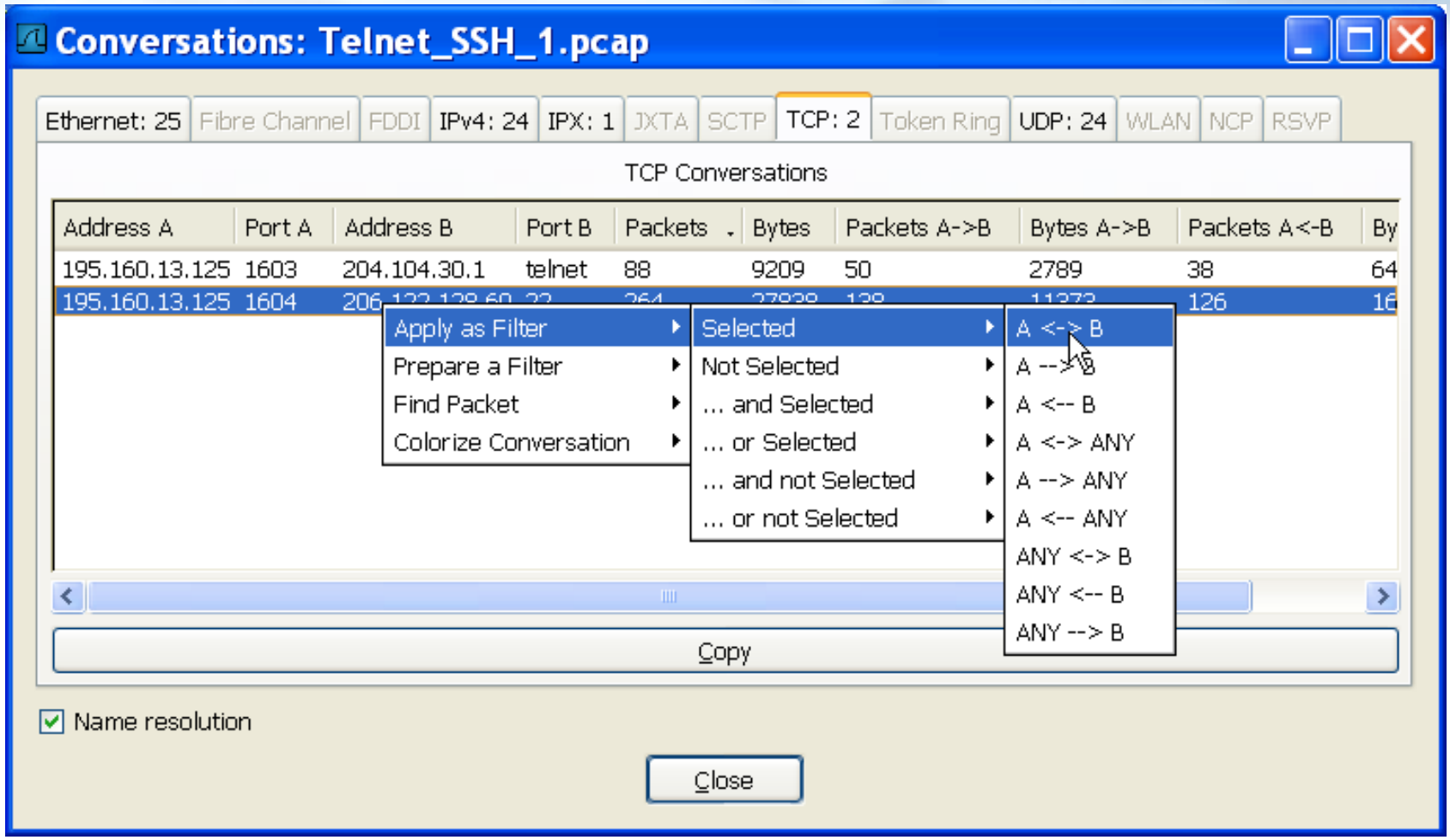
Transmission Control Protocol (TCP) Wireshark Basic Analysis

The Wireshark Function ‚Conversations‘



Transmission Control Protocol (TCP) Wireshark Basic Analysis

The Wireshark Function ‚Conversations‘



TCP Analysis using the Wireshark Expert

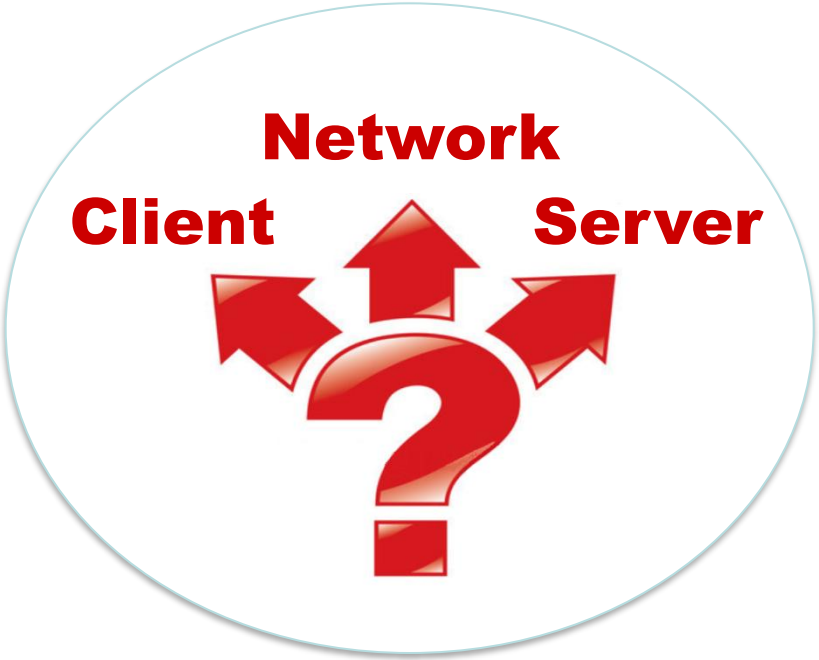
An expert system knows the rules and sequences of a certain process. With that knowledge, errors and abnormalities can be identified and reported.
And the analysis of big amounts of data is made much easier.



© Leutert NetServices 2026 / TCP / V4.1f

TCP Analysis using the Wireshark Expert

- TCP performance is depending on this three main components
- The Wireshark Expert offers great support in isolating bottlenecks
- Expert Messages will lead you to source of performance problems
- Analyzing TCP behavior will solve Finger Pointing situations



© Leutert NetServices 2026 / TCP / V4.1f

TCP Analysis using the Wireshark Expert

The Wireshark Expert is offering great support in analysing TCP sessions:

The Wireshark Expert System is able to analyse TCP sessions, to recognize abnormalities and errors and creates a list sorted by severities.

Wireshark can recognize situations like:

- Segment Lost
- Duplicate Acknowledgments
- Retransmissions
- Fast Retransmissions
- Zero Window
- Window Full
- and many more…

You still need well-founded TCP knowledge to understand the error messages and to draw the right conclusions.

© Leutert NetServices 2026 / TCP / V4.1f

TCP Analysis using the Wireshark Expert

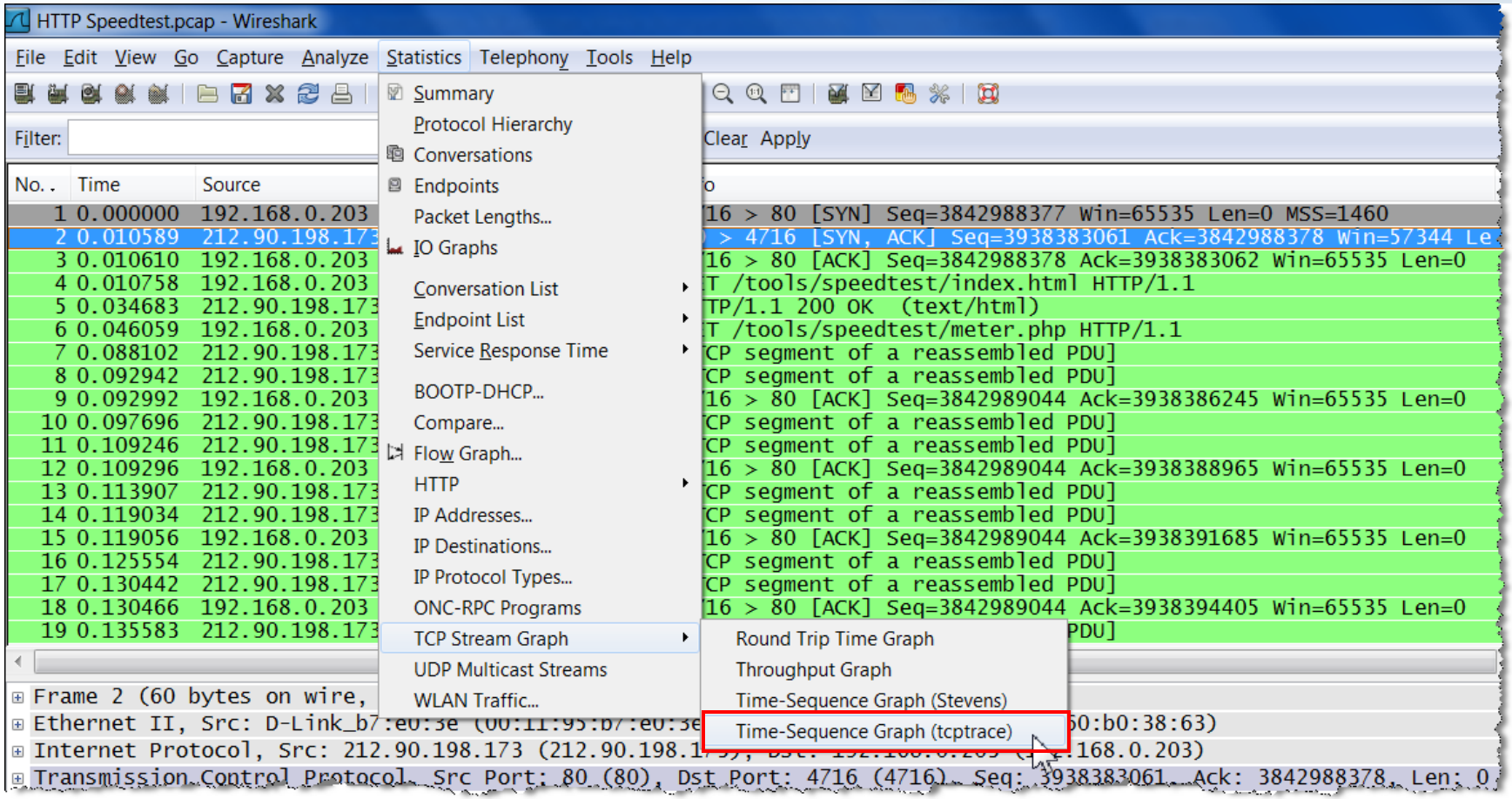
Wireshark offers graphical presentation of a TCP session.

Understanding the TCP Stream Graph allows to recognize all the following abnormalities:

- Lost Frames
- Duplicate Frames
- Out of order Frames
- TCP Sequence number and Segment Sizes
- Acknowledges, Delayed Acknowledges
- Duplicate and Selective Acknowledges
- Retransmissions and Fast Retransmissions
- Windows Sizes, sliding Window, exceeded und frozen Windows Size
- Window Scaling, Zero Window and Window Full Situation
- Slow Start, full Flow rate and Flow throttling

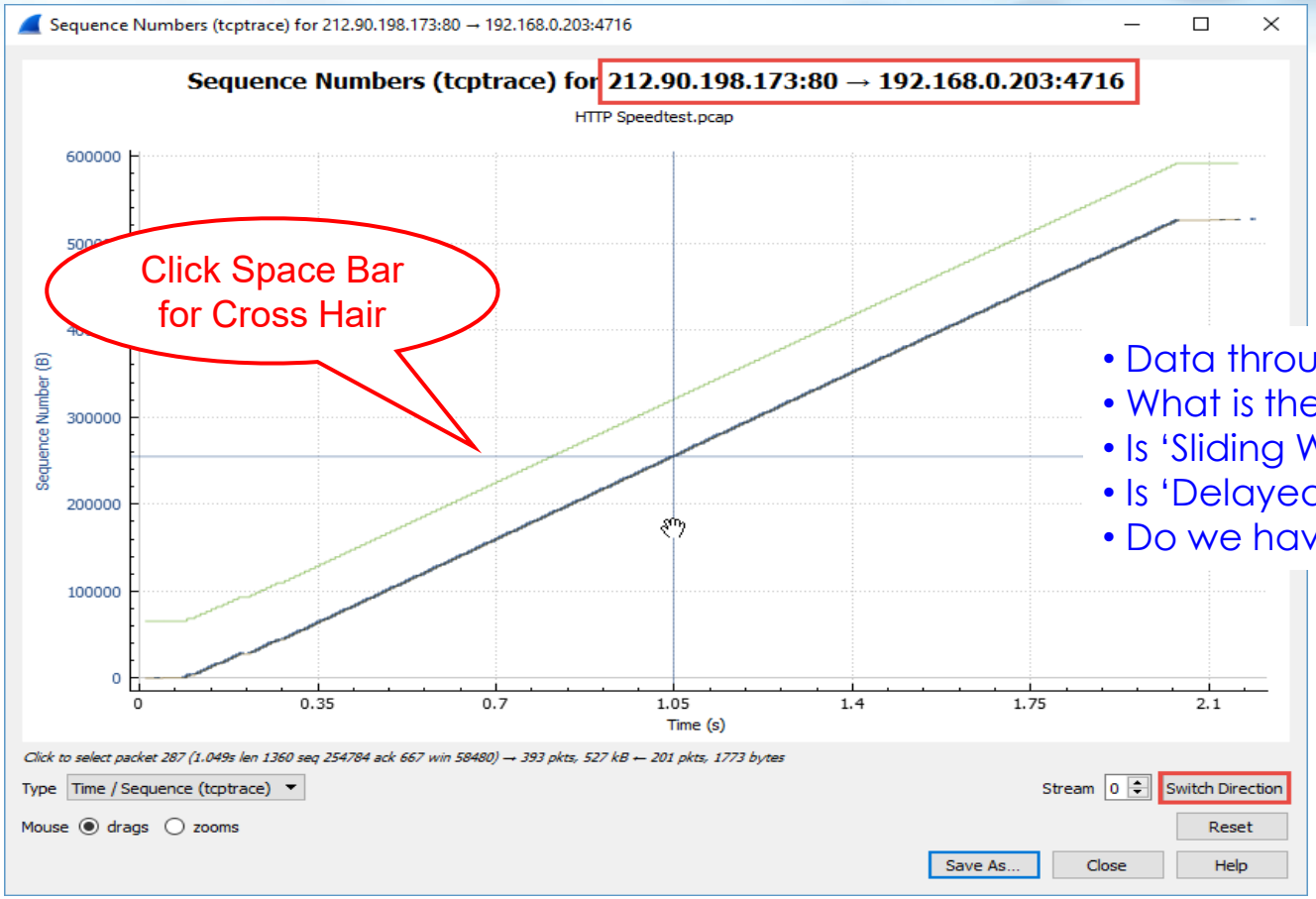
TCP Analysis using the Wireshark Expert

The Wireshark function ‚TCP Stream Graph' (tcptrace)



TCP Analysis using the Wireshark Expert

Exercise: Using the graph, answer the following questions:

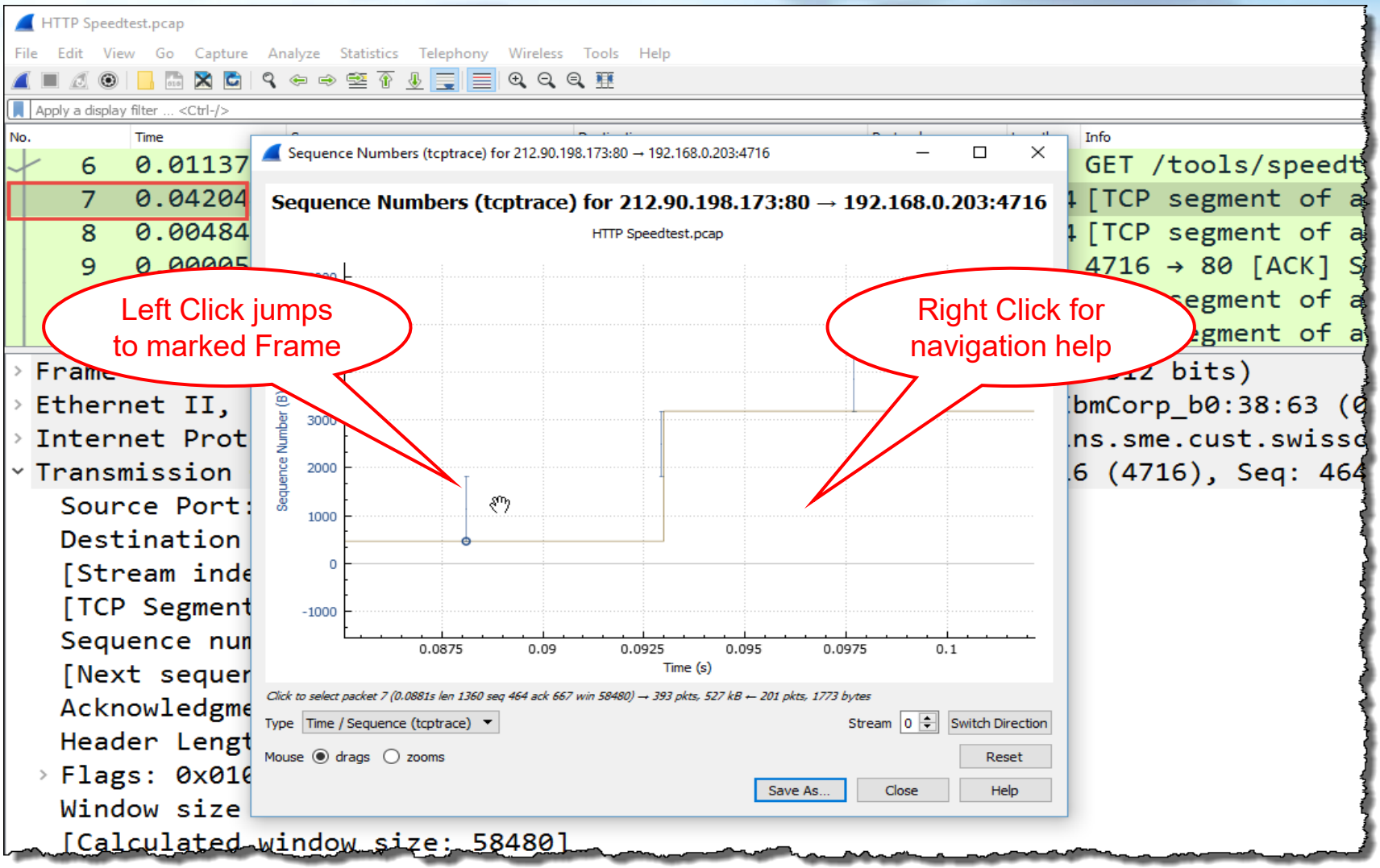


- Data throughput in Mbit/sec
- What is the initial Window Size?
- Is 'Sliding Window' working OK?
- Is 'Delayed Ack' in use?
- Do we have Retransmissions?

Important: Each TCP session consists of two independent Half Sessions, one in each communication direction, the graph is showing **one** Half Session

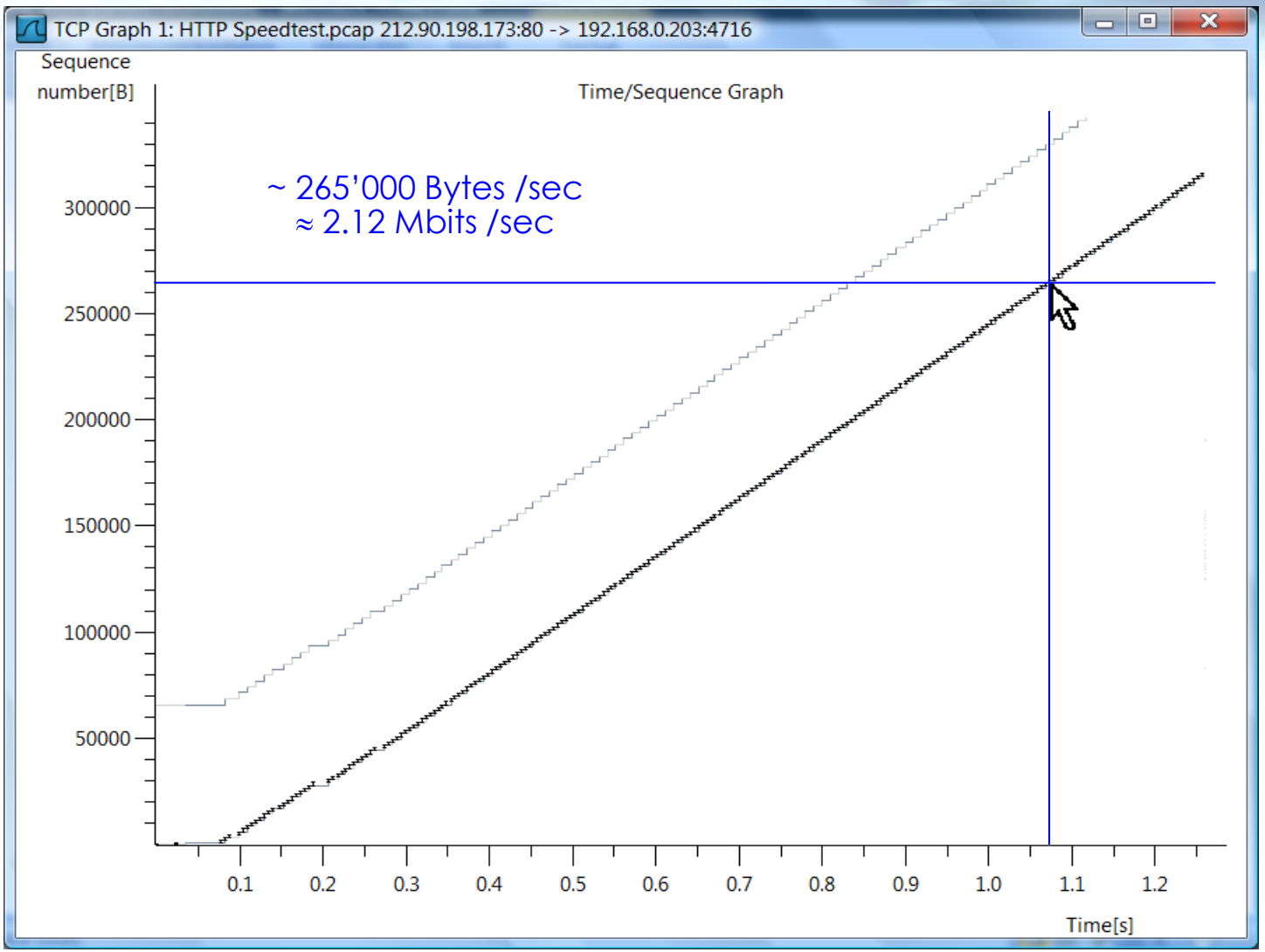
TCP Analysis using the Wireshark Expert

The Wireshark function: Graph navigation



TCP Analysis using the Wireshark Expert

Press Space Bar to show Cross hair

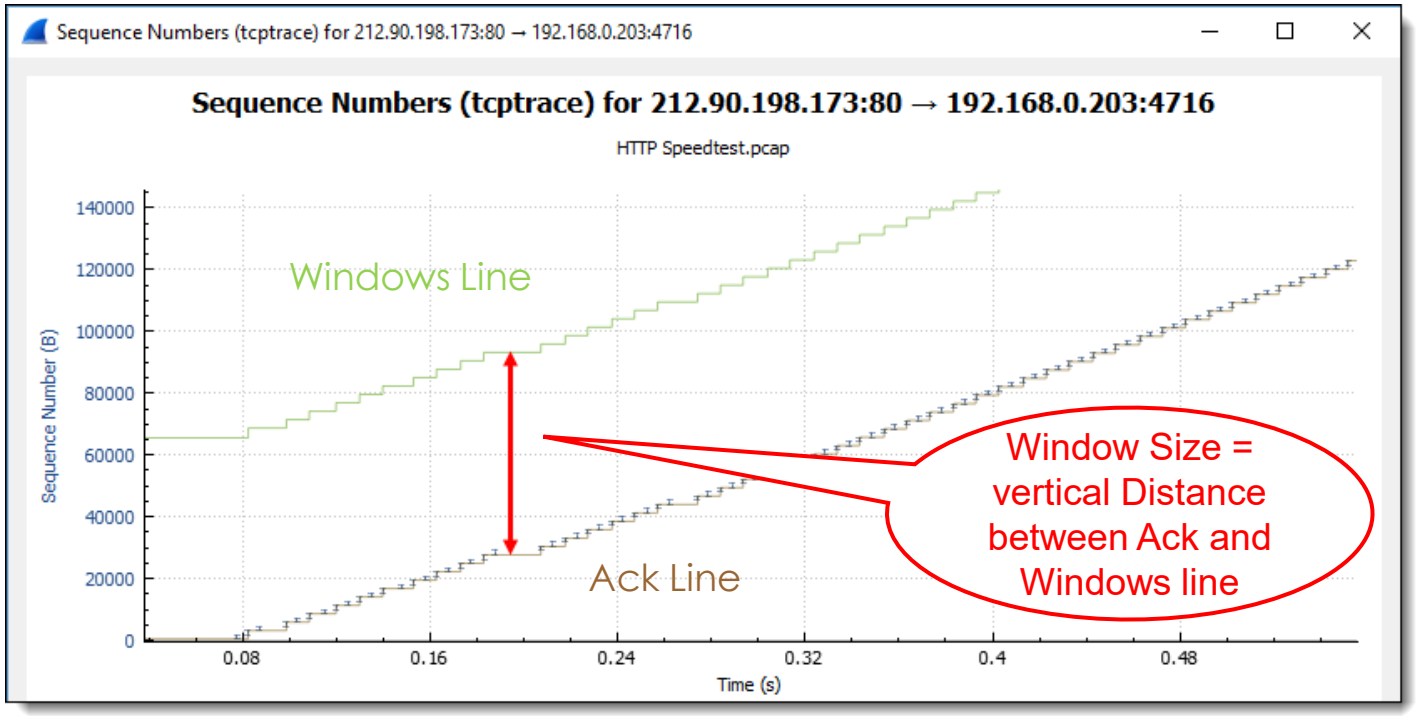


TCP Analysis using the Wireshark Expert

The TCP parameters

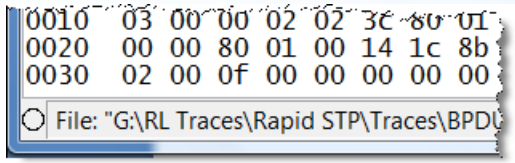
A TCP session is defined by four parameters in each direction (called Half-Session)

- Sequence Number: Is increased by the sender, by the number of sent bytes
- Acknowledge No: Points to the next expected byte and confirms all previous
- Window Size: Indicates the receive buffer size in bytes
- Timing: The size and the delta time between frames specifies the throughput

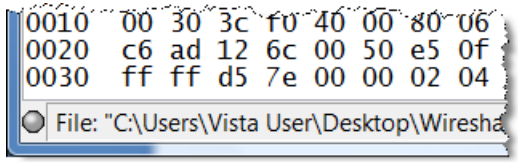


TCP Analysis using the Wireshark Expert

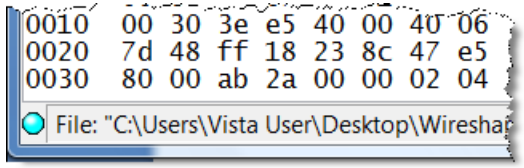
The colour of the ‚Expert Button' on the lower left corner displays the highest severity level present in the trace file.



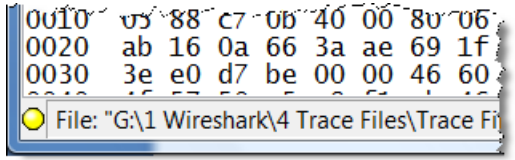
Level 0 = No Expert info available for protocols present in trace file (i.e. for protocols using UDP)



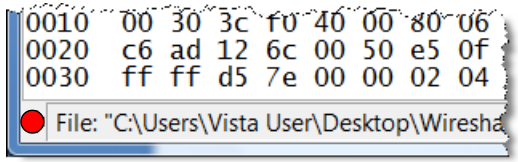
Level 1 = Chat: Information about the normal flow of data, e.g. TCP session establishment and closing. HTTP Get/OK/404 usw.



Level 2 = Note: Reference to slight abnormalities like ‚Duplicate ACK', ‚Retransmissions' etc.



Level 3 = Warning: Informs about abnormalities like ‚Segment lost', Segments out of order' etc.



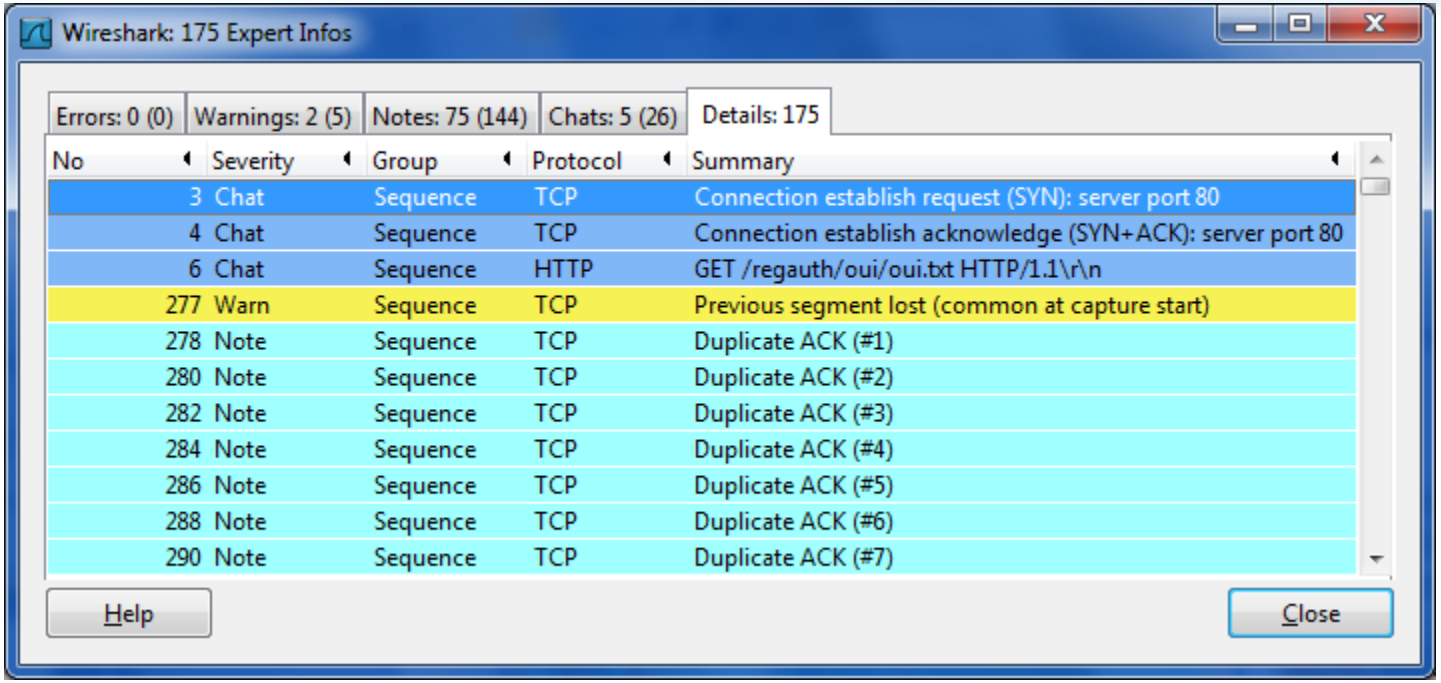
Level 4 = Error: Messages on serious problems like deformed segments (FCS errors)

Click on the ‚Expert Button' to open the window ‚Expert Info Composite'

© Leutert NetServices 2026 / TCP / V4.1f

TCP Analysis using the Wireshark Expert

Clicking on the ‚Expert Button' opens the window ‚Expert Infos'
The four categories of the ‚Expert Info' (severity levels):



Warning about interferences like ‚Segment lost', Segments out of order' etc.

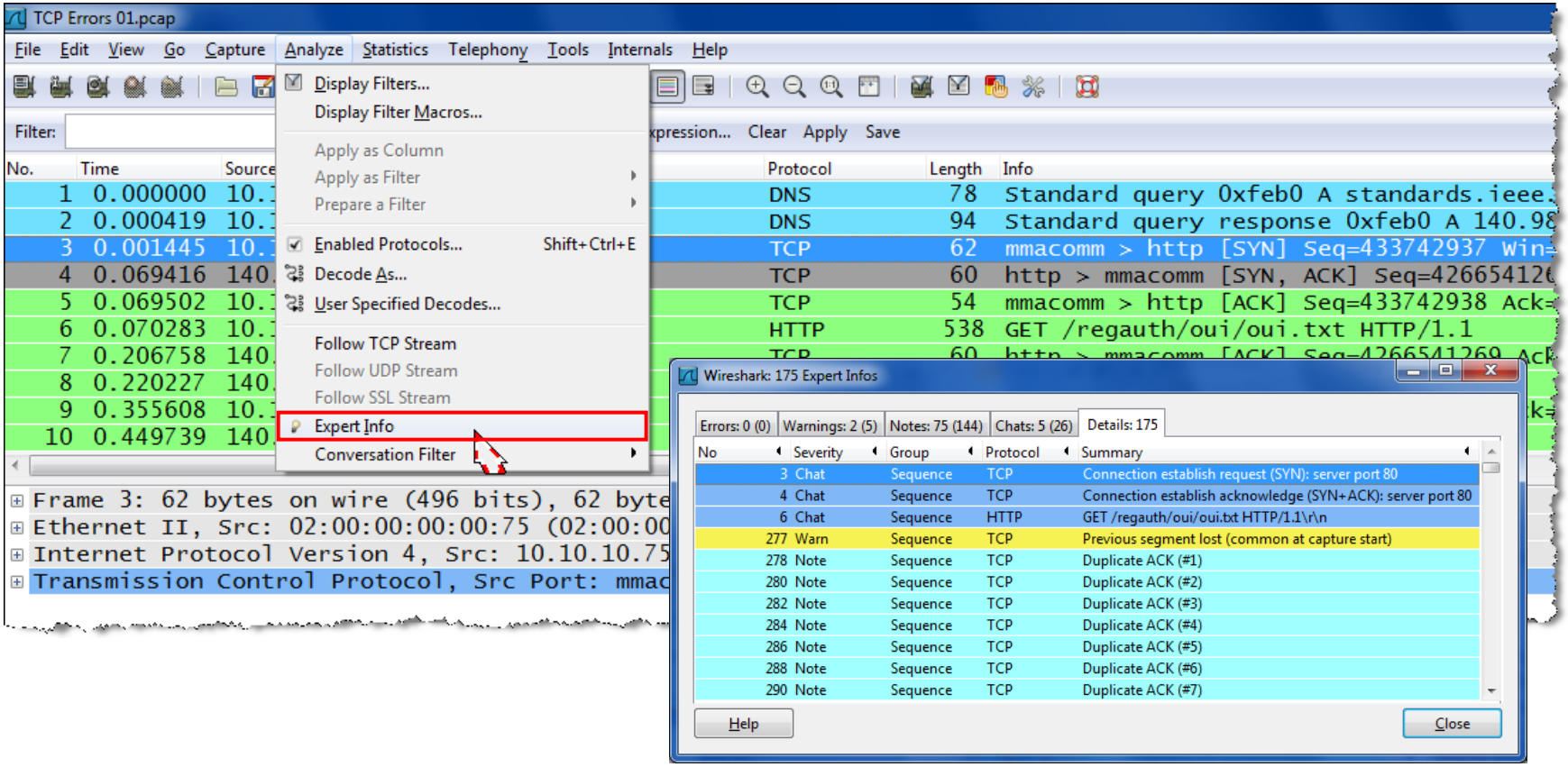
Error: Messages on serious problems like deformed segments (FCS errors)

Chat: Information about the normal flow of data, e.g. TCP session establishment and closing

Note: Reference to slight abnormalities like ‚Duplicate ACK', ‚Retransmissions' etc.

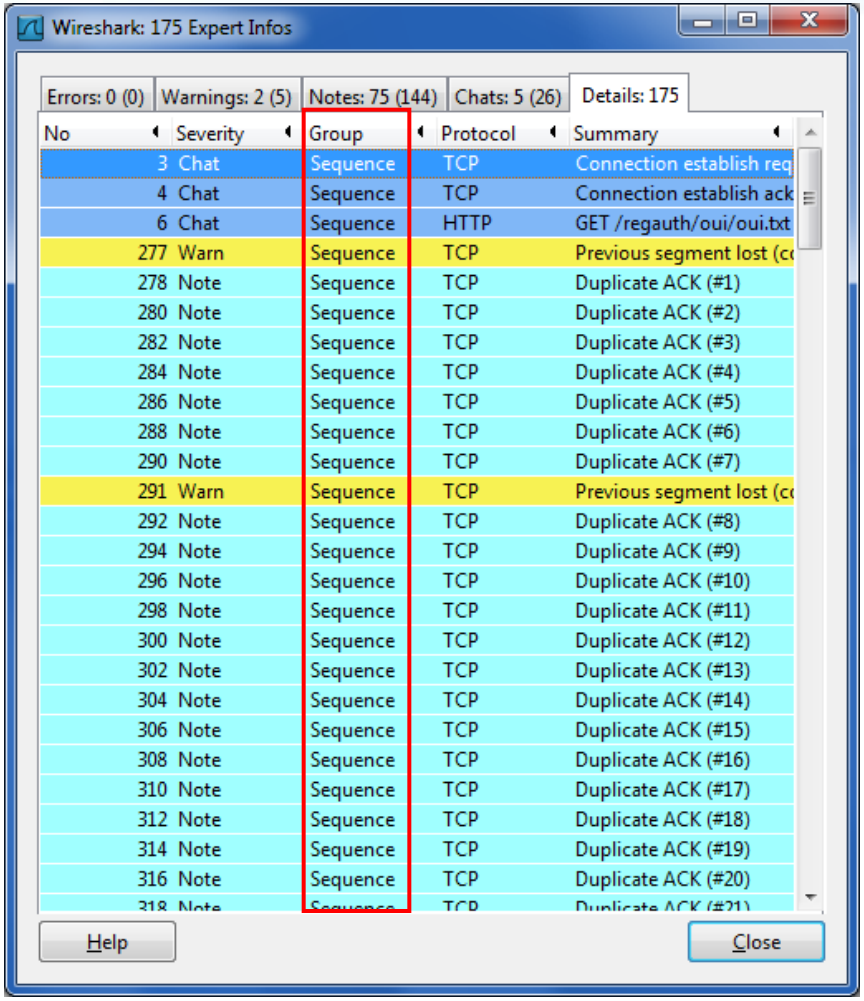
TCP Analysis using the Wireshark Expert

Wireshark detects numerous processes, irregularities and error situations.
According to their importance they are classified in four categories and six groups:



TCP Analysis using the Wireshark Expert

The six groups of ‚Expert Info‘:



Checksum: Incorrect checksum verification

Sequence: Abnormalities in the sequence order

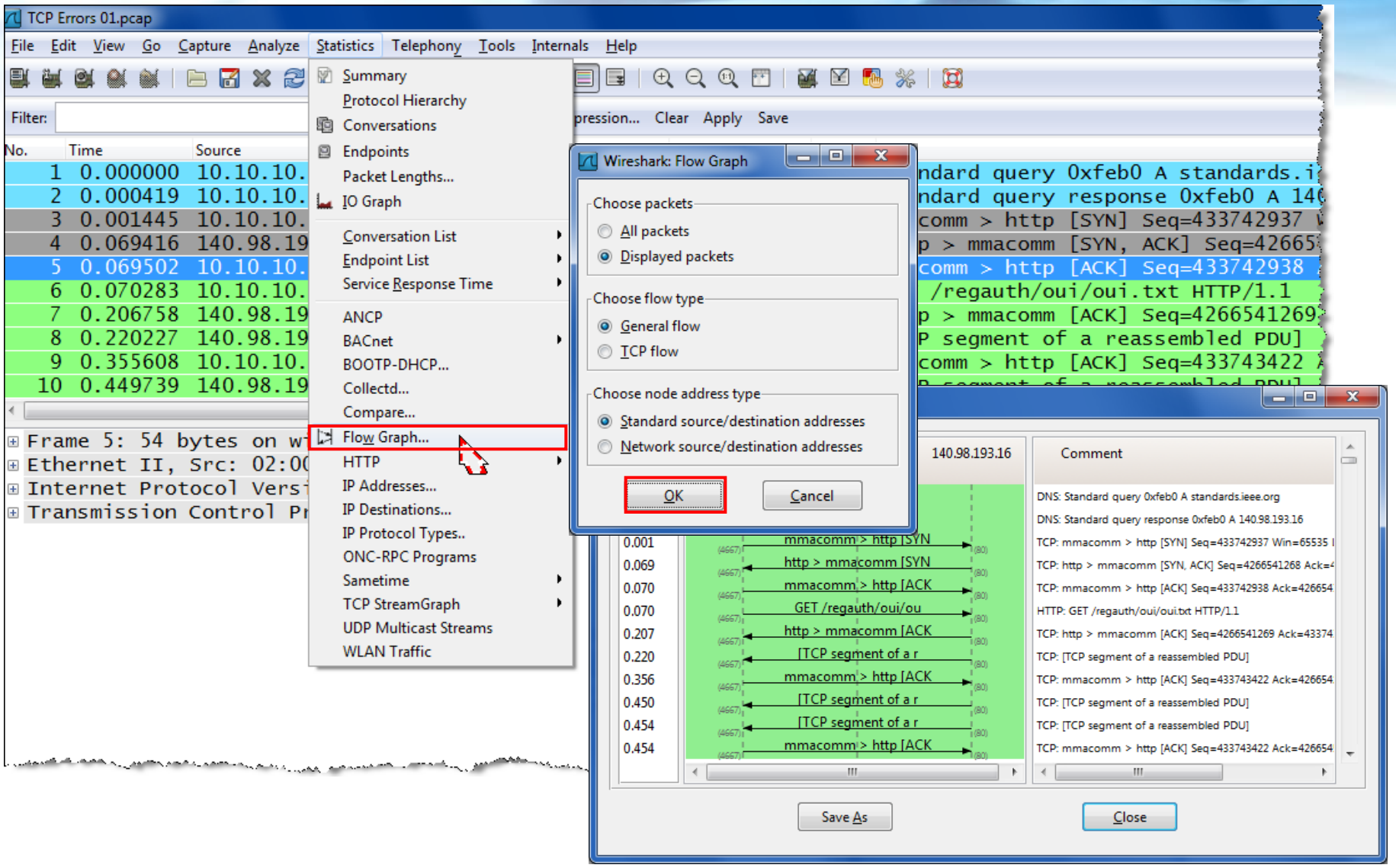
Response Code: Error message of an application, e.g. ‚HTTP 404 page not found‘

Undecoded: Protocol cannot be decoded by the ‚Dissector‘

Reassemble: Problems with the composition of segments

Malformed: Messages on serious problems like deformed segments

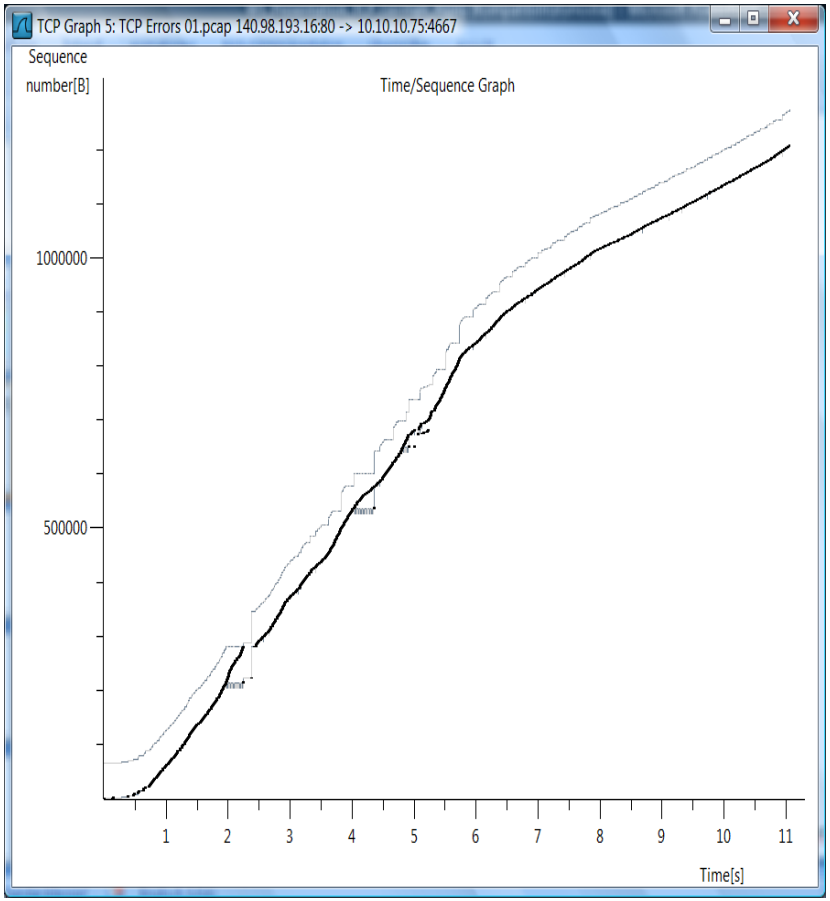
TCP Analysis using the Wireshark Expert



TCP Analysis using the Wireshark Expert

Exercise: Analyse the trace file ‚TCP Errors 01.pcap‘

- Is TCP ‚Slow Start‘ in use?
- What is the highest level of expert message ?
- Which ‚Window Size‘ symptoms do you recognize in the TCP Graph?
- Are ‚Selective Acks‘ in use?
- Is ‚Fast Retransmit‘ in use?
- Are TCP process rules broken?
- What is the average throughput over10 sec in Kbit/sec
- Do you think the TCP session is OK?

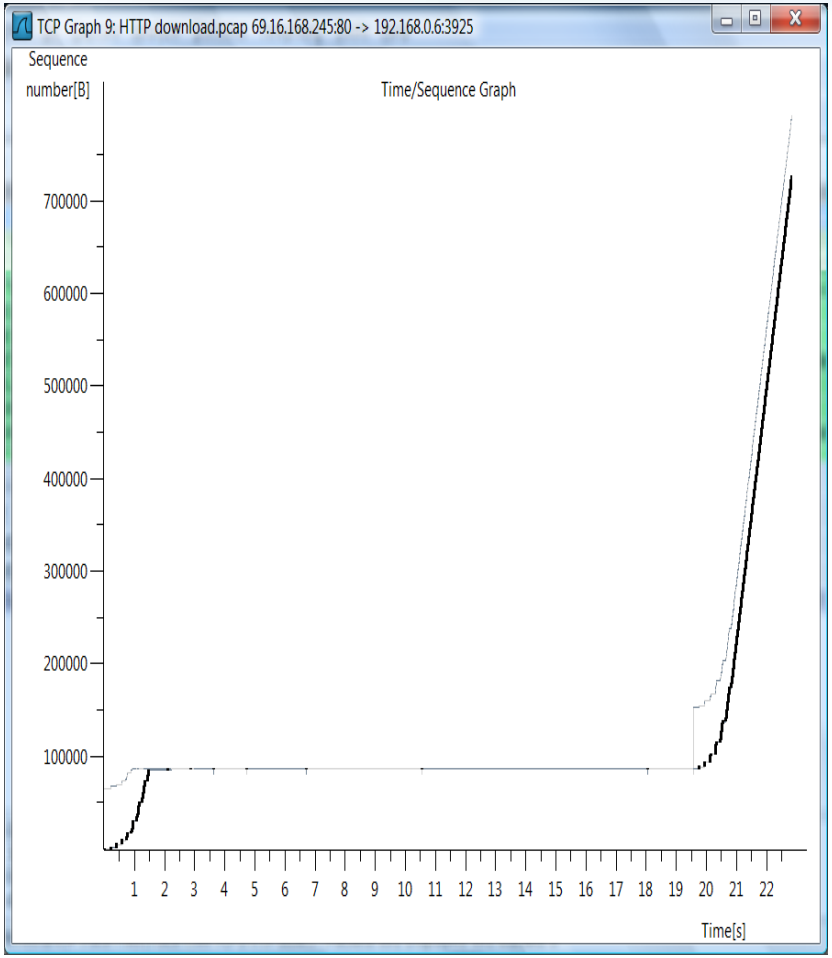


TCP Analysis using the Wireshark Expert

Exercise: Analyse the trace file ‚HTTP download.pcap‘

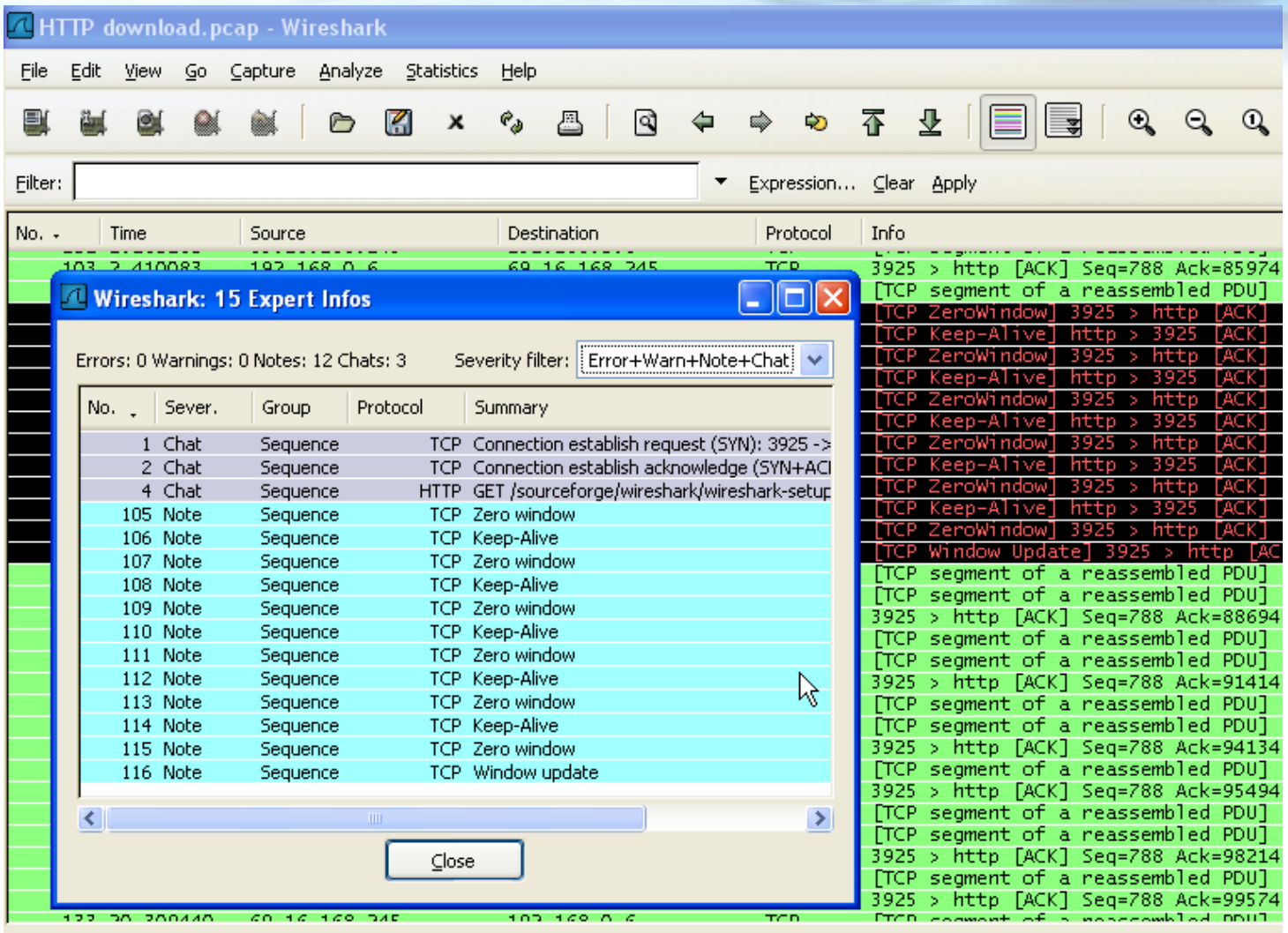
- Is TCP ‚Slow Start‘ in use?
- What is the highest level of expert message ?
- Which ‚Window Size‘ symptoms do you recognize in the TCP Graph?
- Are TCP process rules broken?
- How long the transmission was suspended?
- What caused the suspension?

  (Sender, receiver, network)



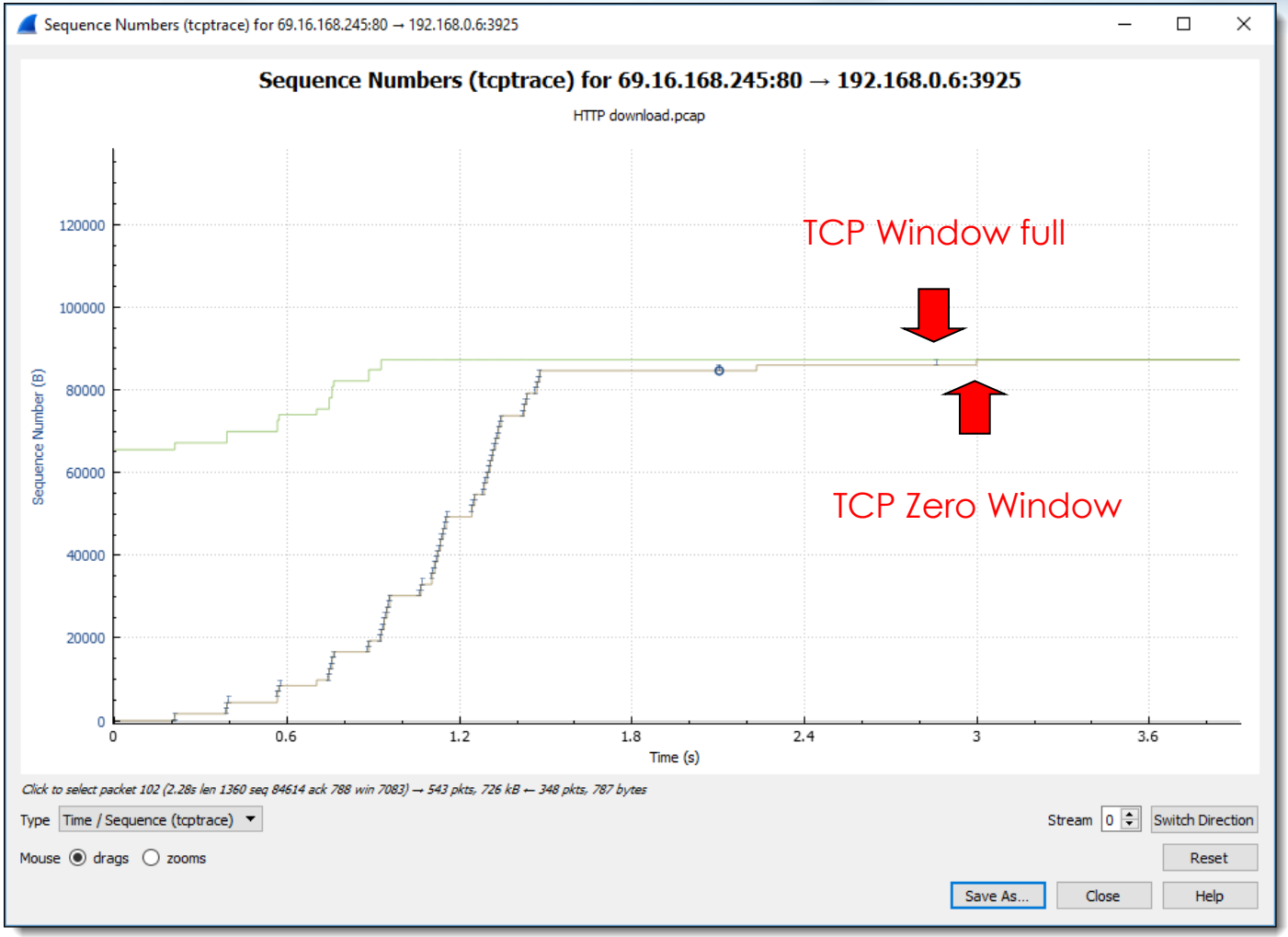
TCP Analysis using the Wireshark Expert

‚TCP Zero Window' Situation



TCP Analysis using the Wireshark Expert

TCP Zero Window situation



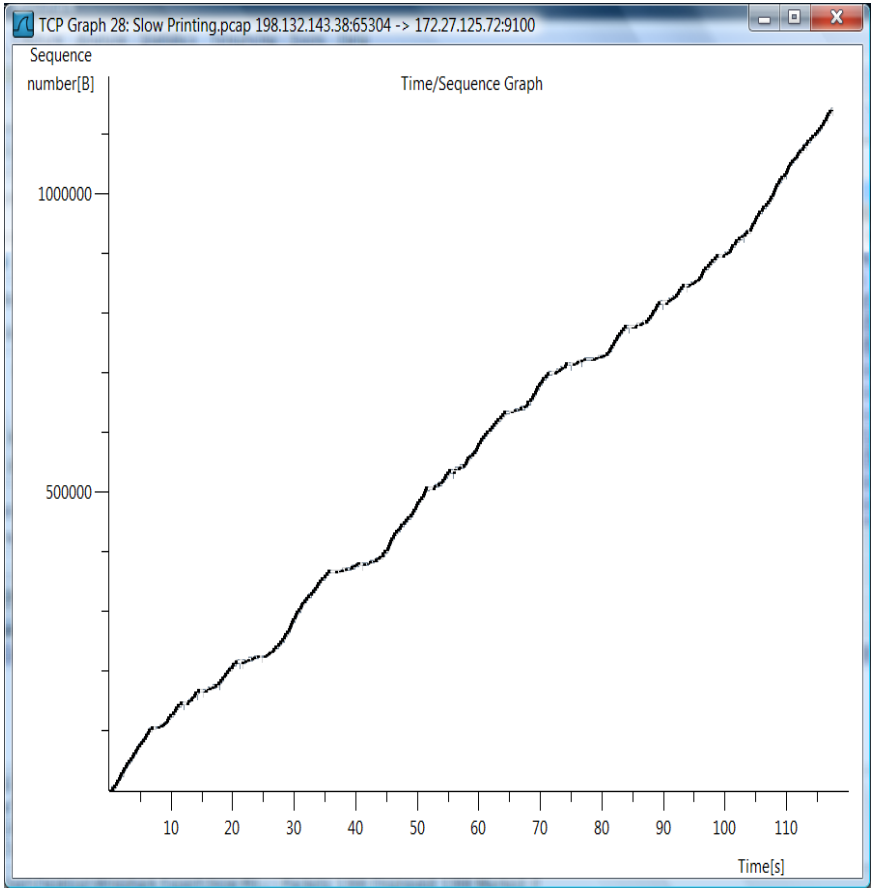
TCP Analysis using the Wireshark Expert

Exercise: Analyse the trace file ‚Slow Printing.pcap‘

A customer is complaining about long delays on printing!

- What is the highest level of expert message ?
- Which ‚Window Size‘ symptoms do you recognize in the TCP Graph?
- Are TCP process rules broken?
- What is the average throughput over100 sec in Kbit/sec
- What did limit throughput?

  (Sender, receiver, network)
- On which side you would do more investigations?



TCP Extensions for High Performance (RFC7323)

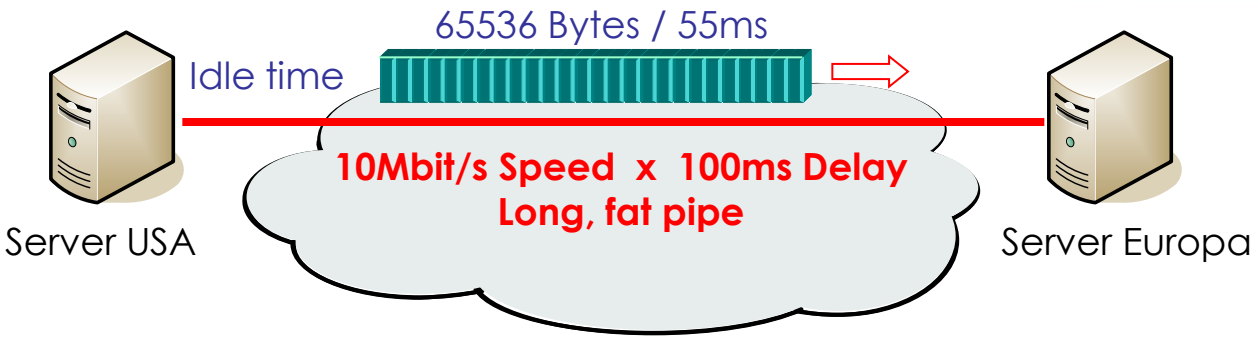
- TCP was designed to operate reliably in the range100bps to 10Mbps and delays of 1ms bis 100sec.
- The introduction of fiber optics is resulting in ever-higher trans-mission speeds paths and are moving out of the domain for which TCP was originally engineered.



- TCP performance depends not upon the transfer rate itself, but rather upon the product of the transfer rate and the round-trip delay. TCP performance problems arise when the bandwidth*delay product is large.
- Internet path operating in this region are called "long, fat pipe", and a network containing this path as an "LFN" (pronounced "elephan(t)").

‘Long, Fat Pipe’ Problems

Maximum standard TCP window size is 65536 Bytes (=2^{16})





‘Long Fat Network’ Problems

- High-capacity packet satellite channels are LFN's. (4 x 35‘800 km = 470ms RTT) (Round Trip Time)
- Terrestrial fiber-optical paths will also fall into the LFN class



There are three fundamental performance problems with the current TCP over LFN paths:

- Window Size Limit (max 64K bytes)

  Remedy: TCP option „Window scale“
- Recovery from Segment Losses

  Remedy: TCP option „Selective acknowledges“
- Round-Trip Measurement

  Remedy: TCP option „Time stamp“

TCP 'Window Scaling' Option

- The TCP Window Size of 65'535 Bytes is too small for LFN's.
- A multiplier called Skaling Factor resolves this limitation.
- During 3-Way hand shake, the Scaling Factor S is negotiated.
- Must not match, each side can offer an individual Scaling Factor.



- The value for the Scaling Factors can be from 0 to14.
- Calculation for the scaled Window Size is as follows:

$$\text{Scaled Window Size} = \text{Window Bytes} \times 2^{S}$$

- Example: Window Size 46 Bytes, Scaling Factor S = 7 $\rightarrow 2^{7} = 128$

46 Bytes x 128 = 5'888 Bytes

- The maximum Window Size is can be: S = 14 $\rightarrow 2^{14} = 16'384$

1'073'741'824 Bytes = 1 Gigabyte (binär)

© Leutert NetServices 2026 / TCP / V4.1f

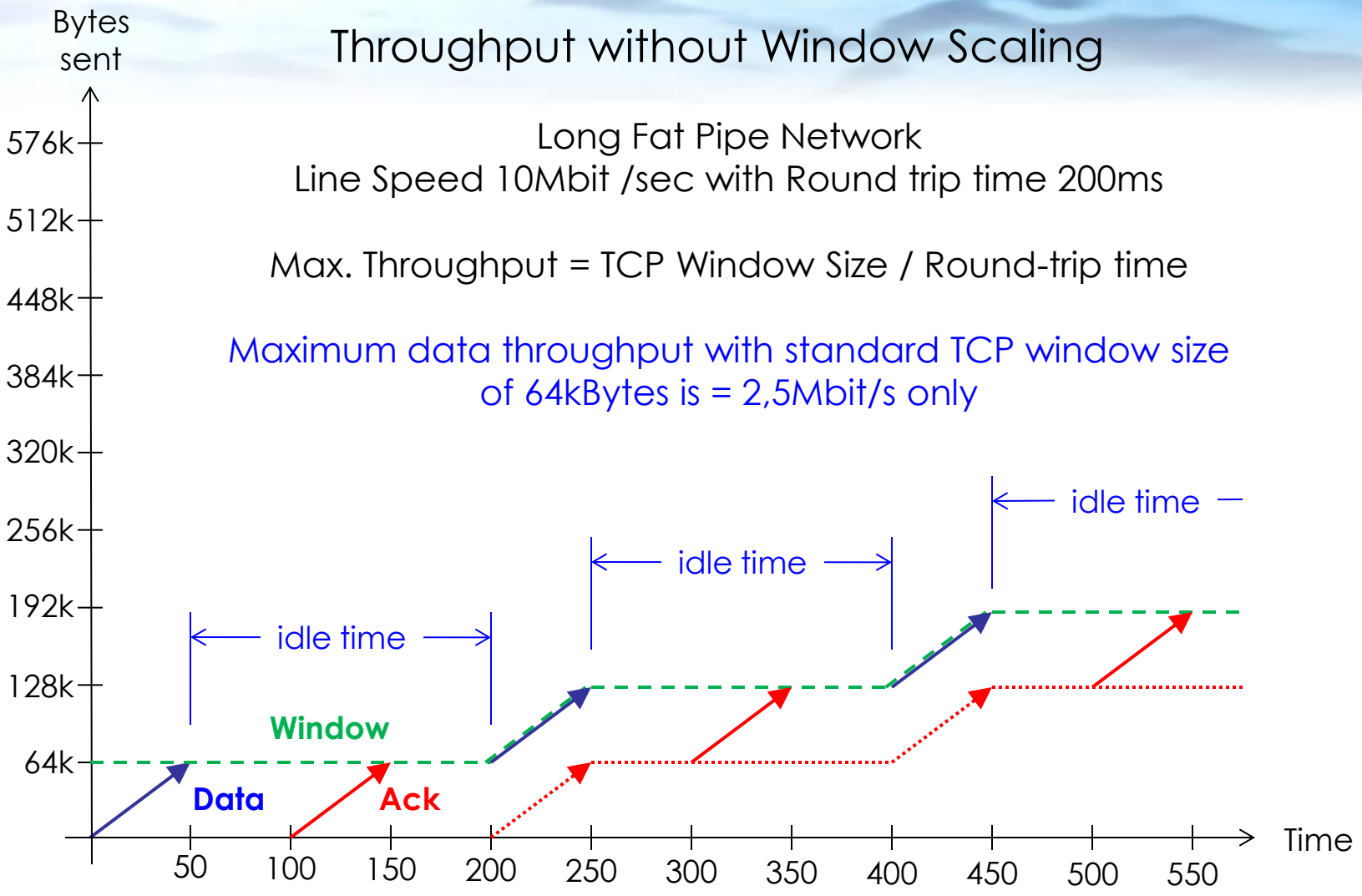
TCP 'Window Scaling' Option

Throughput without Window Scaling

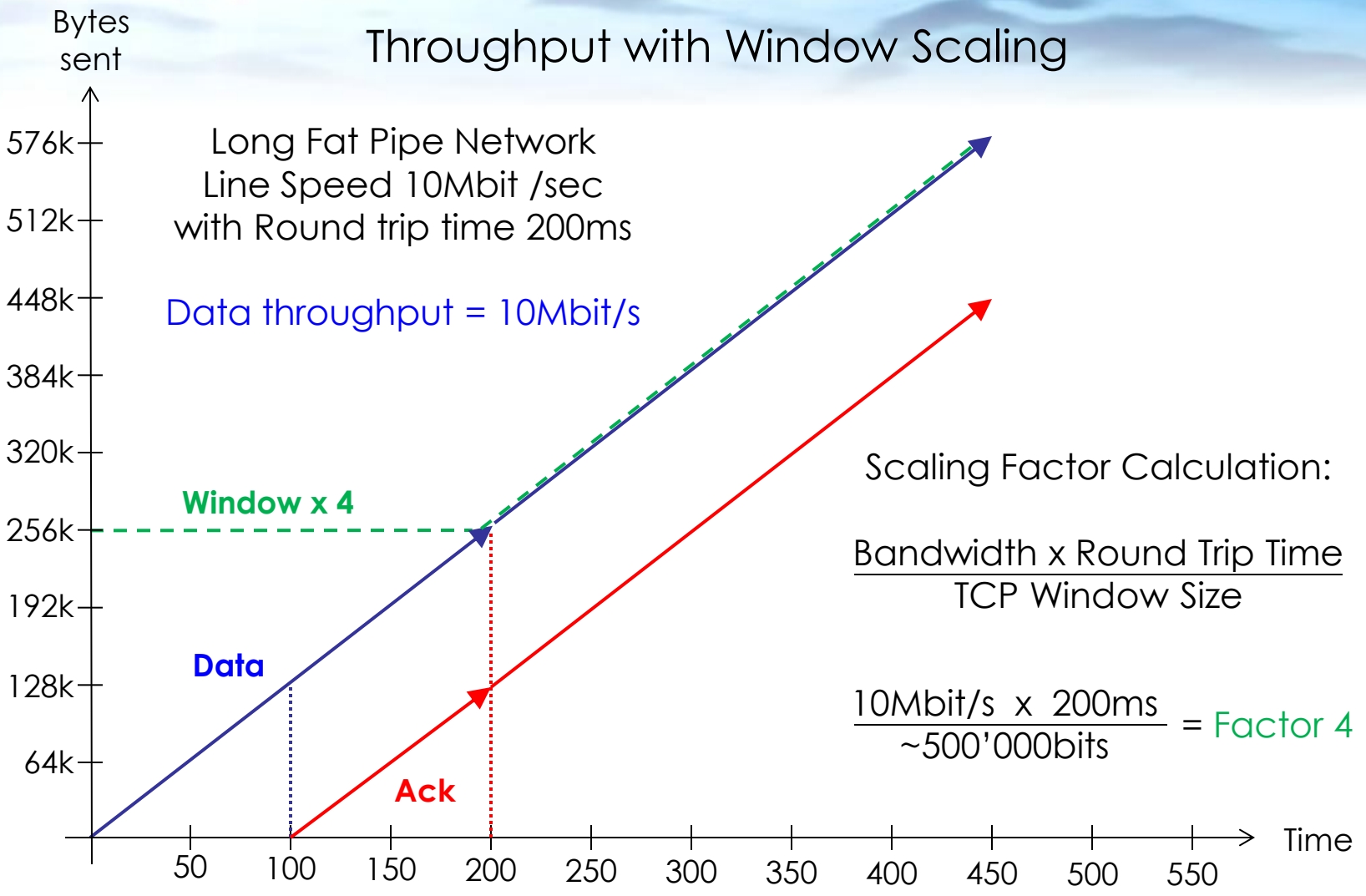
Long Fat Pipe Network
Line Speed 10Mbit /sec with Round trip time 200ms

Max. Throughput = TCP Window Size / Round-trip time

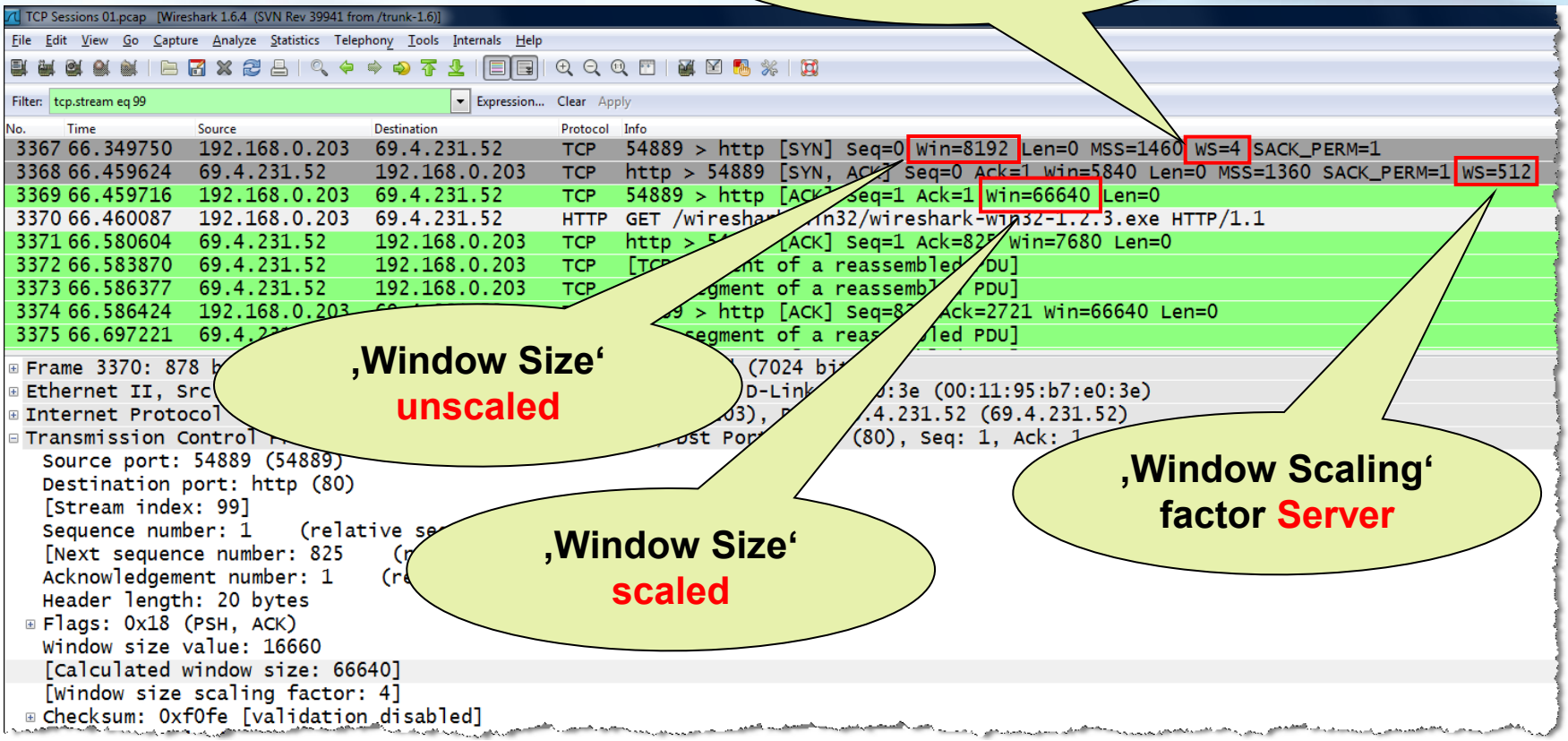
Maximum data throughput with standard TCP window size of 64kBytes is = 2,5Mbit/s only



TCP 'Window Scaling' Option



TCP 'Window Scaling' Option



After the two SYN frames, the window size is announced in the scaled format and Wireshark displays the scaled value.

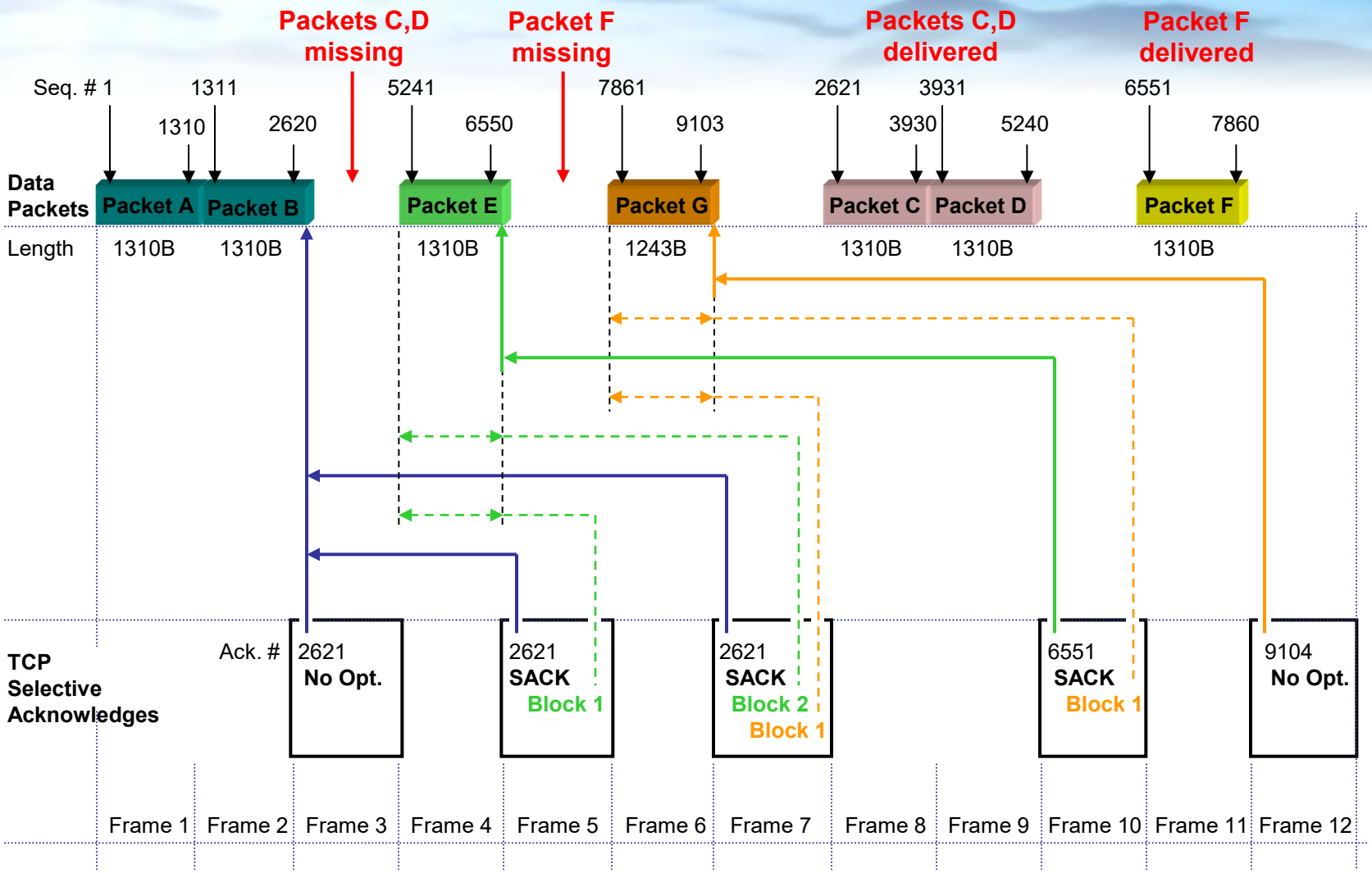
TCP 'Selective Acknowledge' Option (RFC 2018)

- The usage of the TCP SACK option is negotiated during the 3-Way hand shake.
- The SACK option will be offered by the client in the TCP SYN and has be accepted by the server in the TCP SYN ACK packet.
- Without SACK option, only the last received segment of a contiguous series can be acknowledged.



- The SACK Option allows to acknowledge non-contiguous segments of a series and can request for specific segments.
- The SACK Option can improve the throughput of LFN's significantly.

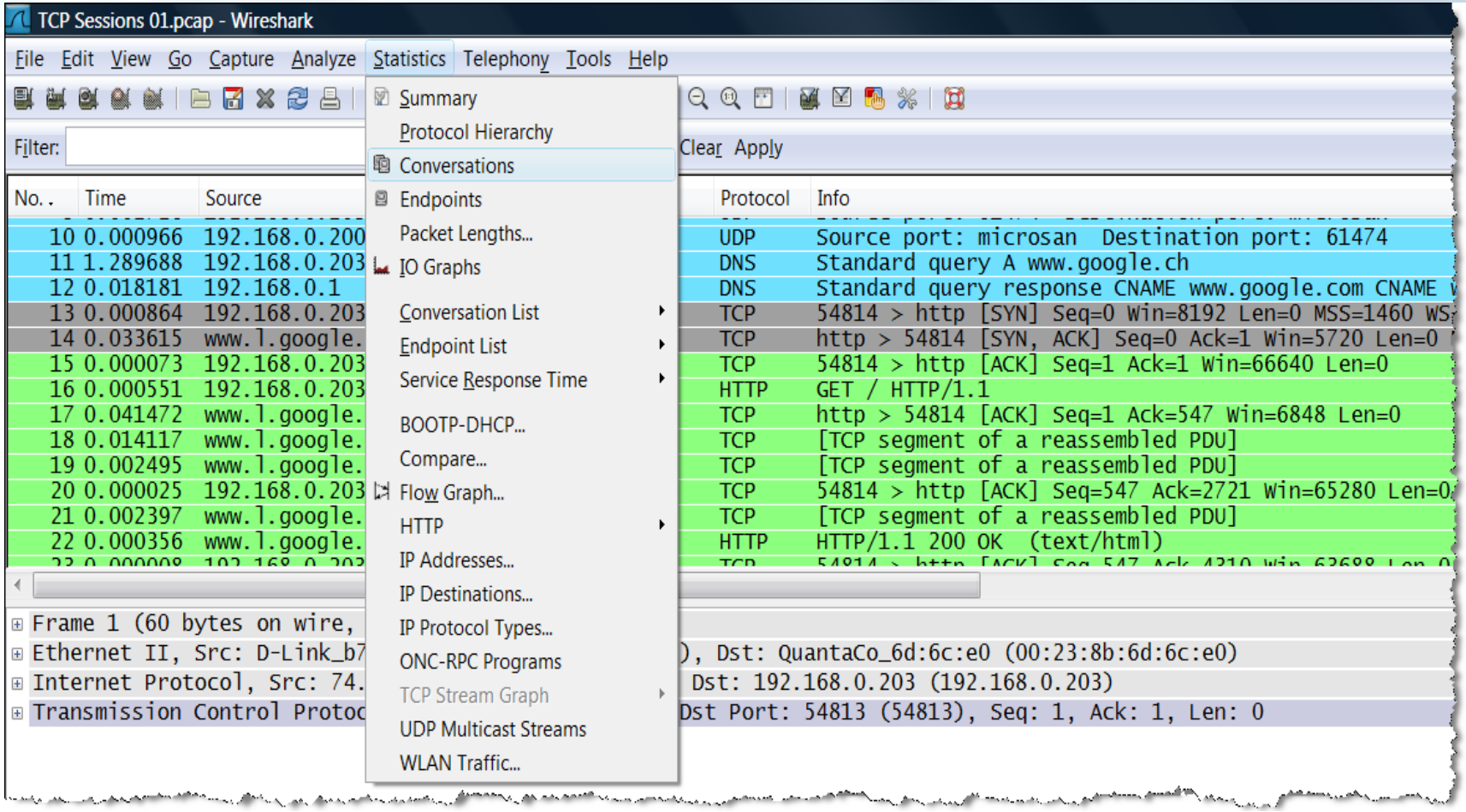
TCP 'Selective Acknowledge' Option



Trace file: TCP SACK

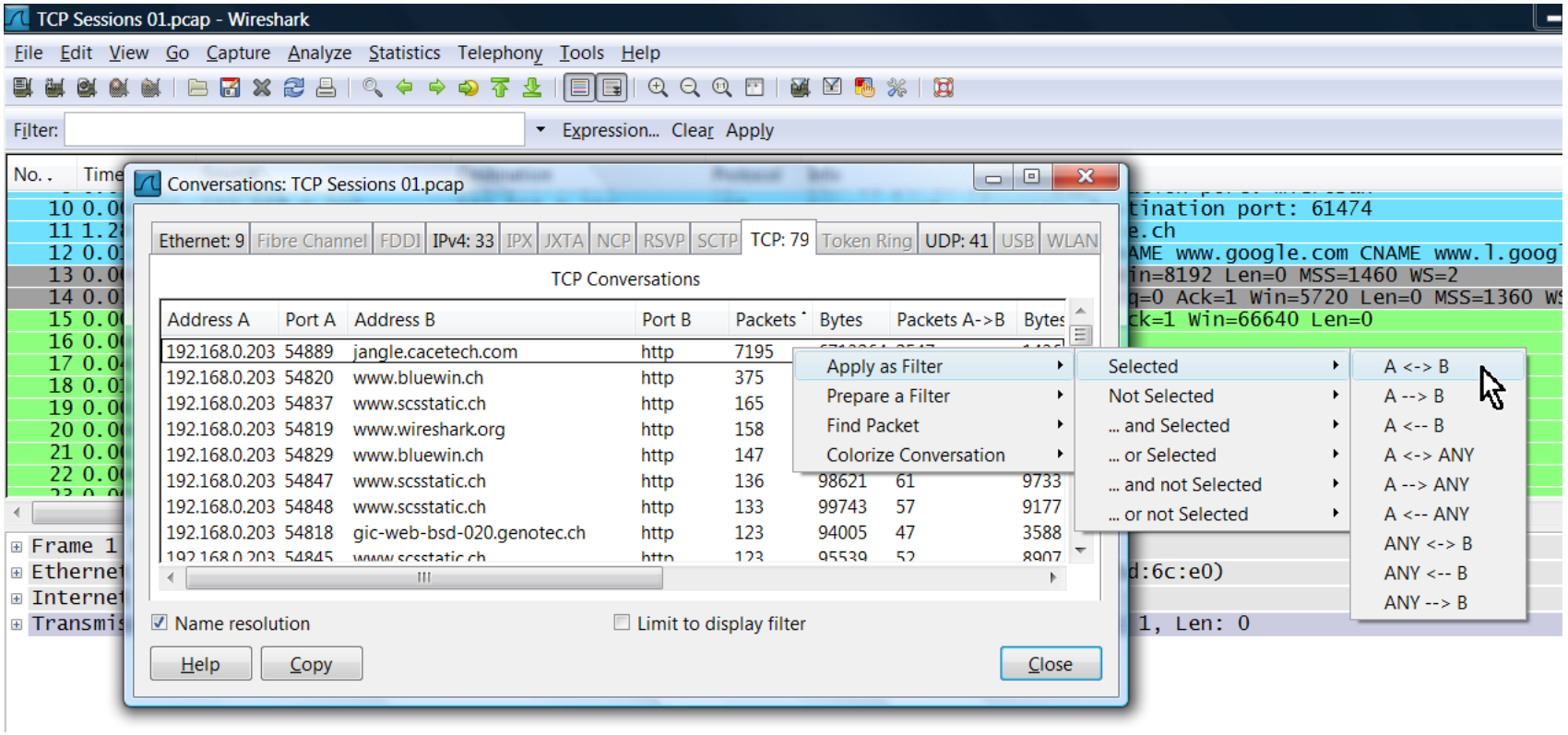
TCP Analysis using the Wireshark Expert

The Wireshark function ‚Conversations' to filter on TCP sessions



TCP Analysis using the Wireshark Expert

The Wireshark function ‚Conversations' to filter on TCP sessions

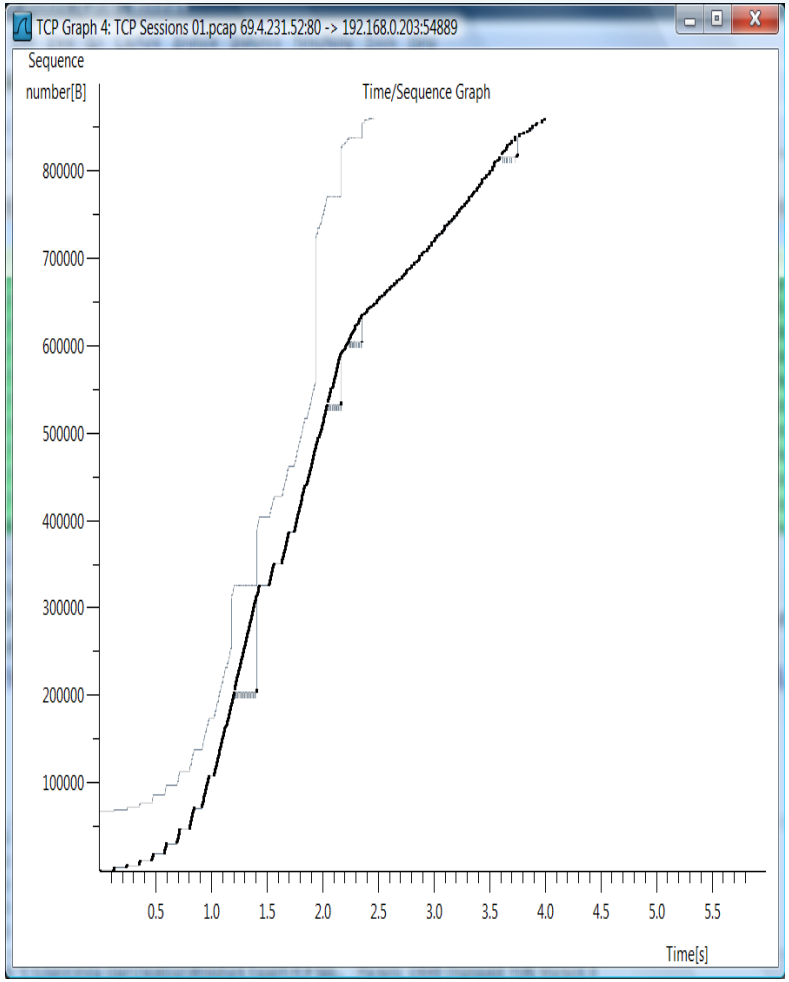


TCP Analysis using the Wireshark Expert

Exercise: Analyse the trace file ‚TCP Sessions 01.pcap‘

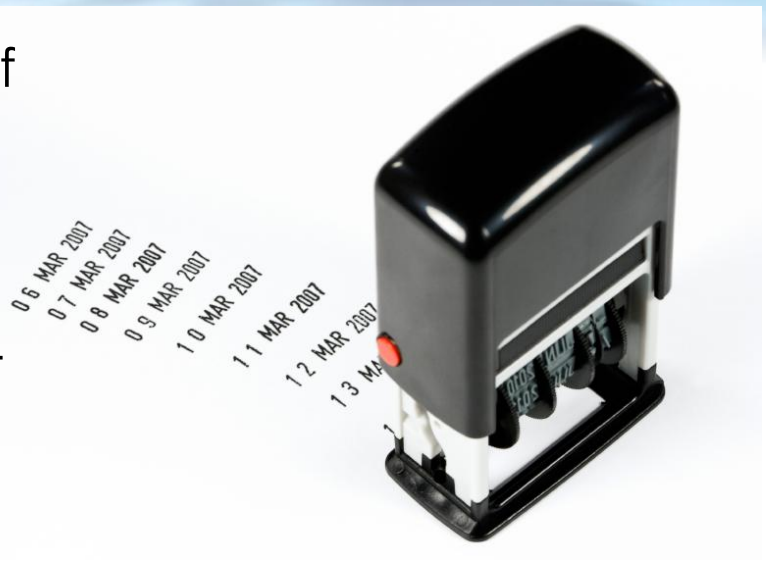
Before you start, filter out the TCP session with the highest date volume transferred.

- On which side the trace was captured?
- What is the initial ‚Window Size‘?
- Is ‚Window Scaling‘ in use?
- Remains the ‚Window Size‘ constant?
- Is ‚Selective Ack‘ in use?
- Who limits the throughput, sender or receiver?



TCP 'Timestamp' Option

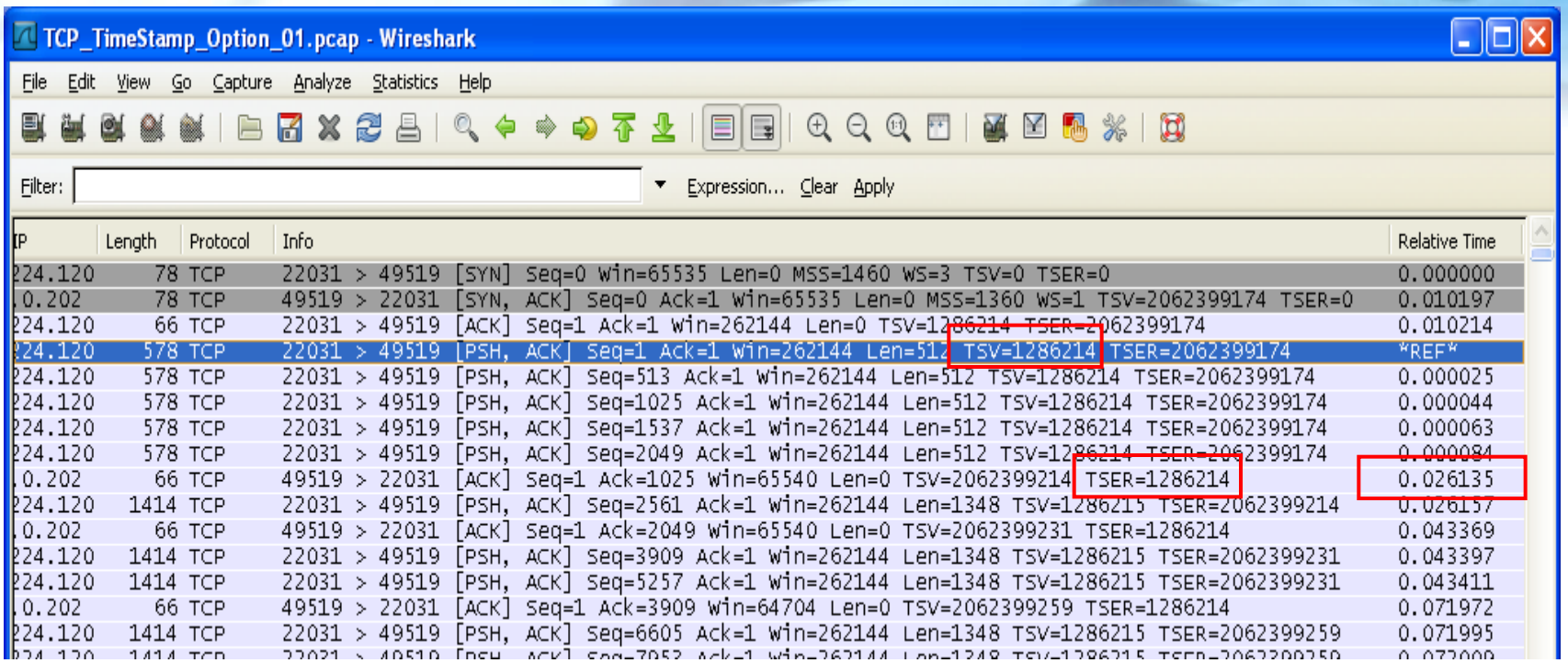
- The Timestamp option improves die round trip time (RTT) measurement of TCP.
- Without this option, RTT is measures once per window size
- For a window size of 64'240 Bytes (44 full segments with 1460 Bytes), only every 44th packet is used for RTT calculation.



- For a window size of up to 65'535 Bytes one measurement per window size was sufficient. It is not anymore with the windows scaling option and maximal window sizes up to 1 Gigabyte.
- Using the Timestamp option, every segment acknowledge pair can be used to calculate RTT.

© Leutert NetServices 2026 / TCP / V4.1f

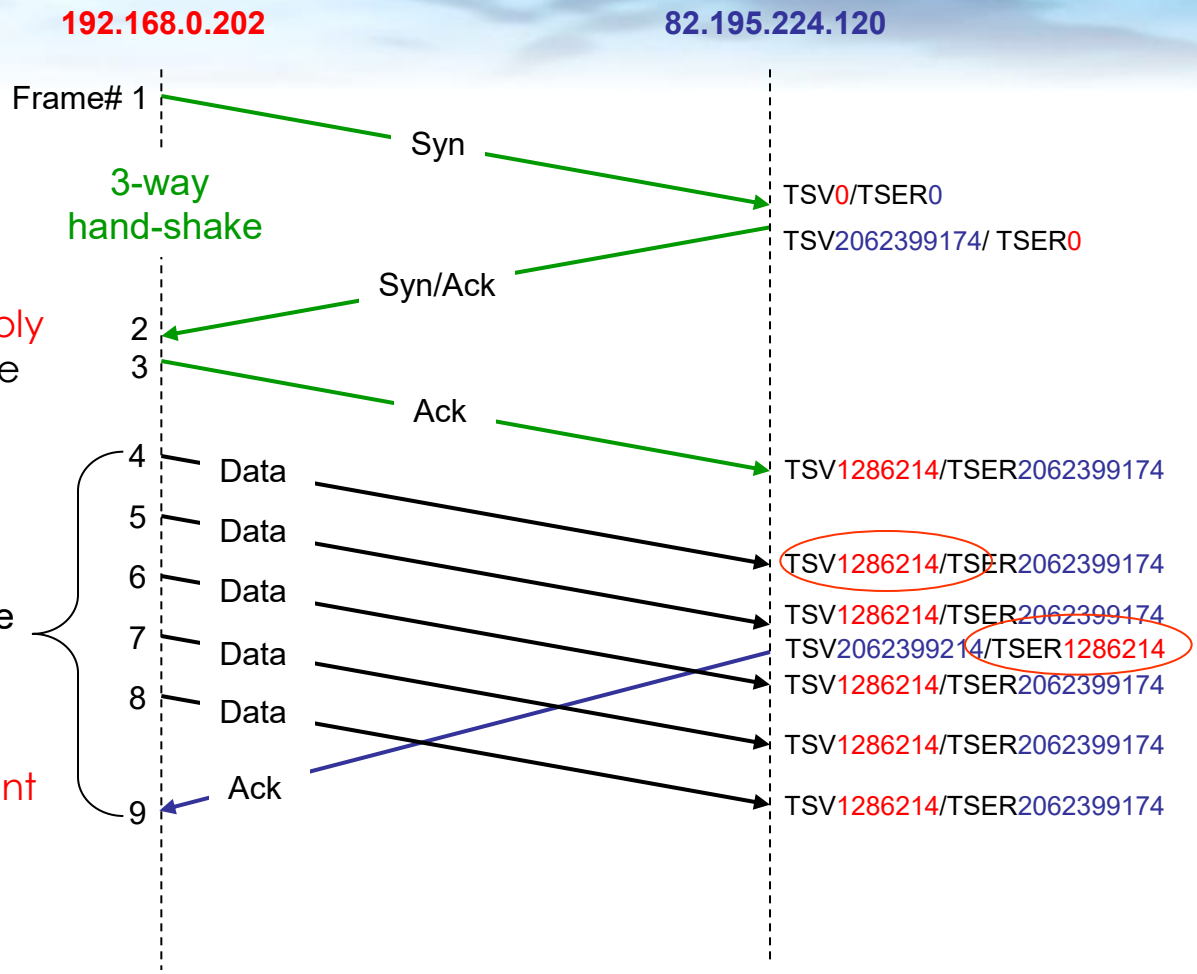
TCP 'Timestamp' Option



1. Note TSV value (1286214) in data frame #4
2. Set time reference *REF* on that frame (right mouse)
3. Search for the value (1286214) in the TSER field of first Ack frame (#9)
4. Value in time column shows Round Trip Time (RTT) (26.135 ms)

© Leutert NetServices 2026 / TCP / V4.1f

TCP 'Timestamp' Option



TSV = Time Stamp Value
TSER = Time Stamp Echo Reply
Die time interval of the value may be different from both sides, in the range from 1ms to 1sec

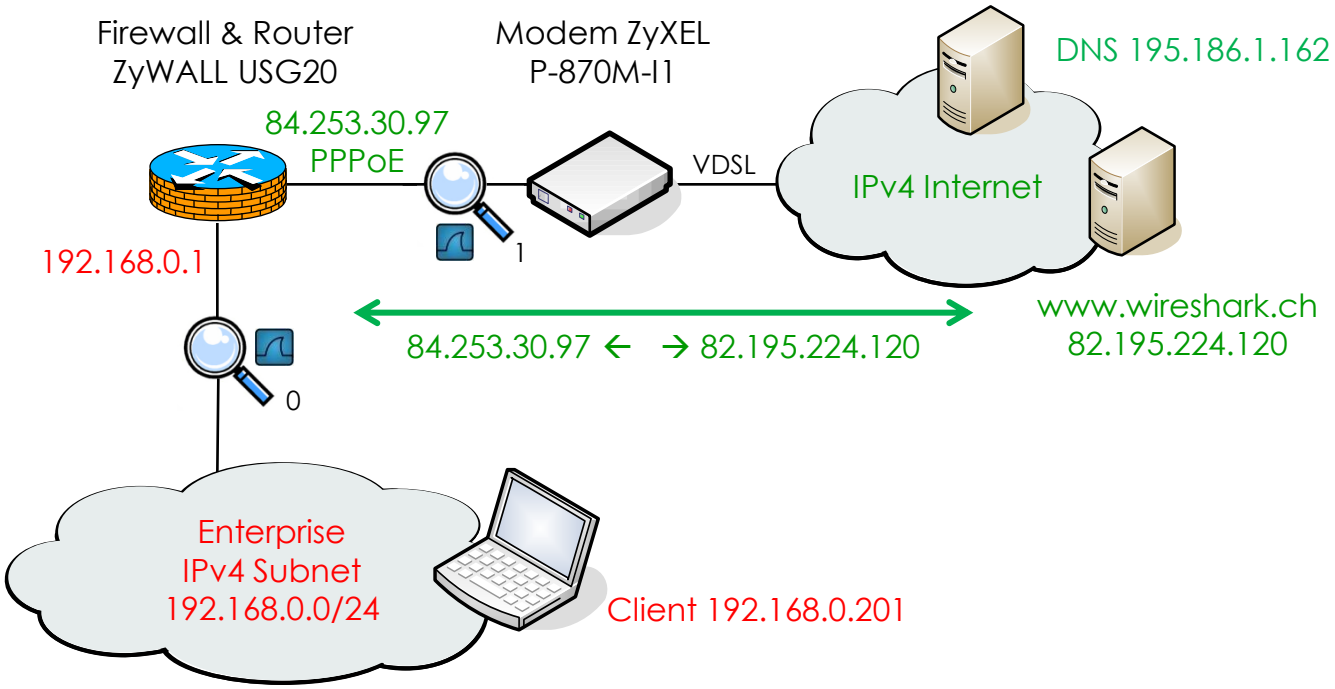
If more than one frame is sent with the same TSV value (frame 4-8), the TSER value refers to the first data frame sent with that value!

Trace file: TCP_TimeStamp_Option_01.pcap

TCP Analysis with two interfaces

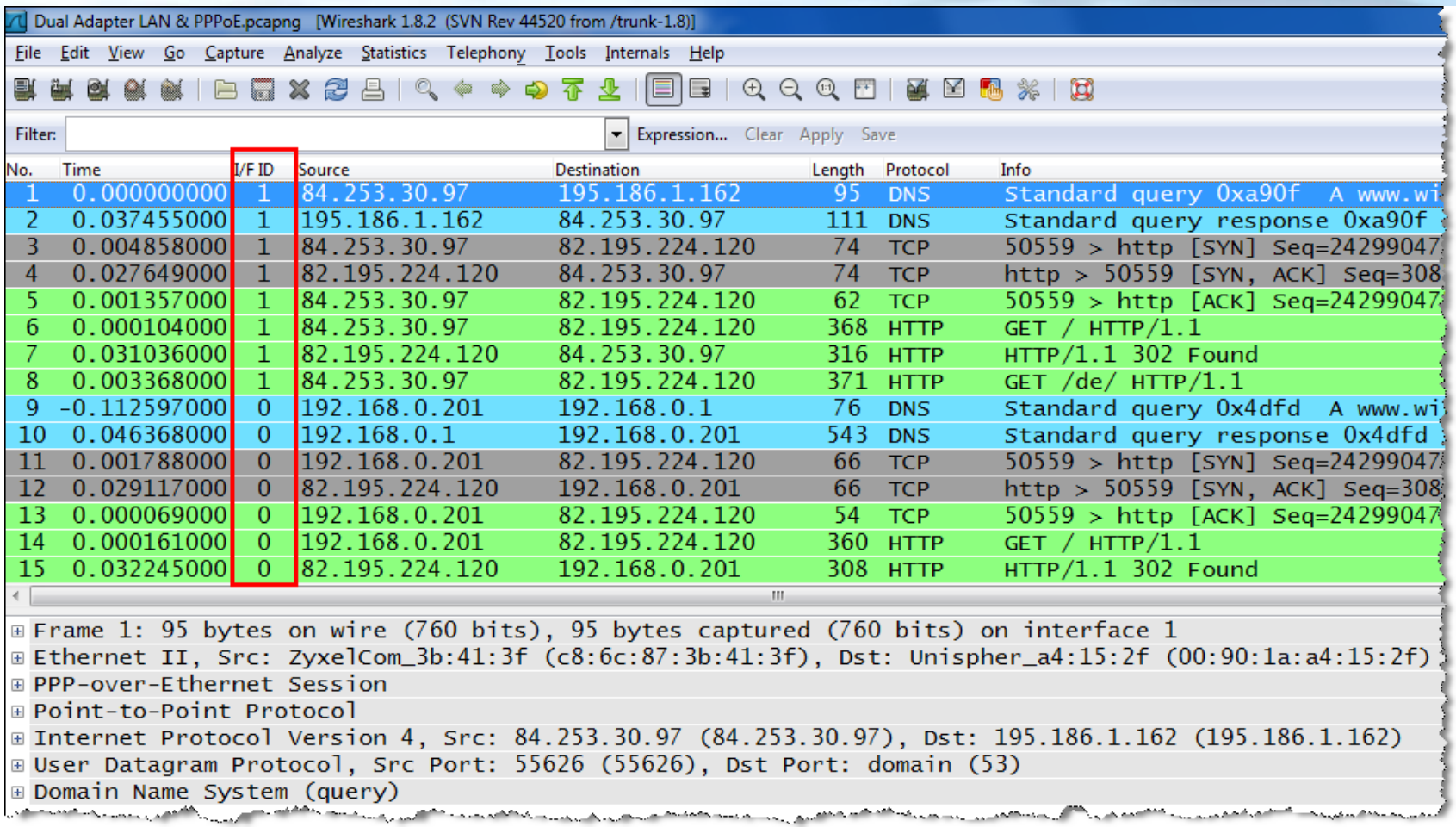
Network devices like Router, Firewalls, Packet Shaper, WAN Accelerator etc. may modify TCP/IP Parameters. Simultaneous capturing at input and output of these components disclose these changes.

Capturing Input and Output of an Internet/NAT Router:



TCP Analysis with two interfaces

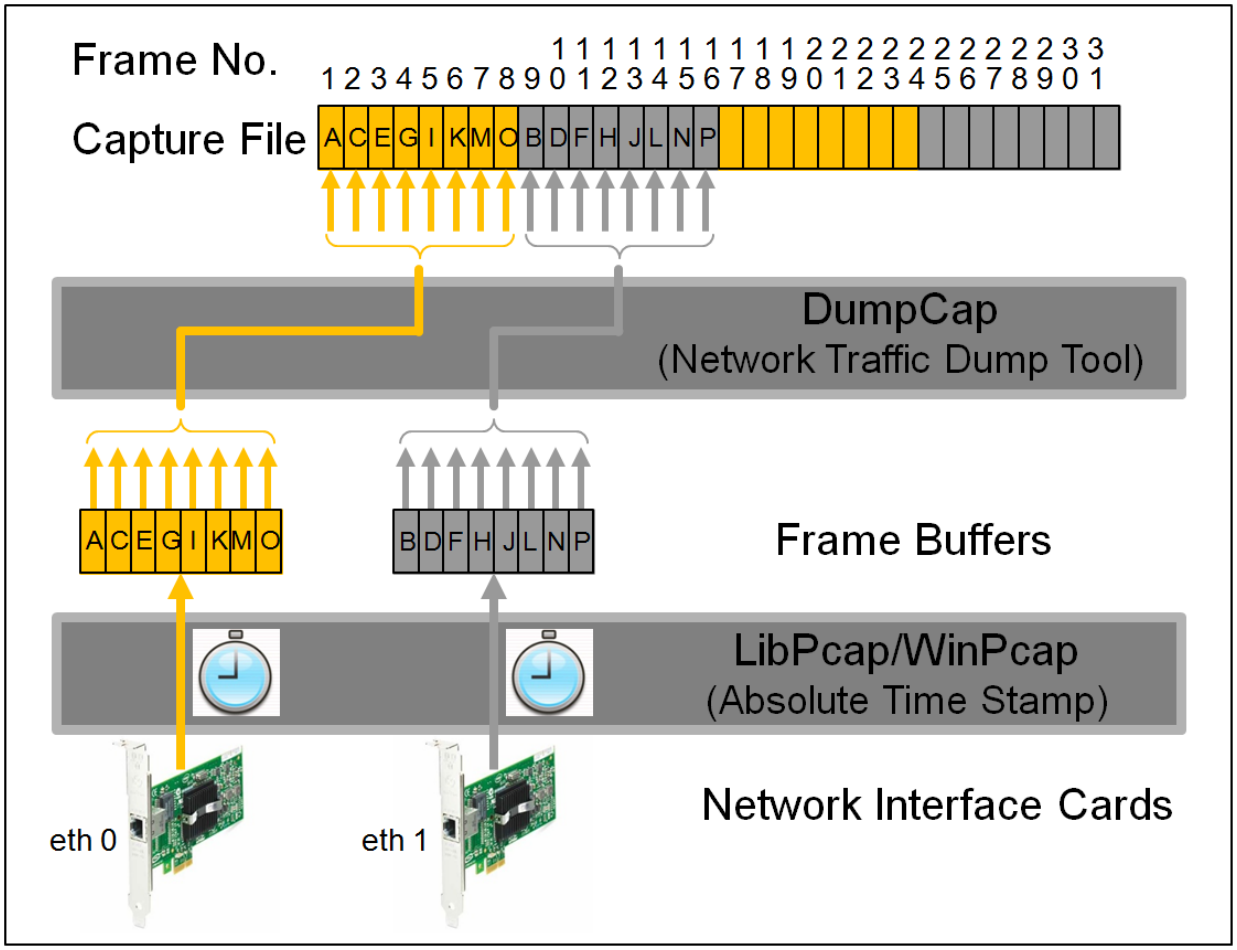
Create a new profile called 'Dual Interfaces' and insert column for I/F ID



Please note the negative delta times in the Time Column!

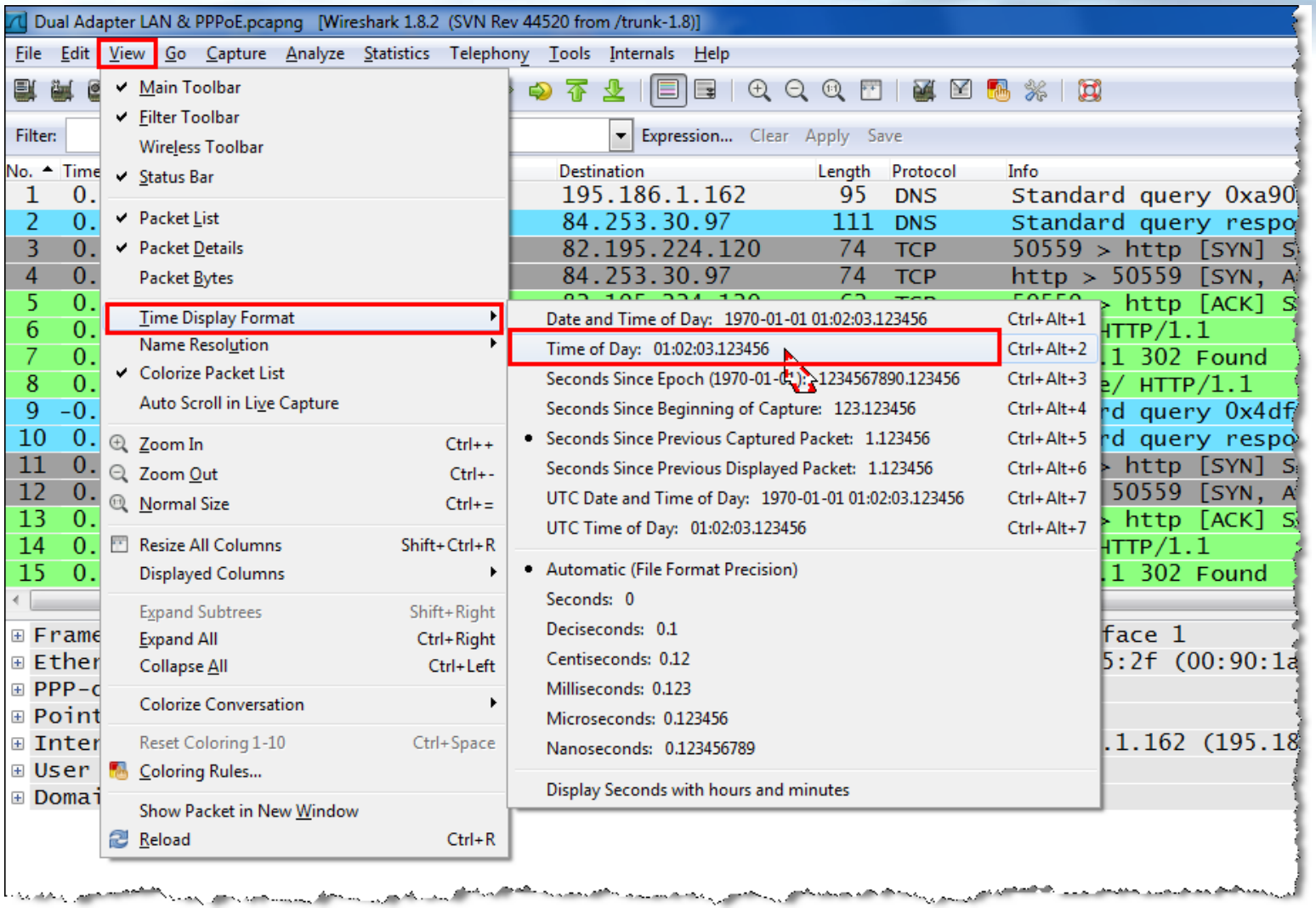
TCP Analysis with two interfaces

The frames are fetched in blocks from the interface buffers and are stored in the capture file. The order of the frame numbers does not correspond with the time stamp of the LibPcap/WinPcap driver.



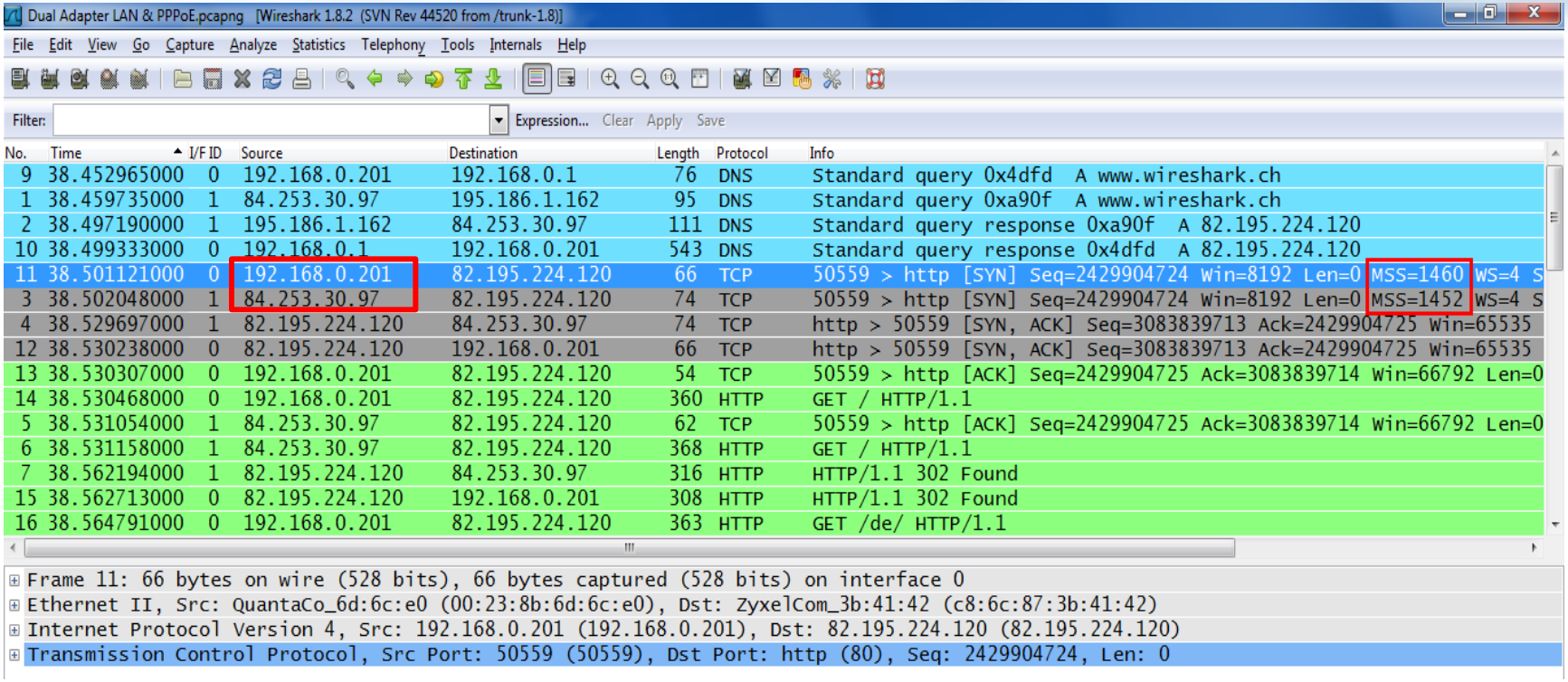
TCP Analysis with two interfaces

Change the display mode of the Time Column to Time of Day



TCP Analysis with two interfaces

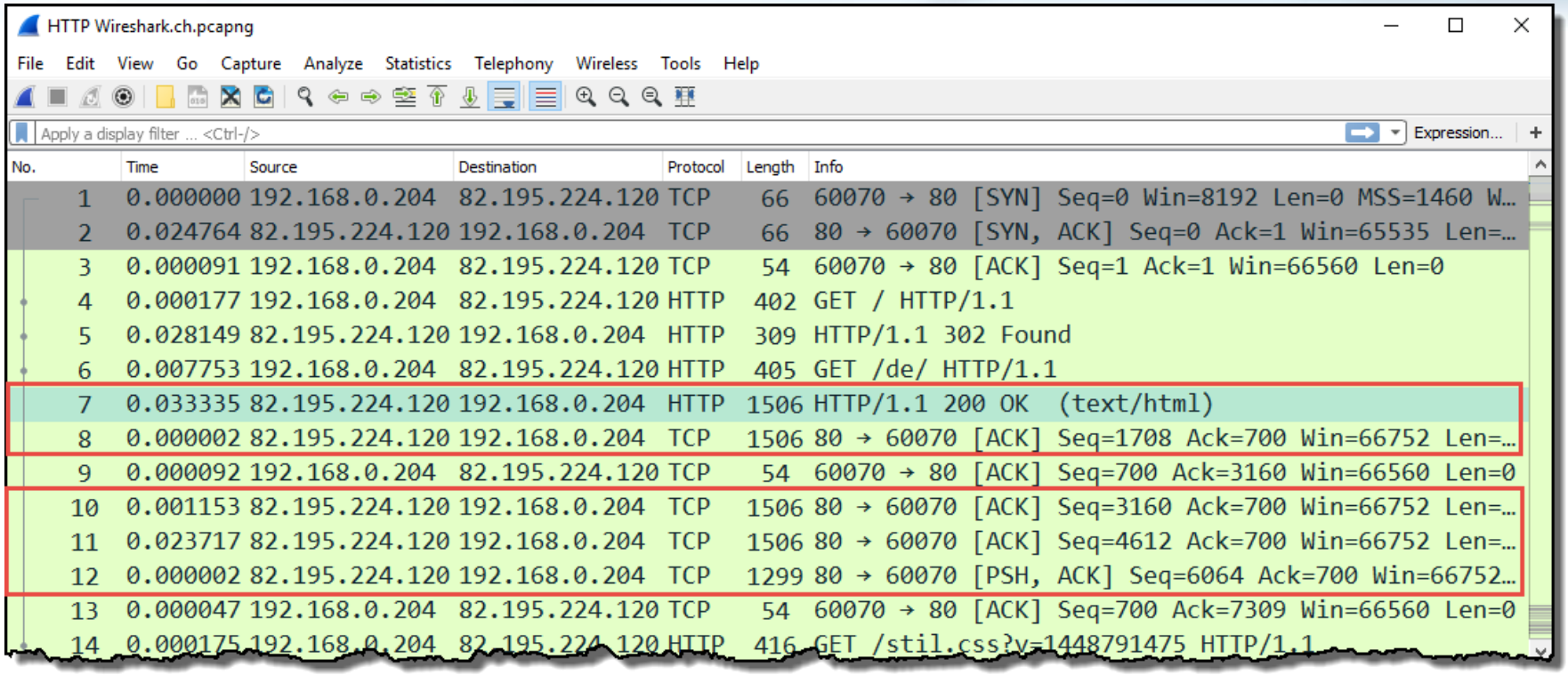
The frames ordered according to their time stamp value



- The router functions as Proxy server for DNS resolution
- The private IP source address is replaced by the public address (NAT)
- The TCP Maximum Segment Size (MSS) is reduced by 8 Byte by the router

TCP Reassembly and Export Function

Unreassembled the frames are shown as individuals

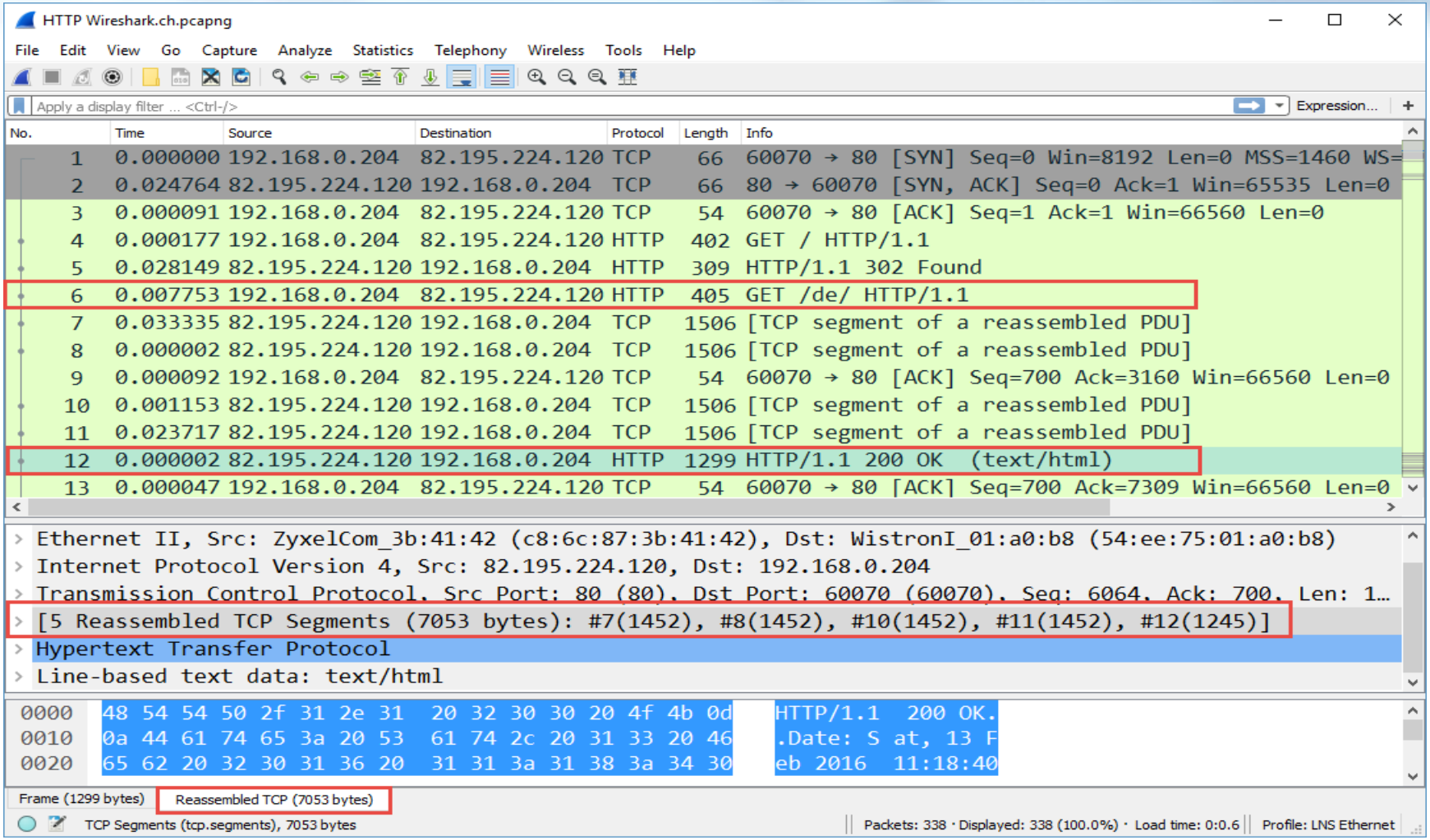


- Wireshark is able to reassemble fragments of a TCP session
- Right mouse click on TCP in Packet List or Packet Detail windows

→ Protocol Preferences → Allow subdissector to reassemble TCP streams

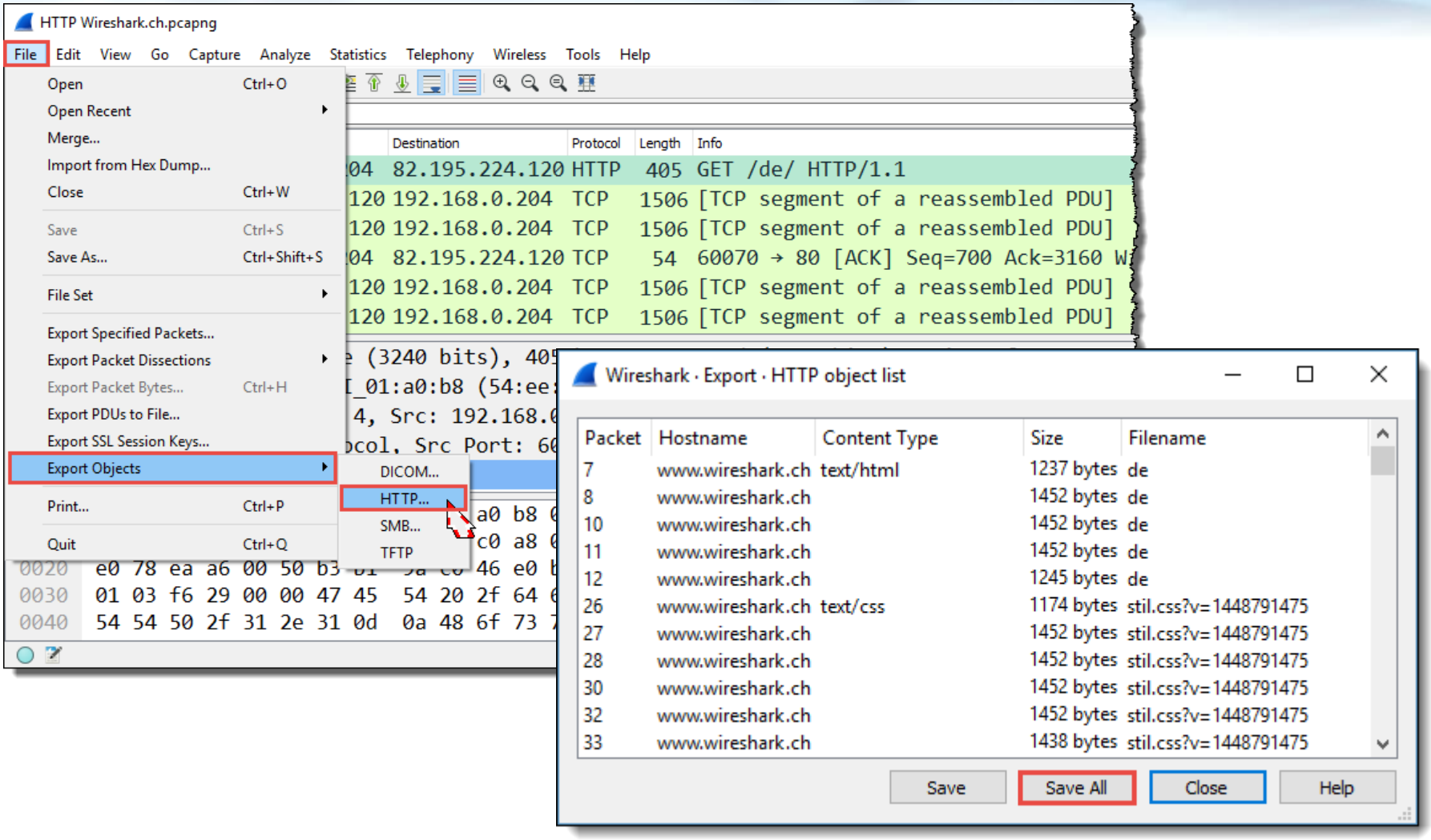
TCP Reassembly and Export Function

The reassembled the frames are shown in the last segment



TCP Reassembly and Export Function

The objects found in the trace file can be exported



Analyzing MTU problems

The Maximum Transmission Unit (MTU) is the size (in bytes) of the largest protocol data unit that the layer can pass onwards.

The MTU depends on protocols and physical layer media.



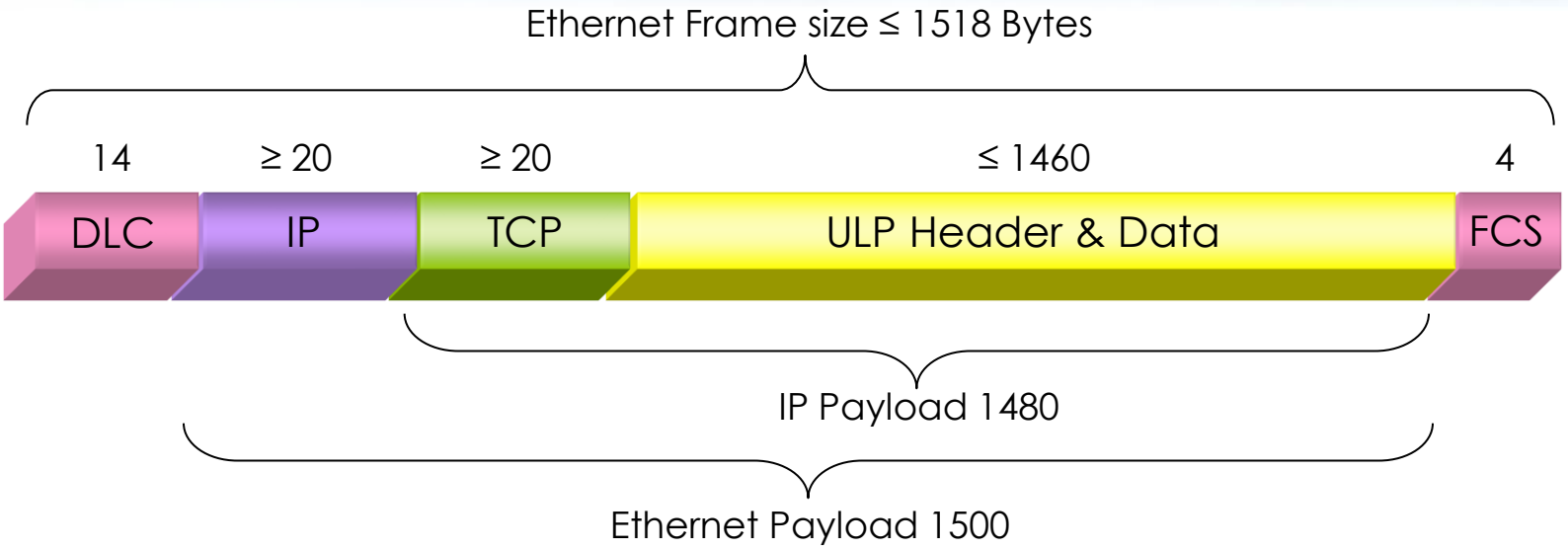
MTU problems have many different appearances and may be difficult to isolate. There are various situations possibly causing MTU problems:

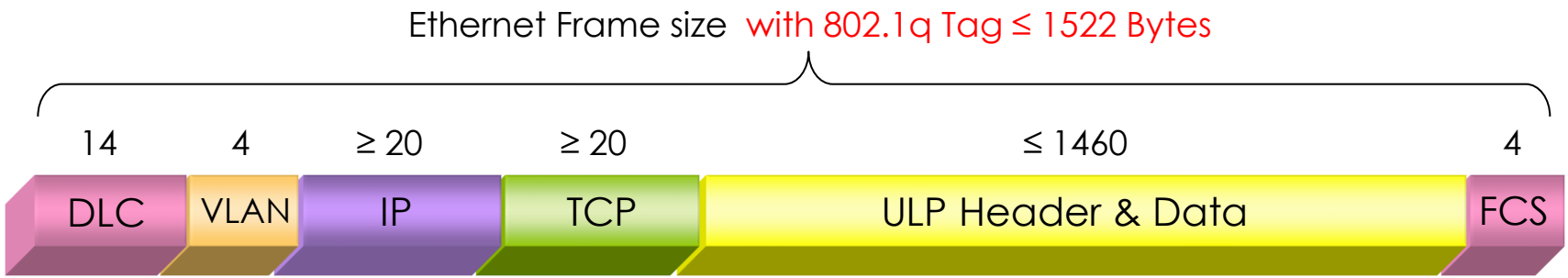
- Path-MTU Black Hole Router discarding packets with too large MTU
- Some WAN provider are using Q in Q VLAN tagging in their backbone
- Tunneling or tagging (i.e. MPLS) increase frame length
- VPN client software is reducing MTU size to reserve header space

© Leutert NetServices 2026 / TCP / V4.1f

Analyzing MTU problems

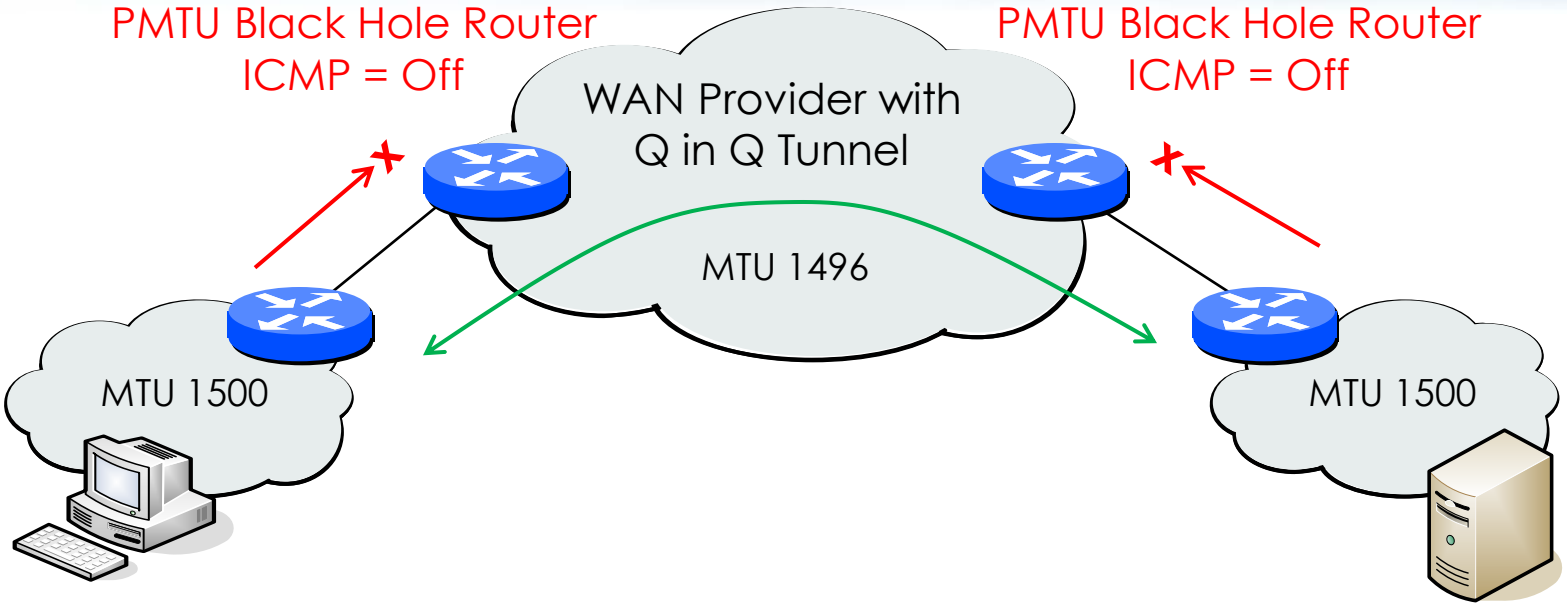
Different frame sizes:





Analyzing MTU problems

Path MTU Black Hole Router will drop frames without notice



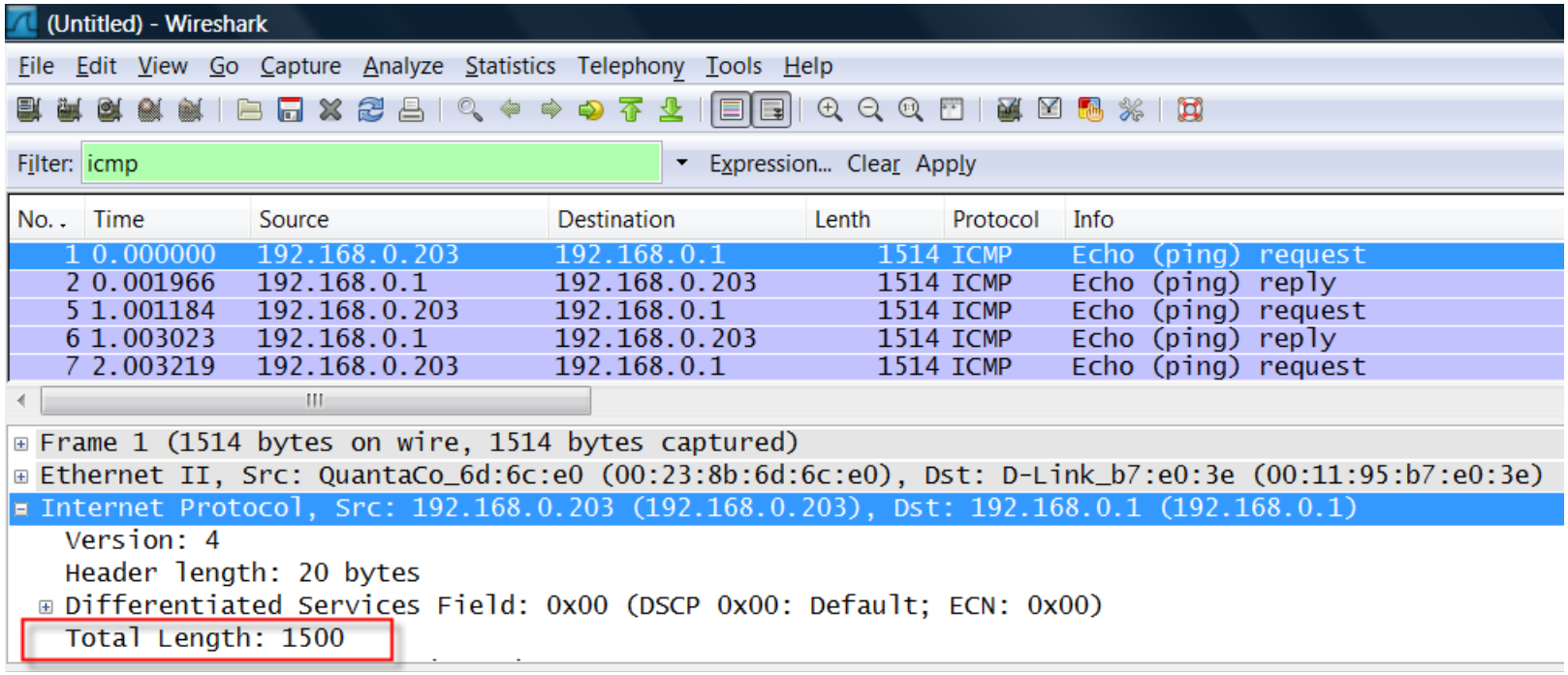
- Frames with MTU ≤1496 are forwarded
- Frames with MTU >1496 and 'Don't Fragment' Bit set will be dropped
- Router does not sent ICMP message, therefore the client will not decrease MTU
- Remedy: Path MTU discovery using ICMP Ping with options

© Leutert NetServices 2026 / TCP / V4.1f

Analyzing MTU problems

Path MTU discovery using ICMP Ping with options

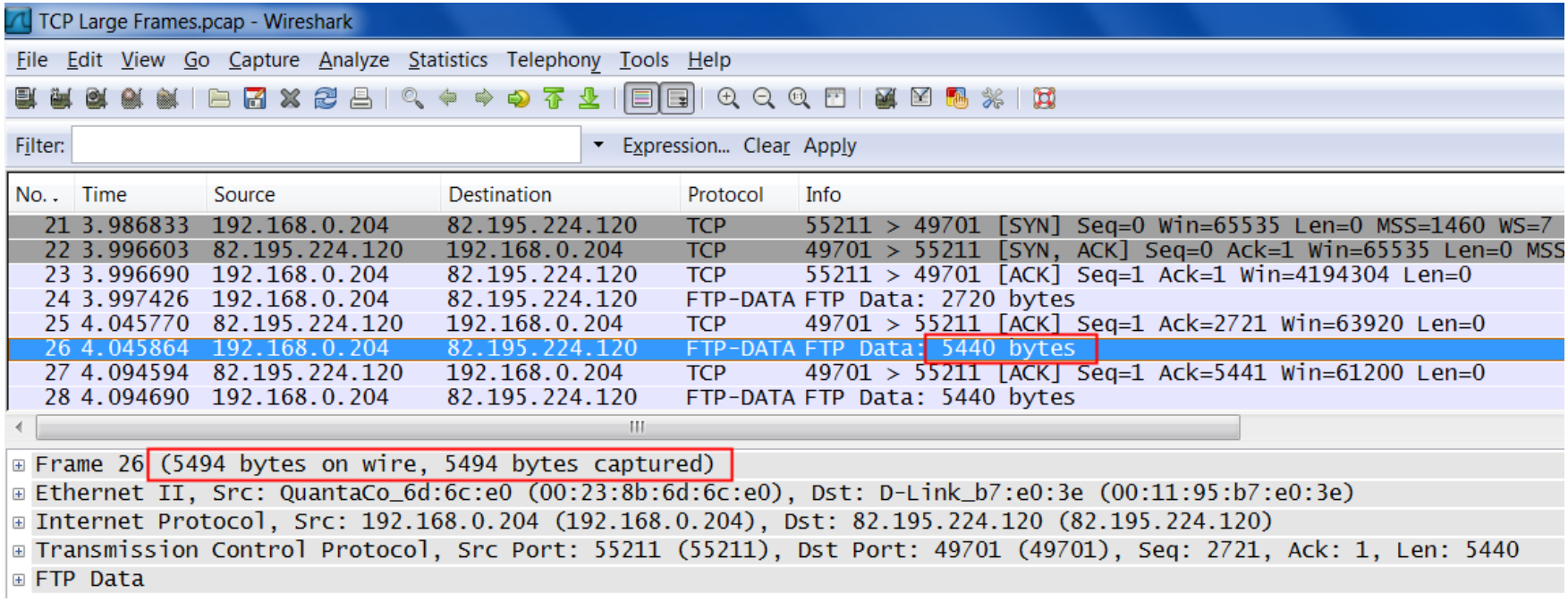
- Use Pings with -l (length) and -f (don't fragment) options
- command: ping 69.4.231.52 -l xxxx -f
- Use Wireshark to specify the maximum length value of xxxx
- Ping your default gateway and start with value -l 1500



TCP Chimney Offload

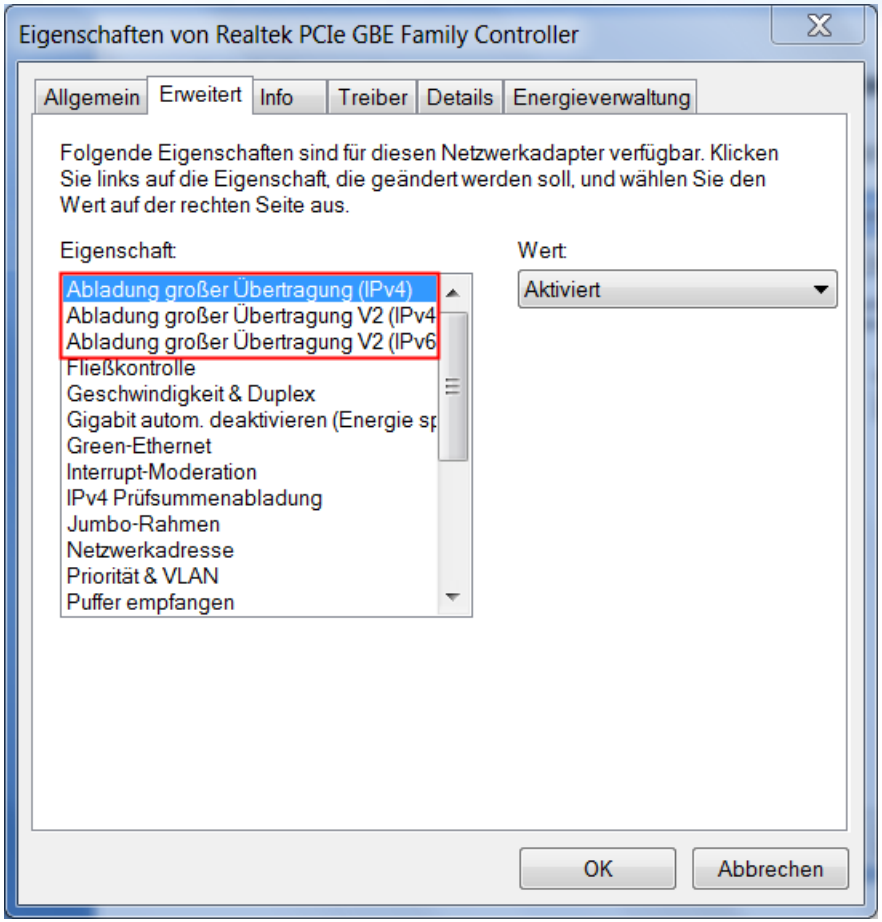
Extended TCP functions in Windows VISTA / 7 / Server 2008

- Wireshark may show TCP segment sizes larger than 1460 bytes
- Jumbo frames may reach up to 15'000 bytes), but are rarely used
- The reason more likely is a function called 'TCP Chimney Offloading'



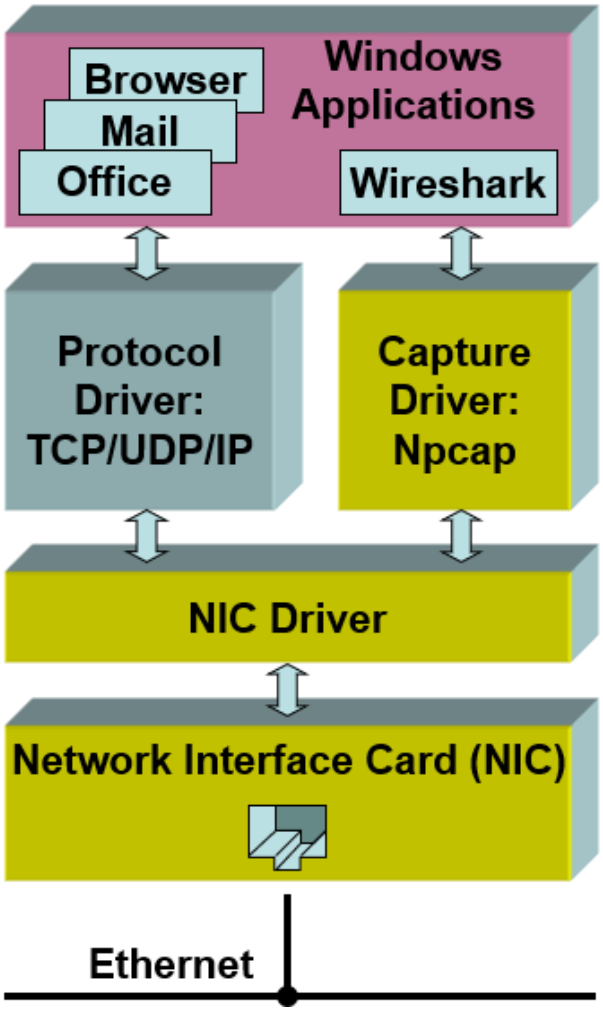
TCP Chimney Offload

Extended TCP functions in Windows VISTA / 7 / Server 2008



- The function may be named differently, depending on the driver manufacturer
- Windows VISTA / 7 supports this function for gigabit Ethernet using IPv4 or IPv6.
- The function offloads tasks from the CPU and should be enabled.
- Option may result in significant throughput improvements on servers.

TCP Chimney Offload



- The TCP driver sends bulk of data down to the driver without segmenting.
- Driver does segmentation of data according to the negotiated segment size.
- Wireshark / WinPcap receives data before they are segmented.
- There is no possibility to display segmented data in Wireshark except turning the function to off (which is not recommended).
- Does only apply for data captured directly on the sending station, not for data captured on the receiver side or from the wire.

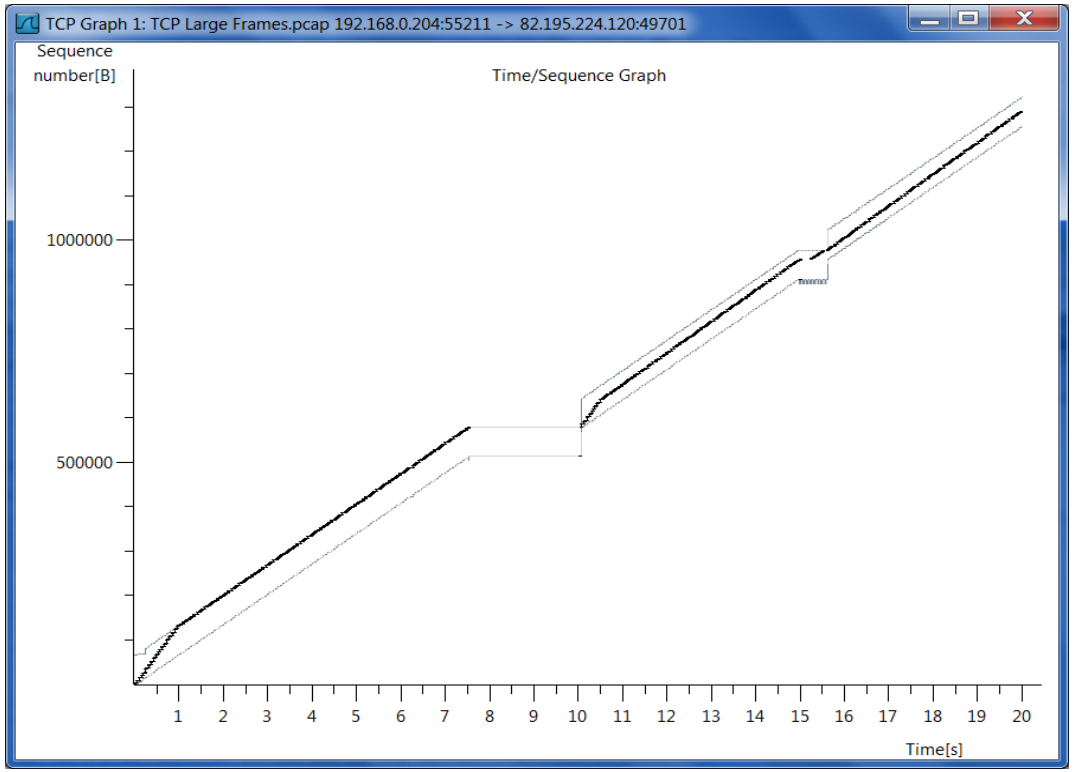
© Leutert NetServices 2026 / TCP / V4.1f

TCP Auto-Tuning in Windows

Extended TCP functions in Windows VISTA / 7 / Server 2008

Three parameters have influence on the optimal TCP throughput :

- Receive Window – size of receive buffer
- Sliding Window – remaining amount of bytes to be possible to send
- Congestion Windows – throttling of sender to avoid congestions



The TCP Auto-Tuning function adapts to the available bandwidth and tries to avoid congestions by throttling the transmission rate.

Domain Name System (DNS)

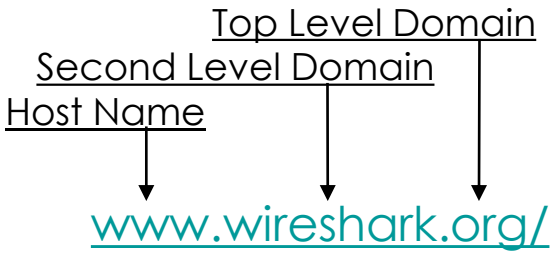
Architecture and Functions

- DNS is one of the most important service of IP-based networks
- The main task is to answer requests for name resolution
- The domain name space consists of a tree data structure with hierarchical domain levels
- The DNS levels are contained in the host name, also called Fully Qualified Domain Name



Domain name syntax:

The **Fully Qualified Domain Name (FQDN)** consist of three (or more) labels:



Other examples of DNS names:

https://ns1.com/
https://klick.hosttech.ch/webhosting/

© Leutert NetServices 2026 / TCP / V4.1f

Domain Name System (DNS)

Architecture and Functions

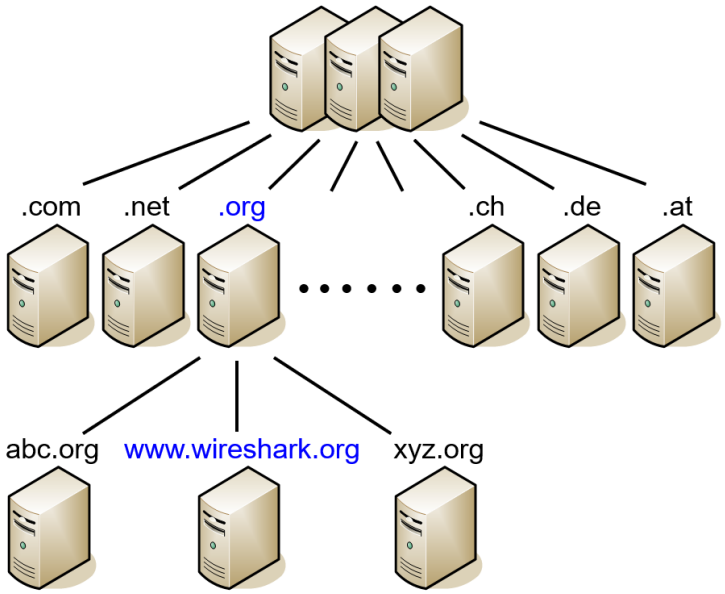
- The hierarchical DNS Architecture consist of thousands of servers worldwide
- DNS can resolve host names to IP addresses, called **Forward lookup**
- DNS can also resolve IP addresses to host Names, called **Reverse lookup**
- There are two DNS methods for name resolution: **Recursive or Iterative**
- **Recursive**: the local DNS server send a query to the next higher DNS server only
- The query is then forwarded levels up from server to server, until a match is found

DNS Server Hierarchy:

Root Level Servers
13 logical a.root-servers.net to m.root-servers.net
(thousand physical server in worldwide 130 locations)

Top Level Domains (TLD)
(actual more than 1500)

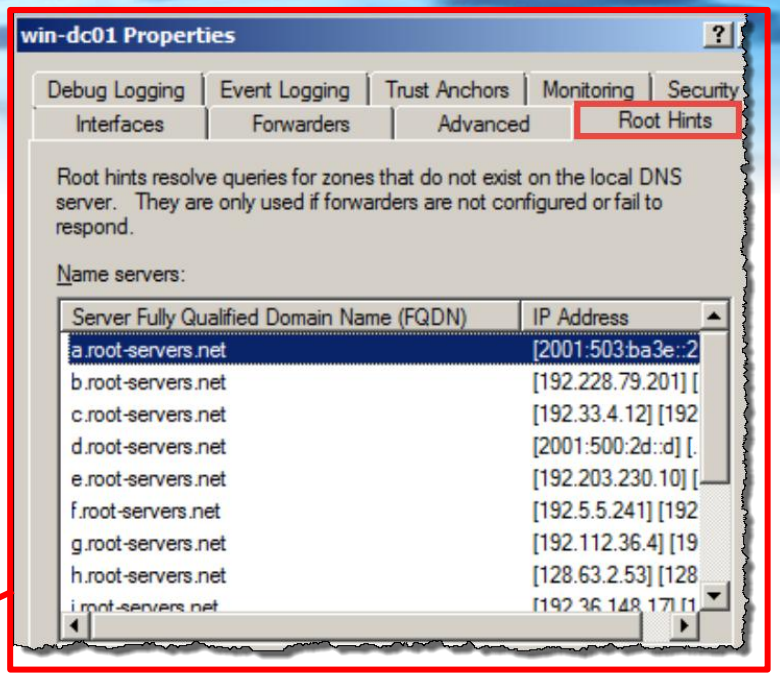
Second Level Domains (SDL)
(actual more than 350 Millions)

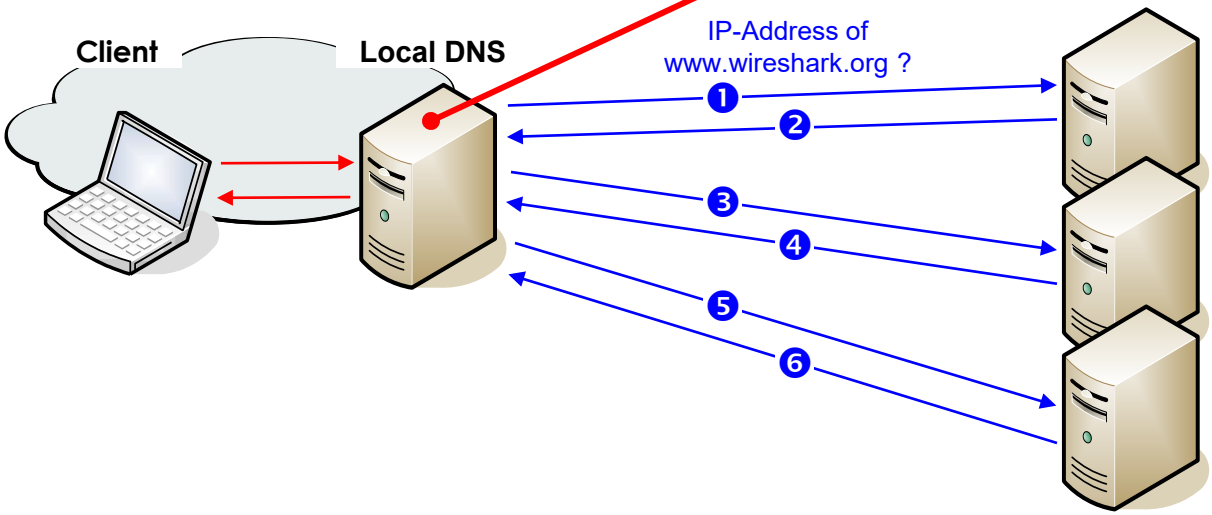


Domain Name System (DNS)

Iterative DNS Name Resolution

- With the iterative name resolution, the Local DNS server contacts all levels of DNS servers directly
- Starting from top, sending the query to a Root Server first
- The local DNS server has a list of the worldwide DNS Root Servers (Root Hints)
- Root servers are using **Anycast** addresses



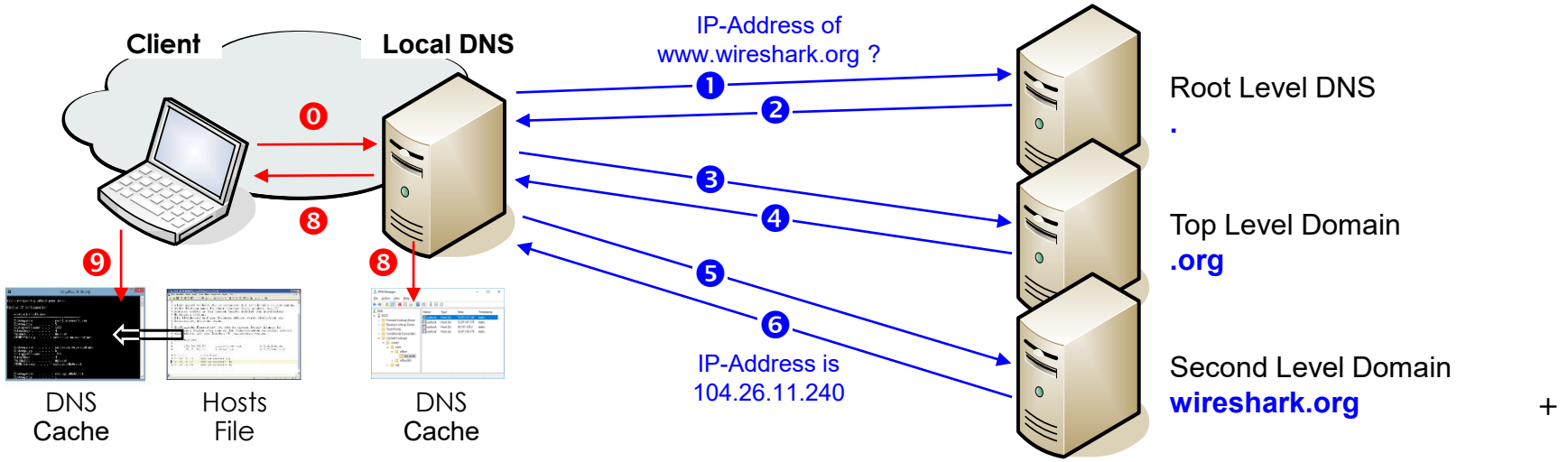


Domain Name System (DNS)

Iterative DNS Name Resolution (File DNS iterative Lookup 1.pcapng)

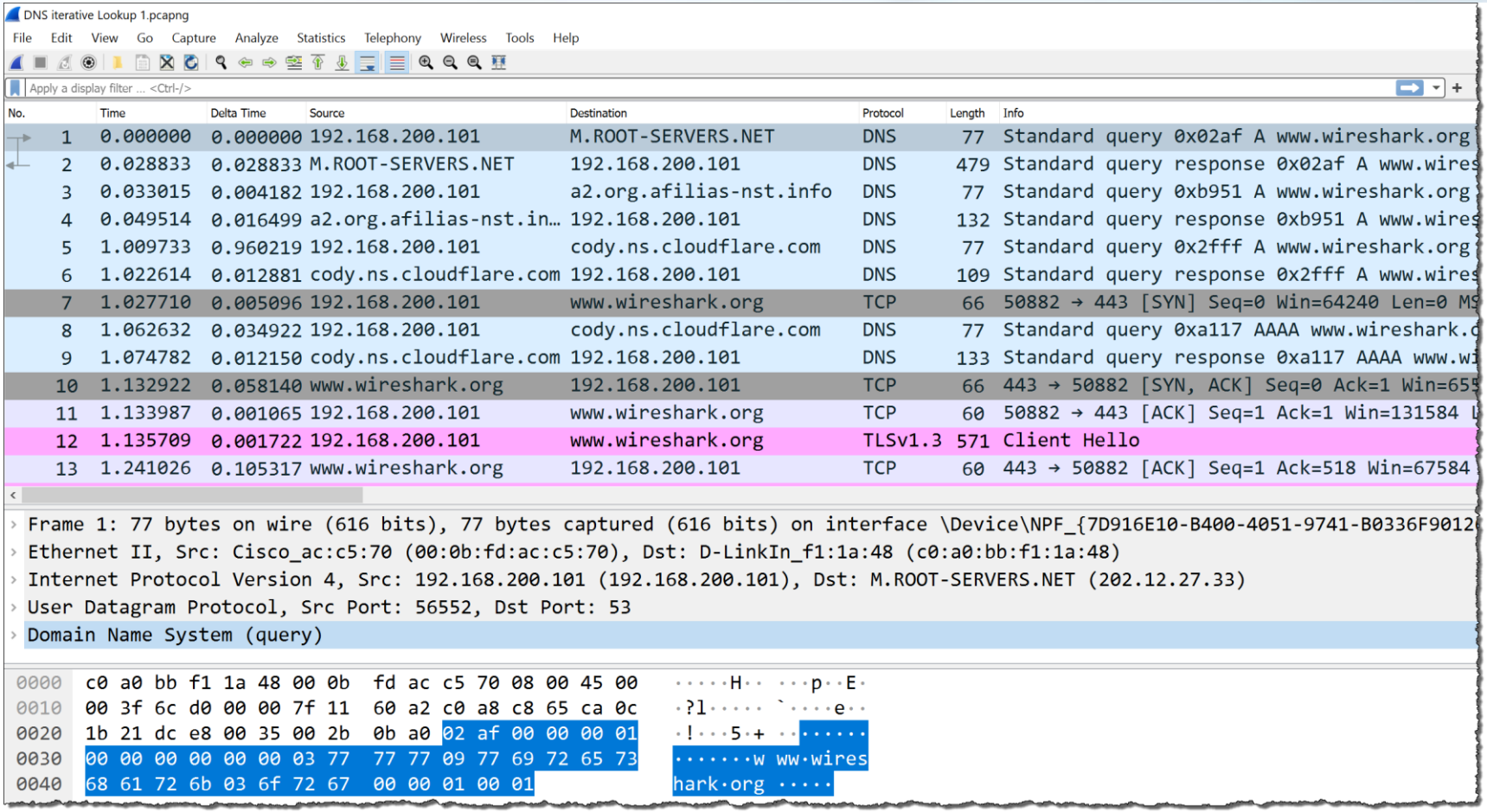
Client wants to connect with **www.wireshark.org**, no DNS match in clients DNS Cache

- ⓿ **Client** sends **DNS Query** for www.wireshark.org to **local DNS Server** (recursive lookup)
- ❶ **Local DNS Server** checks DNS-Cache, if no match found, query is forwarded to a **Root Server**
- ❷ **Root Server** replies with (multiple) IP addresses of **.org Top-Level Domain servers** (iterative lookup)
- ❸ **Local DNS Server** sends query to one of the **Top Level Domain** servers
- ❹ **Top Level Server** replies with (one or multiple) IP address of **Second Level Domain**
- ❺ **Local DNS Server** sends query to **Second Level DNS**
- ❻ **Second Level DNS** replies with IP address of www.wireshark.org server
- ❽ **Local DNS Server** writes IP address to **DNS Cache** and forwards IP address to the **Client**
- ❾ **Client** writes IP address to **DNS Cache** and establishes connection to www.wireshark.org



Domain Name System (DNS)

Iterative DNS Name Resolution



Domain Name System (DNS)

The most used DNS Record Types

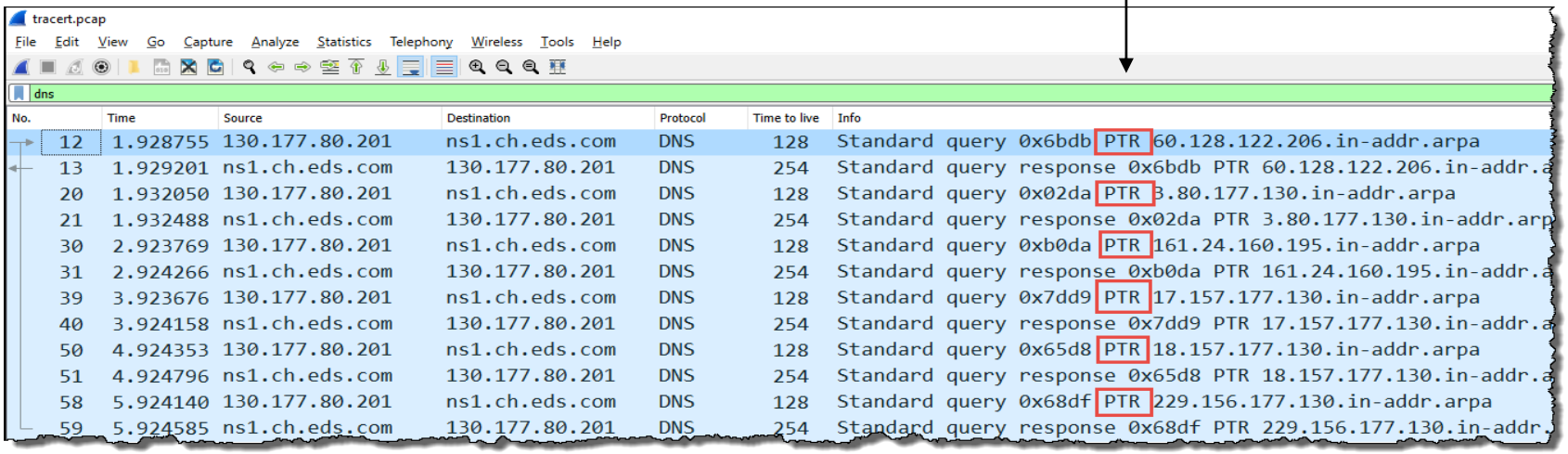
Common DNS Record Types	
Record	**Description**
A	Address record (IPv4)
AAAA	Address record (IPv6)
CNAME	Canonical Name record
MX	Mail Exchanger record
NS	Nameserver record
PTR	Pointer record
SOA	Start of Authority record
SRV	Service Location record
TXT	Text record

Forward lookup: maps FQDN into an IPv4 address
Forward lookup: maps FQDN into an IPv6 address

Reverse lookup: maps IPv4/v6 address into FQDN

Source: www.mustbegeek.com

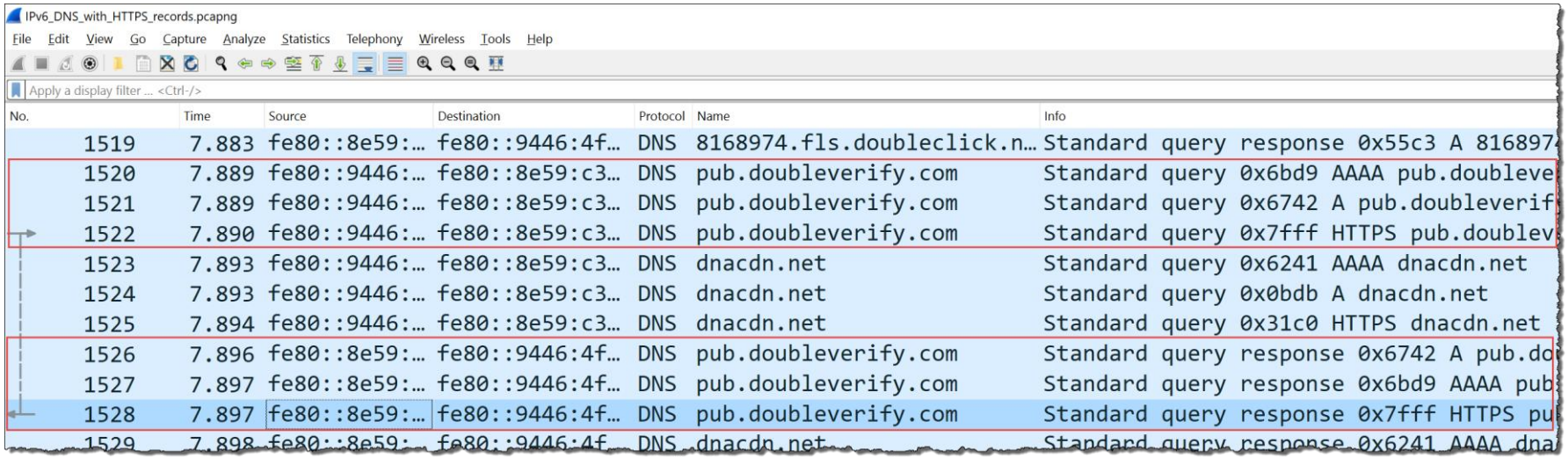
DNS query contains the record type



Domain Name System (DNS)

New DNS Record Type **HTTPS**

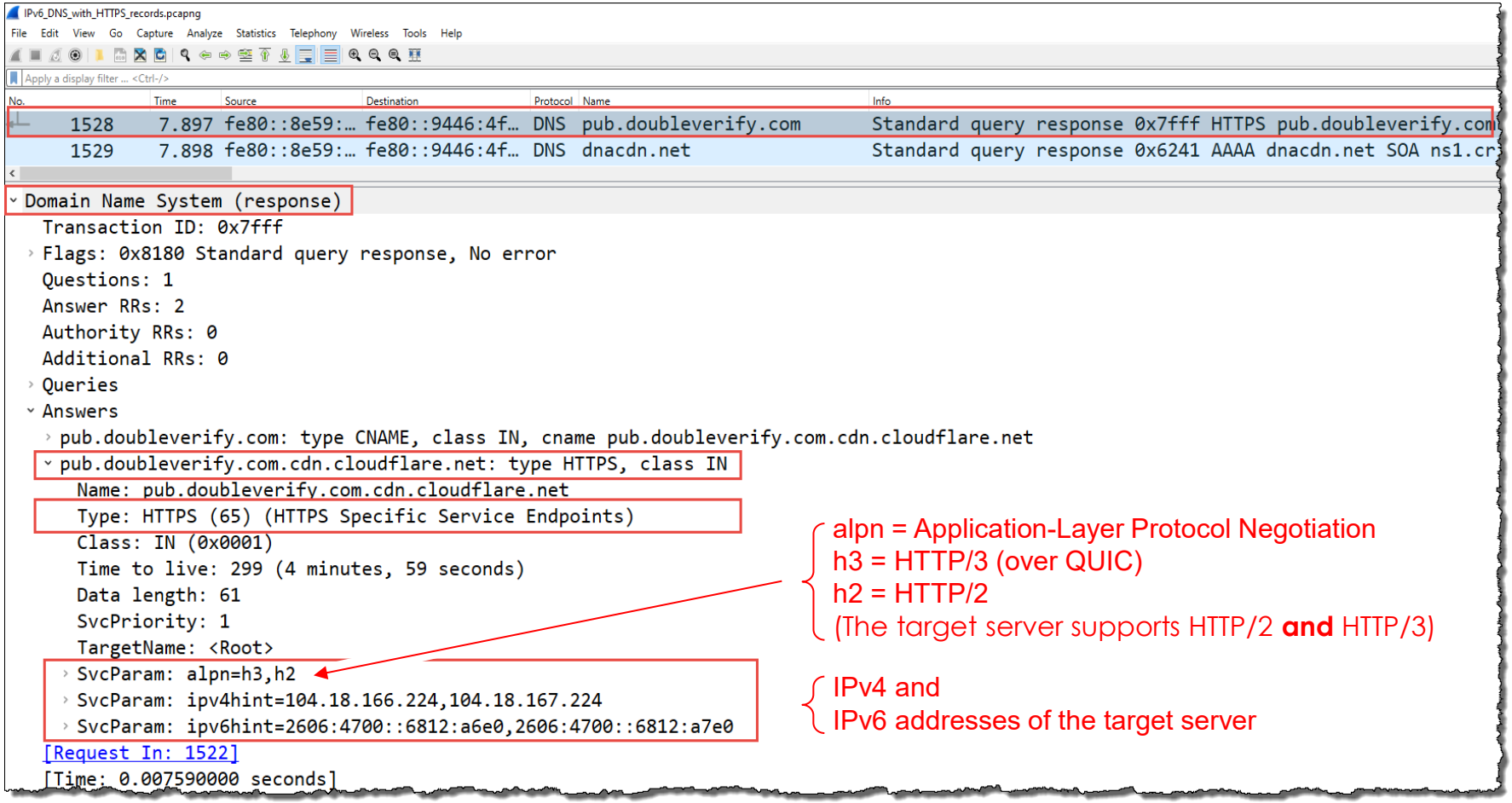
- With a **DNS A or AAAA response**, a browser receives the appropriate IP address.
- Within these records, **there is no information**, whether the target supports **HTTPS**
- The browser will first try to fetch the target server using **HTTP**
- Only if the website **redirects** to an **HTTPS URL**, the browser will then fetch the resource again over a secure connection, causing an additional delay.
- The new DNS resource record **HTTPS** is more flexible and can provide **IPv4/IPv6 addresses** as well as supported **protocol types H2/H3** (HTTP2/HTTP3) and more.
- cloudflare.com the leading DNS (IP 1.1.1.1) already **supports the HTTPS record**



Domain Name System (DNS)

New DNS Record Type **HTTPS**

- The HTTPS record can contain **several pieces of information** about the target server. Therefore, **only one DNS query** is required to get all the information.

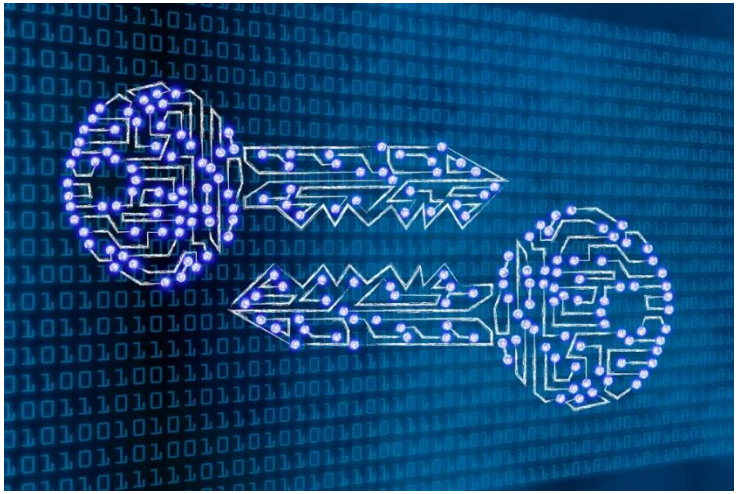


Transport Layer Security (TLS)

Introduction to Encryption / Decryption and Authentication

- TLS is a cryptographic protocol that provides end-to-end security of data sent between applications over the Internet
- TLS is the standard used by HTTPS, E-Mail, Secure Shell (SSH) and many more applications
- TLS does not secure data on end systems, it ensures secure delivery of data over the Internet
- TLS uses a combination of **symmetric and asymmetric cryptography**; this provides a good compromise between performance and security when transmitting data securely

- TLS is also used to validate a server's identity
- Public servers own a personal **Digital Certificate**
- These are issued by a **Certificate Authority (CA**)
- A server provider buys a certificate from a CA
- The certificate consist of a **Private & Public key**
- Each key pair is worldwide unique
- The Private key must be kept secret
- The public key can be distributed to anybody
- The client contacts the CA to verify validity



- This worldwide system is known as ITU-T X.509 standard for **Public Key Infrastructure (PKI)**

Transport Layer Security (TLS)

Introduction to Encryption / Decryption and Authentication

Data Encryption Techniques

- Symmetric: the same key is used for encryption as well as decryption (private-key cryptography)
- Asymmetric: a key is used for encryption, a related key for decryption (public-key cryptography)
- Hybrid: a combination of both methods (used i.e., by TLS)

- **Symmetric Encryption** (lower CPU consumption)
- Client and server must negotiate (or be provided with) the same **Pre-shared Key (PSK)**
- The PSK is used for encryption and decryption of transmitted data

- **Asymmetric Encryption** (higher CPU consumption)
- Two different, but mathematically related, keys are used (Public and Private Key)
- The Public Key is used for encryption, the Private Key for decryption
- A client is using the Public Key from the server to encrypt the data sent to the server
- The server owning the Private Key only can decrypt the received data

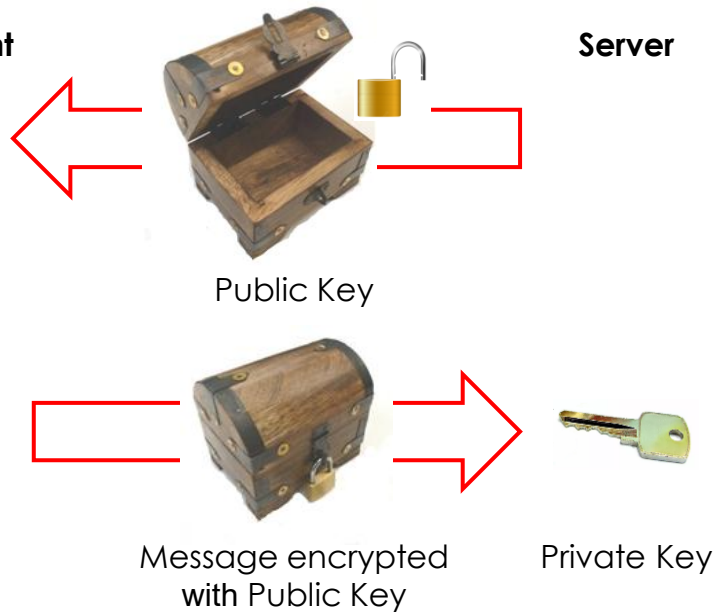
- **Hybrid Method** (used by TLS)
- Asymmetric method is used only initially for authentication and to negotiate a symmetric key
- Symmetric method is used for encryption & decryption during data transfer

Transport Layer Security (TLS)

The relationship between Public and Private key

- The secret with asymmetric encryption lies in the relation between public and private key
- The so-called **KEYS** are very large, random **Prime Numbers** (recommended > 1'024 bits)

- Servers have a Public and a Private key
- The Public Key can be sent to anyone
- The client encrypts the message with the Private key from the server and sends it to the server
- Only the owner of the Private Key (the Server) can decrypt the message
- This is a one-direction encryption only
- It is used to negotiate a symmetric Pre-shared key (PSK) between client and server



- There are many different algorithms to create a public/private key pairs, most popular are:
- RSA (Rivest–Shamir–Adleman), DSA (Digital Signature Algorithm), Elliptic Curve Cryptography
- TLS 1.3 is using Elliptic Curve Diffie-Hellman Ephemeral (ECDHE) with Courve25519

© Leutert NetServices 2026 / TCP / V4.1f

Transport Layer Security (TLS)

Introduction to Encryption / Decryption and Authentication

Key Exchange, Key Negotiation

- The client uses the public key from to server to send a random number (called Master Secret)
- Both sides are using the Master Secret then to calculate a common (symmetric) Session Key
- The session key is calculated using multiple parameter, but will never be transmitted
- For each TCP session, a unique so-called session key is used to encrypt data

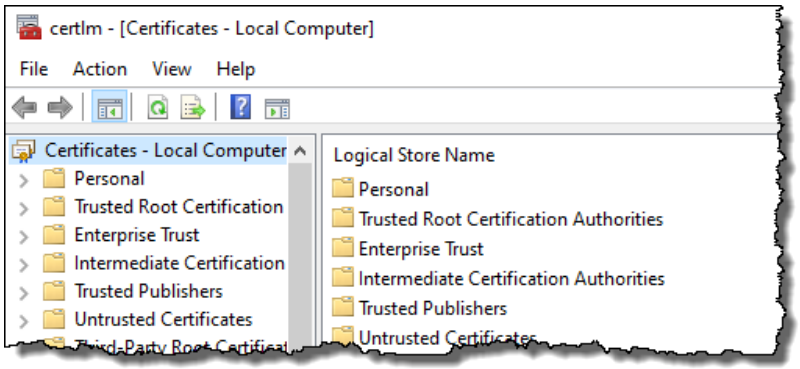
Authentication



- At begin of a connection, the server sends his Digital Certificate to the client
- The client verifies the validity of the certificate using the local certificates list
- If the certificate is invalid or can not be validated, the connection is refused
- The client's operation system updates the list regularly (Windows once a week)

To display the local list of certificates with Windows:

→ Run: certlm.msc



Transport Layer Security (TLS)

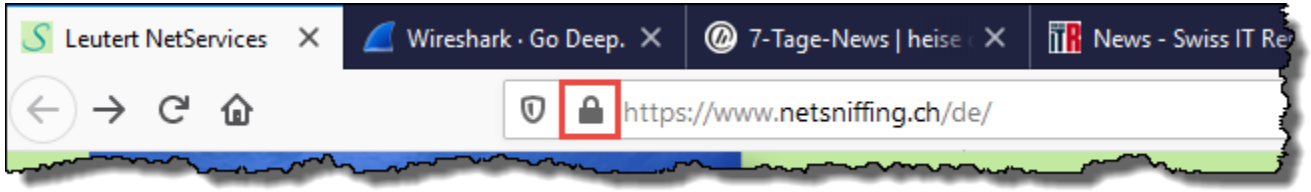
Introduction to Encryption / Decryption and Authentication

Cypher Suites

- A cipher suite is a **set of algorithms** that help secure a network connection
- In the TLS Client Hello, the client offers all by the client supported Cypher Suites
- In the TLS Server Hello, the server replies with the selected Cypher Suite

Encrypted Communication and Client Authentication

- After the successful TLS-handshake, the encrypted TCP connection is established
- The Lock Icon in the URL bar confirms the encrypted communication with the verified server
- Clicking on the Lock Icon displays encryption type and many more information



- On the Internet, normally only the server is authenticated with TLS, using the server certificate
- The client (user) is authenticated after the TLS process, using encrypted communication
- The applications are using login-passwords and optional Two-Factor Authentication (2FA)

© Leutert NetServices 2026 / TCP / V4.1f

Transport Layer Security (TLS)

Summary of negotiation processes

Key Exchange Method

- The way a Symmetric Key is negotiated between server and client

Authentication Method

- The way the identity of a counterpart of a communication is verified
- The decision if both sides, server and client, or the server only will be verified

Encryption Algorithm

- Which type of encryption will be utilized by using the negotiated Symmetric key

Integrity Hash Algorithm:

- Which Message Authentication Code (MAC) will be applied to prevent manipulation

---

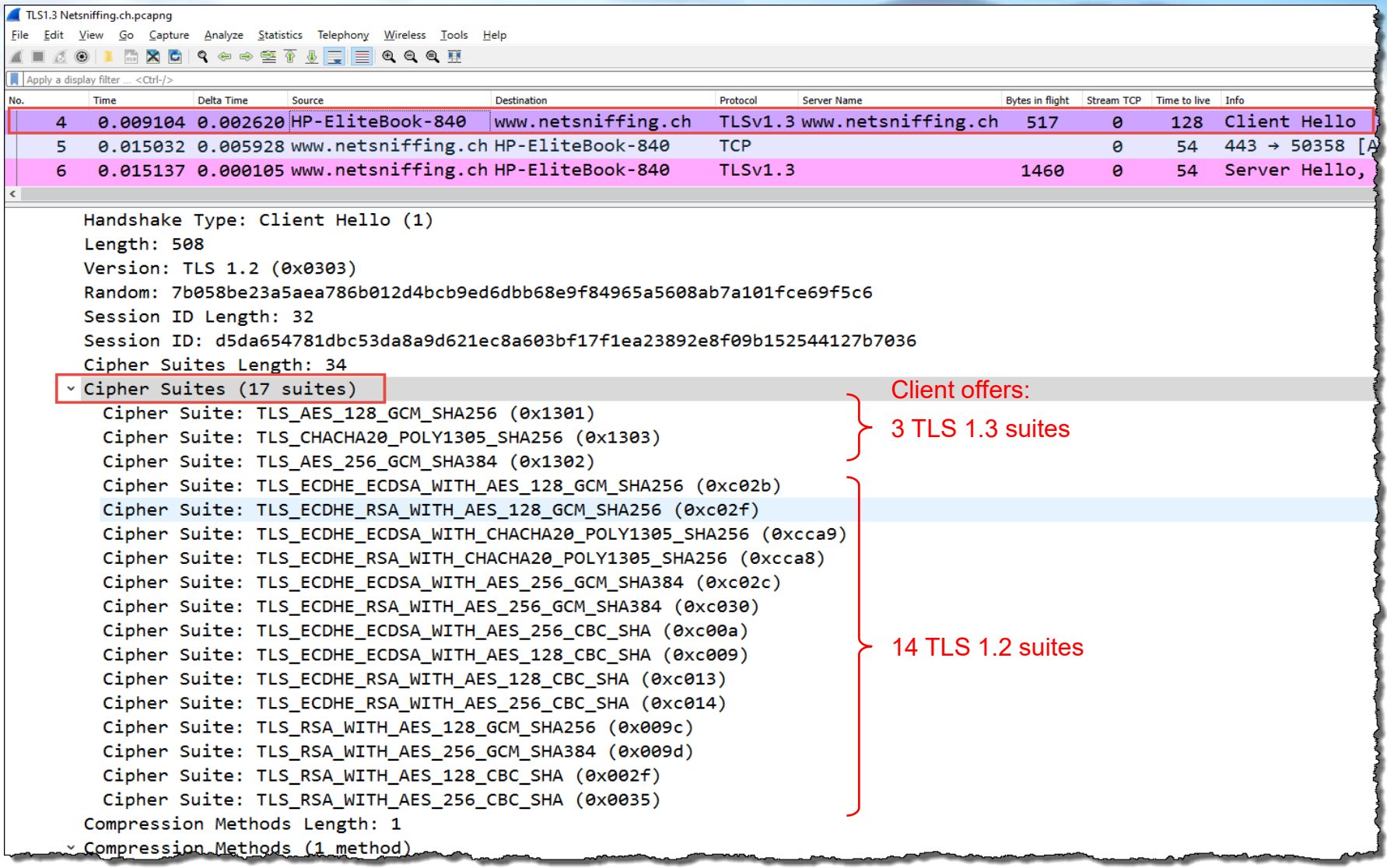
With TLS 1.2 all four above option (called cyphers) are negotiated at session start

Cypher Suite Example: TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256

With TLS 1.3 Key Exchange and Auth. are fix, Encryption and Integrity Hash type are negotiated

Cypher Suite Example: TLS_AES_128_GCM_SHA256

Transport Layer Security (TLS)



Transport Layer Security (TLS)

TLS1.3 Netsniffing.ch.pcapng

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter ... <Ctrl-/>

No.	Time	Delta Time	Source	Destination	Protocol	Server Name	Bytes in flight	Stream TCP	Time to live	Info
4	0.009104	0.002620	HP-EliteBook-840	www.netsniffing.ch	TLSv1.3	www.netsniffing.ch	517	0	128	Client Hello
5	0.015032	0.005928	www.netsniffing.ch	HP-EliteBook-840	TCP			0	54	443 → 50358
6	0.015137	0.000105	www.netsniffing.ch	HP-EliteBook-840	TLSv1.3		1460	0	54	Server Hello

```
Transport Layer Security
  TLSv1.3 Record Layer: Handshake Protocol: Server Hello
    Content Type: Handshake (22)
    Version: TLS 1.2 (0x0303)
    Length: 122
    Handshake Protocol: Server Hello
      Handshake Type: Server Hello (2)
      Length: 118
      Version: TLS 1.2 (0x0303)
      Random: f9f8ef645889faff50590b0653f934dacfcaec0ac9131d6f5e7591a8ba9655a8
      Session ID Length: 32
      Session ID: d5da654781dbc53da8a9d621ec8a603bf17f1ea23892e8f09b152544127b7036
      Cipher Suite: TLS_AES_128_GCM_SHA256 (0x1301)
      Compression Method: null (0)
      Extensions Length: 46
    > Extension: supported_versions (len=2)
    > Extension: key_share (len=36)
      [JA3S Fullstring: 771,4865,43-51]
      [JA3S: f4febc55ea12b31ae17cfb7e614afda8]
  TLSv1.3 Record Layer: Change Cipher Spec Protocol: Change Cipher Spec
    Content Type: Change Cipher Spec (20)
    Version: TLS 1.2 (0x0303)
    Length: 1
    Change Cipher Spec Message
  TLSv1.3 Record Layer: Application Data Protocol: http-over-tls
    Opaque Type: Application Data (23)
    Version: TLS 1.2 (0x0303)
    Length: 36
```

Server has chosen a TLS 1.3 suite

Comparison: TLS 1.2 versus TLS 1.3

TLS 1.2 features

Handshake: 2-RTT

Key Exchange methods:
- RSA
- Diffie Hellman (DH)
- Elliptic Curve Diffie-Hellman Ephemeral

Authentication methods:
- RSA, ECDSA

Encryption Algorithm:
- RC4, 3DES and many others older ones

Integrity Hash Algorithms:
- MD5, SHA1, SHA-256, SHA-384

Four cyphers are negotiated in so-called **CYPHER SUITES**

Cypher Suites offered by client:
TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 (0xc02b)
TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 (0xc02f)
TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 (0xc02c)
TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 (0xc030)
and many more...

TLS 1.3 features

Handshake: 1-RTT and 0-RTT

Key Exchange method*:
- Elliptic Curve Diffie-Hellman Ephemeral (ECDHE with curve X25519 or secp256)

Authentication: (included in Key Exchange)

Encryption Algorithm (five only):
- AES (128 or 256 bits) GCM
- AES (128 or 256 bits) CCM
- CHACHA20_POLY1305

Integrity Hash Algorithms:
- SHA-256, SHA-384

Only two cyphers are negotiated in max. five CYPHER SUITES
*Key Exchange is fixed to ECDHE and therefore not negotiated
Authentication is included in Key Exchange Method

TLS_AES_128_GCM_SHA256 (0x1301)
TLS_AES_256_GCM_SHA384 (0x1302)
TLS_CHACHA20_POLY1305_SHA256 (0x1303)
TLS_AES_128_CCM_SHA256 (0x1304)
TLS_AES_128_CCM_8_SHA256 (0x1305)

© Leutert NetServices 2026 / TCP / V4.1f

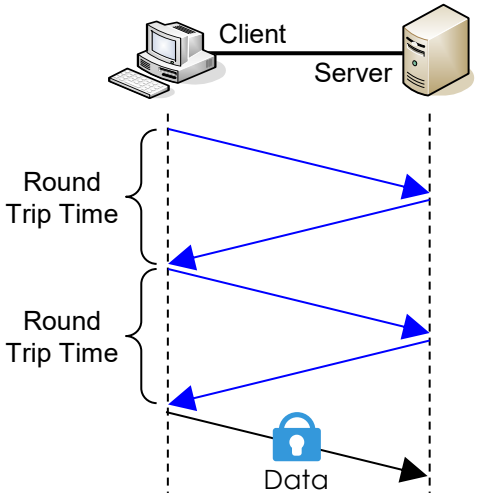
Transport Layer Security (TLS)

History and Versions

• SSL 1.0	1994	Netscape	**S**ecure **S**ocket **L**ayer, never released
• SSL 2.0	1995	Netscape	Released, but contains errors
• SSL 3.0	1996	Netscape	Has been widely used
• TLS 1.0	1999	IETF	SSL 3.1 → renamed to TLS by IETF
• TLS 1.1	2006	IETF	SSL 3.2 → renamed to TLS by IETF
• TLS 1.2	2008	IETF	SSL 3.3 → currently most used
• TLS 1.3	2018	IETF	Recommended standard (supported by current browsers)

Do not use anymore

TLS 1.2 Handshake

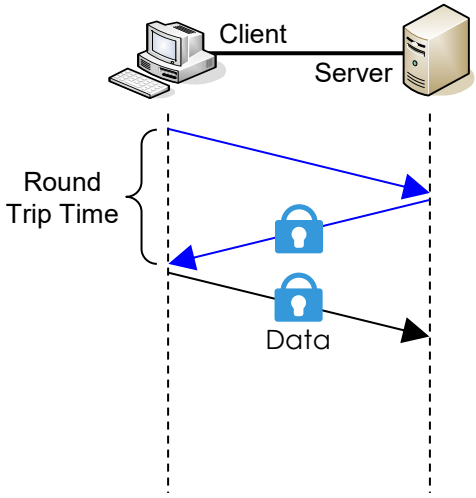


TLS 1.2 Features

- 2 Round Trip Times
- TLS Handshake unencrypted
- 37 Cypher Suites
- Using the Server public/private keys
- Certificate unencrypted
- Cypher Downgrade Forcing possible

https://tls.ulfheim.net/

TLS 1.3 Handshake



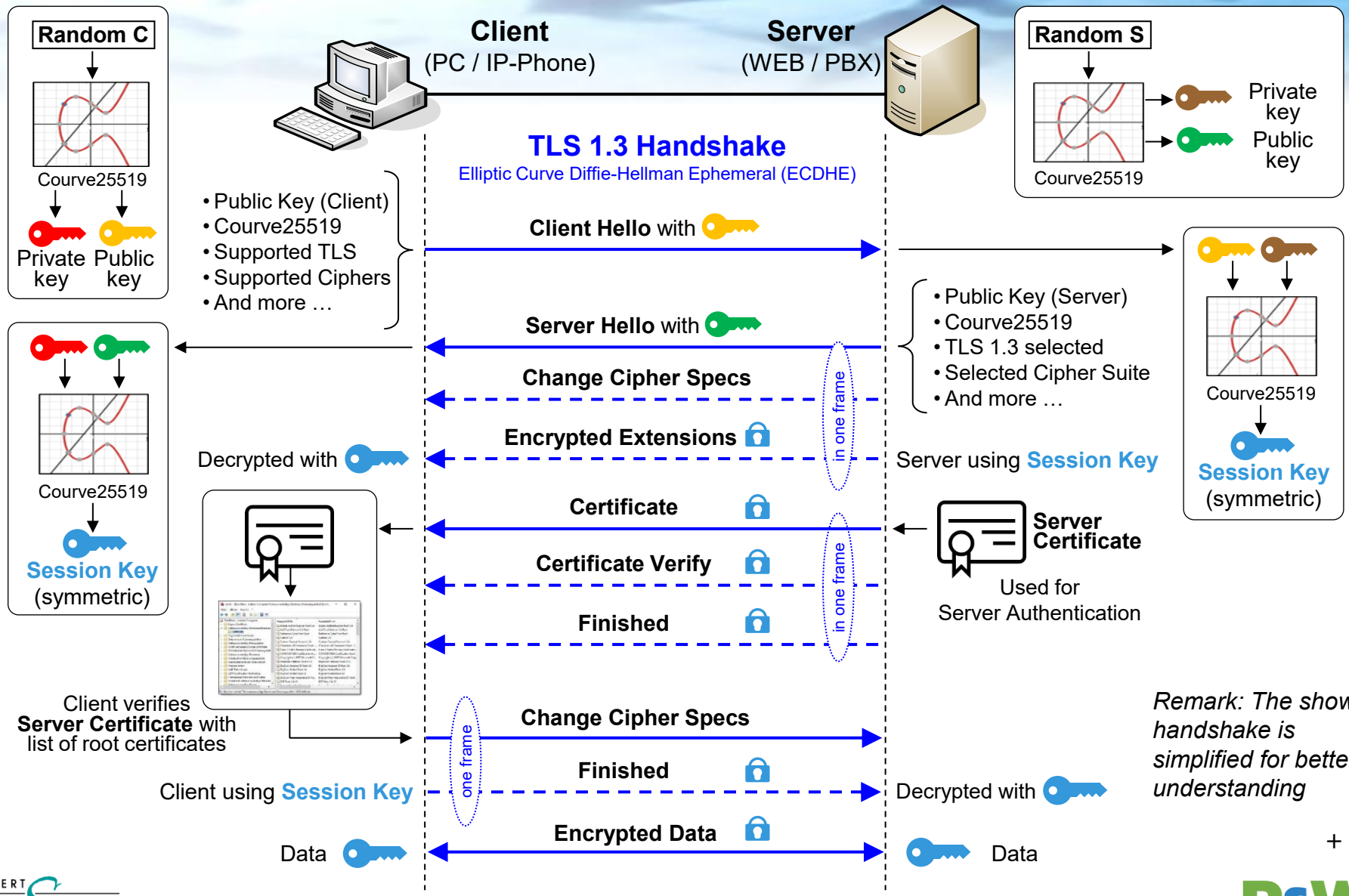
TLS 1.3 Features

- 1 Round Trip Time
- Handshake from Server encrypted
- 5 strong Cypher Suites
- Using generated public/private keys
- Certificate encrypted
- Cypher Downgrade Forcing not possible
- Forward Secrecy, sessions recorded in the past cannot be decrypted
- And many more…

https://tls13.ulfheim.net/

© Leutert NetServices 2026 / TCP / V4.1f

TLS 1.3 Encryption & Authentication



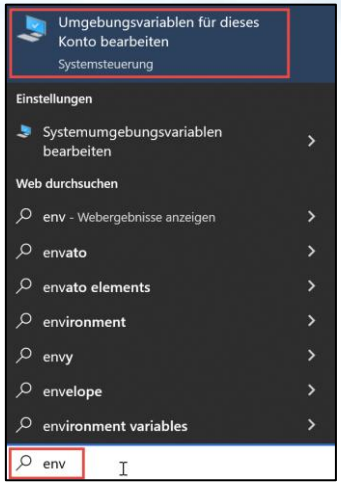
Remark: The shown handshake is simplified for better understanding

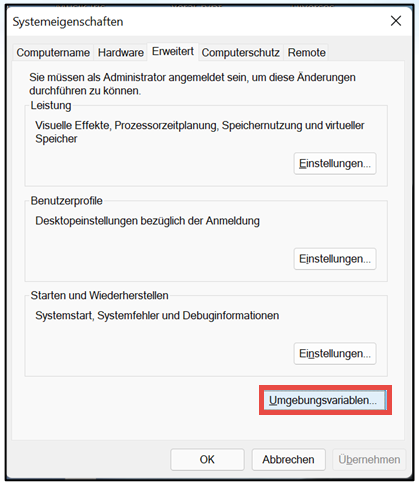
+

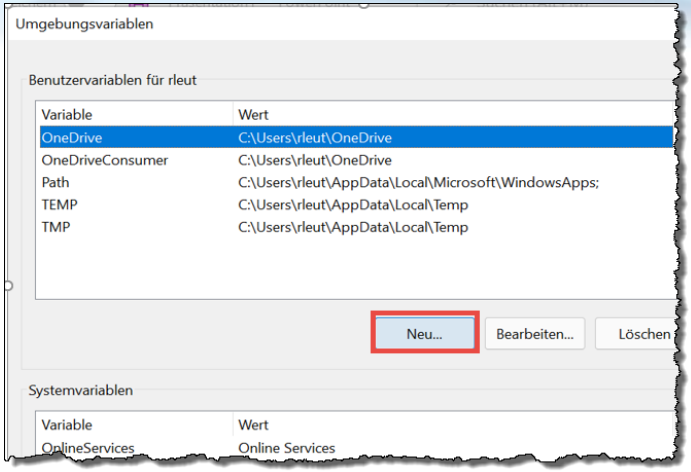
© Leutert NetServices 2026 / TCP / V4.1f

Transport Layer Security (TLS)

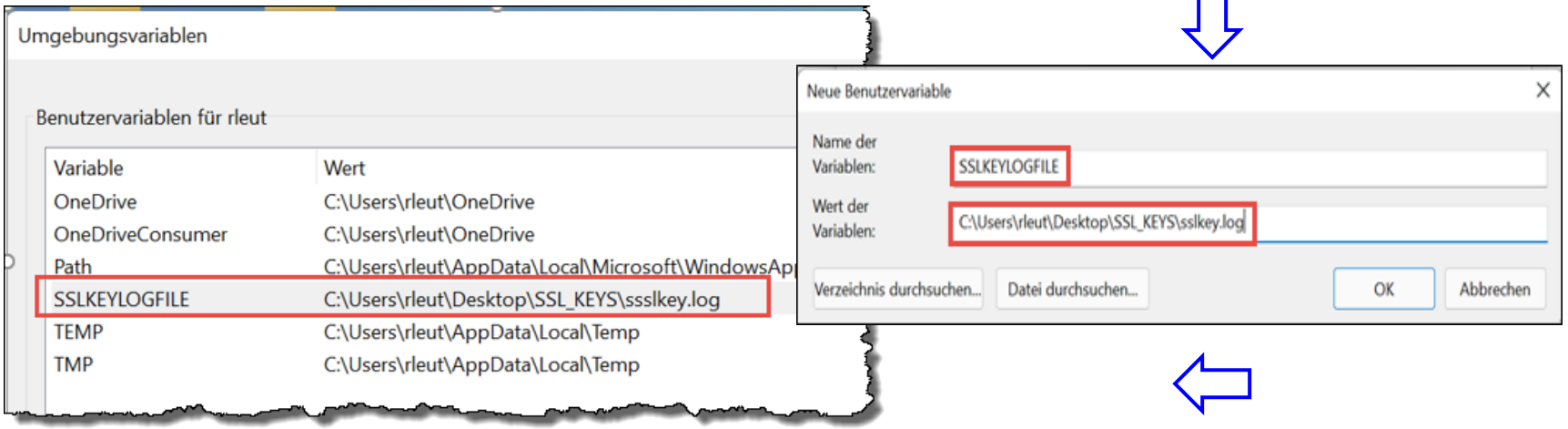
TLS Decryption with SSLKEYLOGFILE (Example: Windows with Firefox 98.0.2)





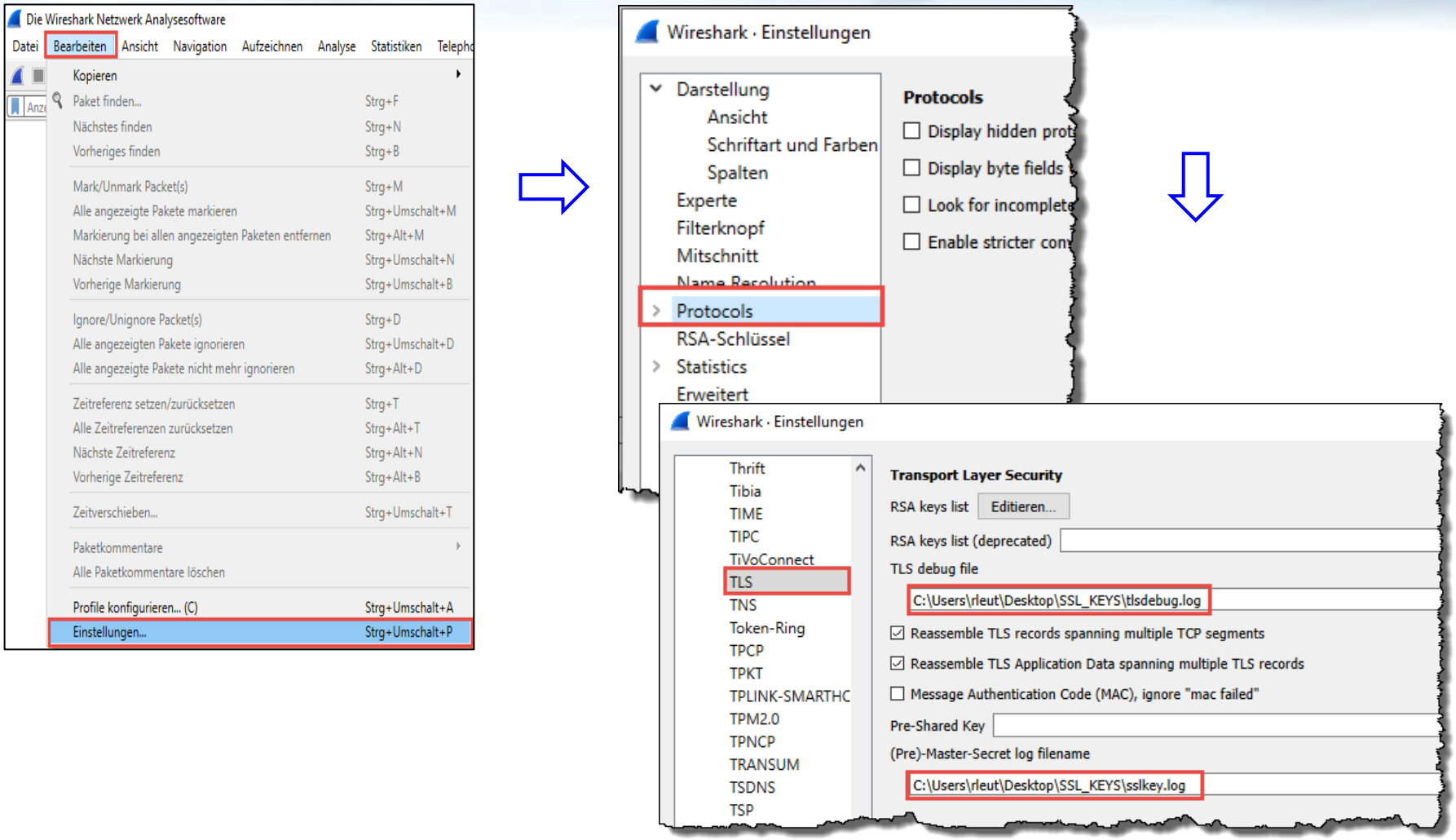


→ **Note**: Close the Browser before entering the user environment variable **SSLKEYLOGFILE** ←



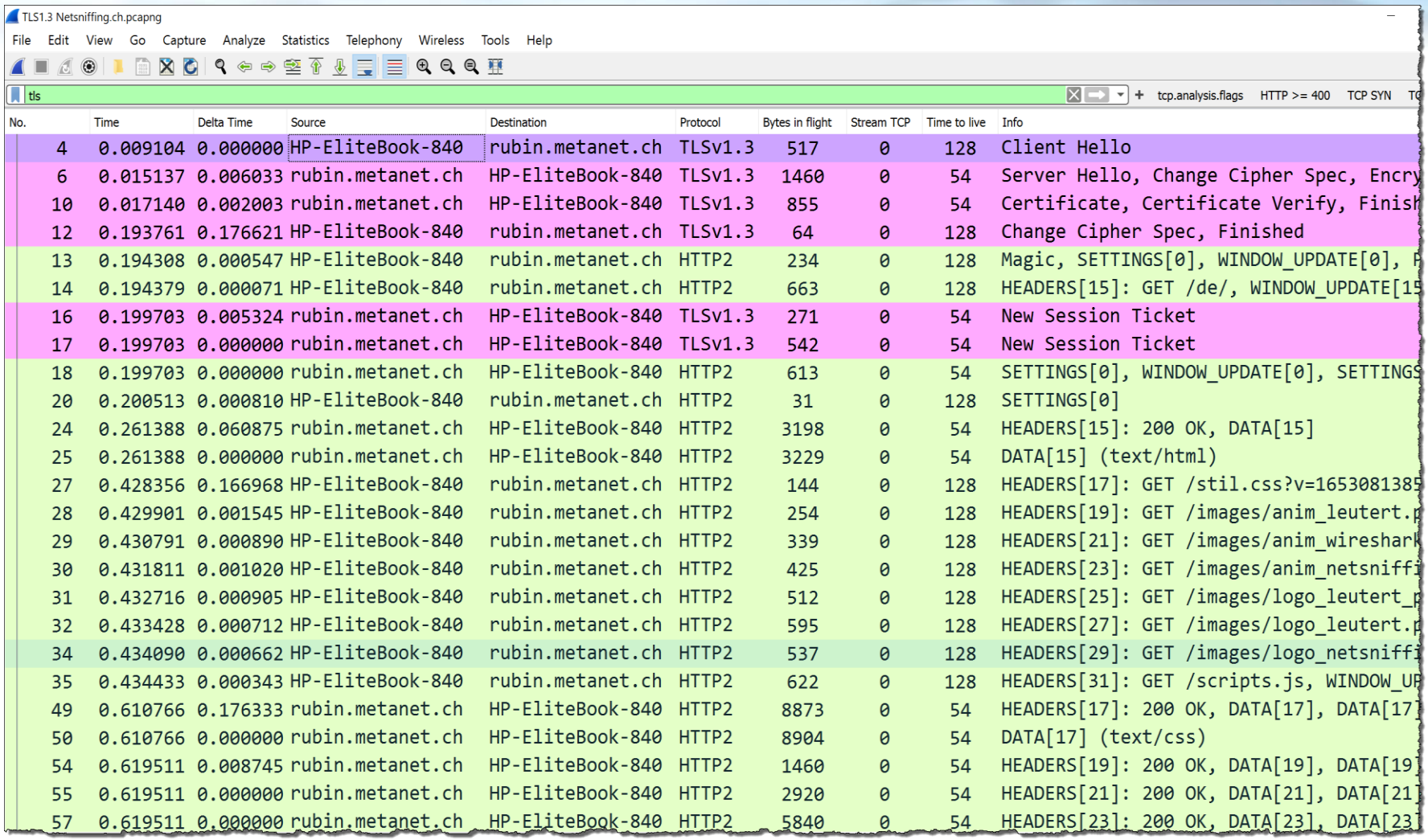
Transport Layer Security (TLS)

TLS Decryption with SSLKEYLOGFILE (Example: Windows with Firefox 98.0.2)



Transport Layer Security (TLS)

TLS Decryption with SSLKEYLOGFILE (Example: Windows with Firefox 98.0.2)



No.	Time	Delta Time	Source	Destination	Protocol	Bytes in flight	Stream TCP	Time to live	Info
4	0.009104	0.000000	HP-EliteBook-840	rubin.metanet.ch	TLSv1.3	517	0	128	Client Hello
6	0.015137	0.006033	rubin.metanet.ch	HP-EliteBook-840	TLSv1.3	1460	0	54	Server Hello, Change Cipher Spec, Encry
10	0.017140	0.002003	rubin.metanet.ch	HP-EliteBook-840	TLSv1.3	855	0	54	Certificate, Certificate Verify, Finish
12	0.193761	0.176621	HP-EliteBook-840	rubin.metanet.ch	TLSv1.3	64	0	128	Change Cipher Spec, Finished
13	0.194308	0.000547	HP-EliteBook-840	rubin.metanet.ch	HTTP2	234	0	128	Magic, SETTINGS[0], WINDOW_UPDATE[0], F
14	0.194379	0.000071	HP-EliteBook-840	rubin.metanet.ch	HTTP2	663	0	128	HEADERS[15]: GET /de/, WINDOW_UPDATE[15
16	0.199703	0.005324	rubin.metanet.ch	HP-EliteBook-840	TLSv1.3	271	0	54	New Session Ticket
17	0.199703	0.000000	rubin.metanet.ch	HP-EliteBook-840	TLSv1.3	542	0	54	New Session Ticket
18	0.199703	0.000000	rubin.metanet.ch	HP-EliteBook-840	HTTP2	613	0	54	SETTINGS[0], WINDOW_UPDATE[0], SETTINGS
20	0.200513	0.000810	HP-EliteBook-840	rubin.metanet.ch	HTTP2	31	0	128	SETTINGS[0]
24	0.261388	0.060875	rubin.metanet.ch	HP-EliteBook-840	HTTP2	3198	0	54	HEADERS[15]: 200 OK, DATA[15]
25	0.261388	0.000000	rubin.metanet.ch	HP-EliteBook-840	HTTP2	3229	0	54	DATA[15] (text/html)
27	0.428356	0.166968	HP-EliteBook-840	rubin.metanet.ch	HTTP2	144	0	128	HEADERS[17]: GET /stil.css?v=1653081385
28	0.429901	0.001545	HP-EliteBook-840	rubin.metanet.ch	HTTP2	254	0	128	HEADERS[19]: GET /images/anim_leutert.p
29	0.430791	0.000890	HP-EliteBook-840	rubin.metanet.ch	HTTP2	339	0	128	HEADERS[21]: GET /images/anim_wireshark
30	0.431811	0.001020	HP-EliteBook-840	rubin.metanet.ch	HTTP2	425	0	128	HEADERS[23]: GET /images/anim_netsniffi
31	0.432716	0.000905	HP-EliteBook-840	rubin.metanet.ch	HTTP2	512	0	128	HEADERS[25]: GET /images/logo_leutert_p
32	0.433428	0.000712	HP-EliteBook-840	rubin.metanet.ch	HTTP2	595	0	128	HEADERS[27]: GET /images/logo_leutert.p
34	0.434090	0.000662	HP-EliteBook-840	rubin.metanet.ch	HTTP2	537	0	128	HEADERS[29]: GET /images/logo_netsniffi
35	0.434433	0.000343	HP-EliteBook-840	rubin.metanet.ch	HTTP2	622	0	128	HEADERS[31]: GET /scripts.js, WINDOW_UP
49	0.610766	0.176333	rubin.metanet.ch	HP-EliteBook-840	HTTP2	8873	0	54	HEADERS[17]: 200 OK, DATA[17], DATA[17]
50	0.610766	0.000000	rubin.metanet.ch	HP-EliteBook-840	HTTP2	8904	0	54	DATA[17] (text/css)
54	0.619511	0.008745	rubin.metanet.ch	HP-EliteBook-840	HTTP2	1460	0	54	HEADERS[19]: 200 OK, DATA[19], DATA[19]
55	0.619511	0.000000	rubin.metanet.ch	HP-EliteBook-840	HTTP2	2920	0	54	HEADERS[21]: 200 OK, DATA[21], DATA[21]
57	0.619511	0.000000	rubin.metanet.ch	HP-EliteBook-840	HTTP2	5840	0	54	HEADERS[23]: 200 OK, DATA[23], DATA[23]

Secure Shell (SSH) vs SSL/TLS

Establishes a secure connection from an SSH-Client to an SSH-Server

- Like **SSL/TLS**, SSH also works with a client-server connection, based on **TCP**
- SSH is best suited for highly secure remote access to servers and devices
- **SSH** is requiring a **piece of software** on the client, **TLS** is using any **Browser**
- Like **TLS1.3**, SSH has mutual authentication, with a public/private key on **both sides**
- Different to TLS, an **SSH client must be registered on the server** to allow access

Frames (Trace file SSH_session.pcapng)

1 The client initiates a TCP session to the server, **default TCP port 22**

4 The server responds with the **supported SSH versions**

5 The client sends **SSH version, encryption and key exchange method**

6 The client starts the **Key Exchange Init**

9,10 The server replies to **the Key Exchange Init**

12 The client starts the **Diffie-Hellman Key Exchange Init**

13 The server replies to the **Diffie-Hellman Key Exchange Init** and sends **New Key**

15 The client replies with **New Key**

16 → The communication starts with encrypted data packets

Secure Shell (SSH)

Establishes a secure connection from an SSH-Client to an SSH-Server

SSH_session.pcapng

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter ... <Ctrl-/> tcp.analysis.flags

	Time	Delta Time	Source	Destination	Protocol	Stream TCP	Bytes in flight	Length	Stream UDP	Seq No	Next sequence	Ack	Info
1	0.000000	0.000000	Client	Server	TCP	0		66		0	1	0	54140 → 22 [SYN, ECE, CWR] Seq=0 Win=64240 Len=0 MSS
2	0.000527	0.000527	Server	Client	TCP	0		66		0	1	1	22 → 54140 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 M
3	0.000565	0.000038	Client	Server	TCP	0		54		1	1	1	54140 → 22 [ACK] Seq=1 Ack=1 Win=262656 Len=0
4	0.008850	0.008285	Server	Client	SSHv2	0	44	98		1	45	1	Server: Protocol (SSH-2.0-OpenSSH_6.6.1p1 Ubuntu-2u
5	0.008920	0.000070	Client	Server	SSHv2	0	20	74		1	21	45	Client: Protocol (SSH-2.0-Devart-6.0)
6	0.009036	0.000116	Client	Server	SSHv2	0	340	374		21	341	45	Client: Key Exchange Init
7	0.009399	0.000363	Server	Client	TCP	0		60		45	45	21	22 → 54140 [ACK] Seq=45 Ack=21 Win=29312 Len=0
8	0.009440	0.000041	Server	Client	TCP	0		60		45	45	3…	22 → 54140 [ACK] Seq=45 Ack=341 Win=30336 Len=0
9	0.010262	0.000822	Server	Client	TCP	0	1460	1514		45	1505	3…	22 → 54140 [ACK] Seq=45 Ack=341 Win=30336 Len=1460
10	0.010262	0.000000	Server	Client	SSHv2	0	1648	242		1505	1693	3…	Server: Key Exchange Init
11	0.010295	0.000033	Client	Server	TCP	0		54		341	341	1…	54140 → 22 [ACK] Seq=341 Ack=1693 Win=262656 Len=0
12	0.021854	0.011559	Client	Server	SSHv2	0	272	326		341	613	1…	Client: Diffie-Hellman Key Exchange Init
13	0.026305	0.004451	Server	Client	SSHv2	0	848	902		1693	2541	6…	Server: Diffie-Hellman Key Exchange Reply, New Keys
14	0.036956	0.010651	Client	Server	TCP	0		54		613	613	2…	54140 → 22 [ACK] Seq=613 Ack=2541 Win=261888 Len=0
15	0.039933	0.002977	Client	Server	SSHv2	0	16	70		613	629	2…	Client: New Keys
16	0.040079	0.000146	Client	Server	SSHv2	0	68	106		629	681	2…	Client: Encrypted packet (len=52)
17	0.040526	0.000447	Server	Client	TCP	0		60		2541	2541	6…	22 → 54140 [ACK] Seq=2541 Ack=681 Win=31360 Len=0
18	0.040551	0.000025	Server	Client	SSHv2	0	52	106		2541	2593	6…	Server: Encrypted packet (len=52)
19	0.040634	0.000083	Client	Server	SSHv2	0	76	130		681	757	2…	Client: Encrypted packet (len=76)
20	0.046693	0.006059	Server	Client	SSHv2	0	60	114		2593	2653	7…	Server: Encrypted packet (len=60)
21	0.046756	0.000063	Client	Server	SSHv2	0	92	146		757	849	2…	Client: Encrypted packet (len=92)

```
> Frame 4: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface \Device\NPF_{071D3E0B-EE5F-4213-A4E5-28FE07AF071D}, id 0
> Ethernet II, Src: e6:73:ca:89:0e:f0 (e6:73:ca:89:0e:f0), Dst: VMware_a8:a1:01 (00:50:56:a8:a1:01)
> Internet Protocol Version 4, Src: Server (10.49.66.180), Dst: Client (10.49.66.137)
> Transmission Control Protocol, Src Port: 22, Dst Port: 54140, Seq: 1, Ack: 1, Len: 44
v SSH Protocol
    Protocol: SSH-2.0-OpenSSH_6.6.1p1 Ubuntu-2ubuntu2.13
    [Direction: server-to-client]
```

SSH session establishment

User Datagram Protocol (UDP)

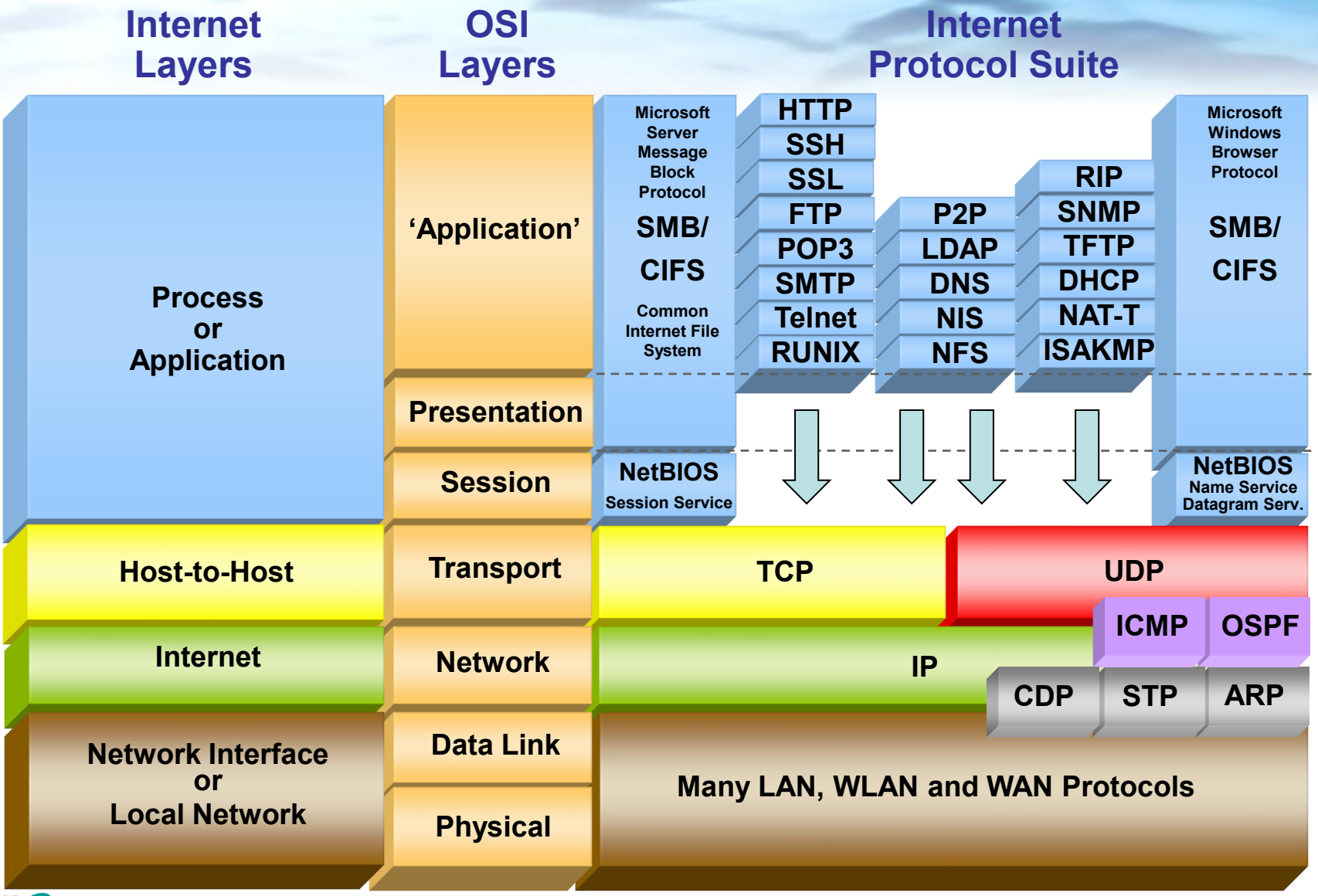
UDP offers a variation to data transmission without connection establishment and without guaranteed delivery. Securing the data transmission is left to the higher protocols (Application Layer). That way UDP gets more efficient with short queries like DNS, SNMP etc. than TCP.

VoIP applications are also based on UDP



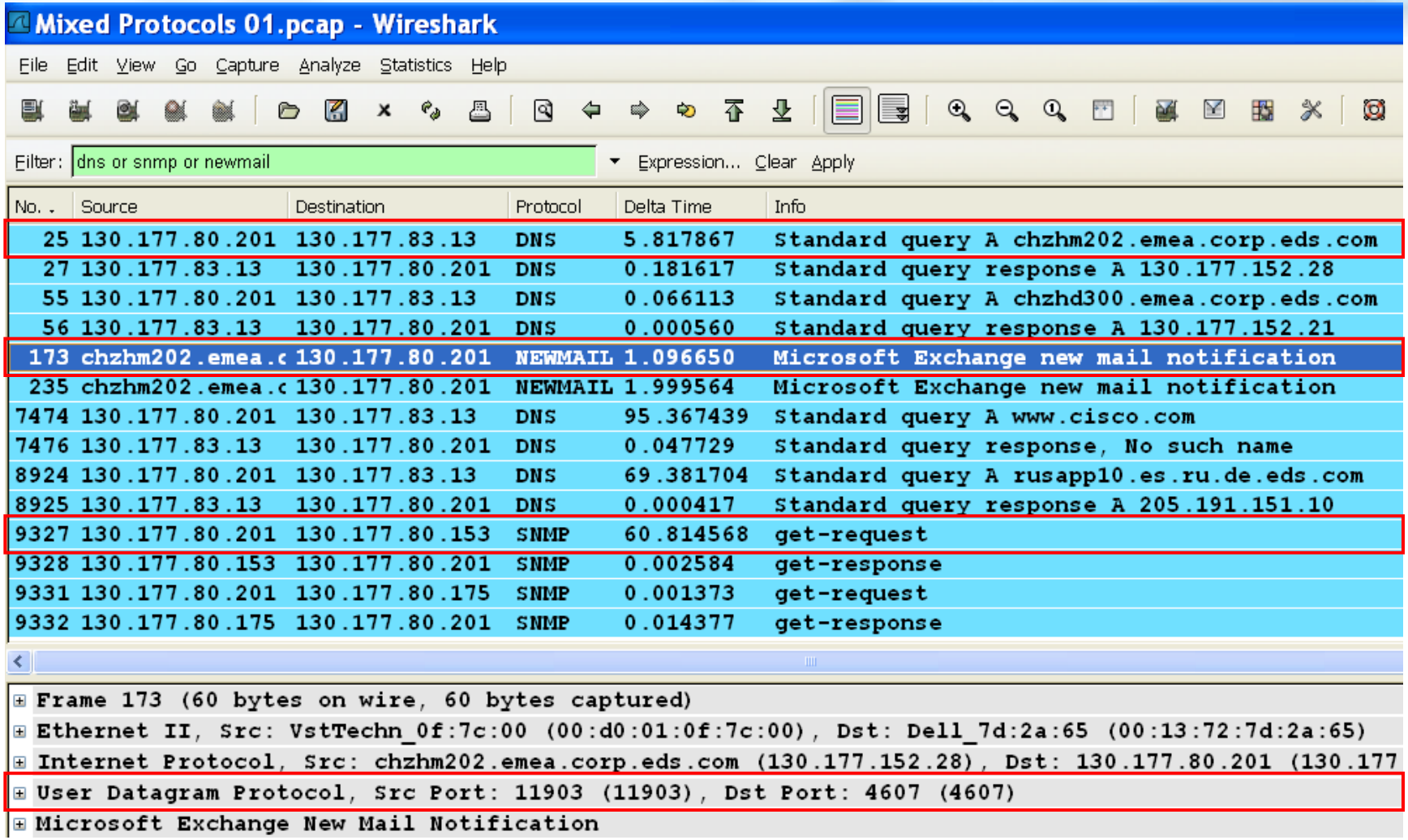
© Leutert NetServices 2026 / TCP / V4.1f

User Datagram Protocol (UDP)



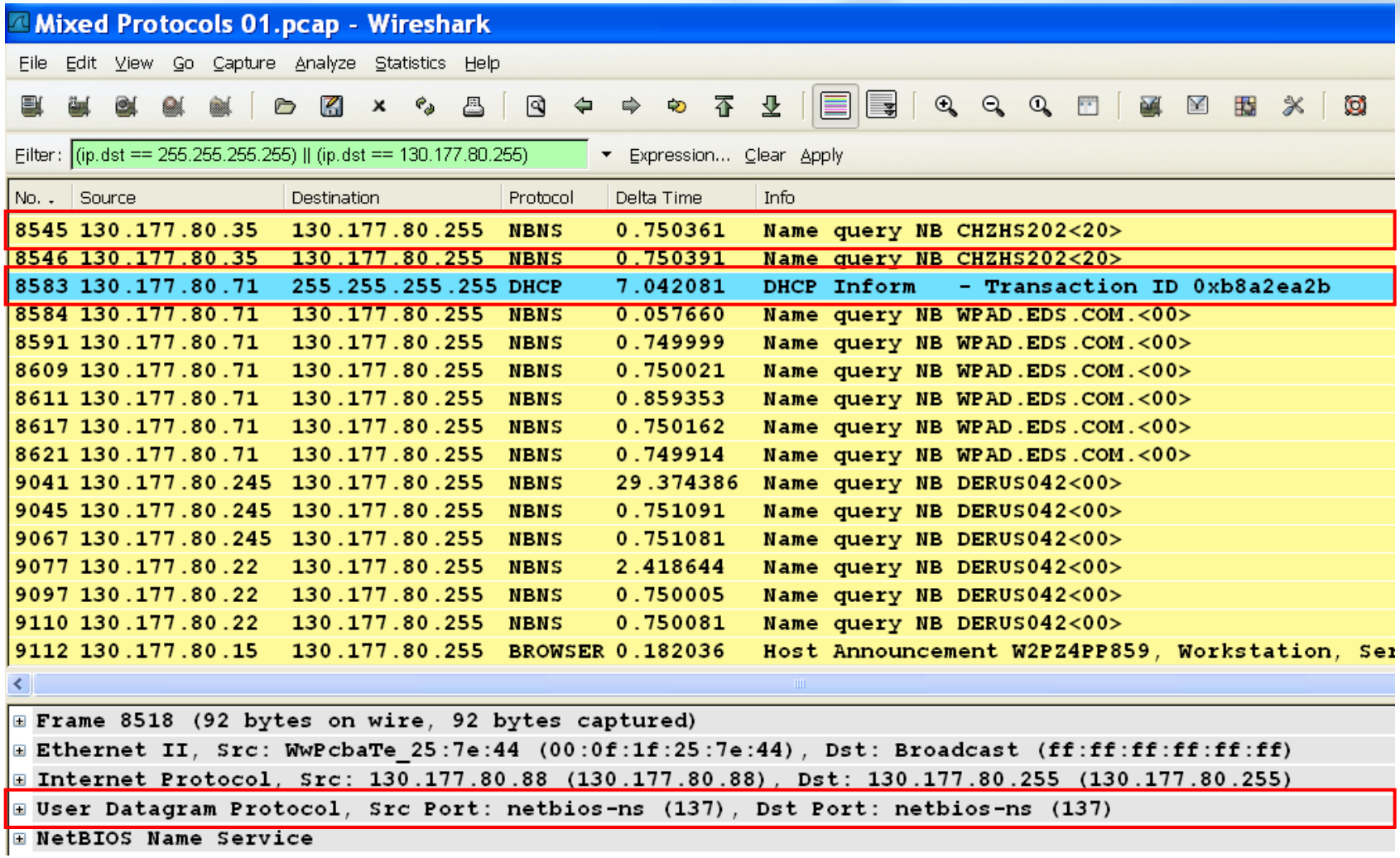
User Datagram Protocol (UDP)

Polling-orientated protocols like DNS, SNMP etc. use UDP



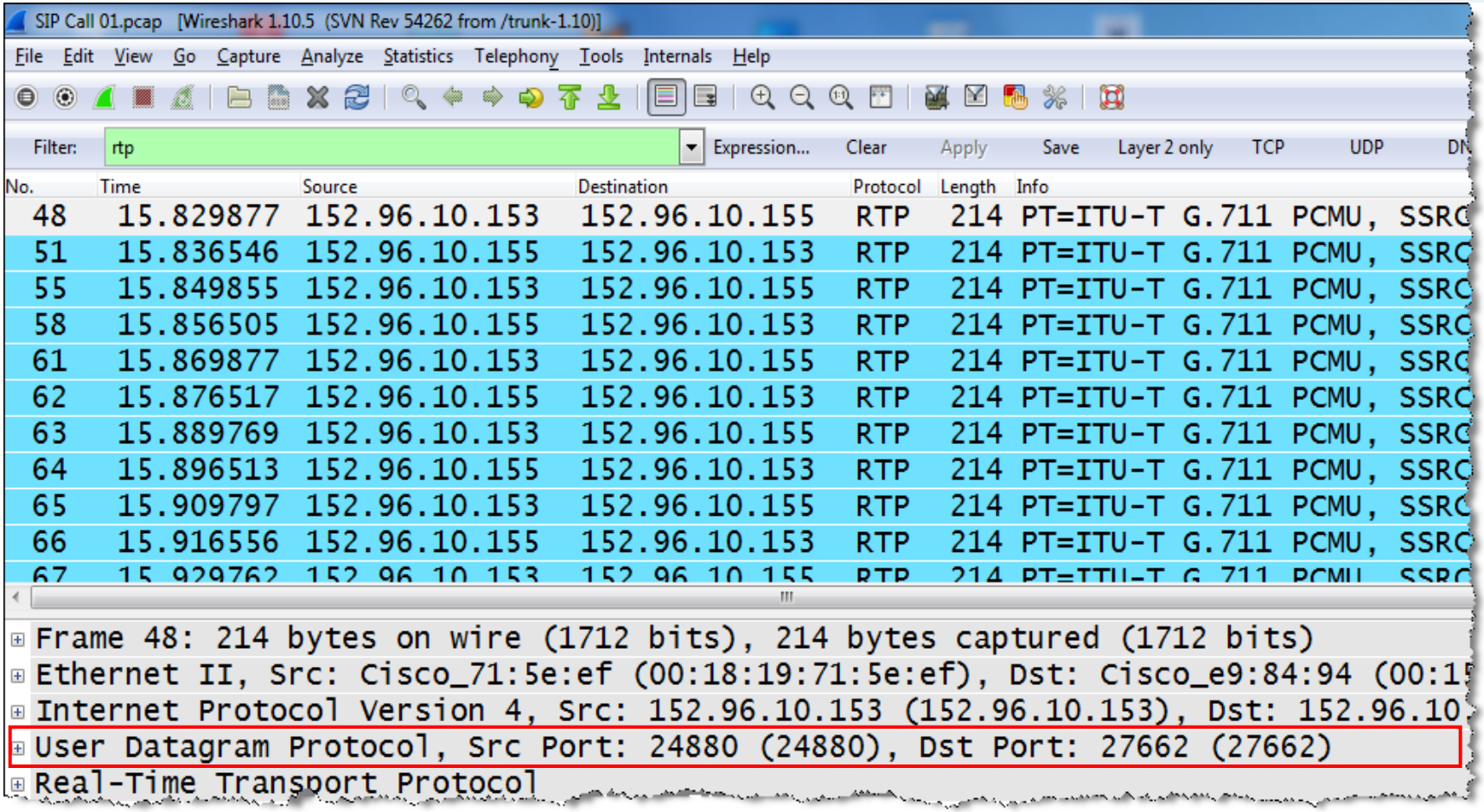
User Datagram Protocol (UDP)

Broadcast protocols like DHCP, NBNS, BROWSER etc. use UDP



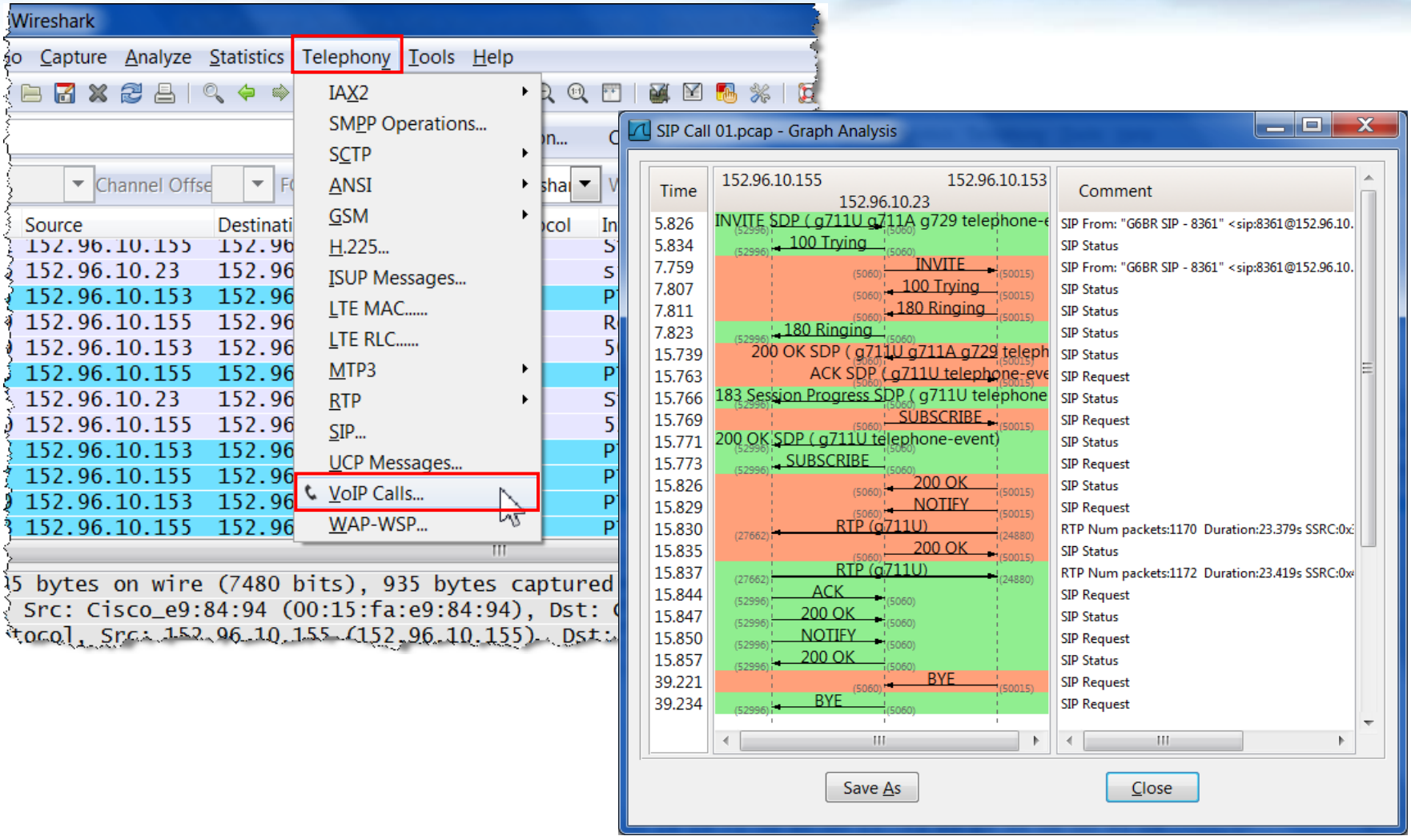
User Datagram Protocol (UDP)

VoIP protocols like Real Time Protocol (RTP) are based on UDP



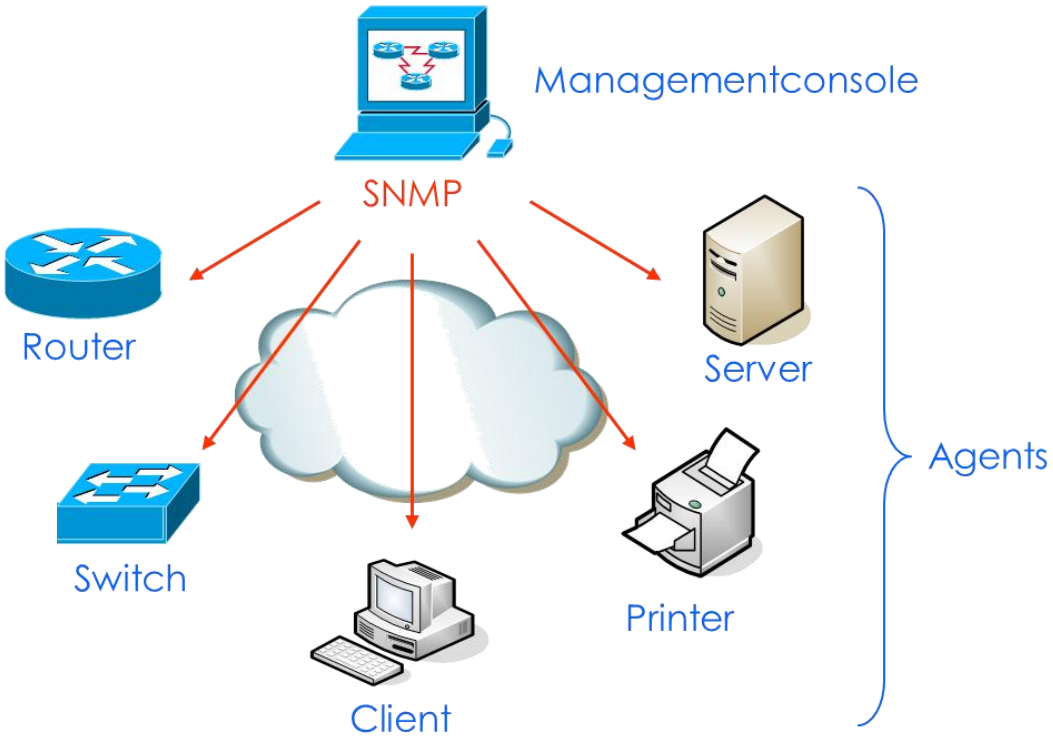
User Datagram Protocol (UDP)

Dialog-oriented protocols like SIP, NFS are based on UDP or TCP

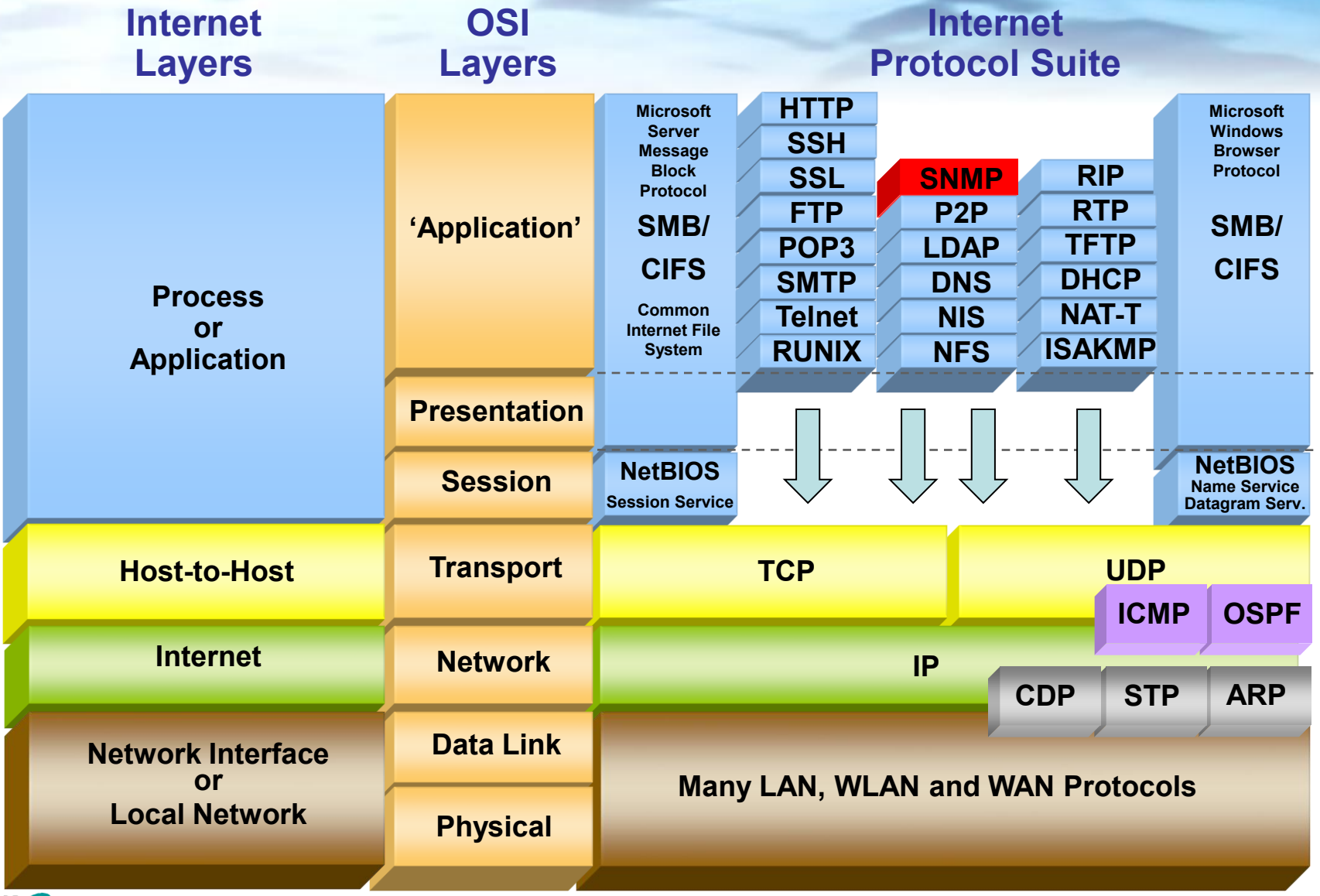


Simple Network Management Protocol (SNMP)

- SNMP is an Internet Standard (IETF) protocol for **collecting and organizing information** about managed devices on IP networks and for modifying that information to change device behaviour.
- Devices that typically support SNMP include **cable modems, routers, switches, servers, workstations, printers** and more.



Simple Network Management Protocol (SNMP)



Simple Network Management Protocol (SNMP)

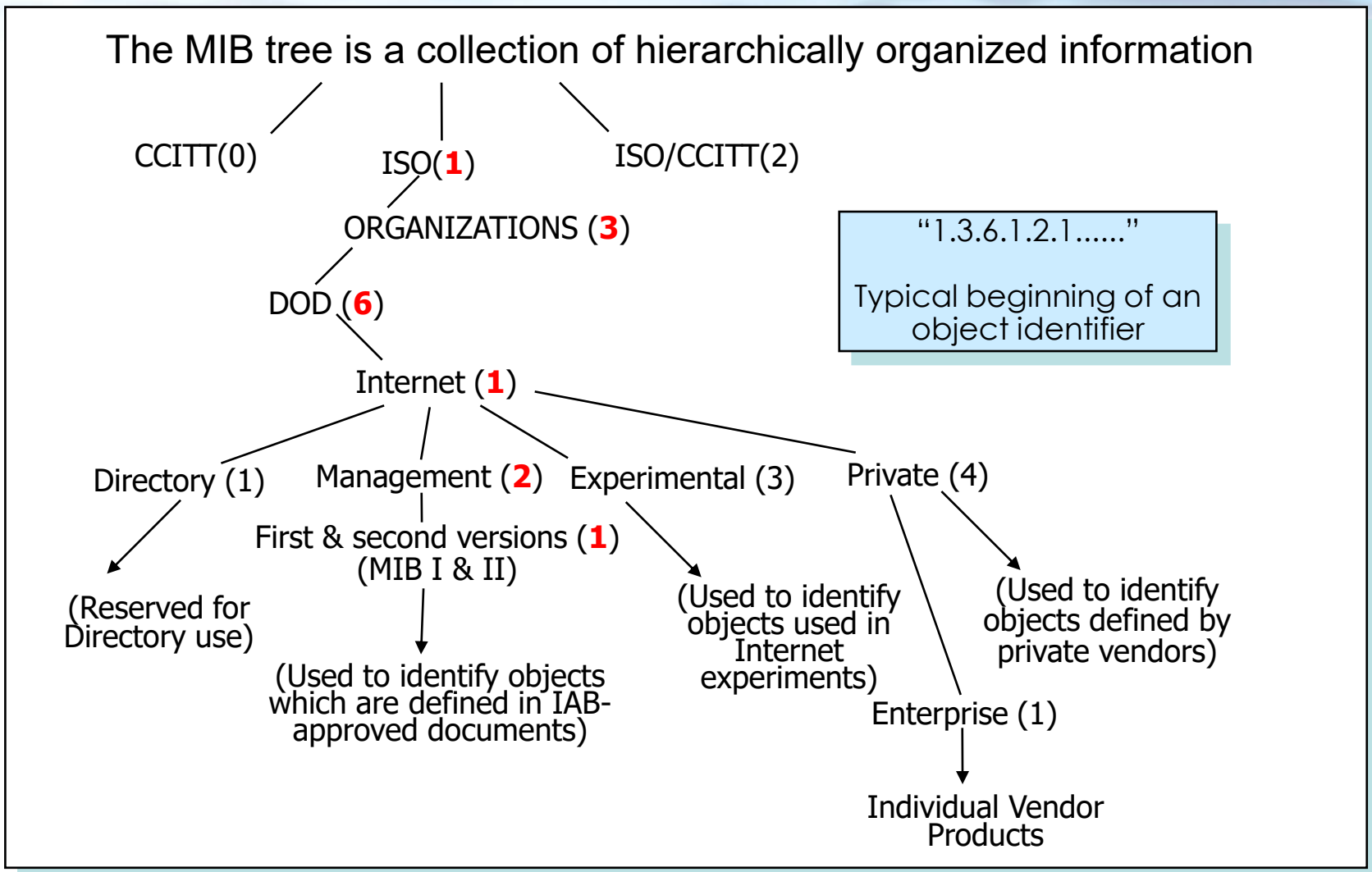
Different SNMP versions offers different security levels

Model	Level	Authentication	Encryption	What Happens
v1	noAuthNoPriv	Community String	No	Uses a community string match for authentication.
v2c	noAuthNoPriv	Community String	No	Uses a community string match for authentication.
v3	noAuthNoPriv	Username	No	Uses a username match for authentication.
v3	authNoPriv	MD5 or SHA	No	Provides authentication based on the HMAC-MD5 or HMAC-SHA algorithms.
v3	authPriv	MD5 or SHA	DES	Provides authentication based on the HMAC-MD5 or HMAC-SHA algorithms. Provides DES 56-bit encryption in addition to authentication based on the CBC-DES (DES-56) standard.

© Leutert NetServices 2026 / TCP / V4.1f

Simple Network Management Protocol (SNMP)

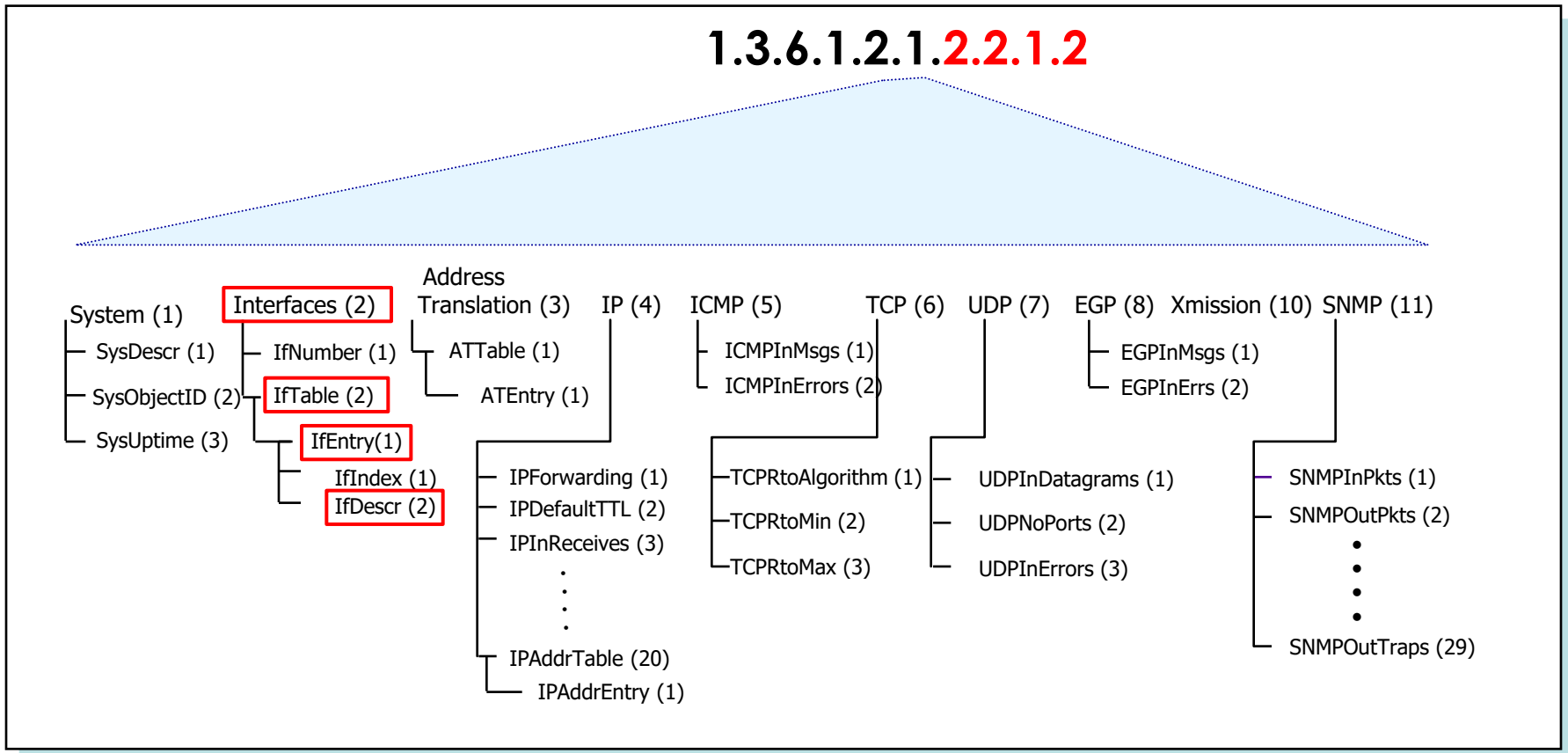
The Management Information Base (MIB)



Simple Network Management Protocol (SNMP)

The Management Information Base (MIB)

Example: Object Identifier (OID) 1.3.6.1.2.1.2.2.1.2

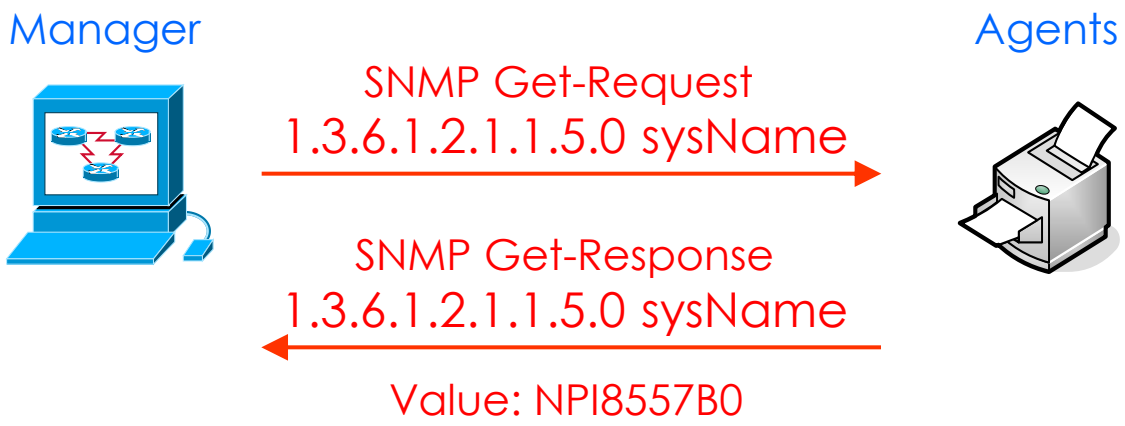


Simple Network Management Protocol (SNMP)

SNMP messages are used to communicate between the manager and the agent.

From Manager:
- **Get:** Requests the value of a specific MIB variable
- **Get-Next:** Requests the next value of a MIB variable
- **Set:** Sets the value of a specific MIB variable

From Agent:
- **Get-Response:** Returns the value of the requested MIB variable
- **Trap:** Alarm message containing the value of an exceeded MIB variable



Simple Network Management Protocol (SNMP)

Get-Request Message

SNMP_Including_fails.pcapng

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

snmp

No.	Time	Delta Time	Source	Destination	Protocol	error-status	Length	Time to live	Source Port	Destination Port	Data	request-id
2405	41.205557	0.000000	10.49.128.113	10.49.128.113	SNMP	noError	75	128	58568	161	get-request	989705
2406	41.206695	0.001138	10.49.128.113	10.49.128.113	SNMP	noError	76	128	161	58568	get-response	989705
2407	41.207361	0.000666	10.49.128.113	10.49.128.113	SNMP	noError	74	128	58569	161	get-request	989706
2408	41.207549	0.000188	10.49.128.113	10.49.128.113	SNMP	noError	78	128	161	58569	get-response	989706
2939	53.196818	11.989269	10.49.128.113	10.49.128.113	SNMP	noError	90	128	57784	161	get-request	989707
2940	53.196988	0.000170	10.49.128.113	10.49.128.113	SNMP	noError	92	128	161	57784	get-response	989707
4484	77.197030	24.000042	10.49.128.113	10.49.128.113	SNMP	noError	76	128	52262	161	get-request	989718
4485	77.197309	0.000279	10.49.128.113	10.49.128.113	SNMP	noError	77	128	161	52262	get-response	989718
4574	80.193796	2.996487	10.49.128.113	10.49.128.113	SNMP	noError	76	128	52263	161	get-request	989719

```
> Frame 2405: 75 bytes on wire (600 bits), 75 bytes captured (600 bits) on interface \Device\NPF_Loopback, id 0
> Null/Loopback
> Internet Protocol Version 4, Src: 10.49.128.113 (10.49.128.113), Dst: 10.49.128.113 (10.49.128.113)
> User Datagram Protocol, Src Port: 58568, Dst Port: 161
v Simple Network Management Protocol
    version: v2c (1)
    community: public
  v data: get-request (0)
    v get-request
        request-id: 989705
        error-status: noError (0)
        error-index: 0
      v variable-bindings: 1 item
        v 1.3.6.1.2.1.25.1.5.0: Value (Null)
            Object Name: 1.3.6.1.2.1.25.1.5.0 (iso.3.6.1.2.1.25.1.5.0)
            Value (Null)
  [Response In: 2406]
```

← Object Identifier (OID) 0 = ITU
1 = ISO

Simple Network Management Protocol (SNMP)

Get-Response Message

SNMP_Including_fails.pcapng

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

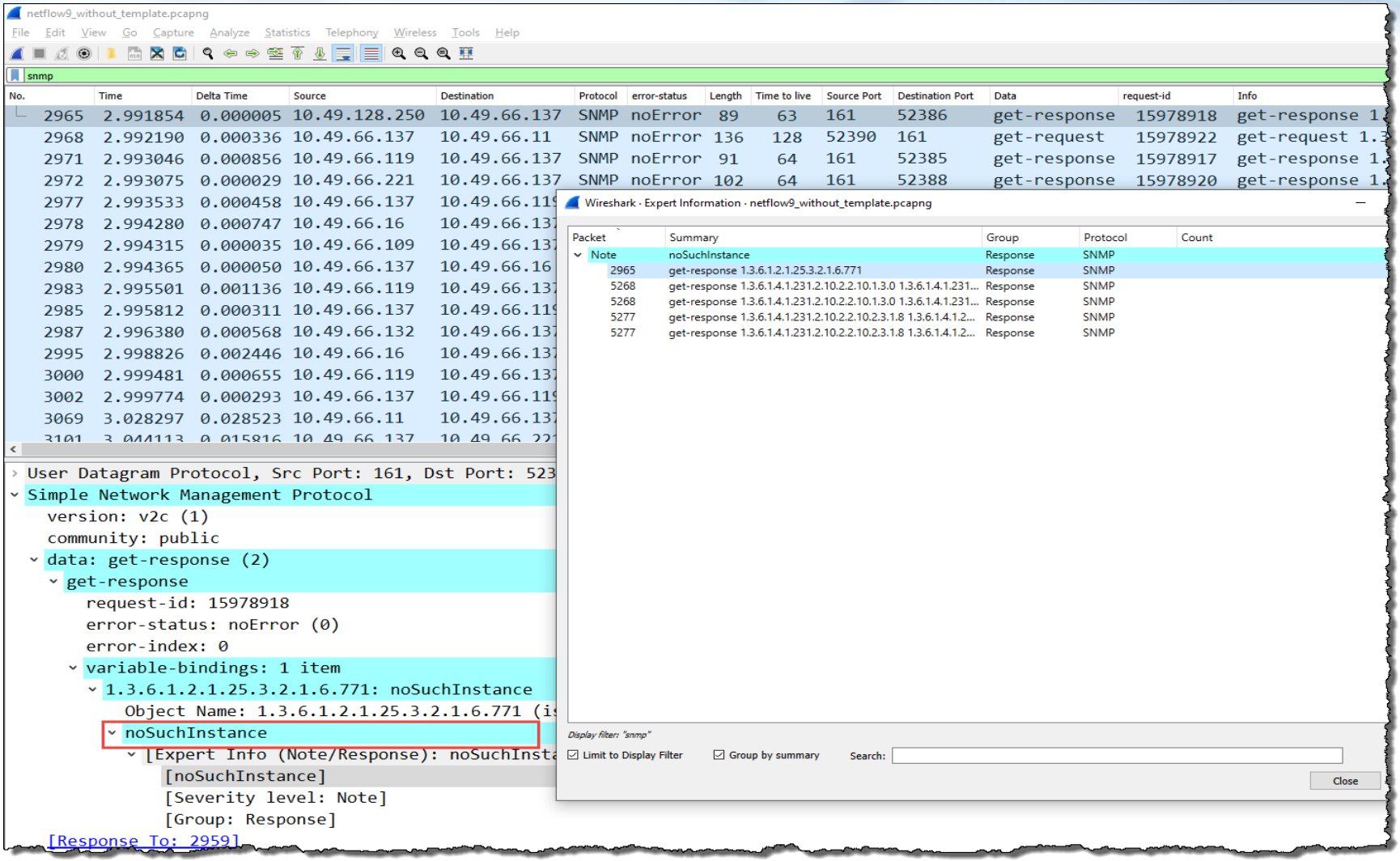
snmp

No.	Time	Delta Time	Source	Destination	Protocol	error-status	Length	Time to live	Source Port	Destination Port	Data	request-id
2405	41.205557	0.000000	10.49.128.113	10.49.128.113	SNMP	noError	75	128	58568	161	get-request	989705
2406	41.206695	0.001138	10.49.128.113	10.49.128.113	SNMP	noError	76	128	161	58568	get-response	989705
2407	41.207361	0.000666	10.49.128.113	10.49.128.113	SNMP	noError	74	128	58569	161	get-request	989706
2408	41.207549	0.000188	10.49.128.113	10.49.128.113	SNMP	noError	78	128	161	58569	get-response	989706
2939	53.196818	11.989269	10.49.128.113	10.49.128.113	SNMP	noError	90	128	57784	161	get-request	989707
2940	53.196988	0.000170	10.49.128.113	10.49.128.113	SNMP	noError	92	128	161	57784	get-response	989707
4484	77.197030	24.000042	10.49.128.113	10.49.128.113	SNMP	noError	76	128	52262	161	get-request	989718
4485	77.197309	0.000279	10.49.128.113	10.49.128.113	SNMP	noError	77	128	161	52262	get-response	989718
4574	80.193796	2.996487	10.49.128.113	10.49.128.113	SNMP	noError	76	128	52263	161	get-request	989719

```
> Frame 2406: 76 bytes on wire (608 bits), 76 bytes captured (608 bits) on interface \Device\NPF_Loopback, id 0
> Null/Loopback
> Internet Protocol Version 4, Src: 10.49.128.113 (10.49.128.113), Dst: 10.49.128.113 (10.49.128.113)
> User Datagram Protocol, Src Port: 161, Dst Port: 58568
v Simple Network Management Protocol
    version: v2c (1)
    community: public
  v data: get-response (2)
    v get-response
        request-id: 989705
        error-status: noError (0)
        error-index: 0
      v variable-bindings: 1 item
        v 1.3.6.1.2.1.25.1.5.0: 92
            Object Name: 1.3.6.1.2.1.25.1.5.0 (iso.3.6.1.2.1.25.1.5.0)
            Value (Gauge32): 92
    [Response To: 2405]
    [Time: 0.001138000 seconds]
```

Simple Network Management Protocol (SNMP)

Get-Response Message: no such instance



Simple Network Management Protocol (SNMP)

SNMP message types

GetRequest – The most common SNMP message that an SNMP manager sends out to request data. The targeted device returns the requested value with a Response message.

GetNextRequest – The SNMP manager can send this message type to discover what information is available from the device. By starting at OID 0, the manager can continue to send a request for the next available data until there is no more "next" data.

GetBulkRequest – Added in SNMP Version 2, this is a newer, optimized version of GetNextRequest

SetRequest – This is a manager-initiated command to set or change the value of a parameter via SNMP on the agent device.

Response – The Response is the message that a device agent sends upon a Request from the manager. When sent in response to a GetRequest type, the packet contains the requested data or values.

Trap – A trap is an unsolicited message sent by the SNMP agent without having been requested by the manager. Often used if a defined threshold value is exceeded.

InformRequest – This message type was added in SNMP v2 to give the manager the possibility to confirm that it received an agent's trap message. Some agents are configured to continue to send a trap until an inform message is received.

Report – SNMP v3 is needed to use Report messages. They allow an SNMP manager to determine what kind of problem was detected by the remote SNMP agent.

Source: www.paessler.com

Simple Network Management Protocol (SNMP)

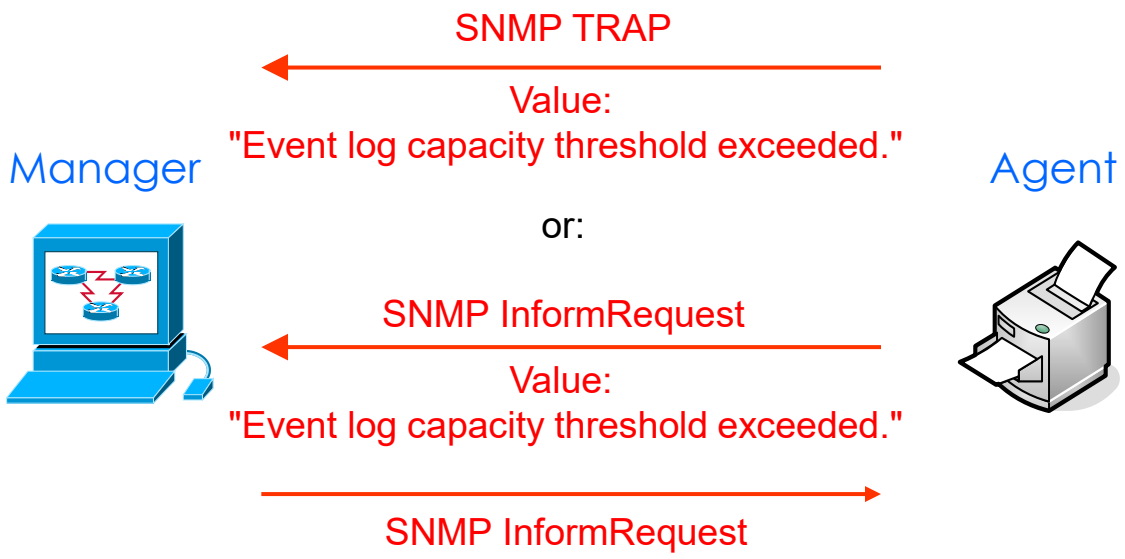
SNMP TRAP Message

A trap is an unsolicited message sent by an SNMP agent without having been requested by the manager. Used to report error conditions or threshold violations.

The manager does not confirm traps; therefore the transmission is not granted.

For reliable transmission, an InformRequest is used instead.

The reception of a InformRequest is acknowledged by the manager.



Simple Network Management Protocol (SNMP)

SNMP TRAP Message

SNMP_Trap.pcap

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

snmp.data == 4

No.	Time	Delta Time	Source	Destination	Protocol	error-status	Length	Time to live	Source Port	Destination Port	Data	request-id	Info
8	0.129419	0.000000	172.16.196.1	172.16.138.28	SNMP		123	47	161	162	trap		trap iso.3.6.1.4.1.64
35	0.279813	0.150394	172.16.196.1	172.16.138.28	SNMP		125	47	161	162	trap		trap iso.3.6.1.4.1.64
201	1.292525	1.012712	172.16.196.1	172.16.138.28	SNMP		123	47	161	162	trap		trap iso.3.6.1.4.1.64
227	1.437052	0.144527	172.16.196.1	172.16.138.28	SNMP		125	47	161	162	trap		trap iso.3.6.1.4.1.64
345	2.100145	0.663093	172.16.196.1	172.16.138.28	SNMP		123	47	161	162	trap		trap iso.3.6.1.4.1.64
346	2.421450	0.321305	172.16.196.1	172.16.138.28	SNMP		125	47	161	162	trap		trap iso.3.6.1.4.1.64

```
> Frame 8: 123 bytes on wire (984 bits), 123 bytes captured (984 bits)
> Ethernet II, Src: DecLocal_00:cb:04 (aa:00:04:00:cb:04), Dst: Oracle_0d:b1:97 (08:00:20:0d:b1:97)
> Internet Protocol Version 4, Src: 172.16.196.1 (172.16.196.1), Dst: 172.16.138.28 (172.16.138.28)
> User Datagram Protocol, Src Port: 161, Dst Port: 162
v Simple Network Management Protocol
    version: version-1 (0)
    community: monitor
  v data: trap (4)
    v trap
        enterprise: 1.3.6.1.4.1.64 (iso.3.6.1.4.1.64)
        agent-addr: 172.16.196.1 (172.16.196.1)
        generic-trap: enterpriseSpecific (6)
        specific-trap: 101
        time-stamp: 286530243
      v variable-bindings: 1 item
        v 1.3.6.1.4.1.64.3.2.3: "Card  8: Card removed."
            Object Name: 1.3.6.1.4.1.64.3.2.3 (iso.3.6.1.4.1.64.3.2.3)
            Value (OctetString): "Card  8: Card removed."
```

NetFlow

Introduced by **Cisco** in 1996, **NetFlow** is used to collect data from network devices, such as routers, switches etc. and sending them to a collecting point for further analysing or to create statistics.

A typical NetFlow monitoring system consists of three main components:

Flow exporter (PRTG Sensors on Routers, Switches etc.): are monitoring device parameters and sending them to one or more flow collector(s).

Flow collector (PRTG Probes): pre-processes received flows and forwards to them to the analysis application.

Analysis application (PRTG Core Server): analyzes and stores received flow data. Creates reports, alerts etc.

Version	Comment
v1	First implementation, now obsolete, and restricted to IPv4 (without IP mask and AS Numbers).
v2	Cisco internal version, never released.
v3	Cisco internal version, never released.
v4	Cisco internal version, never released.
v5	Most common version, available (as of 2009) on many routers from different brands, but restricted to IPv4 flows.
v6	No longer supported by Cisco. Encapsulation information (?).
v7	Like version 5 with a source router field. Used (only?) on Cisco Catalyst switches.
v8	Several aggregation form, but only for information that is already present in version 5 records
v9	Template Based, available (as of 2009) on some recent routers. Mostly used to report flows like IPv6, MPLS, or even plain IPv4 with BGP nexthop.
v10	Used for identifying IPFIX. Although IPFIX is heavily based on NetFlow, v10 does not have anything to do with NetFlow.

Source: https://en.wikipedia.org/wiki/NetFlow

Up to Version 9, the NetFlow technology is based on Cisco's proprietary protocol.
The **IPFIX** protocol is derived from NetFlow v9, but is a vendor independent standard (RFC 5153)

© Leutert NetServices 2026 / TCP / V4.1f

NetFlow

NetFlow Version 9: Record Types

Template Record (FlowSet ID range ≥ 256)

- Templates records are used to describe the type and length of individual fields contained within a NetFlow data record
- Multiple templates can be used simultaneously by an export device
- Each template is marked with a unique Template ID
- Templates periodically expire if they are not refreshed.
- Templates can be refreshed in two ways: A template can be resent every N number of export packets. A template can also be sent on a timer, so that it is refreshed every N number of minutes. Both options are user configurable.
- A **Template FlowSet** is a collection of one or more template records, grouped together in an export packet.

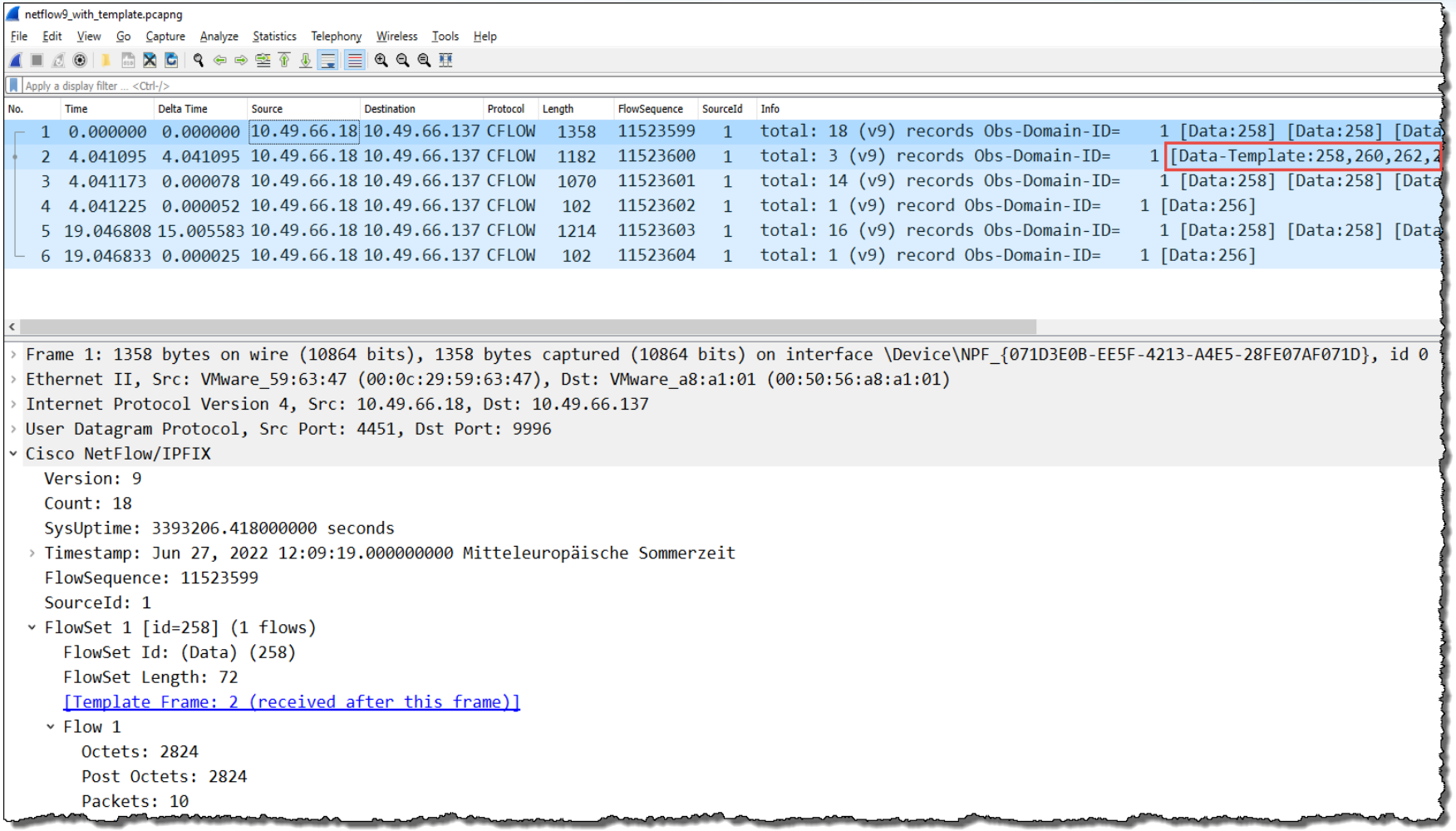
Data Record (FlowSet ID Range 0-255)

- Data records consists of fields containing data about an IP flow
- Each data record refers to a template ID and the fields are arranged according to the corresponding template
- A **Data FlowSet** is a collection of one ore more data records, grouped together in an export packet.

© Leutert NetServices 2026 / TCP / V4.1f

NetFlow

NetFlow Version 9: Data and Template Recors



NetFlow

NetFlow Version 9 Record Types

Data Record

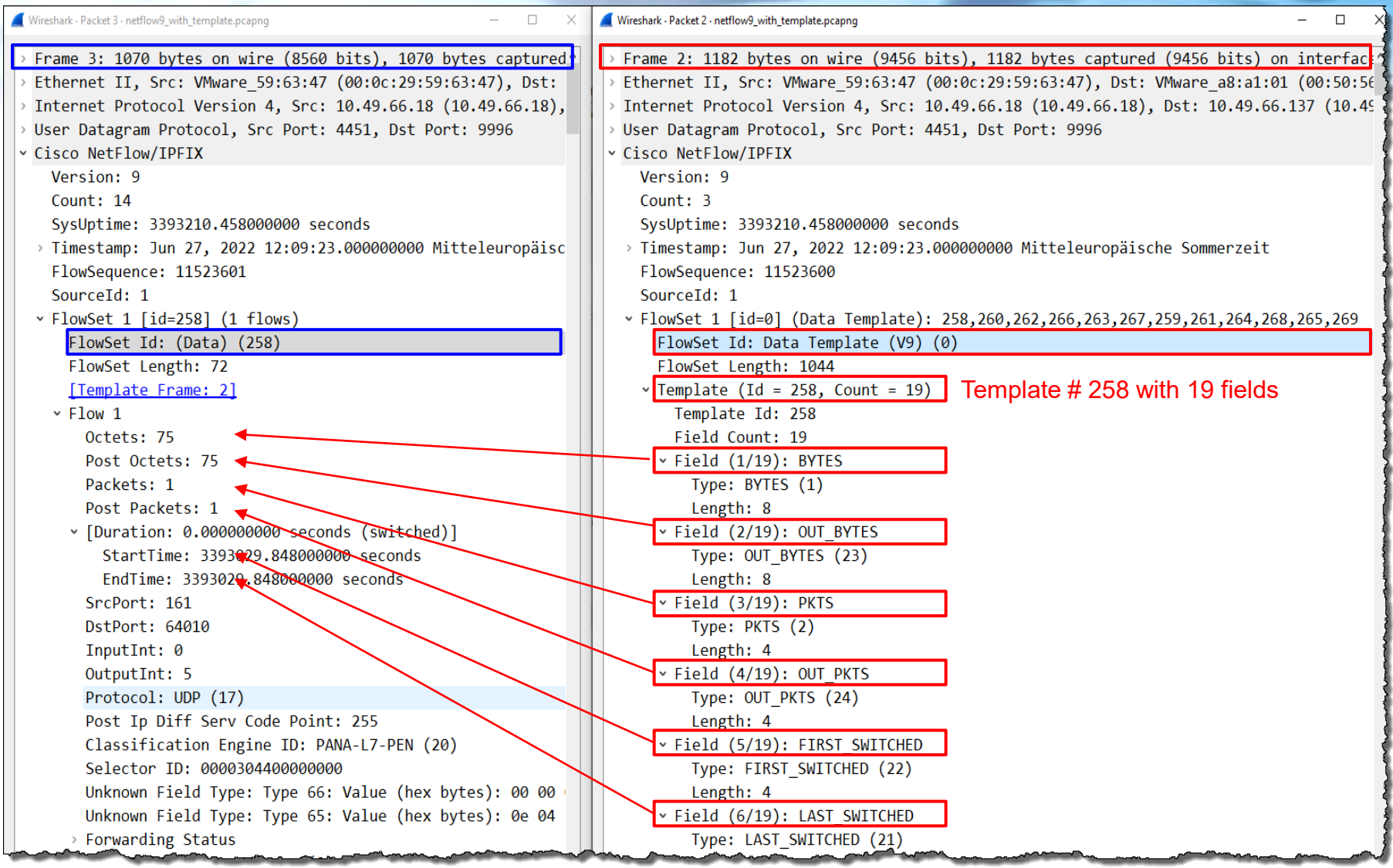
```
FlowSet ID: Data (256)
FlowSet Length: 52

StartTime: 7141.236000 seconds
EndTime: 7141.436000 seconds
Octets: 80
Packets: 2
InputInt: 1
OutputInt: 0
SrcAddr: 192.168.0.220
DstAddr: 255.255.255.255
Protocol: UDP (17)
IP ToS: 0x00
SrcPort: 53805
DstPort: 53805
SamplerID: 0
FlowClass: 0
NextHop: 0.0.0.0
DstMask: 32
SrcMask: 24
TCP Flags: 0x10, ACK
Direction: Ingress (0)
DstAS: 0
SrcAS: 0
```

Template Record (example)

```
FlowSet ID: Template
    Template Id: 256                    Field
    Field Count: 21                     Length
    Field (1/21): LAST_SWITCHED           4
    Field (2/21): FIRST_SWITCHED          4
    Field (3/21): BYTES                   4
    Field (4/21): PKTS                    4
    Field (5/21): INPUT_SNMP              2
    Field (6/21): OUTPUT_SNMP             2
    Field (7/21): IP_SRC_ADDR             4
    Field (8/21): IP_DST_ADDR             4
    Field (9/21): PROTOCOL                1
    Field (10/21): IP_TOS                 1
    Field (11/21): L4_SRC_PORT            2
    Field (12/21): L4_DST_PORT            2
    Field (13/21): FLOW_SAMPLER_ID        1
    Field (14/21): FLOW_CLASS             1
    Field (15/21): IP_NEXT_HOP            4
    Field (16/21): DST_MASK               1
    Field (17/21): SRC_MASK               1
    Field (18/21): TCP_FLAGS              1
    Field (19/21): DIRECTION              1
    Field (20/21): DST_AS                 2
    Field (21/21): SRC_AS                 2
```

NetFlow



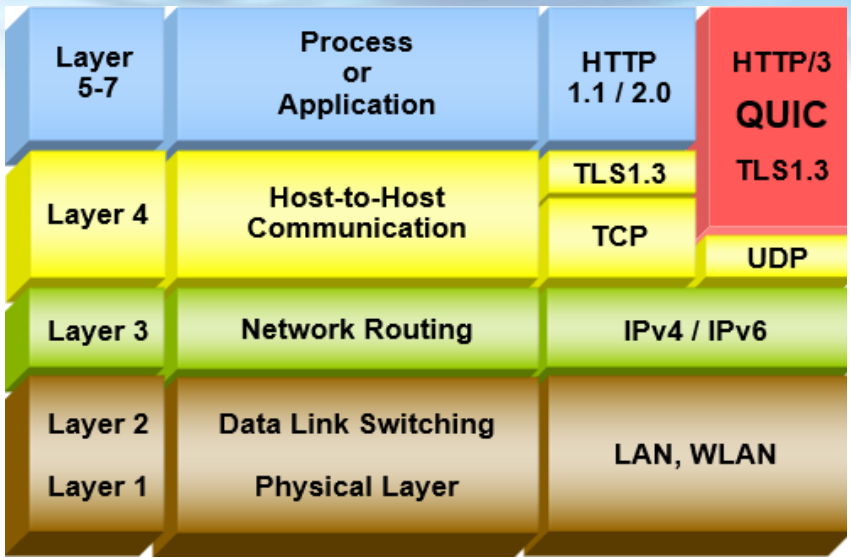
Data Record

Template Record

Quick UDP Internet Connections (QUIC)

TCP vs QUIC

- QUIC is a new network protocol that has great chances of replacing TCP, which is more than **50 years old**.
- Presented by Google in 2013, designed to speed up overall internet traffic.
- **IETF standard (RFC 9000) since May 2021**
- QUIC is currently supported by all popular browsers.
- QUIC breaks the layer model and combines several layers.



- QUIC integrates **TCP functions, HTTP/3 and TLS 1.3 encryption** (no longer an option)
- Faster connection setup due to fewer handshake cycles **(1-RTT)**
- Faster recovery in case of connection interruptions **(0-RTT)**
- Strong **TLS1.3 encryption**, except initial handshake
- Parallel connections (multiplexed streams) **within one QUIC packet**
- On the fly changes of client **IP address and UDP port** (roaming)
- No blocking of transmission due to lost packets **(no Head-of-line blocking)**
- Uses **UDP** instead of TCP. Acknowledgments, flow control etc. with QUIC Control Packets

QUIC

QUIC is already supported (and preferred!) by all popular browsers



- Some sources claim, that QUIC already accounts for up to 50% of the global network traffic.
- As early as October 2020, Facebook switched its servers and Android/IOS apps to QUIC. Currently, the QUIC share of Facebook traffic is already 75%.

  https://engineering.fb.com/2020/10/21/networking-traffic/how-facebook-is-bringing-quic-to-billions

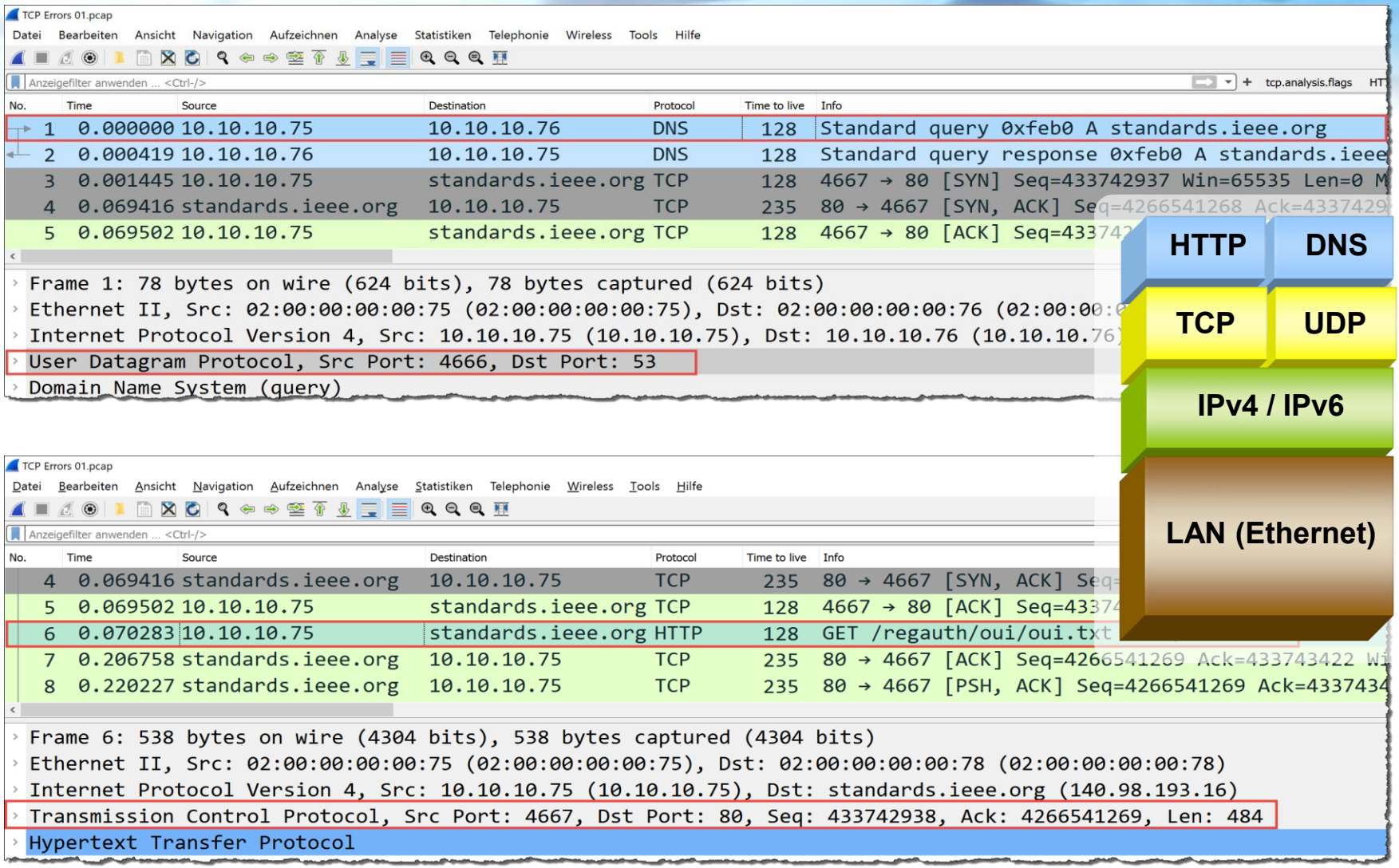
Why should we care?

- Because many network devices such as load balancers, traffic shapers, etc. are based on TCP.
- Because network security (e.g. firewalls) today is mostly TCP-oriented
- Because deep packet inspection (DPI) and intrusion detection systems (IDS) do not support QUIC.
- Because QUIC is already present in our networks and is preferred by many browsers.
- Because applications that are supposed to use H3/QUIC may have to be adapted.

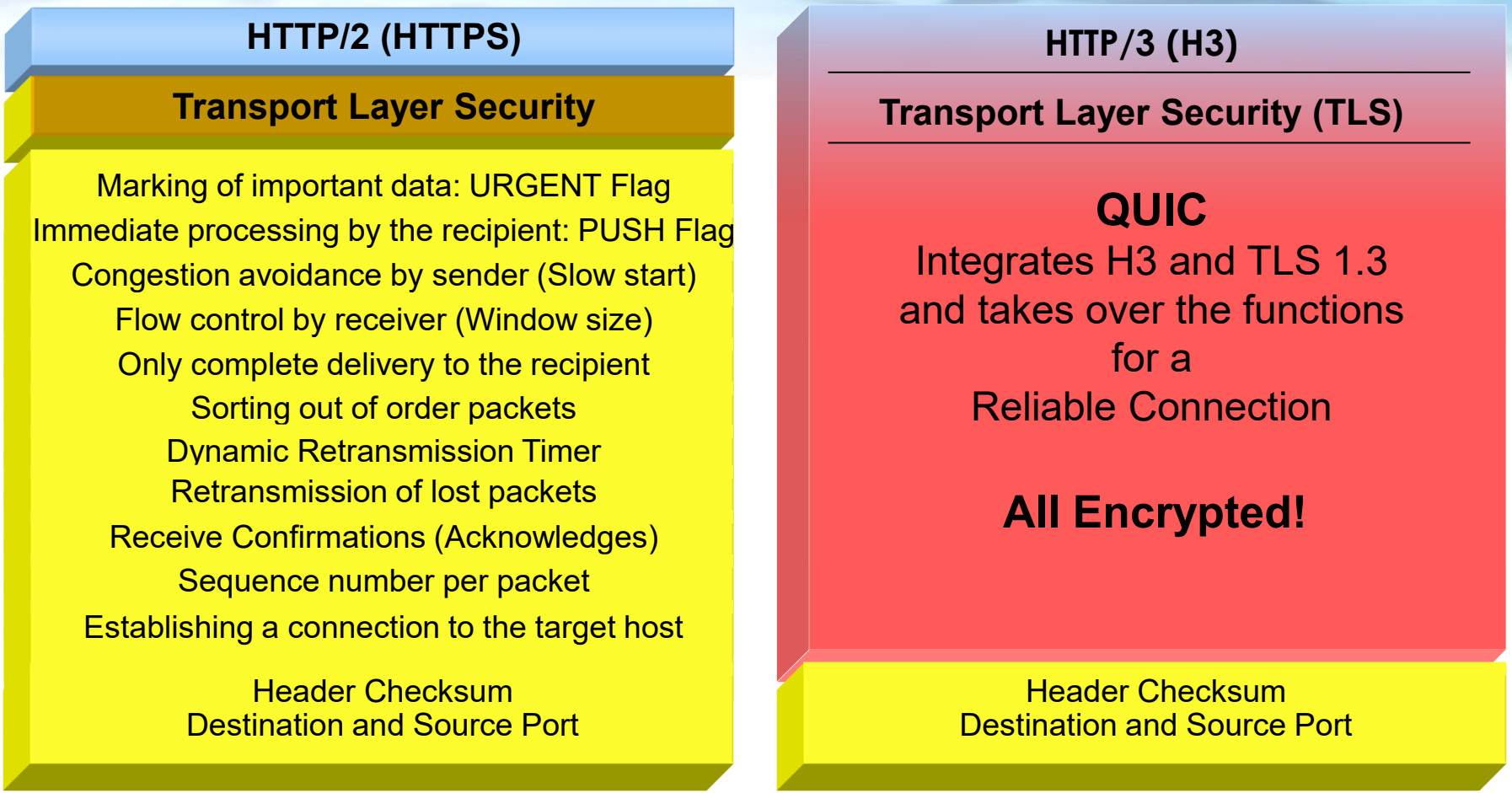
+

© Leutert NetServices 2026 / TCP / V4.1f

TCP vs. UDP


TCP Errors 01.pcap
Datei Bearbeiten Ansicht Navigation Aufzeichnen Analyse Statistiken Telephonie Wireless Tools Hilfe
Anzeigefilter anwenden ... <Ctrl-/>
tcp.analysis.flags
No. Time Source Destination Protocol Time to live Info
1 0.000000 10.10.10.75 10.10.10.76 DNS 128 Standard query 0xfeb0 A standards.ieee.org
2 0.000419 10.10.10.76 10.10.10.75 DNS 128 Standard query response 0xfeb0 A standards.ieee
3 0.001445 10.10.10.75 standards.ieee.org TCP 128 4667 → 80 [SYN] Seq=433742937 Win=65535 Len=0 M
4 0.069416 standards.ieee.org 10.10.10.75 TCP 235 80 → 4667 [SYN, ACK] Seq=4266541268 Ack=4337429
5 0.069502 10.10.10.75 standards.ieee.org TCP 128 4667 → 80 [ACK] Seq=433742
Frame 1: 78 bytes on wire (624 bits), 78 bytes captured (624 bits)
Ethernet II, Src: 02:00:00:00:00:75 (02:00:00:00:00:75), Dst: 02:00:00:00:00:76 (02:00:00:0
Internet Protocol Version 4, Src: 10.10.10.75 (10.10.10.75), Dst: 10.10.10.76 (10.10.10.76)
User Datagram Protocol, Src Port: 4666, Dst Port: 53
Domain Name System (query)
HTTP
DNS
TCP
UDP
IPv4 / IPv6
LAN (Ethernet)
TCP Errors 01.pcap
Datei Bearbeiten Ansicht Navigation Aufzeichnen Analyse Statistiken Telephonie Wireless Tools Hilfe
Anzeigefilter anwenden ... <Ctrl-/>
No. Time Source Destination Protocol Time to live Info
4 0.069416 standards.ieee.org 10.10.10.75 TCP 235 80 → 4667 [SYN, ACK] Seq=
5 0.069502 10.10.10.75 standards.ieee.org TCP 128 4667 → 80 [ACK] Seq=433742
6 0.070283 10.10.10.75 standards.ieee.org HTTP 128 GET /regauth/oui/oui.txt
7 0.206758 standards.ieee.org 10.10.10.75 TCP 235 80 → 4667 [ACK] Seq=4266541269 Ack=433743422 Wi
8 0.220227 standards.ieee.org 10.10.10.75 TCP 235 80 → 4667 [PSH, ACK] Seq=4266541269 Ack=4337434
Frame 6: 538 bytes on wire (4304 bits), 538 bytes captured (4304 bits)
Ethernet II, Src: 02:00:00:00:00:75 (02:00:00:00:00:75), Dst: 02:00:00:00:00:78 (02:00:00:00:00:78)
Internet Protocol Version 4, Src: 10.10.10.75 (10.10.10.75), Dst: standards.ieee.org (140.98.193.16)
Transmission Control Protocol, Src Port: 4667, Dst Port: 80, Seq: 433742938, Ack: 4266541269, Len: 484
Hypertext Transfer Protocol

TCP vs. UDP



Transmission Control Protocol

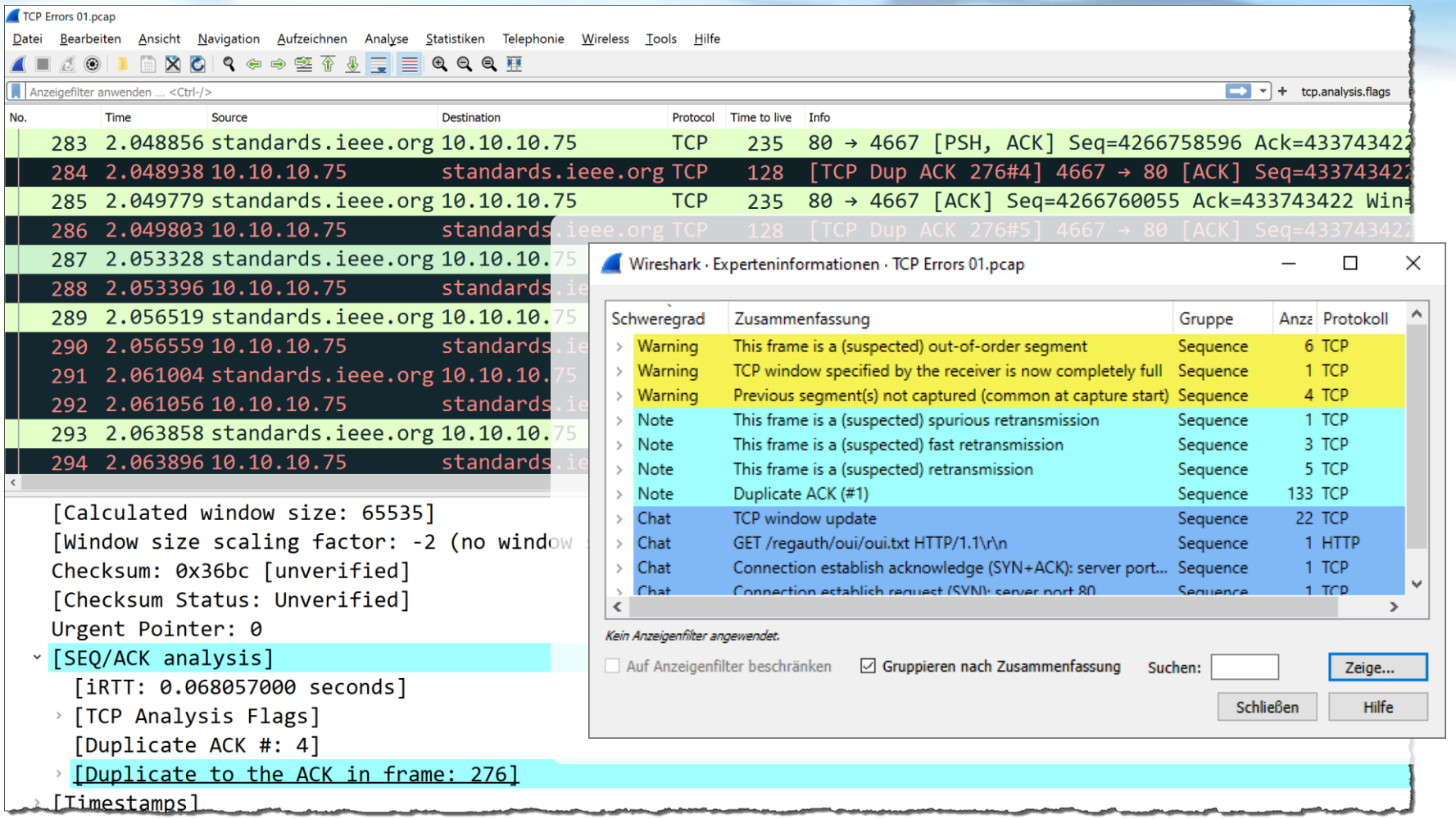
→ Reliable Connection

User Datagram Protocol & QUIC

→ Reliable Connection

+

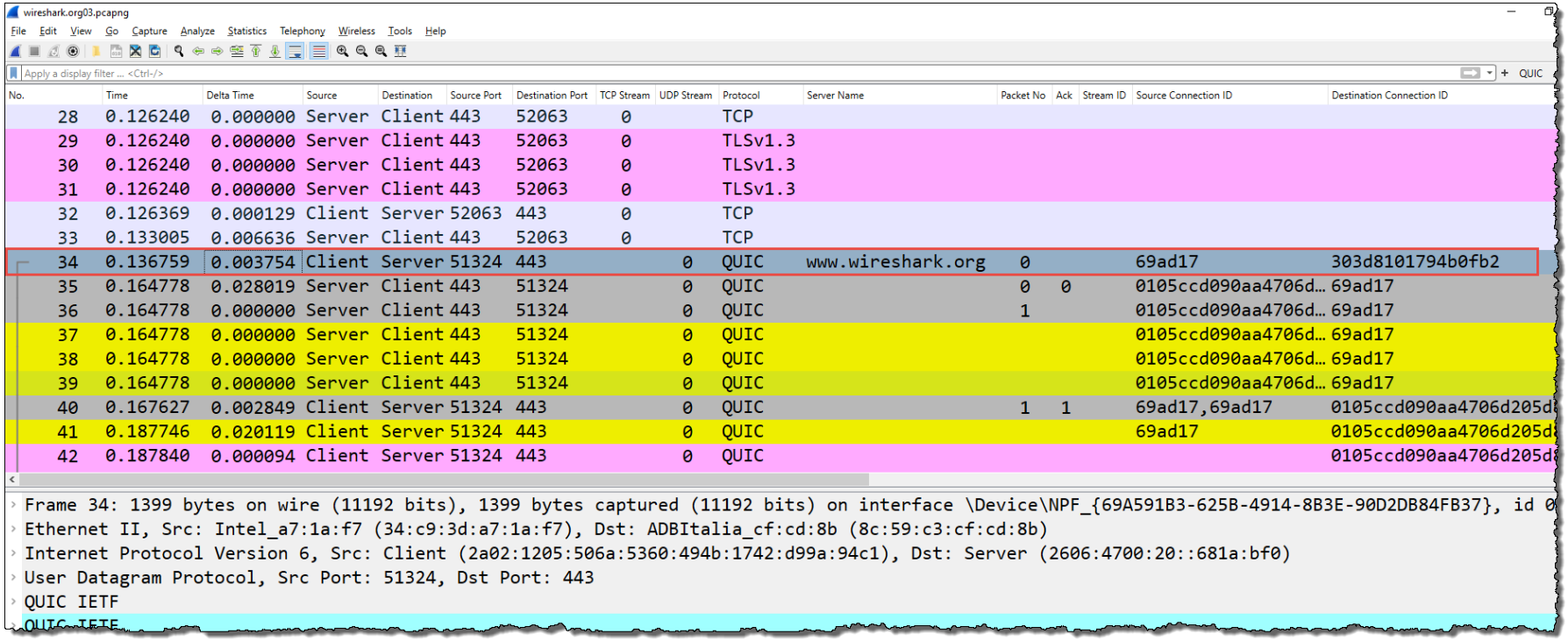
TCP vs. UDP


TCP Errors 01.pcap
Datei Bearbeiten Ansicht Navigation Aufzeichnen Analyse Statistiken Telephonie Wireless Tools Hilfe
Anzeigefilter anwenden ... <Ctrl-/>
tcp.analysis.flags
No. Time Source Destination Protocol Time to live Info
283 2.048856 standards.ieee.org 10.10.10.75 TCP 235 80 → 4667 [PSH, ACK] Seq=4266758596 Ack=433743422
284 2.048938 10.10.10.75 standards.ieee.org TCP 128 [TCP Dup ACK 276#4] 4667 → 80 [ACK] Seq=433743422
285 2.049779 standards.ieee.org 10.10.10.75 TCP 235 80 → 4667 [ACK] Seq=4266760055 Ack=433743422 Win=
286 2.049803 10.10.10.75 standards.ieee.org TCP 128 [TCP Dup ACK 276#5] 4667 → 80 [ACK] Seq=433743422
287 2.053328 standards.ieee.org 10.10.10.75
288 2.053396 10.10.10.75 standards.ie
289 2.056519 standards.ieee.org 10.10.10.75
290 2.056559 10.10.10.75 standards.ie
291 2.061004 standards.ieee.org 10.10.10.75
292 2.061056 10.10.10.75 standards.ie
293 2.063858 standards.ieee.org 10.10.10.75
294 2.063896 10.10.10.75 standards.ie
[Calculated window size: 65535]
[Window size scaling factor: -2 (no windo
Checksum: 0x36bc [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
[SEQ/ACK analysis]
[iRTT: 0.068057000 seconds]
[TCP Analysis Flags]
[Duplicate ACK #: 4]
[Duplicate to the ACK in frame: 276]
[Timestamps]
Wireshark · Experteninformationen · TCP Errors 01.pcap
Schweregrad Zusammenfassung Gruppe Anza Protokoll
Warning This frame is a (suspected) out-of-order segment Sequence 6 TCP
Warning TCP window specified by the receiver is now completely full Sequence 1 TCP
Warning Previous segment(s) not captured (common at capture start) Sequence 4 TCP
Note This frame is a (suspected) spurious retransmission Sequence 1 TCP
Note This frame is a (suspected) fast retransmission Sequence 3 TCP
Note This frame is a (suspected) retransmission Sequence 5 TCP
Note Duplicate ACK (#1) Sequence 133 TCP
Chat TCP window update Sequence 22 TCP
Chat GET /regauth/oui/oui.txt HTTP/1.1\r\n Sequence 1 HTTP
Chat Connection establish acknowledge (SYN+ACK): server port... Sequence 1 TCP
Kein Anzeigenfilter angewendet.
Auf Anzeigenfilter beschränken
Gruppieren nach Zusammenfassung
Suchen:
Zeige...
Schließen
Hilfe

QUIC (Protocol change from von **TCP** to **QUIC** on the fly)

- Most browsers already **prefer QUIC over TCP** to contact a website
- If the server is **not reachable by QUIC** (if UDP port 443 is blocked), TCP is used.
- QUIC can also **take over** an existing TCP session **without losing the connection**

Trace file is encrypted with TLSv1.3



Frame 34:** Connection changes from **TCP/TLSv1.3** to **H3/QUIC

+

- Without the session key, **QUIC packet numbers** are only visible during session setup

QUIC (Protocol change from von **TCP** to **QUIC** on the fly)

Use the following steps to decrypt the trace file

Adding the sslkey to Wireshark:

→ Edit

→ Preferences

→ Protocols

→ TLS

→ Browse to **sslkey.log**

→ Select OK

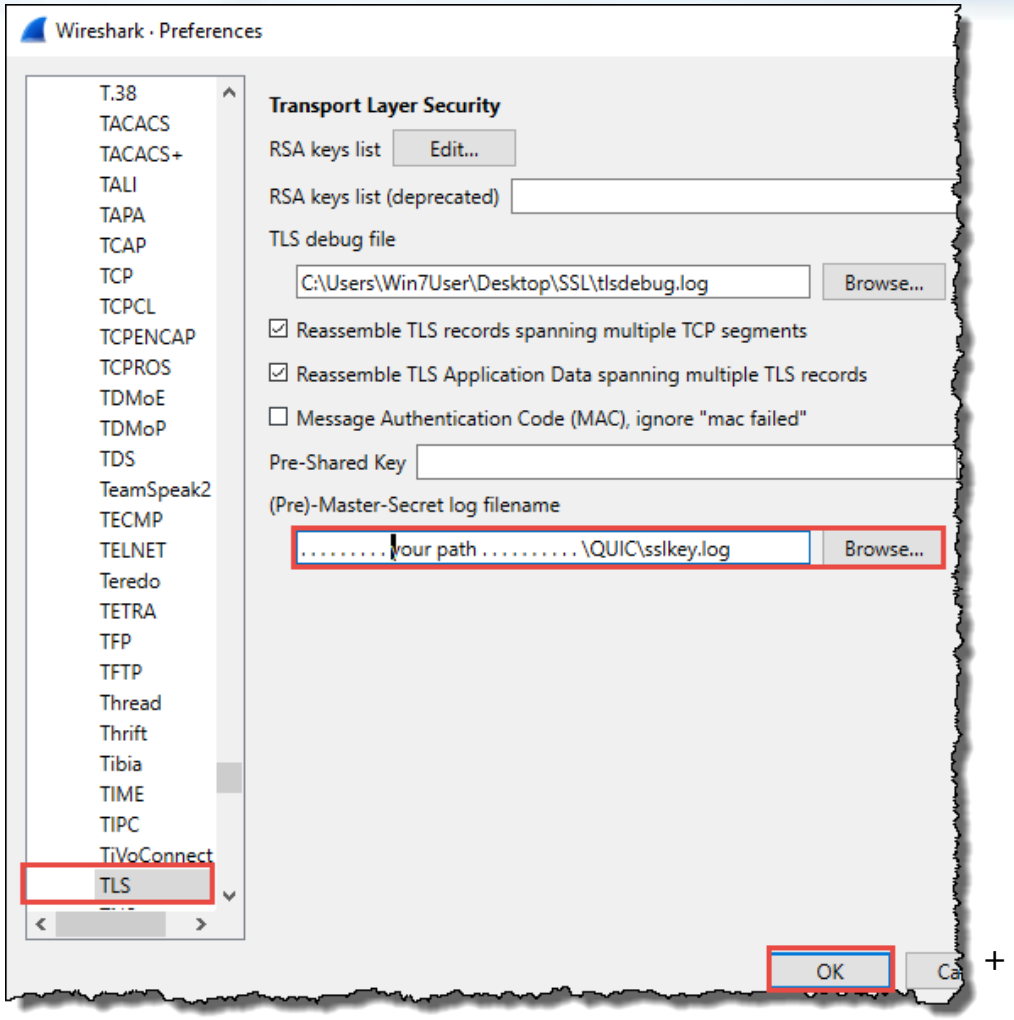
Inserting the sslkey into the trace file:

→ Edit

→ Inject TLS Secrets

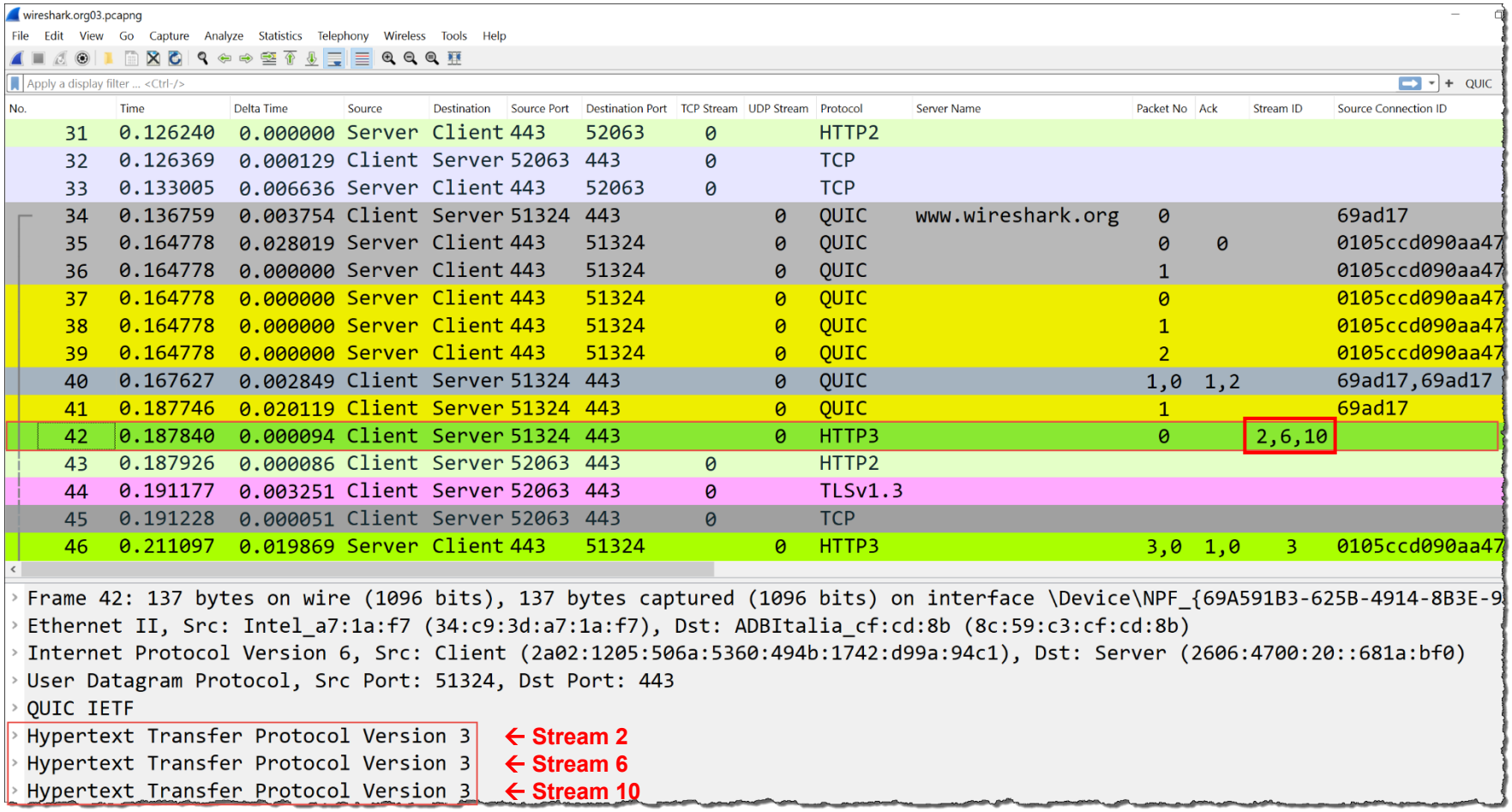
→ Save the trace file

The trace file can now be opened without the sslkey.log file



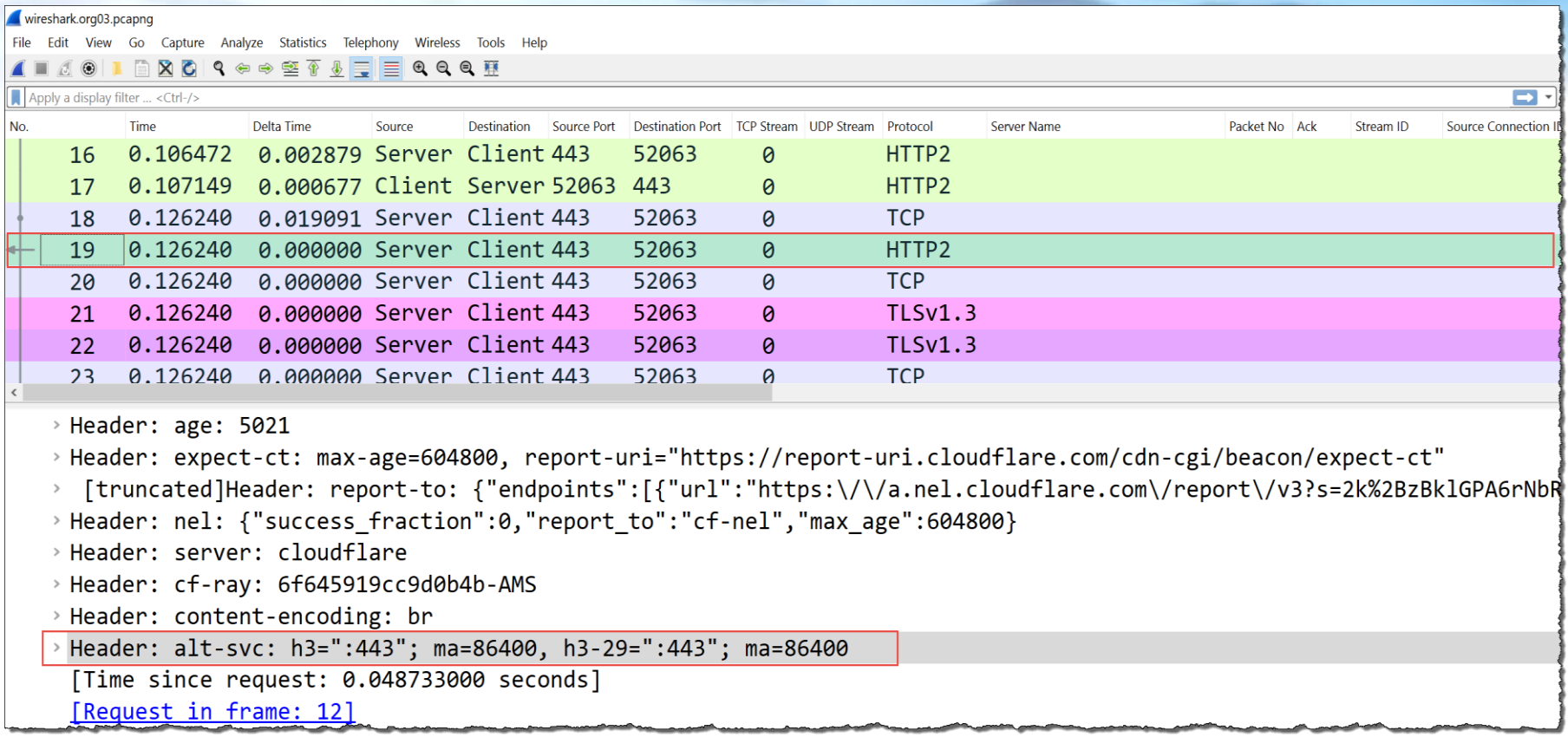
QUIC (Protocol change from von **TCP** to **QUIC** on the fly)

- The decrypted file shows **QUIC packet numbers, HTTP/2/3 payload, Stream ID** etc.
- QUIC supports **multiple streams in one packet** (corresponds to several TCP sessions)



Decrypted file shows more details of QUIC connection

QUIC (Protocol change from von **TCP** to **QUIC** on the fly)

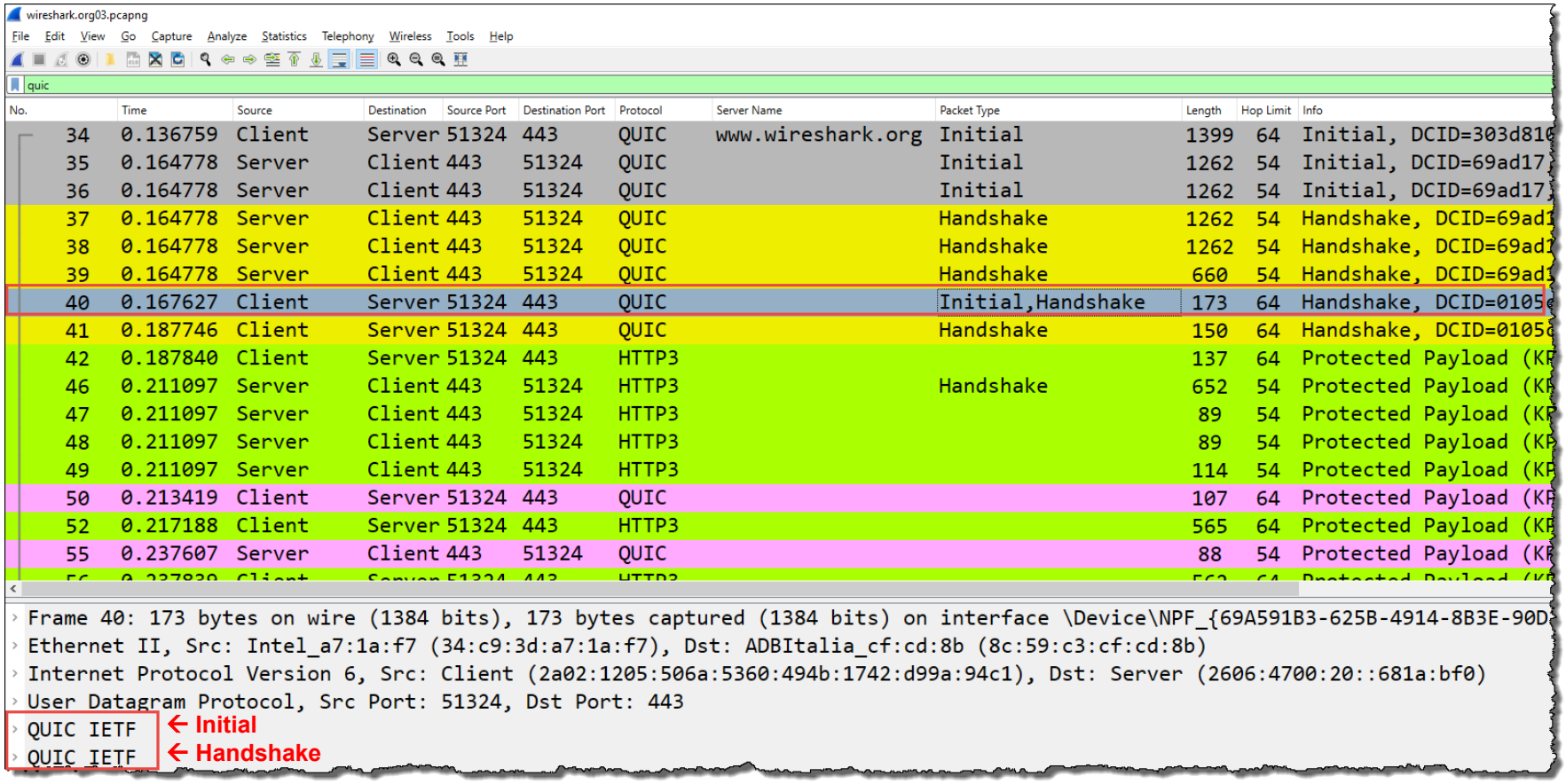


The optional **alt-svc HTTP Header Field** can advertise the availability of **alternative services to clients**

h3=“:443”;h3-29=“:443” → Supports **HTTP/3 RFC** and **HTTP/3 draft version 29,** each on **UDP-port 443**
ma=86400 → Max-age in seconds for which the service is considered fresh (default 86400 sec =24 h)

QUIC

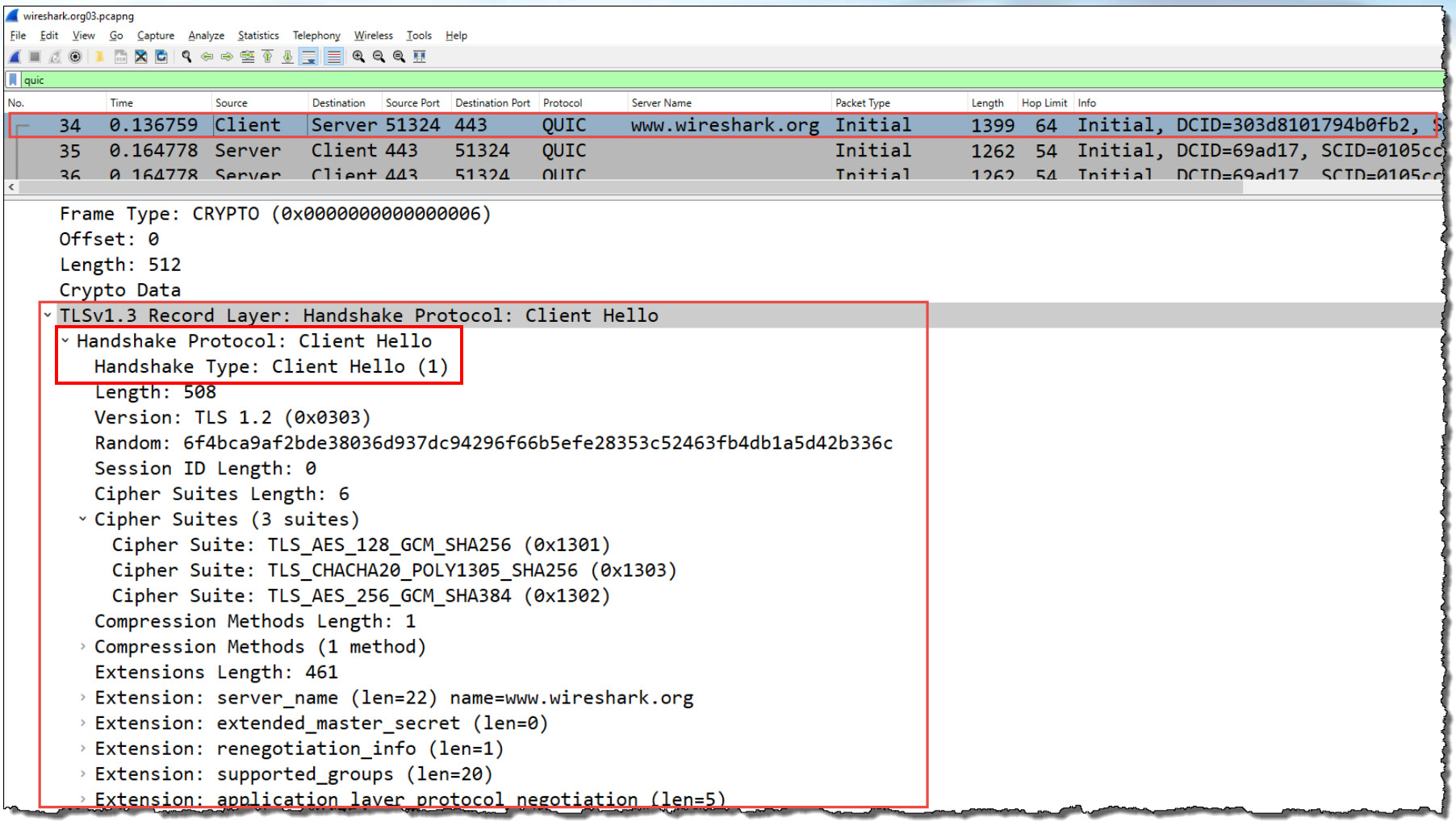
- QUIC uses three types of packets: **Initial, Handshake and Application data**
- Packets of different types can be **combined in the same frame**.



QUIC frame containing different packet types

QUIC

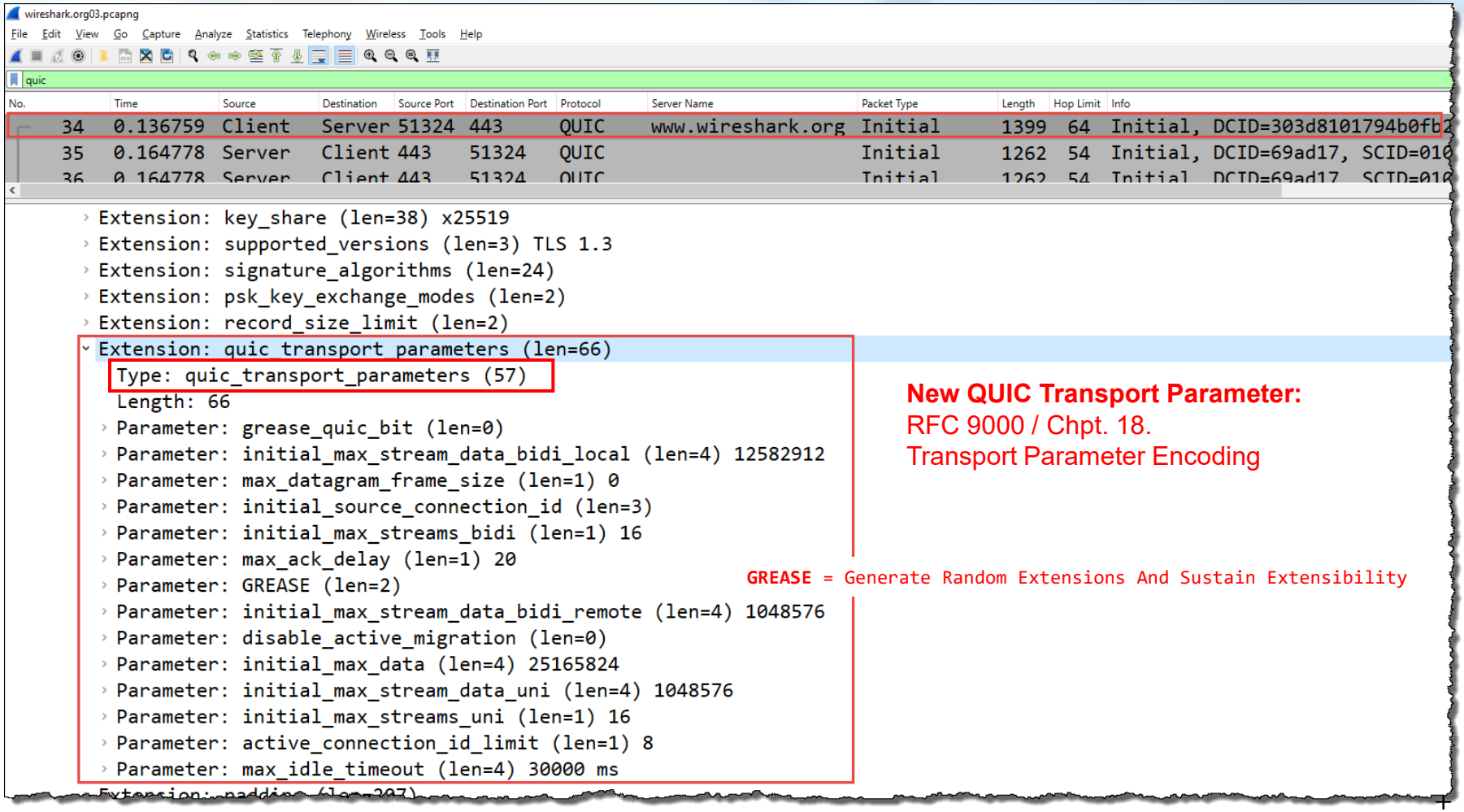
- The TLS1.3 handshake is **included** in the QUIC Initial packets



*The TLS1.3 handshake is included in the **QUIC initial** frames*

QUIC

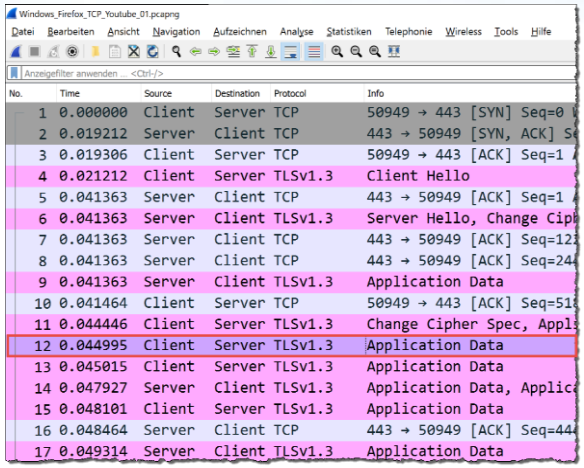
- QUIC adds its transport parameters into the TLS header

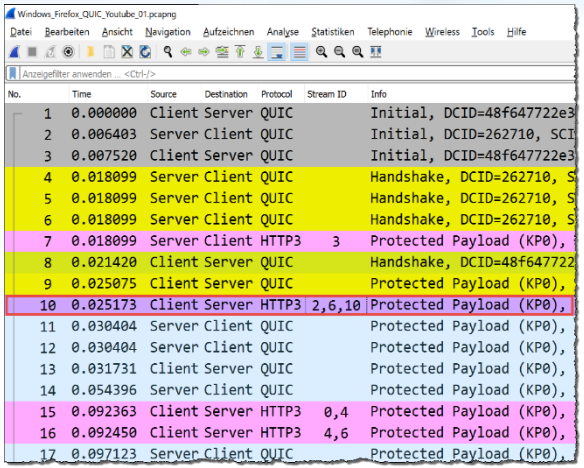


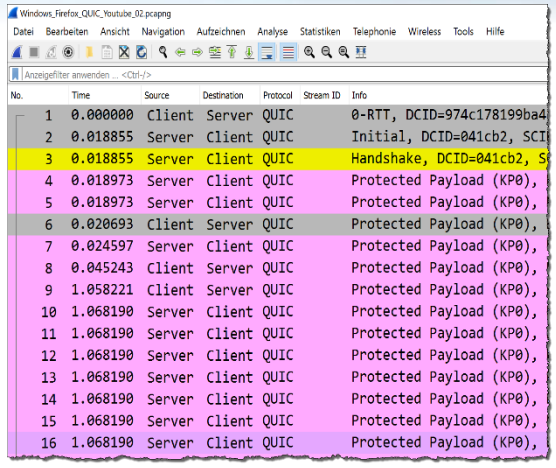
*The TLS header is extended with the **QUIC transport parameters***

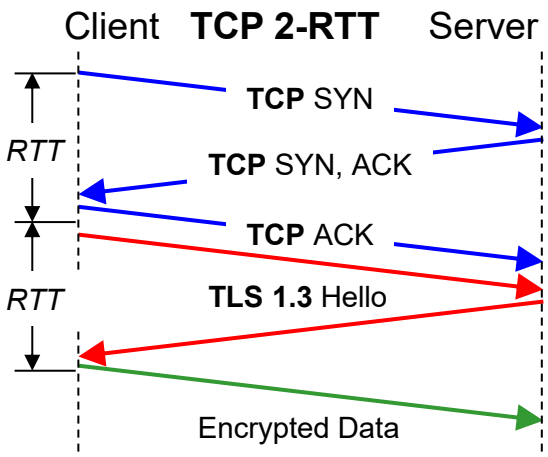
TCP vs. QUIC

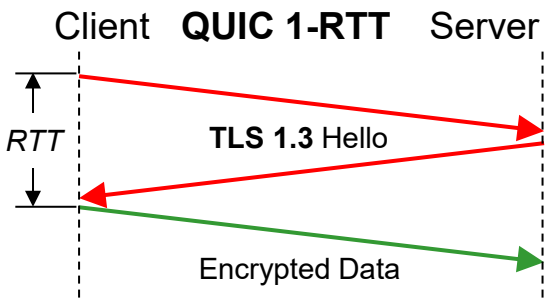
- Faster connection establishment with fewer handshake cycles

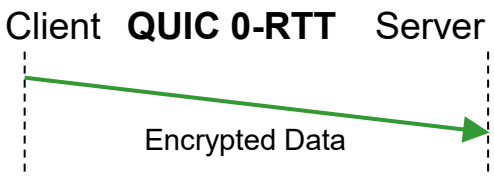










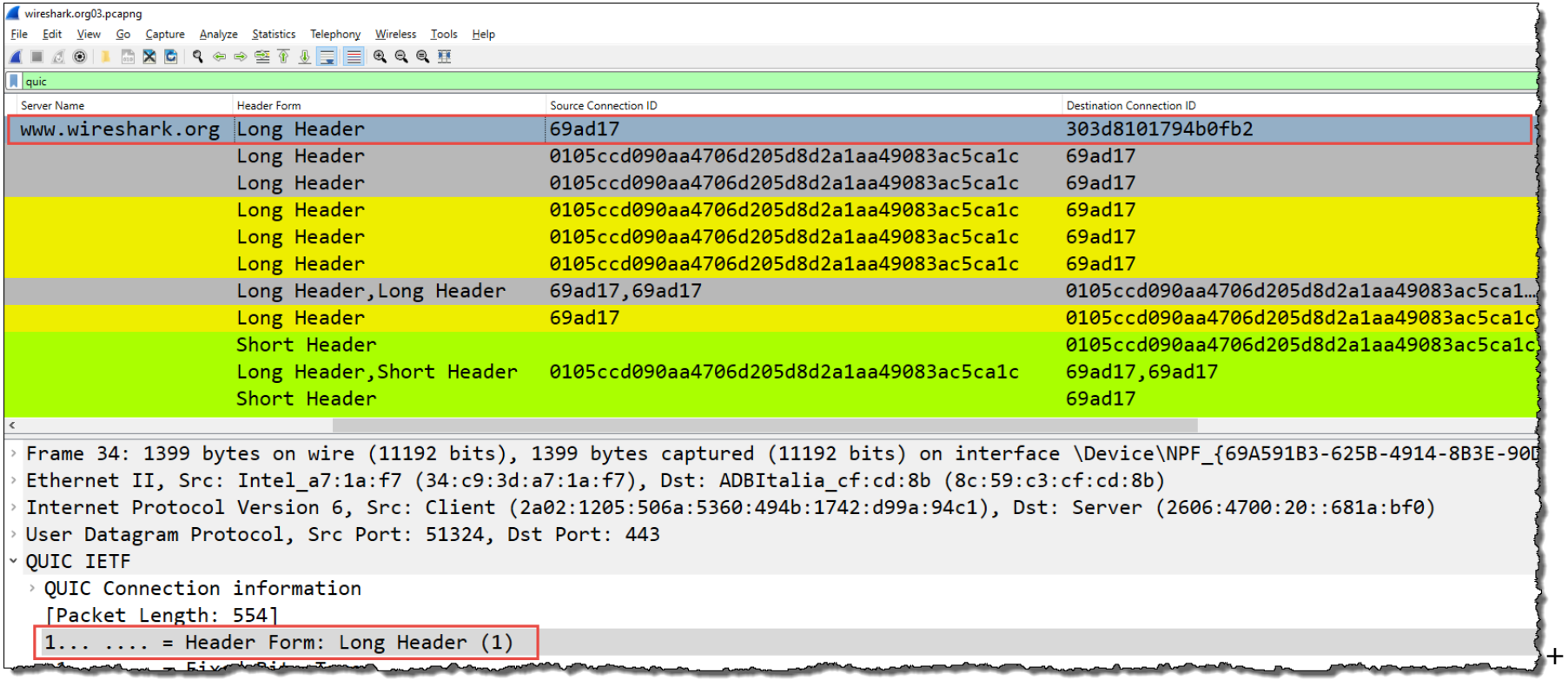


RTT = Round Trip Time
(Packet transit time from client to server and back)

+

QUIC Src. Connection ID (SCID) and Dst. Connection ID (DCID)

- QUIC is using an **own addressing scheme** with **Connection Identifier**
- Src. and Dst. connection ID (up to 20 bytes length) **are negotiated** at session start,
- **Long headers** use **both IDs**, **short headers** only contain the **Destination ID**
- This makes **QUIC independent from IP addresses and UDP Ports** (mobility) **!**
- **The client IP and UDP port could change without loosing the connection !**

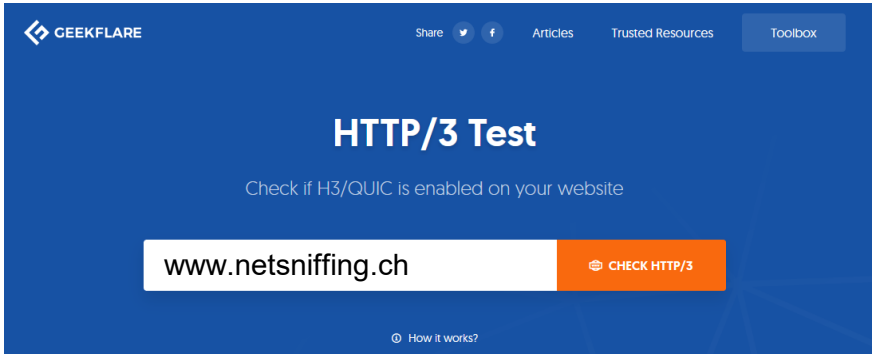


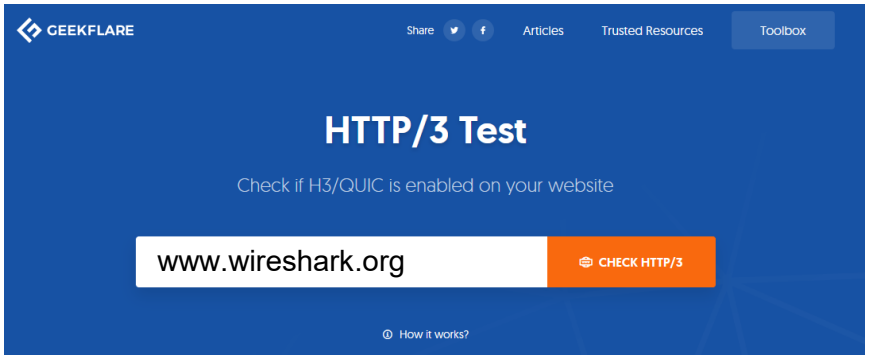
QUIC Source and Destination Connection Identifier

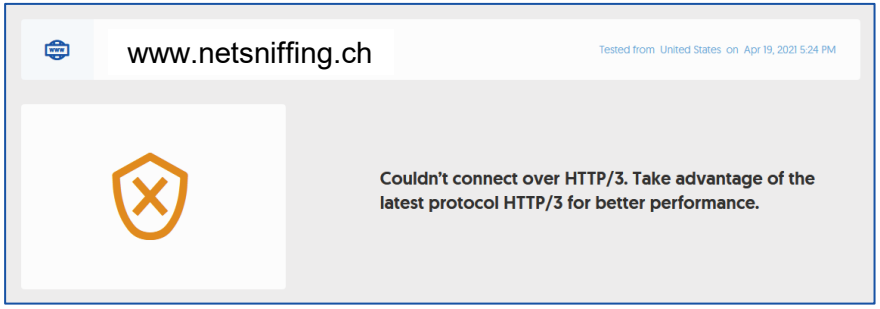
QUIC

Checking if a web server supports QUIC

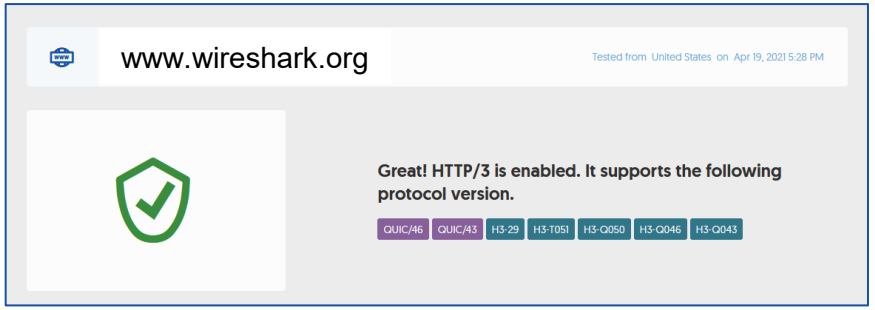
- Accessing the **HTTP/3** (QUIC) test page: https://gf.dev/http3-test
- Enter the **URL** of the web server to be tested and press the **CHECK HTTP/3** button







Sorry **NO**



OK

QUIC



New QUIC Protocol Certificate - TCP aged 50 years now replaced

- IETF Internet Standard QUIC rfc-9000 ratified June 2021 (near 1000 pages)
- QUIC's rapid growth near 50% of Internet, outstrips firewall support
- Blinding Firewalls, vendors recommend Enterprises "block" QUIC

From a weekend experiment in 2012 it become an IETF working group and QUIC was ratified as a standard IETF Transport Protocol Summer of 2021. In the words of a popular load balancer vendor, it seems "QUIC is eating the Internet".

QUIC's success is a good thing, but many enterprise networks don't yet have it managed to mitigate some of QUIC's network firewall issues, as most large firewall vendors, for now, recommend turning QUIC off by applying firewall deny filters. Despite these warnings, QUIC continues to gain adoption due to its compelling features.

+

Source: https://www.austincyber.show/post/ietf-s-new-tcp-quic-certificate

© Leutert NetServices 2026 / TCP / V4.1f

QUIC

Summary

- Since **QUIC is an IETF standard protocol**, the share of Internet traffic is **growing**
- QUIC traffic is fully **TLS1.3 encrypted**, except for a few frames at connection start
- QUIC needs **fewer handshake cycles** and even supports **Zero-Round-Trip-Time**
- Some sources claim, that QUIC is **up to three times faster** than TCP
- The client can **change IP and UDP port** (roaming) **without losing server connection**
- QUIC started with **HTTP/3,** but now also supports **SMB and DNS over QUIC**

Conclusions

- QUIC **blinds firewalls, intrusion detection** and other **TCP based security devices**
- Most **firewall** vendors do **not yet support QUIC** and recommend to **block UDP 443**
- Most **Load Balancer** and traffic regulation/prioritization devices are based on TCP

→ **Network analysing and troubleshooting** becomes very difficult without session key

Internet Control Message Protocol (ICMP)

ICMP messages inform about irregularities during data transmission on the way to the target host. They are generated by routers or (optionally) by clients.

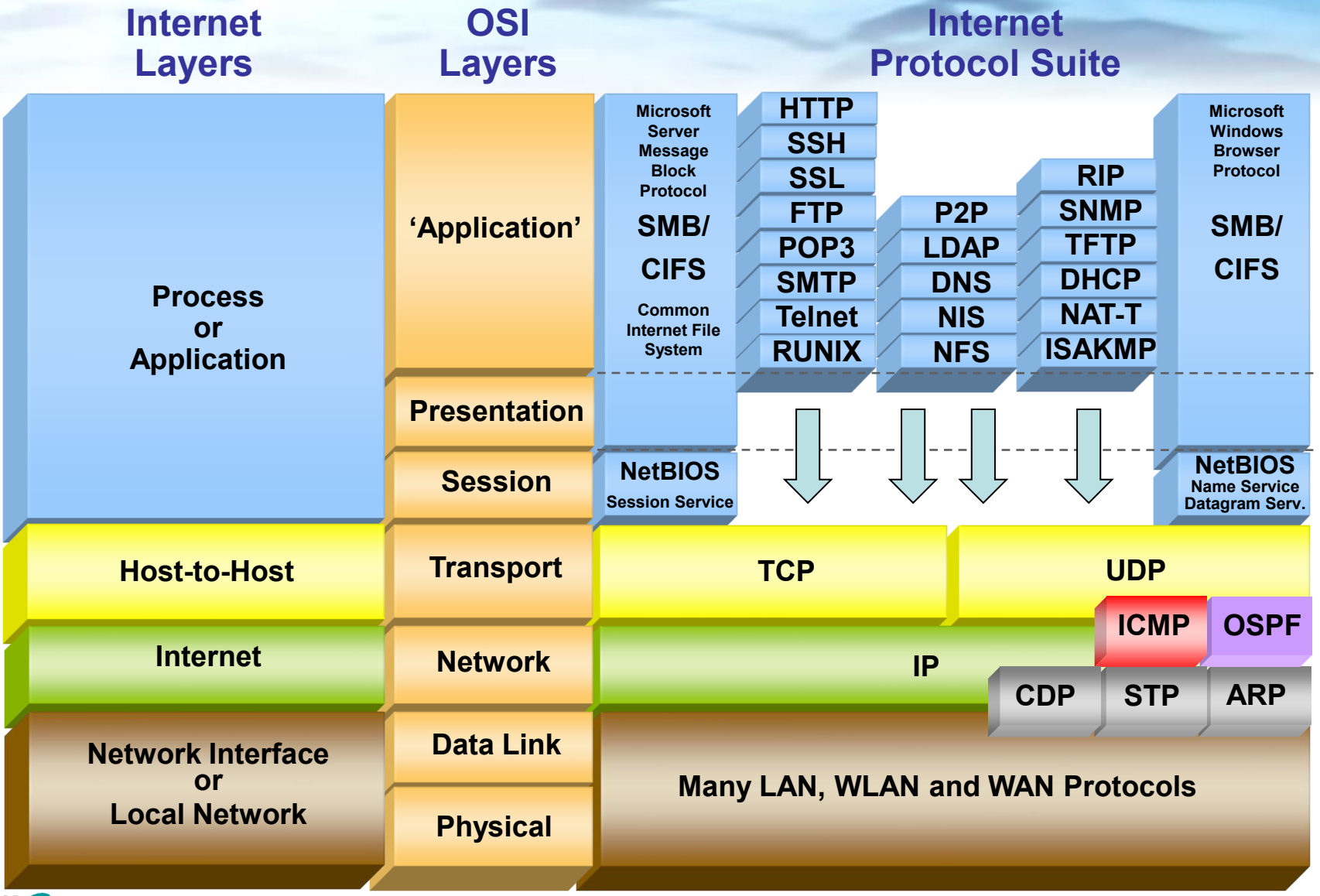
To understand these messages makes it easier to narrow-down problems in the routing area.

ICMP is also used to test a connection (PING).



© Leutert NetServices 2026 / TCP / V4.1f

Internet Control Message Protocol (ICMP)



Internet Control Message Protocol (ICMP)

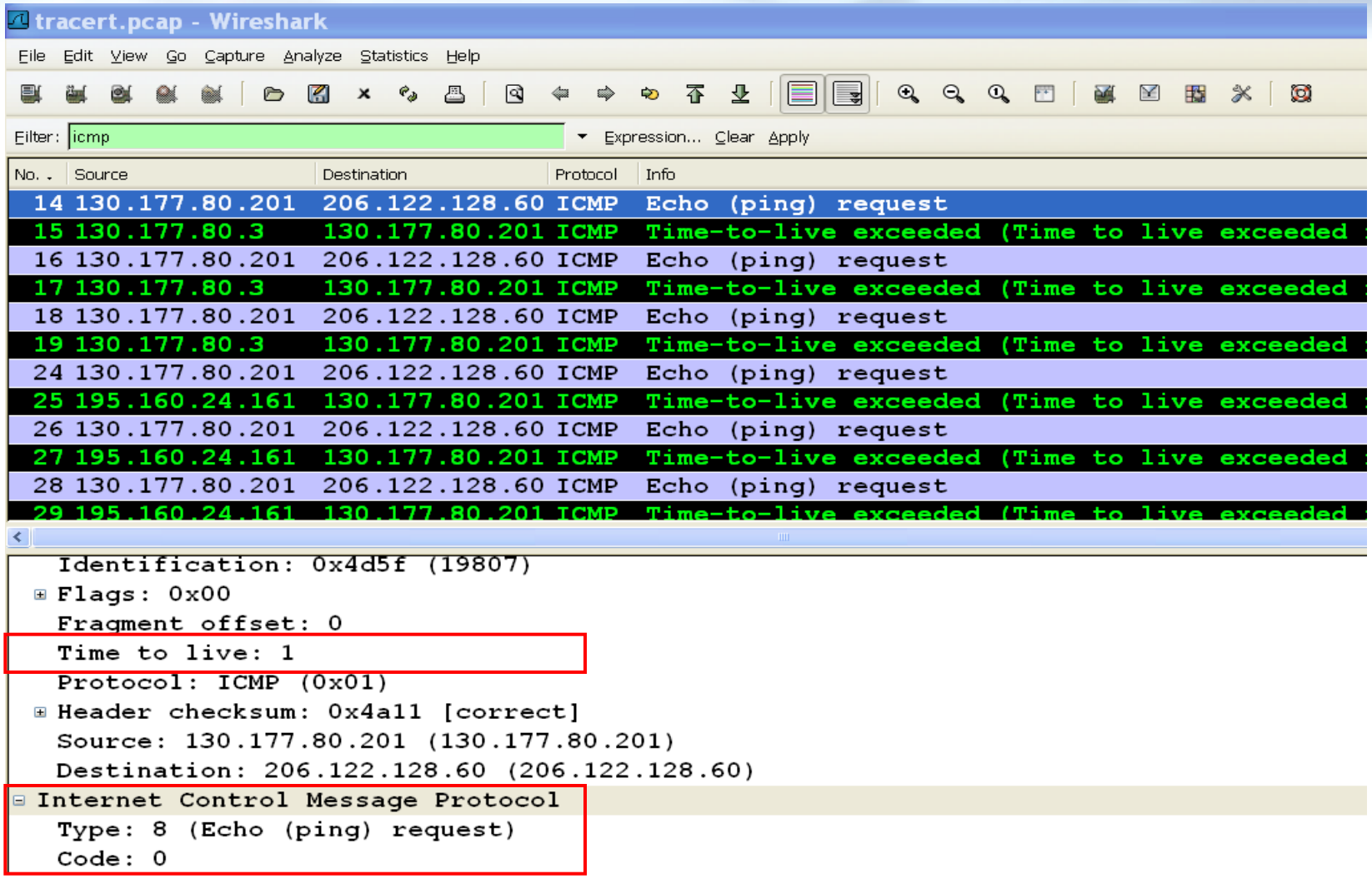
The most important ICMP messages:

- Echo Request/Reply ..... Ping Request/Reply
- TTL exceeded in transit ..... TTL has reached zero
- Redirect ..... There is no more direct path to the target host
- Source Quench ..... Router has buffer problems
- Destination unreachable
  - Destination Network unreachable ... Target net is not reachable
  - Destination Host unreachable ..... Target host is not reachable
  - Protocol unreachable ..... Protocol is not reachable
  - Port unreachable ..... Port is not reachable
  - Fragmentation needed, ... Target is not reachable since but don't fragment flag set fragmentation not allowed

© Leutert NetServices 2026 / TCP / V4.1f

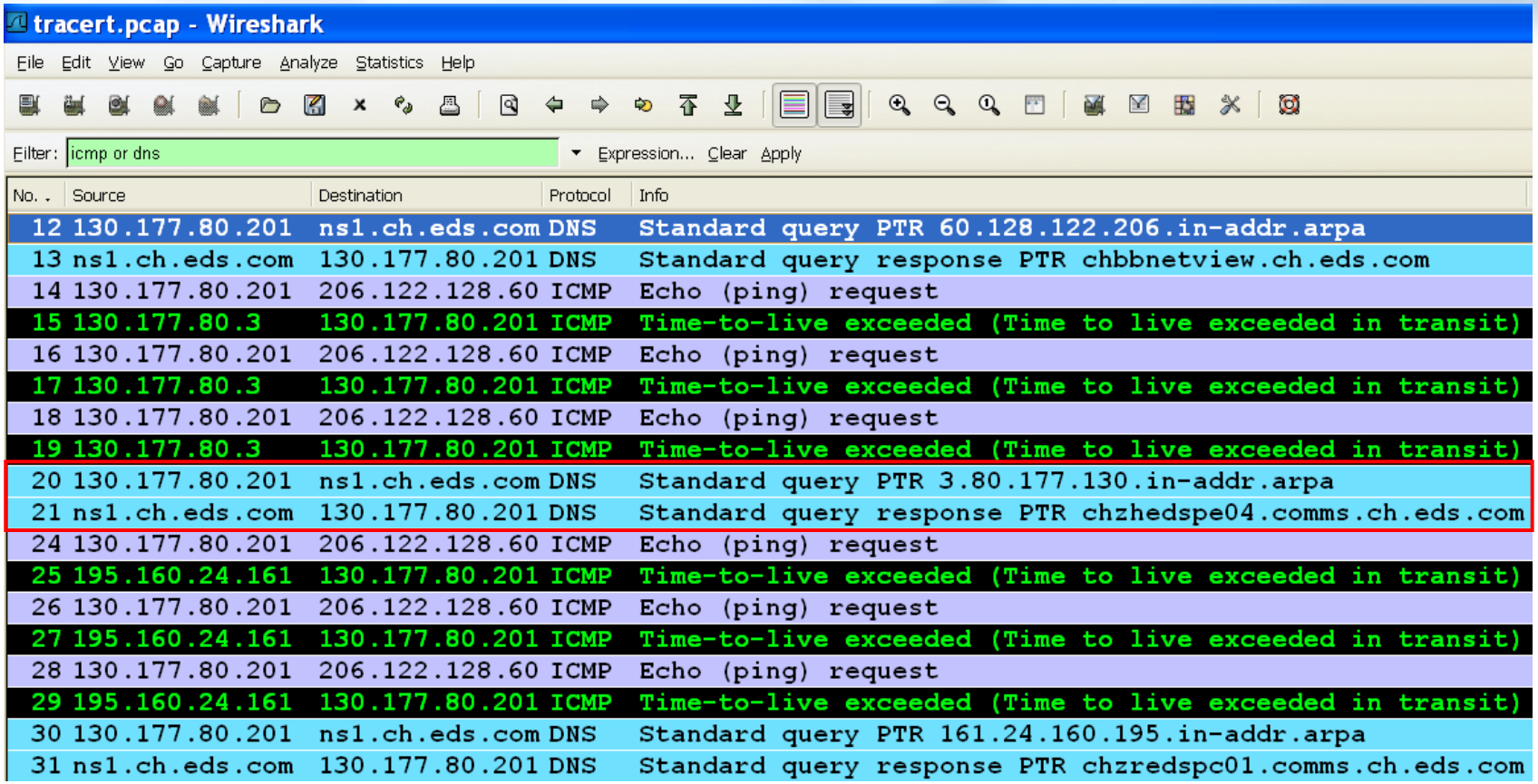
Internet Control Message Protocol (ICMP)

‚Trace Route' (Tracert) uses the ICMP ‚TTL exceeded' message



Internet Control Message Protocol (ICMP)

‚Trace Route' in combination with DNS

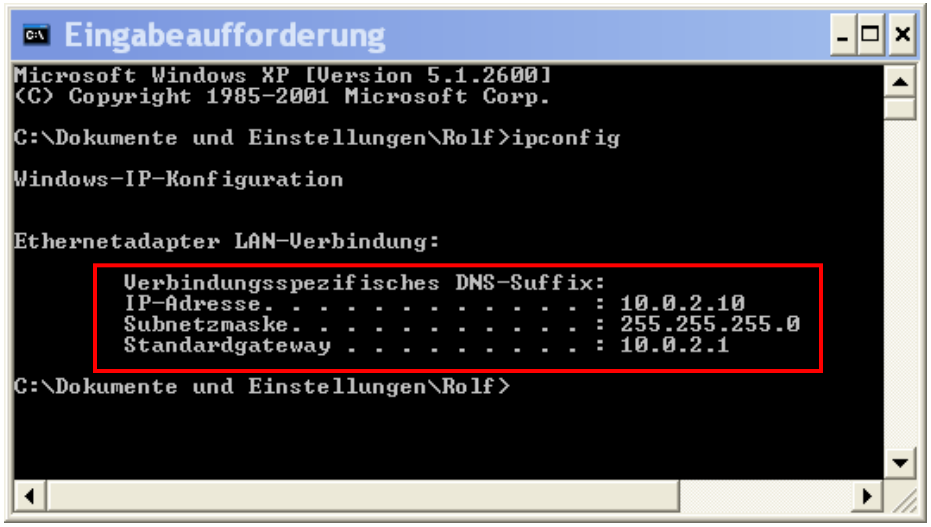


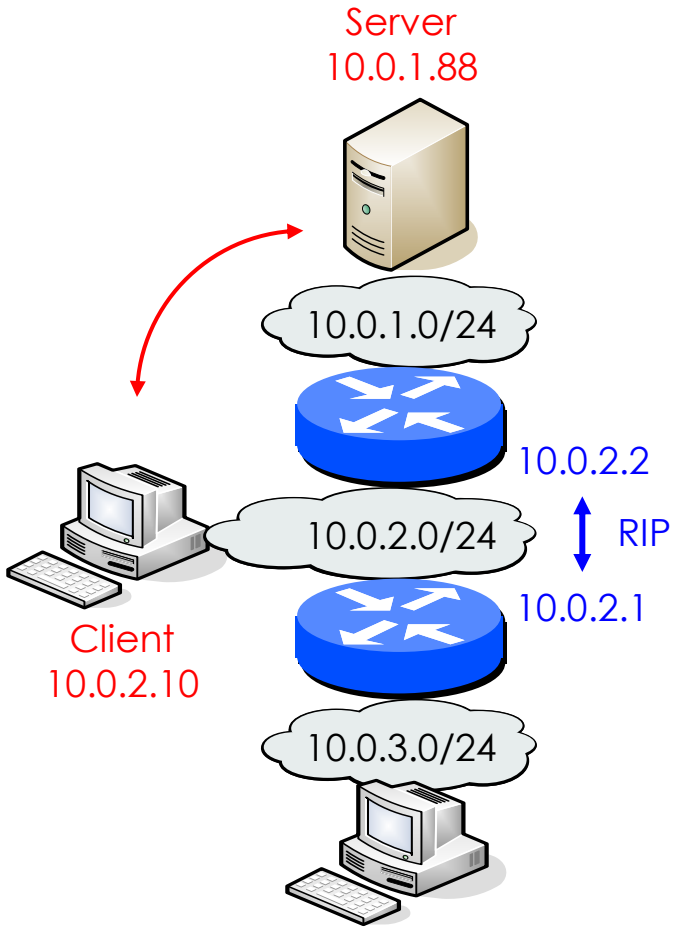
No.	Source	Destination	Protocol	Info
12	130.177.80.201	ns1.ch.eds.com	DNS	Standard query PTR 60.128.122.206.in-addr.arpa
13	ns1.ch.eds.com	130.177.80.201	DNS	Standard query response PTR chbbnetview.ch.eds.com
14	130.177.80.201	206.122.128.60	ICMP	Echo (ping) request
15	130.177.80.3	130.177.80.201	ICMP	Time-to-live exceeded (Time to live exceeded in transit)
16	130.177.80.201	206.122.128.60	ICMP	Echo (ping) request
17	130.177.80.3	130.177.80.201	ICMP	Time-to-live exceeded (Time to live exceeded in transit)
18	130.177.80.201	206.122.128.60	ICMP	Echo (ping) request
19	130.177.80.3	130.177.80.201	ICMP	Time-to-live exceeded (Time to live exceeded in transit)
20	130.177.80.201	ns1.ch.eds.com	DNS	Standard query PTR 3.80.177.130.in-addr.arpa
21	ns1.ch.eds.com	130.177.80.201	DNS	Standard query response PTR chzhedspe04.comms.ch.eds.com
24	130.177.80.201	206.122.128.60	ICMP	Echo (ping) request
25	195.160.24.161	130.177.80.201	ICMP	Time-to-live exceeded (Time to live exceeded in transit)
26	130.177.80.201	206.122.128.60	ICMP	Echo (ping) request
27	195.160.24.161	130.177.80.201	ICMP	Time-to-live exceeded (Time to live exceeded in transit)
28	130.177.80.201	206.122.128.60	ICMP	Echo (ping) request
29	195.160.24.161	130.177.80.201	ICMP	Time-to-live exceeded (Time to live exceeded in transit)
30	130.177.80.201	ns1.ch.eds.com	DNS	Standard query PTR 161.24.160.195.in-addr.arpa
31	ns1.ch.eds.com	130.177.80.201	DNS	Standard query response PTR chzredspc01.comms.ch.eds.com

Internet Control Message Protocol (ICMP)

‚ICMP Redirect' message

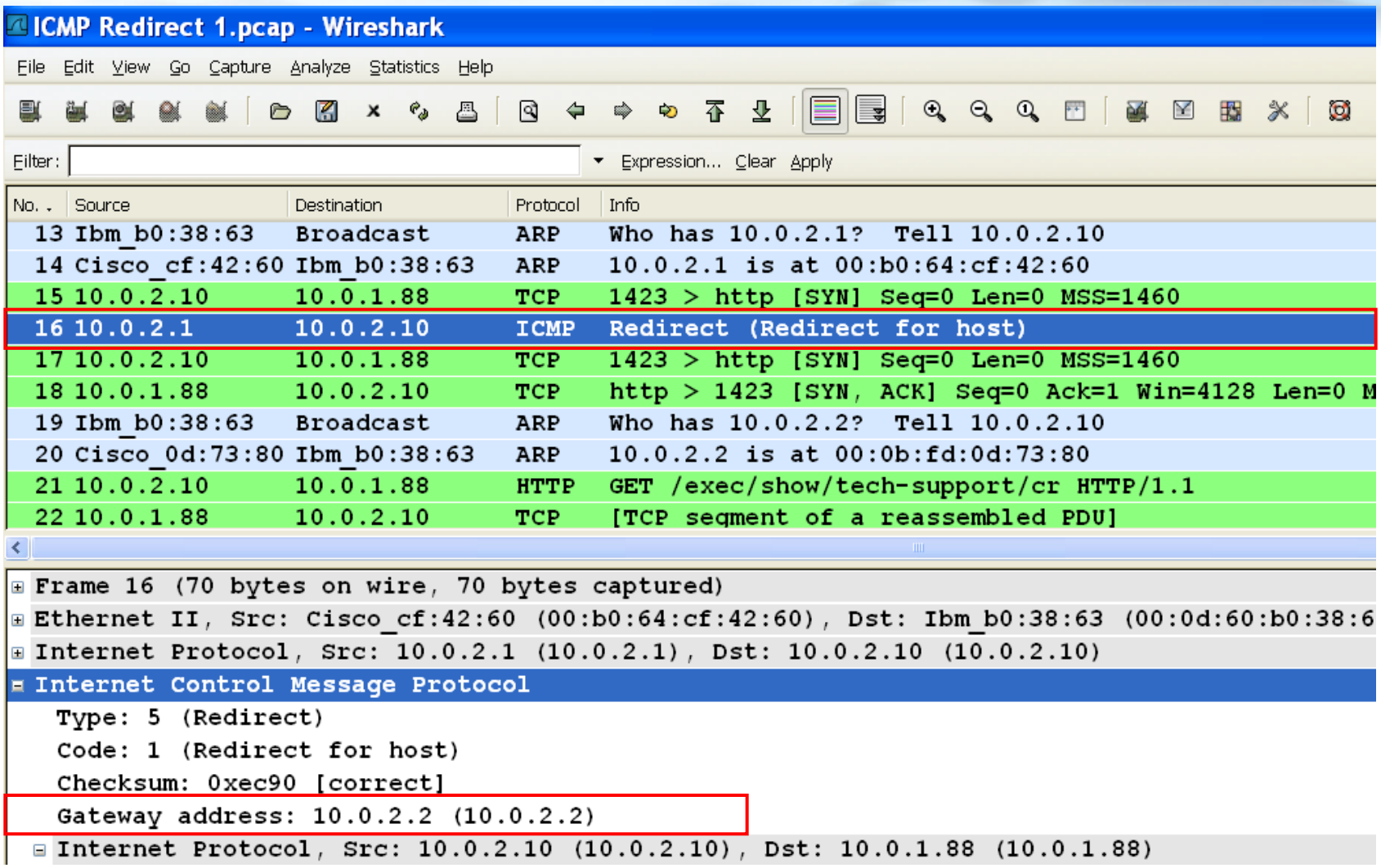
The client establishes a connection to the server.
Which are the particular steps?





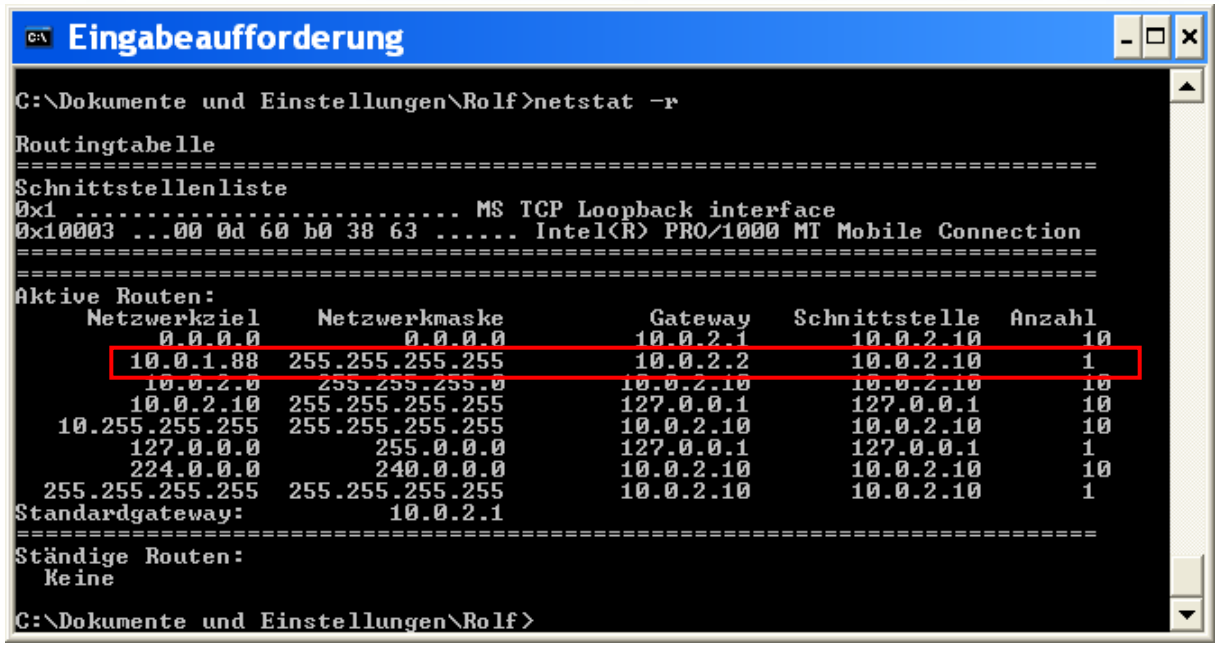
Internet Control Message Protocol (ICMP)

‚ICMP Redirect' message with address of more direct gateway

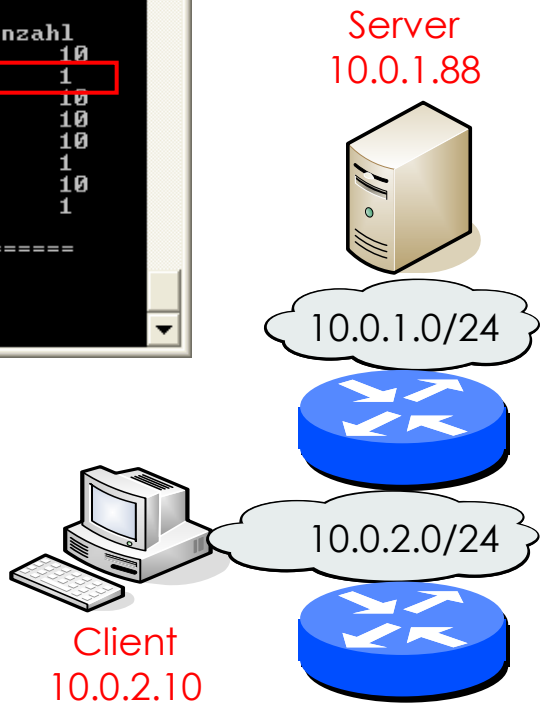


Internet Control Message Protocol (ICMP)

‚ICMP Redirect' message

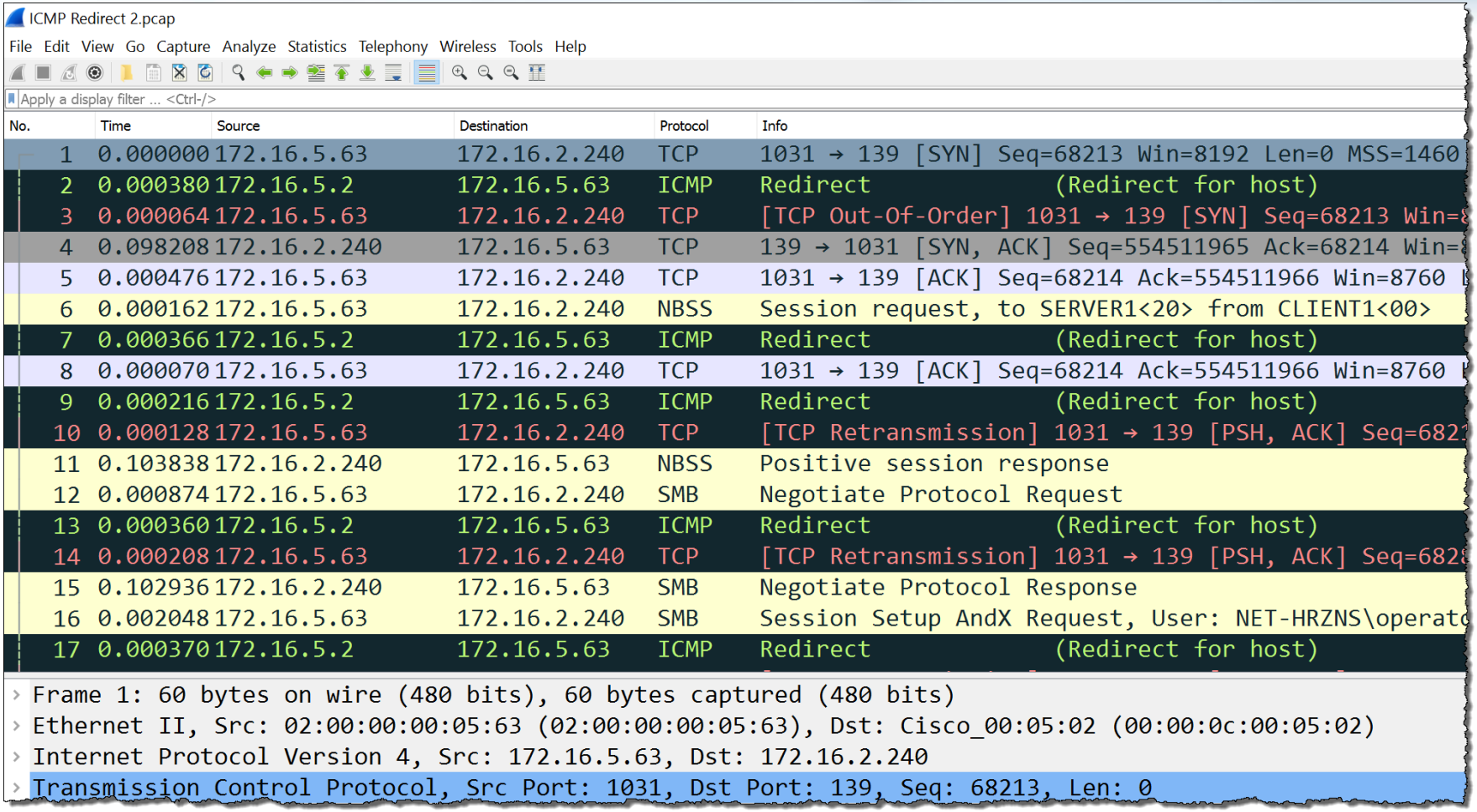


‚netstat -r' shows the routing table of the client after reception of the ‚ICMP Redirect' message



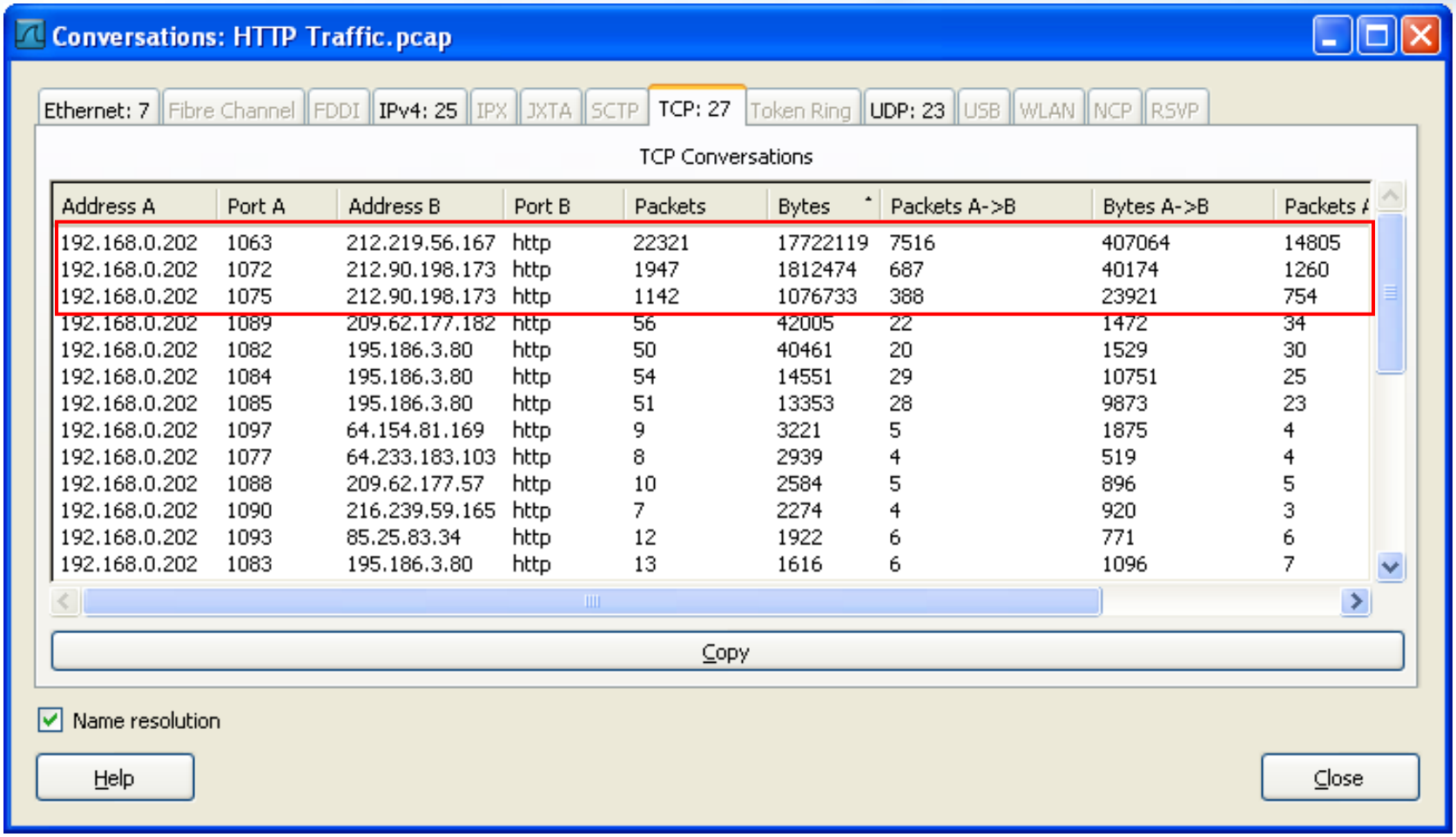
Internet Control Message Protocol (ICMP)

‚ICMP Redirect' message



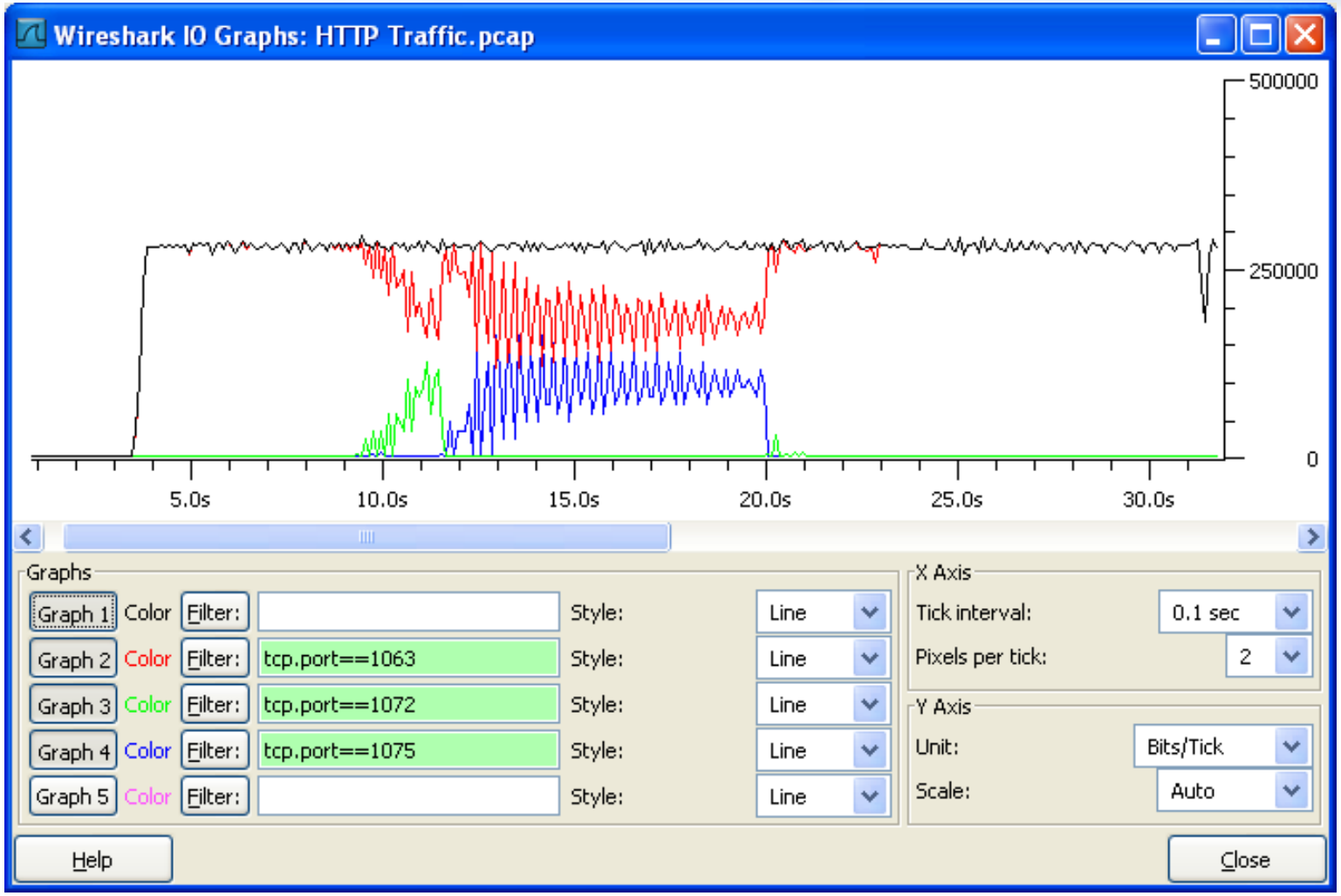
Miscellaneous Functions of Wireshark

Graphical presentation with I/O Graphs



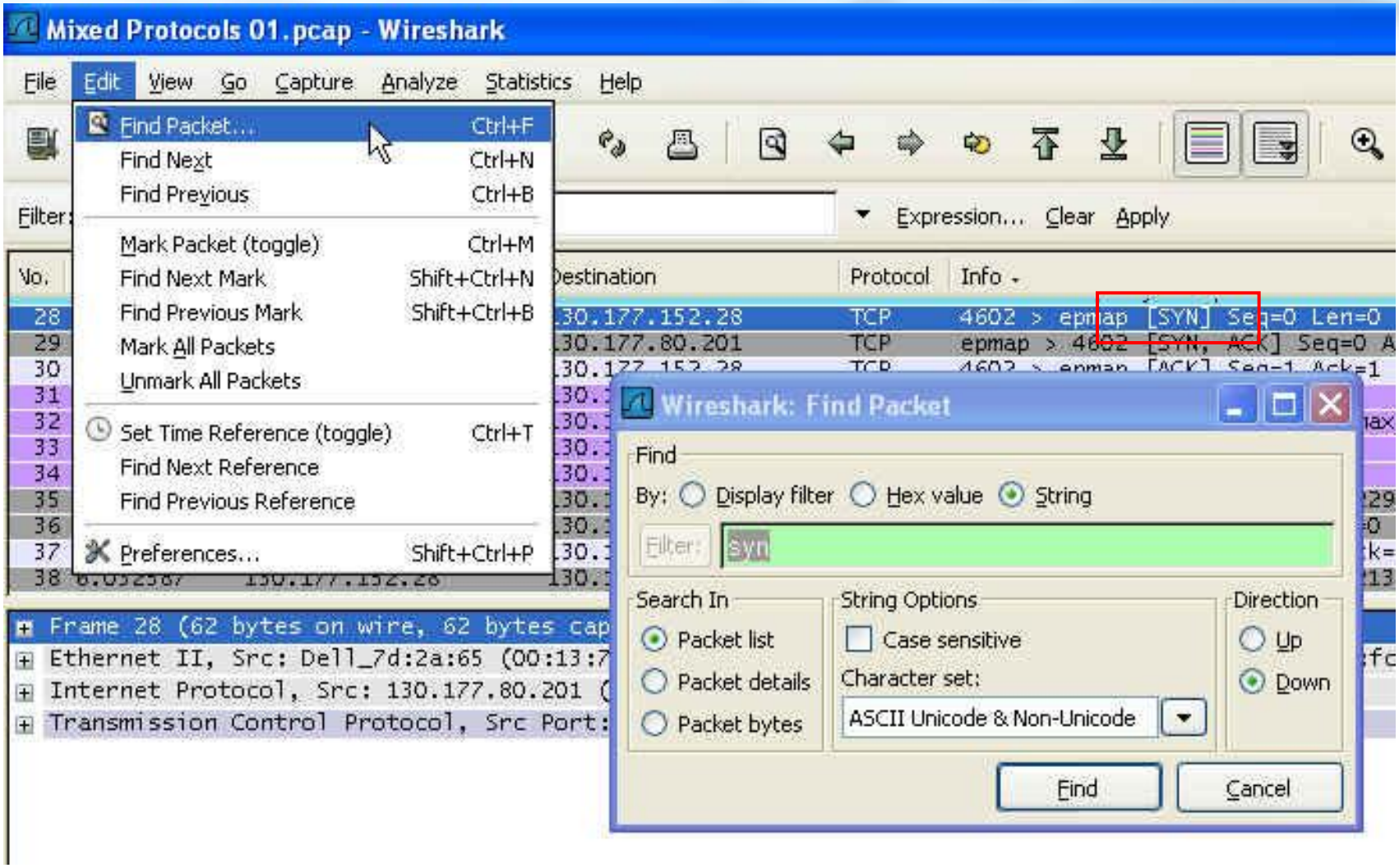
Miscellaneous Functions of Wireshark

Graphical presentation with I/O Graphs



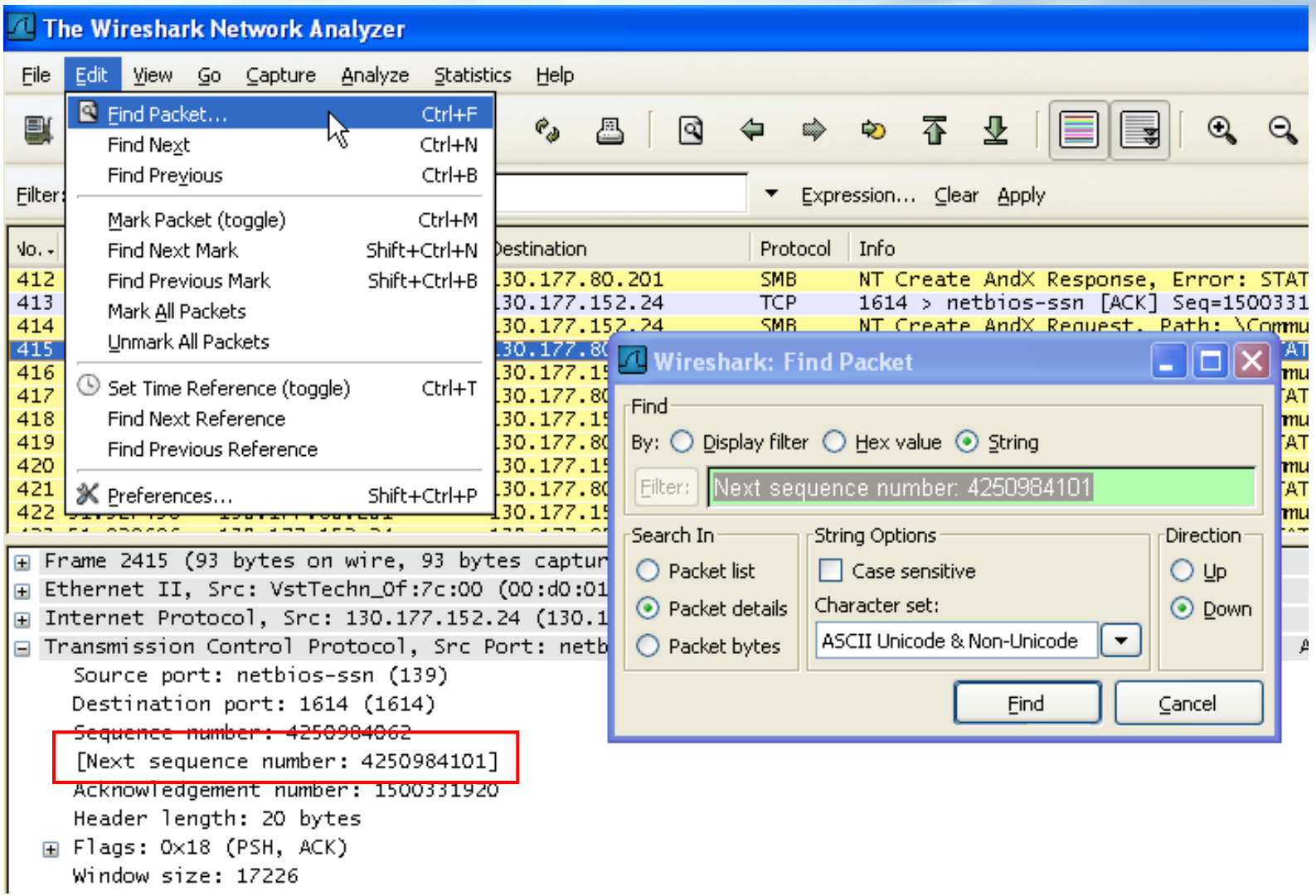
Miscellaneous Functions of Wireshark

Searching for text in the window ‚Packet list'



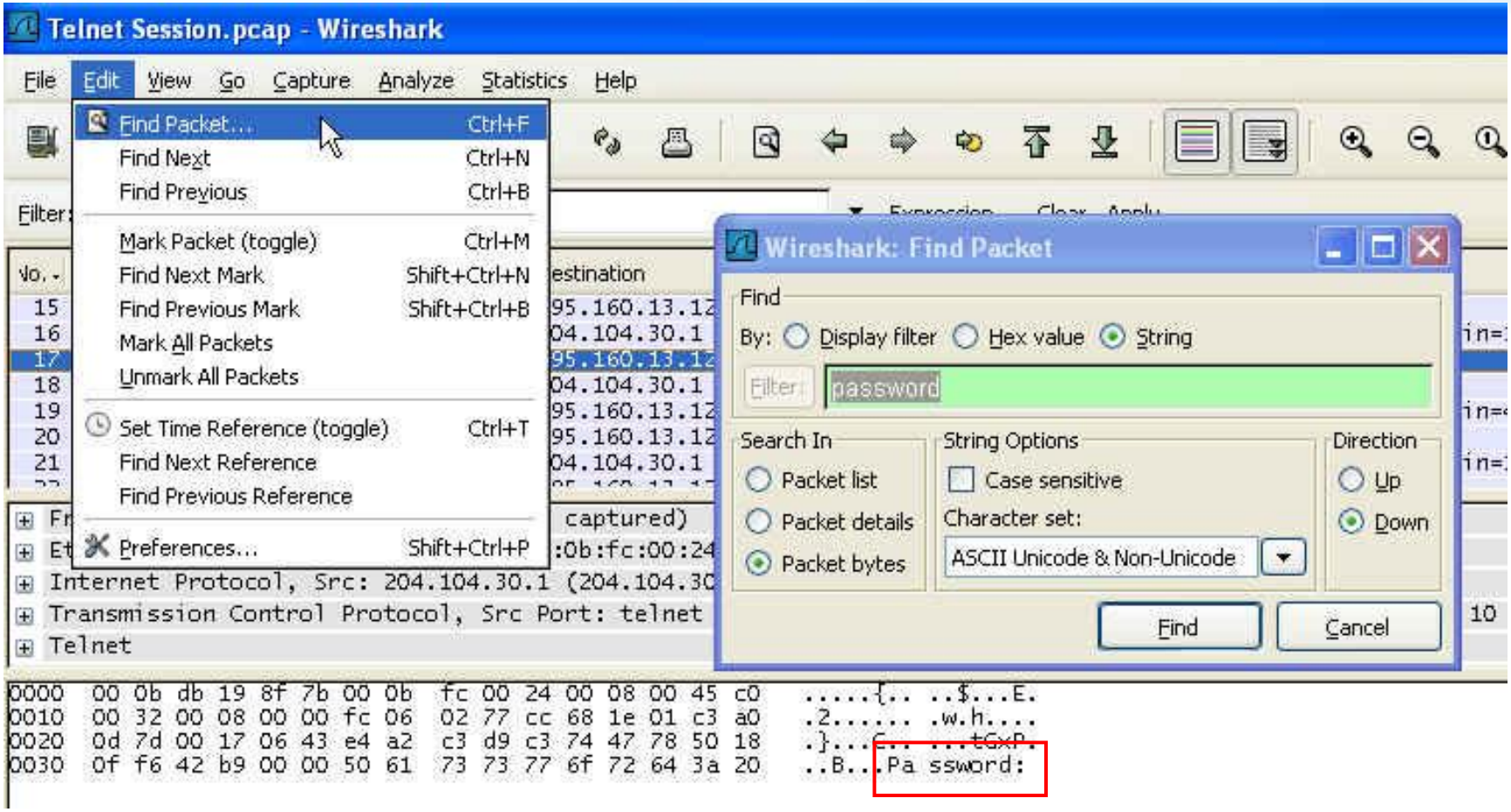
Miscellaneous Functions of Wireshark

Searching for text in the window ‚Packet details'



Miscellaneous Functions of Wireshark

Searching for text in the window ‚Packet bytes'



Capturing Data with Wireshark

‚Capturing' means the process of receiving, catching and storing data packets (frames).

These frames can be captured either from a wired medium (e.g. Ethernet) or from a wireless medium (e.g. Wireless LAN).

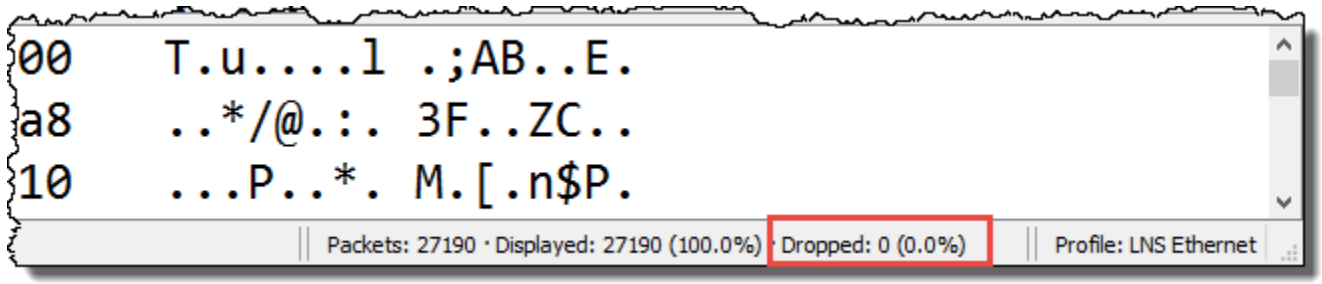


© Leutert NetServices 2026 / TCP / V4.1f

Capture Options

Avoiding or reducing Dropped Packets

- Wireshark displays the Dropped Counter after stopping the capture process.
- Missing packets may create wrong error messages from the Wireshark expert.

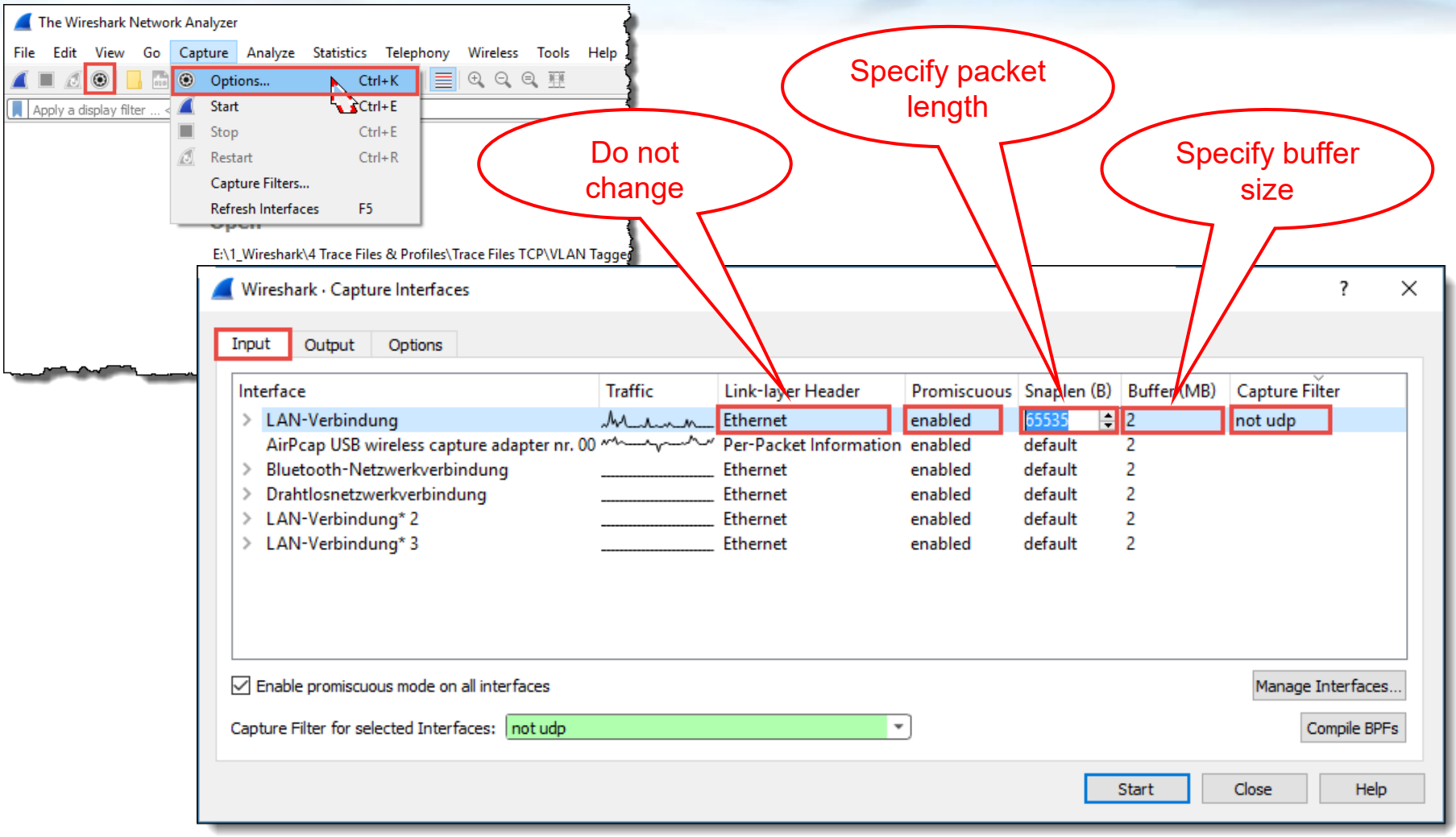


Use the following measures to reduce or prevent packet loss:

- Increase Buffer Size (using RAM storage)
- Reduce Snap Length (Packet Slicing)
- Use Capture Filter
- Replace Hard disk with SSD
- Use Capture Appliance for high rates (e.g. ProfiShark, Allegro Packets)

Capture Options

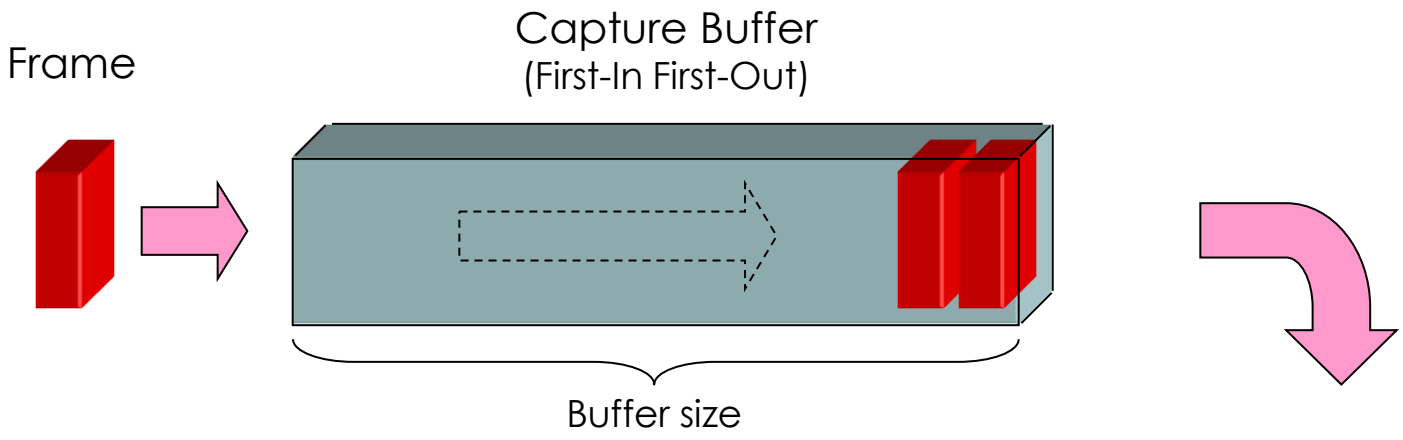
Input options

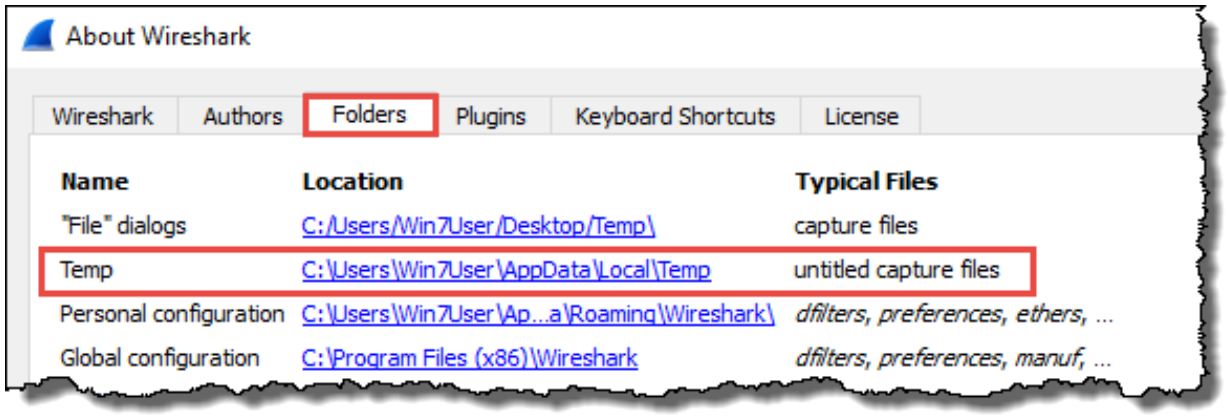


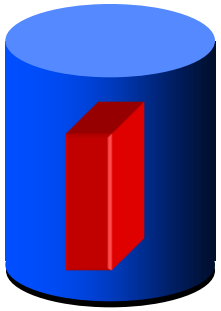
Capture Options

Input options

- When starting capturing, a temporary file will be automatically created.
- The frames will be sent through a FIFO-buffer to the storage media





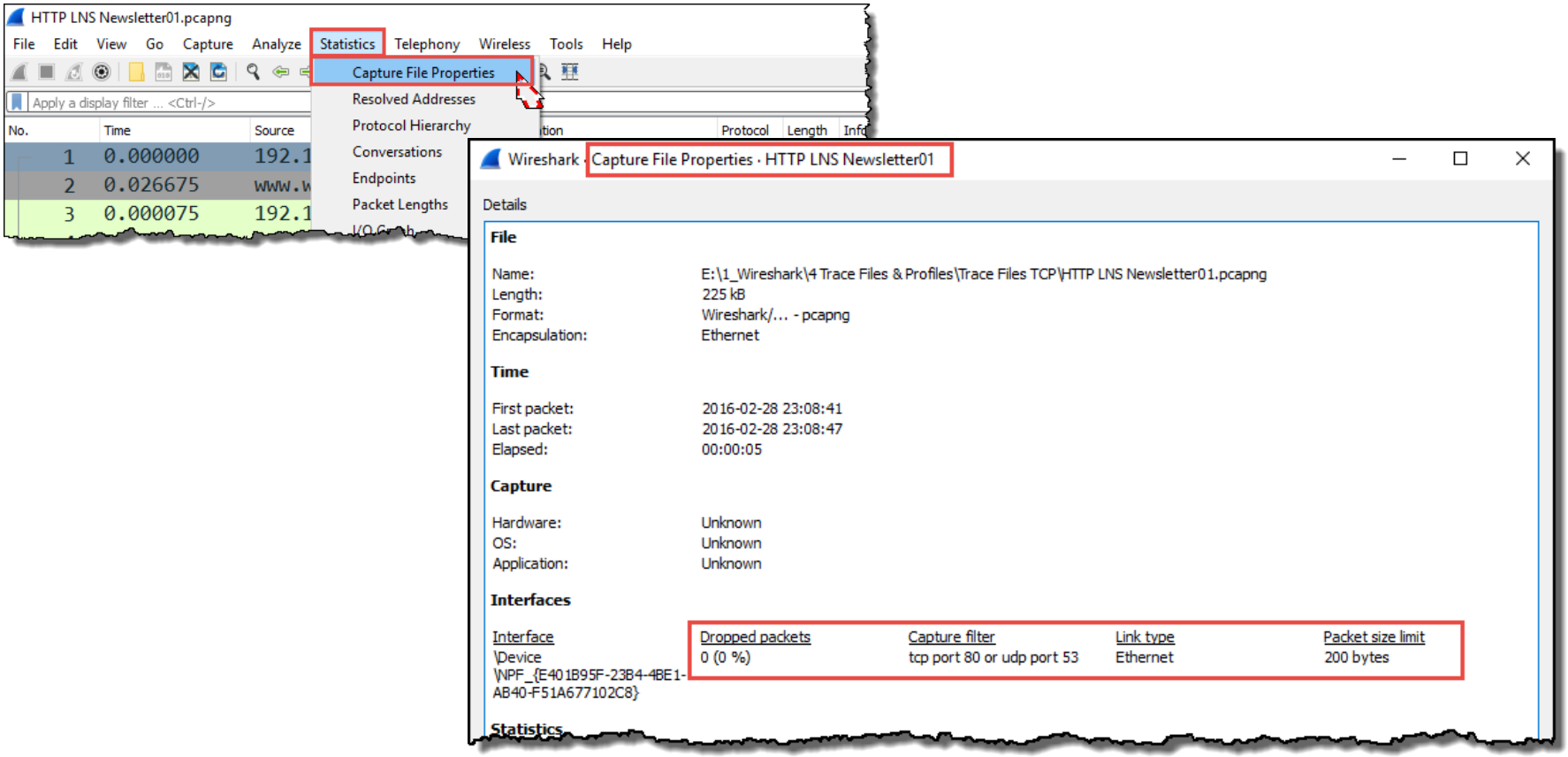


Hard Disk
or SSD

Capture Options

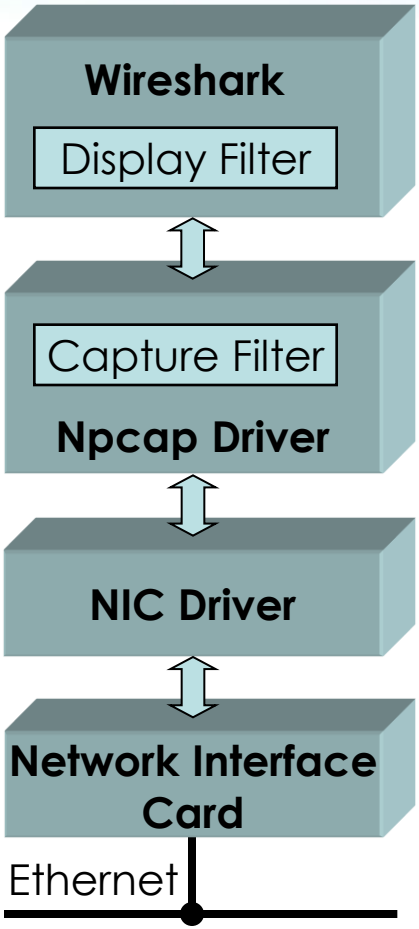
Capture information stored in pcapng file format

- Pcap file format only stores time stamp with frame data
- Pcap-ng (next generation) stores Interface ID, File & Packet Comment, DNS Names, Dropped Counter, Snap Length and Capture filter with the frames.



Capture Filter

Using ‚Display' or ‚Capture' Filter?



Display Filter:
All frames are read in the buffer file, but only those are displayed that meet the ‚Display' Filter conditions. During the saving process you can define which frames should be saved.

Capture Filter:
Forwarded to Wireshark are only frames which meet the filter conditions. This process creates smaller buffer files but consumes more CPU resources (it is also called Berkeley Packet Filter (BPF)).

Capture or Display Filter?

Comparison of ‚Display' und ‚Capture' Filter Syntax

Capture Filter:

ether host 00:11:95:b7:e0:3e
ether src 00:11:95:b7:e0:3e
ether dst 00:11:95:b7:e0:3e

host 192.168.178.1
src host 192.168.178.1
dst host 192.168.178.1

net 192.168.178.0 mask 255.255.255.0
net 192.168.178/24

arp
not ip
tcp
udp port 138

Operators: not, and, or

http://wiki.wireshark.org/CaptureFilters
or Google: tcp dump quick reference

Display Filter:

eth.addr == 00:11:95:b7:e0:3e
eth.src == 00:11:95:b7:e0:3e
eth.dst == 01:00:5e:00:00:09

ip.addr == 192.168.178.1
ip.src == 192.168.178.1
ip.dst == 192.168.178.1

ip.addr == 192.168.178.0/24
ip.host contains "192.168.178"

arp
not ip
tcp
udp.port == 138

Operators: not, and, or

http://wiki.wireshark.org/DisplayFilters

© Leutert NetServices 2026 / TCP / V4.1f

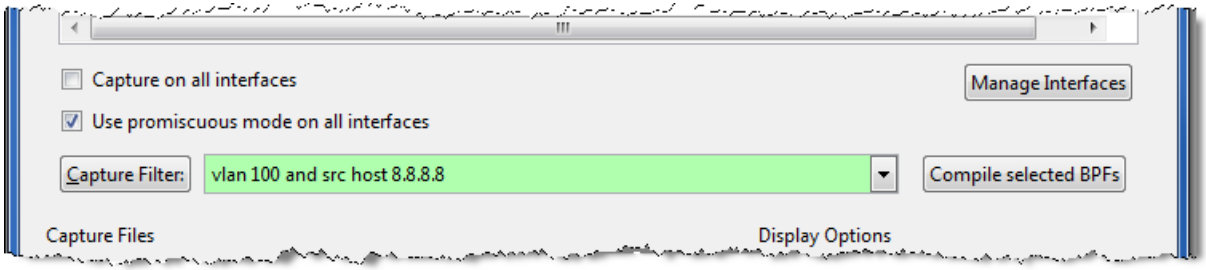
Capture or Display Filter?

Capture Filter

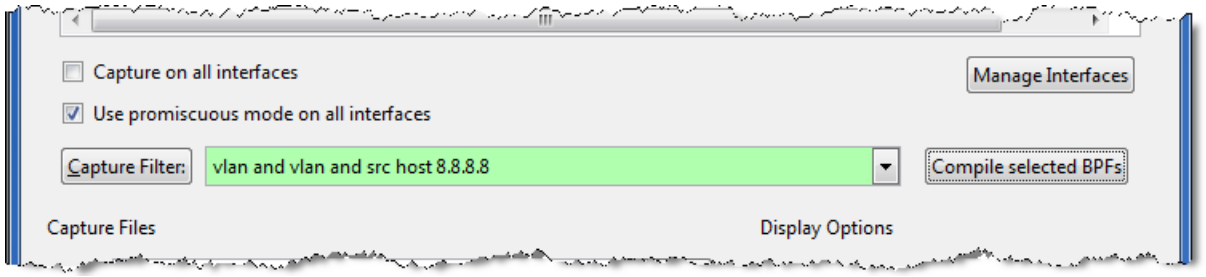
- Capture filter syntax must contain specifications about VLAN tags



Capture Filter on IP 8.8.8.8 with any one VLAN Tag



Capture Filter on IP 8.8.8.8 with VLAN 100 Tag



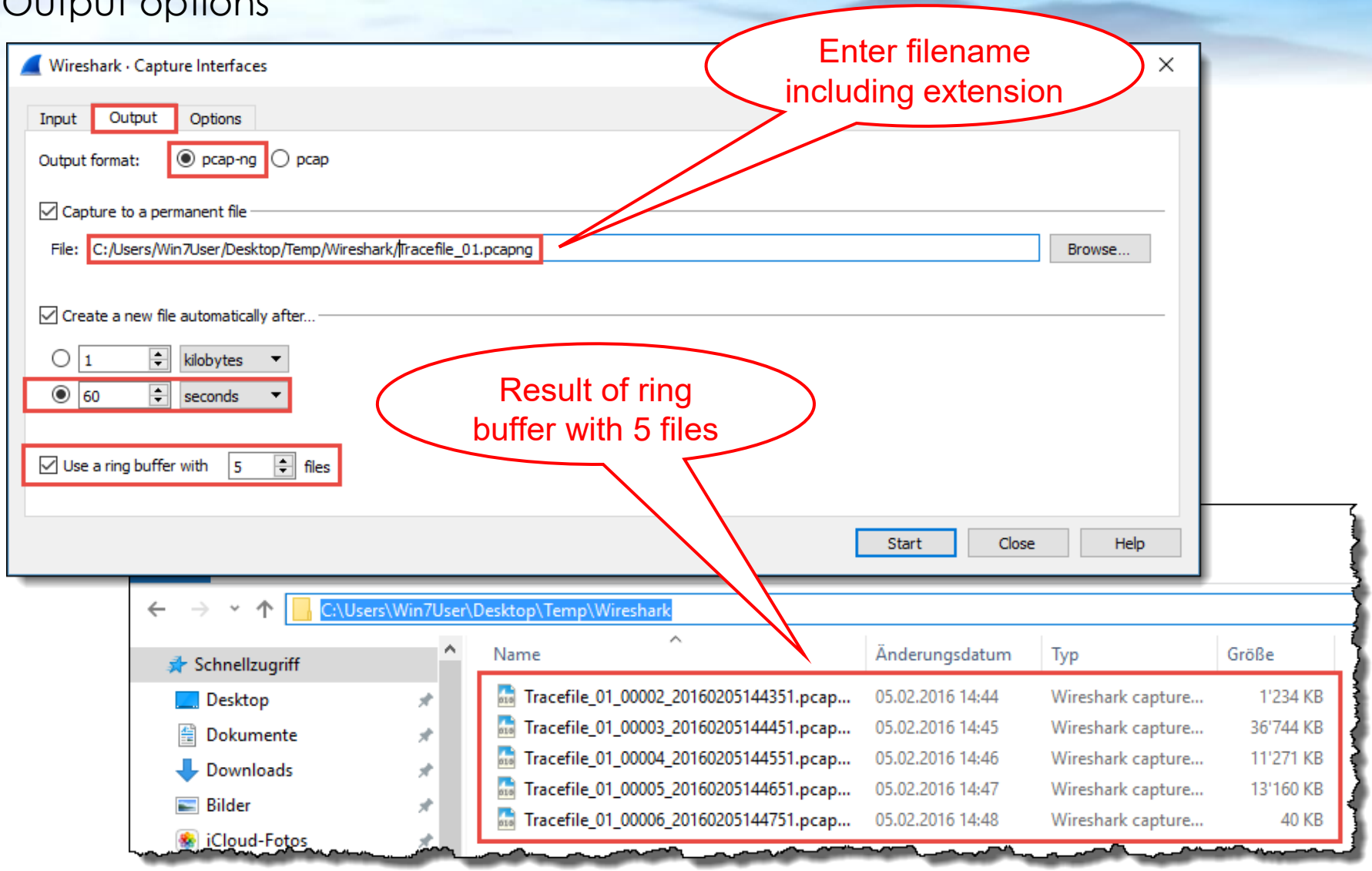
Capture Filter on IP 8.8.8.8 with any two VLAN Tags

+

© Leutert NetServices 2026 / TCP / V4.1f

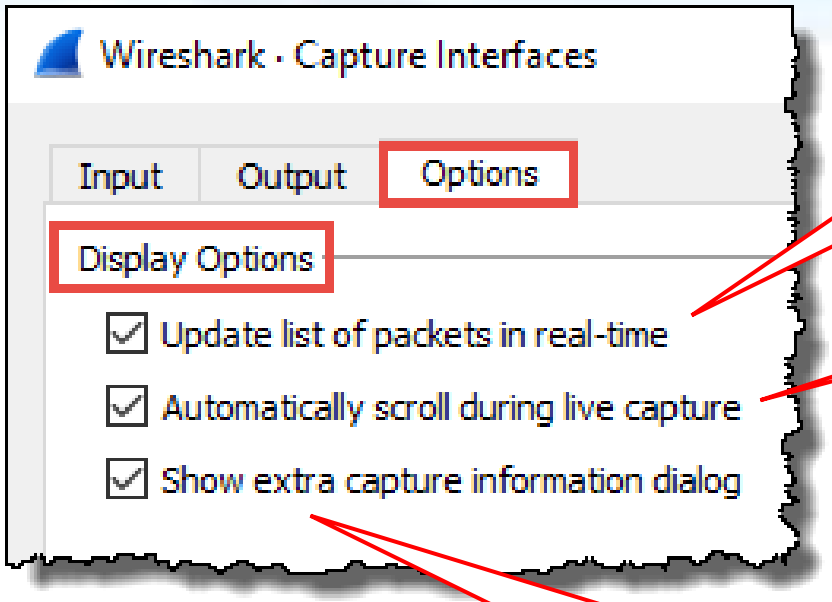
Capture Options

Output options



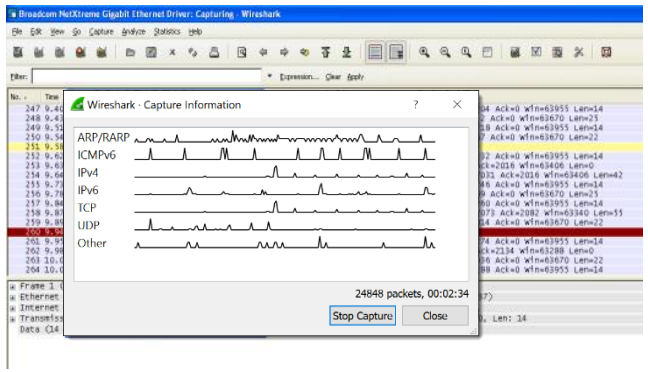
Capture Options

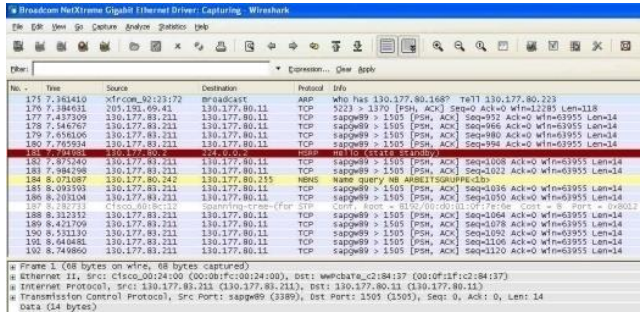
Display settings during capture



Decodes packets during capturing

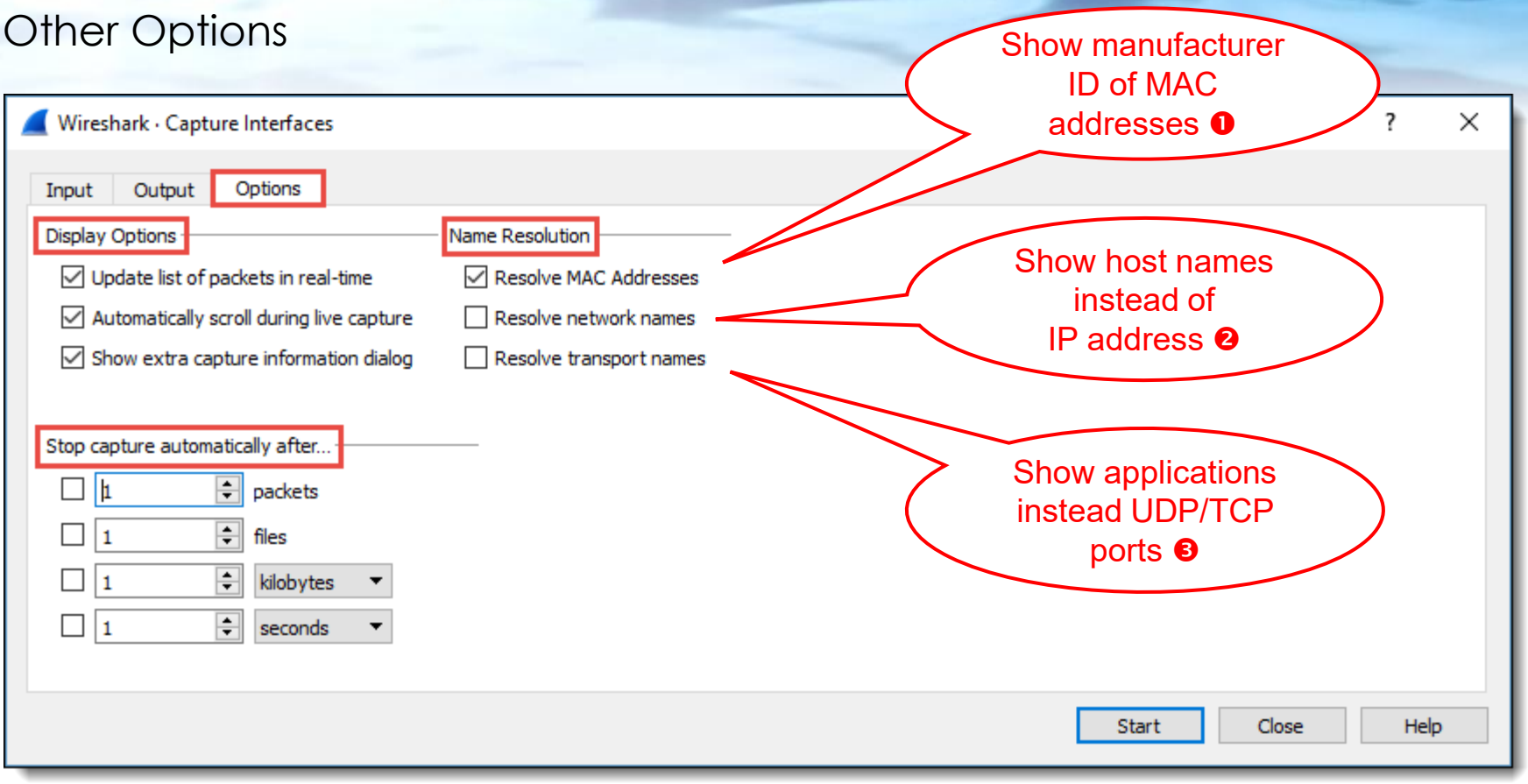
Scrolls packets during capturing





Capture Options

Other Options

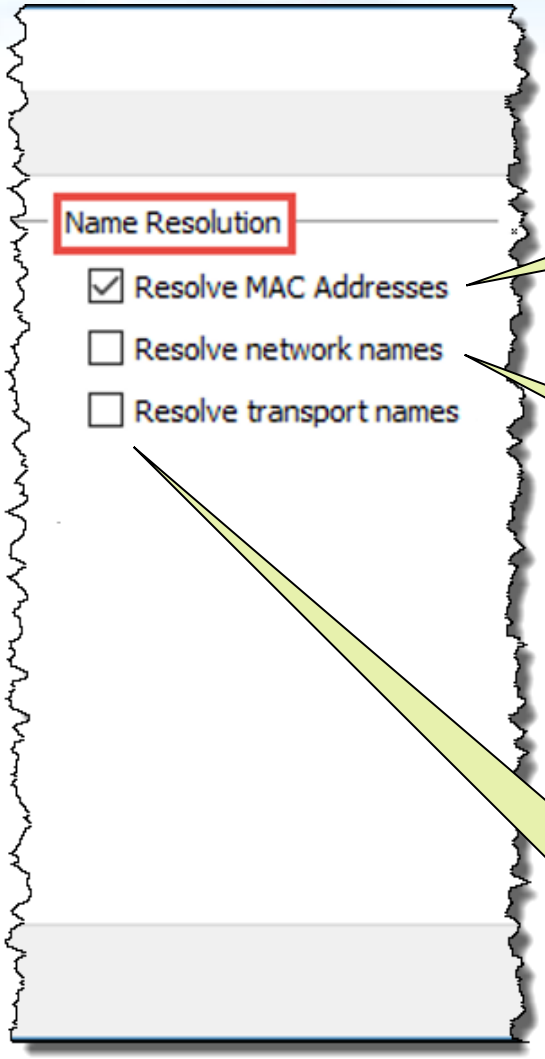


1. Wireshark is using the internal file..\Program Files (x86)\Wireshark\manuf
2. Wireshark becomes active and requires access to DNS server for Reverse Lookup
3. Wireshark is using the internal file..\Program Files (x86)\Wireshark\services

© Leutert NetServices 2026 / TCP / V4.1f

Capture Options

Other Options



Filter: arp

No.	Time	Source
911	12.53	Ibm_ca:f8:a4
912	12.54	Cisco_00:24:00
1257	17.78	WwPcbaTe_a8:23:e5

Filter: arp

No.	Time	Source
1	0.000	00:0f:1f:25:74:b9
5	2.198	00:0b:fc:00:24:00
6	2.739	00:13:72:7d:2a:65

Source	Destination
CHZHSCS101.ch.eds.com	130.177.80.201
130.177.80.201	CHZHSCS101.ch.eds.com

Remark: only possible if DNS is configured!

Source	Destination
130.177.152.24	130.177.80.201
130.177.80.201	130.177.152.24

```
Transmission Control Protocol, Src Port: 3807 (3807),
    Source port: 3807 (3807)
    Destination port: microsoft-ds (445)
```

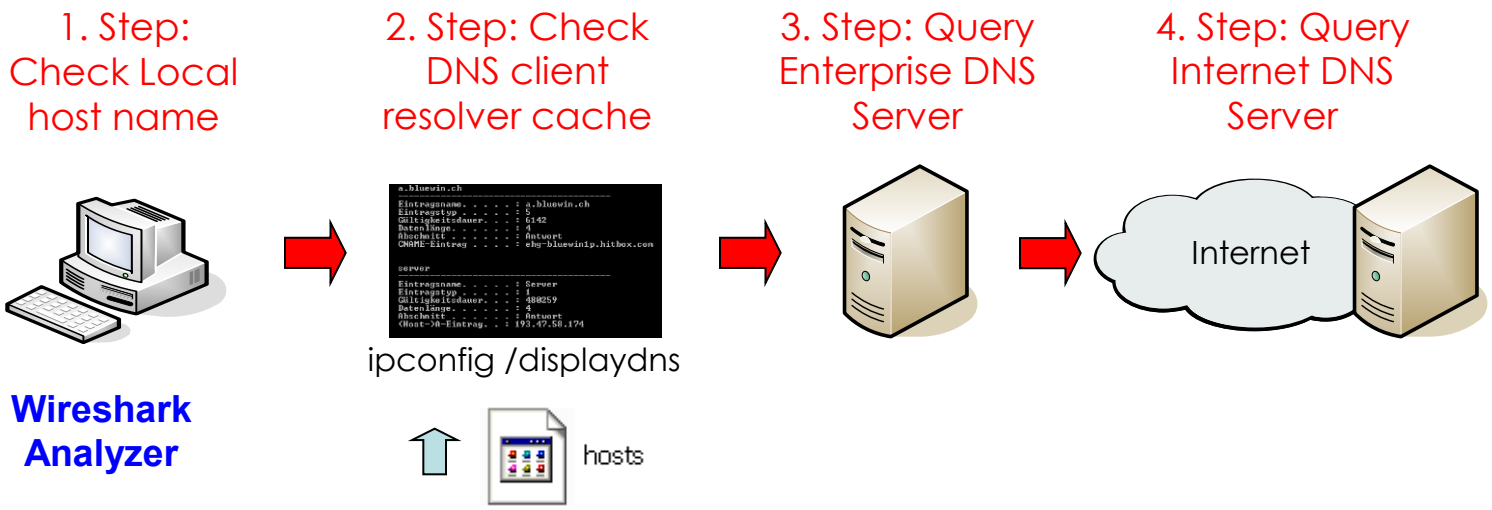
```
Transmission Control Protocol, Src Port: 3807 (3807),
    Source port: 3807 (3807)
    Destination port: 445 (445)
```


© Leutert NetServices 2026 / TCP / V4.1f

Capture Options

Name resolution during capturing data

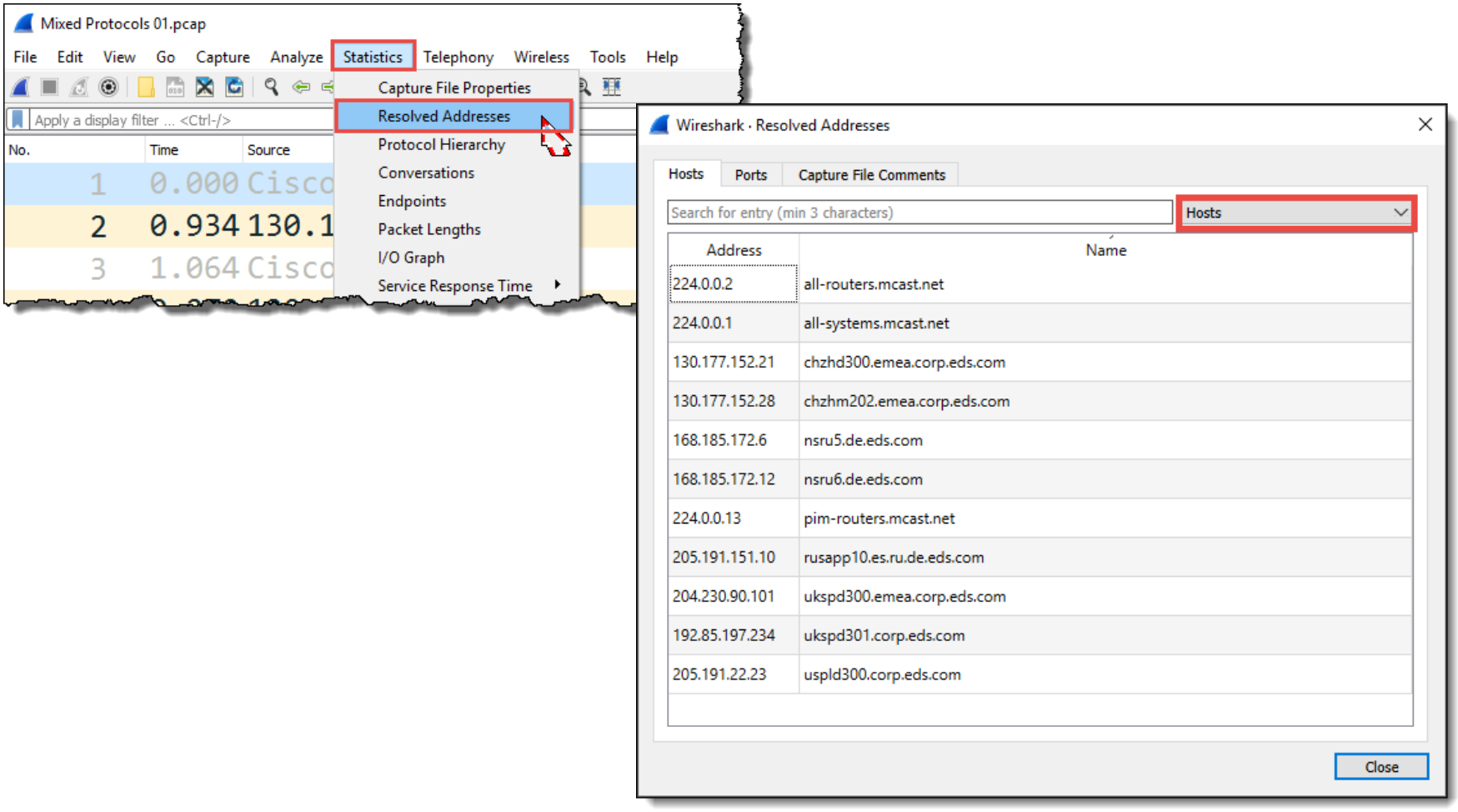
- Wireshark is using standard Windows-DNS-Resolver process
- Client running Wireshark requires full IP configuration with IP address etc.



Display Options

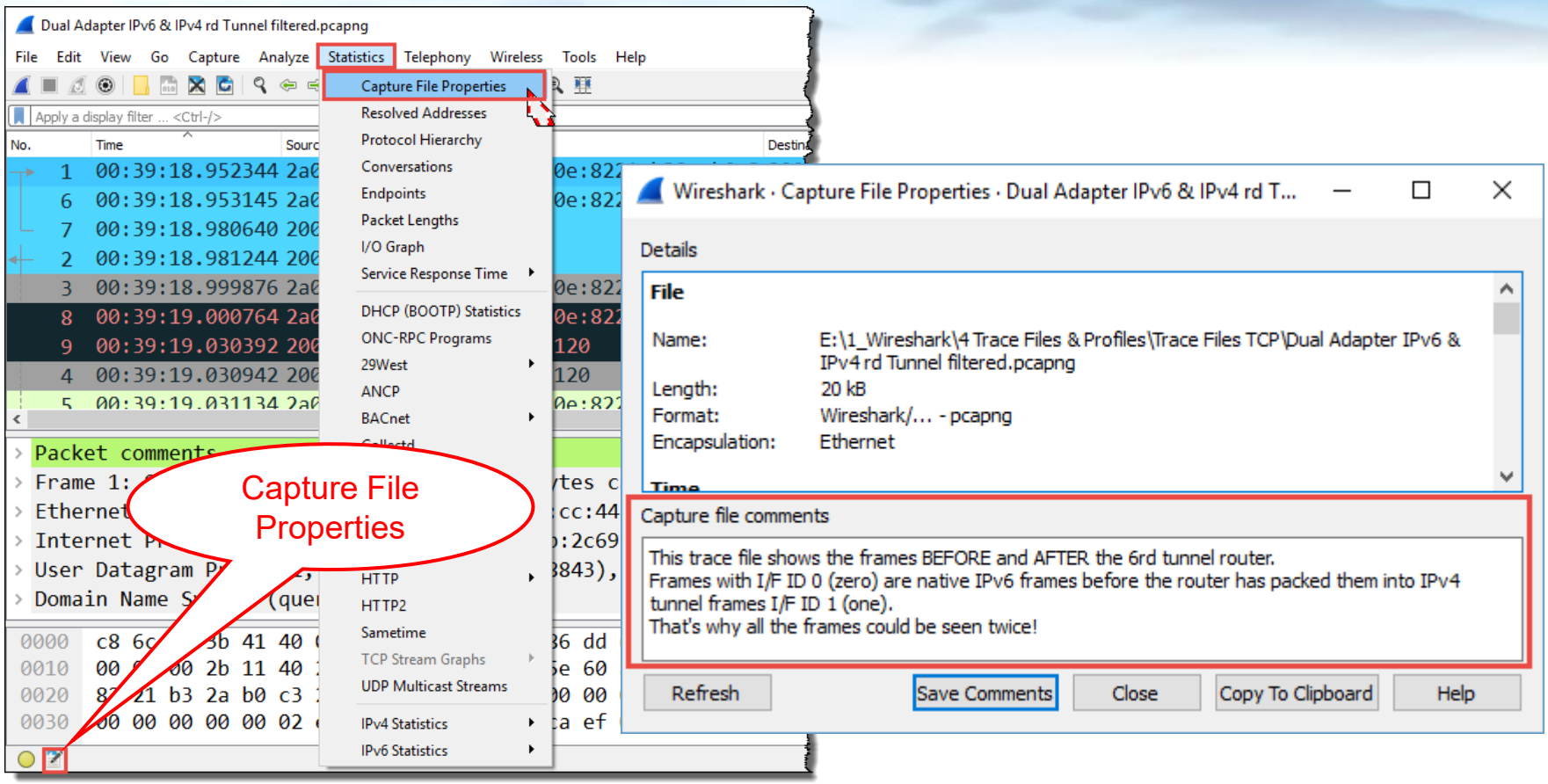
Name resolution

- Wireshark creates a list of resolved names



Capture Options

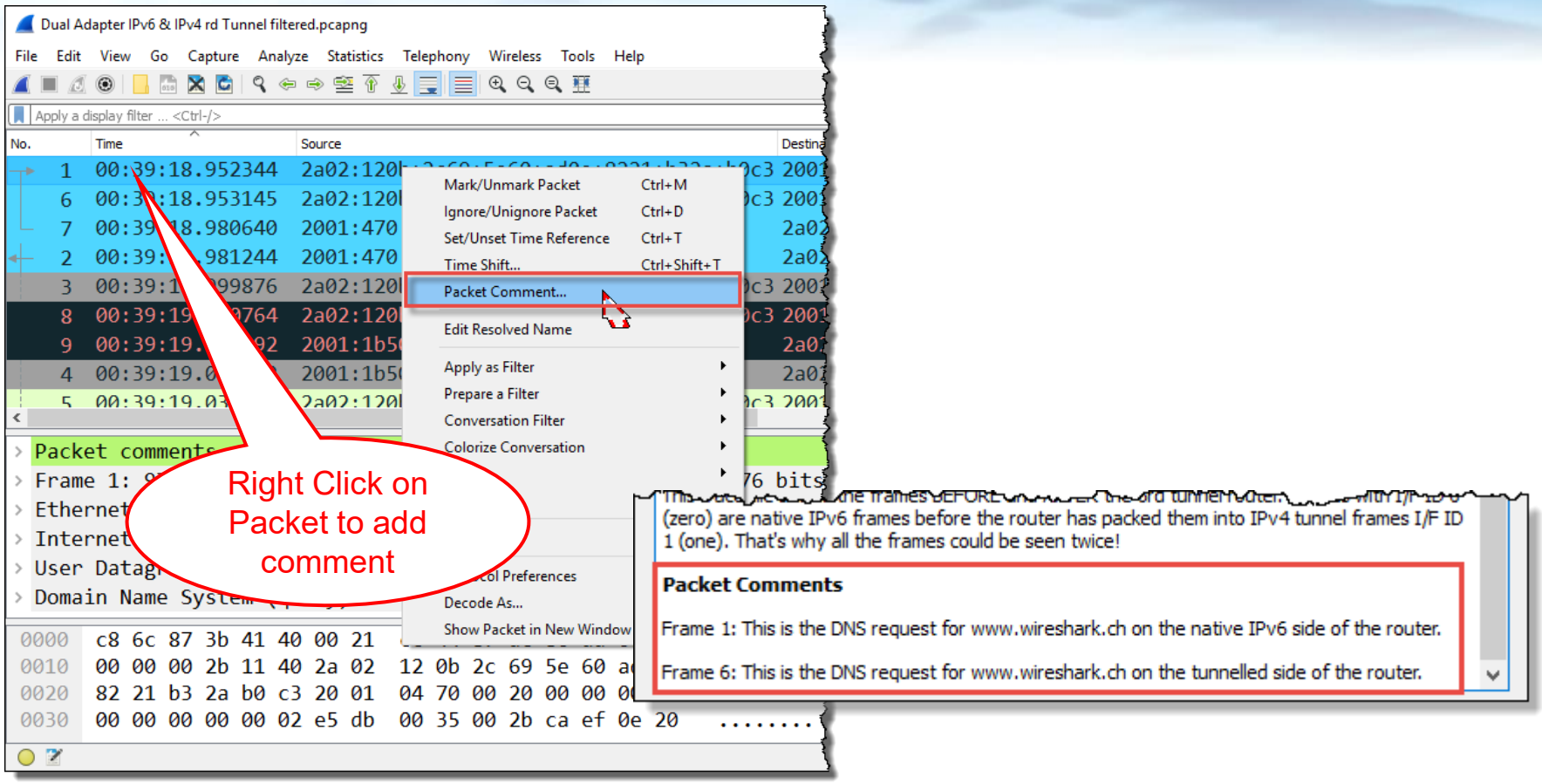
Adding Capture File Comments



- Two ways to add or read Capture File Comments.
- The File Comment is stored only in pcapng file format.

Capture Options

Adding Packet Comments



- Two ways to add or read Packet File Comments.
- The Packet Comment is stored only in pcapng file format

The End

Thank you for your attention!



Useful links:
https://www.netsniffing.ch/en/
www.wireshark.org/
www.wireshark.org/docs/
www.wireshark.org/faq.html
wiki.wireshark.org/
bugs.wireshark.org/bugzilla/

Register for the Wireshark Newsletter in German:
www.netsniffing.ch/de/wireshark-infos/newsletter

© Leutert NetServices 2026 / TCP / V4.1f